Путь Python-разработчика

От полного новичка до первой работы

Владислав Филиппов

1 ноября 2026

0 Содержание

1 Путь Python-разработчика: книга по Python для начинающих **20**

2 Путь Python-разработчика **21**

I 2 Том I. Основы программирования **22**

Быстрые ссылки . . . 23

2.1 Как думает компьютер **24**

После этой главы вы сможете . . . 24
Вы уже сталкивались с программированием . . . 25
Задача, алгоритм и программа . . . 26
Компьютер не умеет догадываться . . . 27
Первая практика . . . 28
Короткий итог . . . 28

2.2 Почему компьютер понимает только 0 и 1 **30**

Почему компьютер не понимает человеческий язык . . . 30
Что находится внутри компьютера . . . 31
Почему именно два состояния . . . 33
Откуда появились 0 и 1 . . . 34
Что такое бит . . . 36
Как из двух состояний получается сложная информация . . . 37
Всё превращается в числа . . . 38
Что происходит технически . . . 40
От транзисторов к логическим операциям . . . 41
Немного истории . . . 42
Почему это важно будущему Python-разработчику . . . 43
Практика . . . 44
Задание 1 . . . 44
Задание 2 . . . 44
Задание 3 . . . 45
Ответы к практике . . . 45
Ответ к заданию 1 . . . 45
Ответ к заданию 2 . . . 46

Ответ к заданию 3 . . . 46
Проверьте себя . . . 47
Ответы для самопроверки . . . 47
Главное, что нужно запомнить . . . 48
Что дальше . . . 49

2.3 Как компьютер хранит данные **50**

Бит — самая маленькая единица информации **51**

Байт **52**

Как компьютер понимает, что хранится в байте? **54**

Как хранится текст **56**

Почему этого оказалось недостаточно **58**

Как хранится число **59**

Как хранится изображение **60**

Как хранится музыка **62**

Как хранится программа **63**

Главное, что нужно запомнить **65**

Что дальше? **66**

2.4 Как компьютер выполняет программы **67**
После этой главы вы сможете . . . 67
Программа — это инструкция для компьютера . . . 67
Где находится программа до запуска . . . 68
Что происходит после запуска . . . 68
Как процессор выполняет инструкции . . . 69
Откуда процессор знает, какую инструкцию выполнять . . . 70
А где здесь Python? . . . 71
Короткий итог . . . 72
Практика . . . 72

2.5 Почему Python **73**
Можно ли писать программы сразу на машинном языке . . . 73
Язык программирования нужен человеку . . . 74
Почему языков программирования так много . . . 76
Почему для обучения выбран Python . . . 77
Python не лучший язык для всего . . . 79
В чём сила Python . . . 80
Где используется Python . . . 80

Веб-разработка . . . 81
Автоматизация . . . 81
Анализ данных . . . 82
Машинное обучение . . . 82
Тестирование . . . 82
Простота не означает примитивность . . . 82
Что именно мы будем изучать . . . 83
Что важно запомнить . . . 83
Что дальше? . . . 84

2.6 Рабочее пространство Python **85**
После этой главы вы сможете . . . 85
Основные инструменты . . . 85
Где скачать Python . . . 86
Python — это интерпретатор . . . 87
Что такое редактор кода . . . 88
Структура рабочего проекта . . . 88
Терминал . . . 89
Виртуальное окружение . . . 89
Короткий итог . . . 89
Практика . . . 90

2.7 Первая программа на Python **91**
Создаём файл программы . . . 91
Пишем первую строку Python . . . 92
Что делает `print` . . . 93
Запускаем программу . . . 94
Что произошло после команды запуска . . . 95
Код и результат — не одно и то же . . . 96
Добавим ещё одну инструкцию . . . 96
Порядок инструкций имеет значение . . . 97
Программа должна быть записана точно . . . 98
Ошибка — часть программирования . . . 99
Читайте сообщения об ошибках . . . 100
Первый маленький эксперимент . . . 101
Практика: программа-визитка . . . 101
Не бойтесь экспериментировать . . . 102
Что мы уже умеем . . . 103
Что важно запомнить . . . 103
Что дальше? . . . 104

II 3 Том II. Базовые конструкции Python **105**
Быстрые ссылки . . . 106

3.1 Переменные **107**
Значению можно дать имя . . . 107
Что означает знак = . . . 108
Переменную можно использовать много раз . . . 110
Значение переменной можно изменить . . . 110
Python использует текущее значение . . . 111
Имена переменных должны быть понятными . . . 112
Как можно называть переменные . . . 113
Попробуйте сами . . . 114
Практика: карточка разработчика . . . 114
Практика: изменение счёта . . . 115
Что важно запомнить . . . 115
Что дальше? . . . 116

3.2 Типы данных **117**
Что такое тип данных . . . 118
Тип принадлежит значению . . . 119
Python определяет тип автоматически . . . 120
Как узнать тип значения . . . 121
Целые числа: `int` . . . 122
Дробные числа: `float` . . . 123
Текст: `str` . . . 124
Одинаково выглядит — разный смысл . . . 126
Один оператор может работать по-разному . . . 126
Что произойдёт, если типы несовместимы . . . 128
Логические значения: `bool` . . . 130
Зачем нужны `True` и `False` . . . 131
Отсутствие значения: `None` . . . 131
Не путайте значение и его представление . . . 132
Тип может измениться при новом присваивании . . . 133
Динамическая типизация не означает отсутствие типов . . . 134
Основные типы, которые мы уже встретили . . . 134
Небольшой эксперимент . . . 135
Найдите ошибку . . . 137
Что важно запомнить . . . 137

3.3 Ввод данных **139**
Первая интерактивная программа . . . 139
Как работает `input()` . . . 140
Текст внутри `input()` . . . 142
Значение нужно сохранить . . . 142
Попробуйте сами . . . 143
Программа может задавать несколько вопросов . . . 144
`input()` возвращает значение . . . 144
Какой тип возвращает `input()` . . . 145

`input()` всегда возвращает строку . . . 147
Почему Python делает именно так . . . 148
Проверим разные значения . . . 149
Строка `"25"` — не число 25 . . . 150
Почему это важно . . . 151
Ошибка снова помогает нам . . . 152
Не спешите исправлять пример . . . 153
Ввод и вывод вместе . . . 154
Практика: небольшая анкета . . . 154
Практика: предскажите результат . . . 155
Практика: найдите проблему . . . 156
Практика: что хранится в переменных . . . 157
Задача: персональное приветствие . . . 158
Задача: мини-интервью . . . 159
Задача со звёздочкой . . . 159
Что важно запомнить . . . 160
Что дальше? . . . 161

3.4 Преобразование типов **163**
Что такое преобразование типов . . . 164
Первое преобразование . . . 165
Функция `int()` . . . 167
Исправляем программу с возрастом . . . 168
Преобразование можно выполнить сразу . . . 169
Попробуйте сами . . . 170
`int()` работает не с любой строкой . . . 171
Дробные числа и `float()` . . . 172
`float()` вместе с `input()` . . . 173
Когда использовать `int()`, а когда `float()` . . . 174
Пример: стоимость покупки . . . 175
Преобразование `float` в `int` . . . 176
Преобразование `int` в `float` . . . 177
Функция `str()` . . . 178
Зачем превращать число в строку . . . 179
Функция `bool()` . . . 180
Строки и `bool()` . . . 181
Тип до и после преобразования . . . 183
Преобразование не всегда возможно . . . 184
Попробуйте предсказать результат . . . 185
Что важно запомнить . . . 186
Что дальше? . . . 187
Практические задания . . . 188

3.5 Арифметические операции **192**
Основные арифметические операторы . . . 193

Сложение . . . 194
Вычитание . . . 195
Умножение . . . 196
Обычное деление . . . 197
Деление может давать дробный результат . . . 198
Деление на ноль . . . 199
Целочисленное деление . . . 200
Остаток от деления . . . 201
Зачем нужен остаток от деления . . . 203
Возведение в степень . . . 205
Несколько операций в одном выражении . . . 206
Приоритет операций . . . 207
Скобки делают вычисления понятнее . . . 209
Операции одинакового приоритета . . . 210
Особенность оператора ** . . . 211
Вычисления с int и float . . . 212
Вычисления с пользовательским вводом . . . 212
Попробуйте сами . . . 214
Что важно запомнить . . . 215
Практические задания . . . 216
Что дальше? . . . 221

3.6 Сравнение значений 223
Первое сравнение . . . 224
Операторы сравнения . . . 224
Равенство . . . 226
= и == — разные вещи . . . 227
Неравенство . . . 229
Больше и меньше . . . 230
Больше или равно . . . 231
Меньше или равно . . . 231
Результат сравнения имеет тип bool . . . 232
Сравнение переменных . . . 233
Сравнение после input() . . . 234
Сравнение строк . . . 236
Регистр имеет значение . . . 237
Сравнение строк с помощью > и < . . . 238
Сравнение разных числовых типов . . . 239
Значение и тип — не одно и то же . . . 239
Сначала вычисление, потом сравнение . . . 240
Приоритет арифметики и сравнения . . . 242
Можно сохранить результат сравнения . . . 243
Несколько полезных примеров . . . 243
Попробуйте сами . . . 245
Типичная ошибка: сравнение строки и числа . . . 245

Цепочки сравнений . . . 246
Что важно запомнить . . . 248
Практические задания . . . 249
Что дальше? . . . 253

3.7 Условия **255**
Первая условная конструкция . . . 256
Как работает if . . . 257
Что произойдёт при False . . . 258
Синтаксис if . . . 259
Отступ — часть синтаксиса Python . . . 260
Блок кода . . . 262
Условие после input() . . . 262
Ветка else . . . 264
Как работает if...else . . . 265
Пример с паролем . . . 266
Пример с бюджетом . . . 267
Несколько вариантов: elif . . . 268
Как работает elif . . . 269
Порядок условий имеет значение . . . 271
elif может быть несколько . . . 272
else не содержит условия . . . 273
elif и else не всегда обязательны . . . 273
Условием уже может быть bool . . . 274
Код после условия . . . 275
Вложенные условия . . . 276
Пример: билет в кино . . . 277
Пример: положительное, отрицательное или ноль . . . 278
Попробуйте сами . . . 279
Типичные ошибки . . . 280
Пропущено двоеточие . . . 280
Нет отступа . . . 280
Перепутаны = и == . . . 281
Неправильный порядок elif . . . 281
Что важно запомнить . . . 281
Практические задания . . . 283
Что дальше? . . . 291

3.8 Логические операции **293**
Три логических оператора . . . 293
Как работает and . . . 294
and внутри условия . . . 296
Как работает or . . . 297
or внутри условия . . . 298
Как работает not . . . 299

Сравнение, and, not и if . . . 300
Приоритет логических операций . . . 301
Сложное условие лучше разделять . . . 302
Короткое вычисление . . . 303
Что важно запомнить . . . 304
Практические задания . . . 305
Что дальше? . . . 308

3.9 Цикл while **309**
Зачем программе повторять действия . . . 310
Первый цикл while . . . 310
Как работает while . . . 311
Счётчик в цикле . . . 312
Граница включается или не включается . . . 314
Обратный счёт . . . 314
Бесконечный цикл . . . 316
while и пользовательский ввод . . . 317
Повторная проверка данных . . . 319
Накопитель . . . 320
Повторный ввод и накопитель . . . 321
Шаг счётчика . . . 322
Что важно запомнить . . . 323
Практические задания . . . 324
Что дальше? . . . 328

3.10 Цикл for **329**
Первый цикл for . . . 330
Как читать цикл for . . . 331
Как работает for . . . 331
Переменная цикла . . . 332
Что делает range() . . . 333
range() с одним аргументом . . . 334
range() с двумя аргументами . . . 335
range() с шагом . . . 336
Шаг может быть больше единицы . . . 338
Обратный отсчёт . . . 339
Важно правильно задать направление . . . 339
Повторение действия несколько раз . . . 341
Отступы работают так же, как в if и while . . . 342
for и while могут решать одну задачу . . . 343
Когда удобен while . . . 343
Когда удобен for . . . 344
for умеет перебирать строку . . . 345
Как перебирается строка . . . 346
Имя переменной цикла должно быть понятным . . . 347

Подсчёт количества итераций . . . 347
Типичная ошибка с правой границей . . . 348
Сумма чисел с for . . . 349
Накопитель внутри for . . . 349
Вычисления внутри цикла . . . 350
Условия внутри for . . . 351
Условие может зависеть от каждого значения . . . 351
Таблица умножения для одного числа . . . 352
Повторный input() с for . . . 353
Переменная цикла после завершения . . . 354
range() задаёт числа по правилам . . . 354
Только stop . . . 354
start и stop . . . 354
start, stop и step . . . 355
Отрицательный шаг . . . 355
range() не включает stop . . . 355
Пустой range . . . 356
Не нужно изменять переменную цикла вручную . . . 357
for — это не просто счётчик . . . 357
Сравним while и for . . . 358
Попробуйте сами . . . 358
Типичные ошибки . . . 359
Неправильная правая граница . . . 359
Неправильный шаг . . . 359
Лишнее изменение переменной . . . 360
Неправильный отступ . . . 360
Пошаговый разбор for . . . 360
Пример: сумма покупок . . . 361
Пример: символы слова . . . 362
Что важно запомнить . . . 362
Практические задания . . . 364
Что дальше? . . . 371

3.11 Строки **372**
Что такое строка . . . 372
Строка состоит из символов . . . 373
Индексы . . . 374
Получение символа по индексу . . . 375
Индекс может храниться в переменной . . . 376
Последний символ . . . 376
Отрицательные индексы . . . 377
Ошибка индекса . . . 378
Длина строки . . . 379
Пробел тоже является символом . . . 380
Пустая строка . . . 380

Последний индекс и длина строки . . . 381
Перебор строки через for . . . 382
Символ тоже является строкой . . . 383
Проверка длины одного символа . . . 384
Объединение строк . . . 384
Строки можно умножать . . . 385
Строку нельзя изменить по индексу . . . 386
Новую строку создать можно . . . 387
Срез строки . . . 388
Правая граница среза не включается . . . 389
Срез с начала строки . . . 390
Срез до конца строки . . . 391
Копия всей строки . . . 391
Отрицательные индексы в срезах . . . 392
Срез с шагом . . . 392
Строка в обратном порядке . . . 393
Проверка наличия текста с in . . . 394
Проверка отсутствия с not in . . . 395
in внутри if . . . 395
Регистр по-прежнему имеет значение . . . 396
Методы строк . . . 396
upper() . . . 397
lower() . . . 398
strip() . . . 399
Несколько методов подряд . . . 399
replace() . . . 400
count() . . . 400
find() . . . 401
startswith() . . . 402
endswith() . . . 403
Методы возвращают новые значения . . . 403
Подсчёт символов через for . . . 405
Подсчёт определённого символа . . . 405
Поиск символа внутри цикла . . . 406
Пример: количество гласных . . . 406
Пример: проверка начала и конца . . . 407
Пример: нормализация ответа . . . 408
Что лучше: индекс или for . . . 408
Что важно запомнить . . . 409
Практические задания . . . 411
Что дальше? . . . 418

3.12 Списки **419**
Первый список . . . 420
Тип list . . . 420

Элементы списка . . . 421
В списке могут храниться числа . . . 422
Список может содержать значения разных типов . . . 422
Пустой список . . . 423
Индексы списка . . . 424
Получение элемента по индексу . . . 425
Отрицательные индексы . . . 425
Ошибка индекса . . . 426
Длина списка . . . 426
Списки можно изменять . . . 427
Изменение элемента . . . 429
append() . . . 429
Добавление пользовательского значения . . . 430
append() внутри цикла . . . 430
insert() . . . 432
Разница между append() и insert() . . . 432
Удаление по значению: remove() . . . 433
remove() удаляет первое совпадение . . . 434
Если значения нет . . . 434
Удаление по индексу: pop() . . . 435
pop() без индекса . . . 435
clear() . . . 436
Проверка элемента с in . . . 437
not in . . . 437
Перебор списка через for . . . 438
Как for перебирает список . . . 439
Когда индекс не нужен . . . 439
Перебор по индексам . . . 440
Изменение элементов по индексам . . . 441
Срезы списков . . . 441
Другие срезы . . . 442
Срез создаёт новый список . . . 443
Объединение списков . . . 443
Повторение списка . . . 444
count() . . . 444
index() . . . 445
Сортировка списка . . . 445
sort() изменяет список . . . 446
Сортировка строк . . . 447
reverse() . . . 447
sum() . . . 448
min() и max() . . . 449
Среднее значение . . . 449
Важный случай: пустой список . . . 450
bool списка . . . 450

Проверка пустого списка . . . 451
Создание списка из пользовательского ввода . . . 452
Почему это лучше одного накопителя . . . 453
Пример: оценки . . . 454
Пример: список покупок . . . 455
Пример: поиск элемента . . . 455
Пример: фильтрация значений . . . 456
Изменение списка во время перебора . . . 457
Копия списка . . . 457
Присваивание — не копия . . . 458
Настоящая простая копия . . . 459
Что важно запомнить . . . 460
Практические задания . . . 462
Что дальше? . . . 471

3.13 Функции **472**
Что такое функция . . . 473
Создание функции с def . . . 474
Вызов функции . . . 476
Функцию можно вызвать несколько раз . . . 476
Несколько команд внутри функции . . . 477
Отступы внутри функции . . . 477
Код после функции . . . 478
Зачем нужны функции . . . 479
Понятные имена функций . . . 480
Параметры функции . . . 480
Как работает параметр . . . 482
Параметр и аргумент . . . 483
Одна функция — разные данные . . . 484
Несколько параметров . . . 485
Порядок аргументов имеет значение . . . 486
Функция для вычисления . . . 487
Функция и input() . . . 488
Возвращаемое значение . . . 488
Первый return . . . 489
Как работает return . . . 490
print() и return — не одно и то же . . . 491
print() . . . 492
return . . . 492
Возвращённое значение можно использовать сразу . . . 493
Пример: стоимость покупки . . . 494
Функция может возвращать строку . . . 495
Функция может возвращать bool . . . 496
return завершает выполнение функции . . . 496
Код после return . . . 497

Несколько return . . . 498
Что возвращает функция без return . . . 499
None как результат функции . . . 499
Параметры можно использовать внутри вычислений . . . 500
Функция может вызывать другую функцию . . . 501
Разделение большой задачи . . . 502
Локальные переменные . . . 503
Переменная внутри функции . . . 503
Параметры тоже локальные . . . 504
Внешняя и локальная переменная могут иметь одинаковое имя . . . 505
Не полагайтесь на глобальные данные без необходимости . . . 506
Аргументы можно брать из переменных . . . 507
Именованные аргументы . . . 508
Значения параметров по умолчанию . . . 509
Значение по умолчанию можно заменить . . . 510
Обязательные параметры идут раньше . . . 510
Функция и список . . . 510
Функция для суммы списка . . . 511
Функция может возвращать список . . . 512
Не смешивайте слишком много задач в одной функции . . . 513
Маленькие функции легче читать . . . 514
Функции уменьшают повторение кода . . . 514
Функции делают основную программу понятнее . . . 515
Типичные ошибки . . . 516
Функция определена, но не вызвана . . . 516
Забыт аргумент . . . 516
Передано слишком много аргументов . . . 517
Перепутаны print() и return . . . 517
Код после return . . . 518
Пошаговый разбор вызова . . . 518
Попробуйте сами . . . 519
Практический пример: итог покупки . . . 520
Практический пример: обработка списка . . . 522
Функции объединяют изученные конструкции . . . 523
Что важно запомнить . . . 524
Завершение тома II . . . 525
Практические задания . . . 529

III Том III. Структуры данных и организация кода 543
Быстрые ссылки . . . 544

4.1 Кортежи 545
Первый кортеж . . . 545
Элементы кортежа . . . 546

Кортеж похож на список . . . 547
Пустой кортеж . . . 548
Кортеж из одного элемента . . . 548
Скобки помогают читать код . . . 549
Индексы кортежа . . . 551
Ошибка индекса . . . 552
Длина кортежа . . . 552
Срезы кортежей . . . 552
Перебор и проверка . . . 553
Кортеж нельзя изменить . . . 554
Что означает неизменяемость . . . 554
Можно создать новый кортеж . . . 556
Объединение кортежей . . . 557
Повторение кортежа . . . 557
count() . . . 557
index() . . . 558
Почему у кортежа мало методов . . . 558
List и tuple . . . 558
Распаковка кортежа . . . 559
Распаковка трёх значений . . . 561
Количество переменных должно совпадать . . . 562
Кортеж можно распаковать сразу . . . 562
Обмен значений переменных . . . 563
Функция может вернуть несколько значений . . . 563
Пример: минимальное и максимальное значение . . . 565
Кортежи внутри списка . . . 566
Перебор списка кортежей . . . 567
Распаковка прямо в for . . . 567
Где list и tuple работают вместе . . . 568
bool кортежа . . . 568
Итоговый ориентир . . . 569
Попробуйте сами . . . 569
Что важно запомнить . . . 571
Практические задания . . . 572
Что дальше? . . . 582

4.2 Множества **584**
Что такое множество . . . 584
Создание множества . . . 586
Пустое множество . . . 586
Создание set из другой коллекции . . . 588
Сколько уникальных значений . . . 589
Проверка элемента . . . 590
У множества нет индексов . . . 591
Добавление элемента . . . 591

Удаление через remove() . . . 592
Удаление через discard() . . . 593
Метод pop() . . . 594
Очистка множества . . . 595
Перебор множества . . . 595
Объединение множеств . . . 596
Пересечение множеств . . . 597
Разность множеств . . . 598
Направление разности имеет значение . . . 599
Симметрическая разность . . . 600
Четыре операции рядом . . . 600
Пример: интересы пользователей . . . 602
Пример: уникальные посетители . . . 603
Итоговый пример: студенты двух курсов . . . 604
Set, list и tuple . . . 604
Что важно запомнить . . . 606
Практические задания . . . 607
Что дальше? . . . 615

4.3 Словари **617**
Что такое словарь . . . 617
Ключ и значение . . . 619
Создание словаря . . . 619
Пустой словарь . . . 620
Получение значения по ключу . . . 621
Ключи — не индексы . . . 621
Если ключа нет . . . 623
Добавление новой пары . . . 624
Изменение значения . . . 624
Один синтаксис — два результата . . . 625
Значения могут иметь разные типы . . . 626
Ключи должны быть уникальными . . . 627
Количество пар . . . 627
Проверка наличия ключа . . . 628
Удаление пары . . . 629
Пример: карточка товара . . . 630
Итоговый пример . . . 631
Список или словарь . . . 631
Что важно запомнить . . . 633
Практические задания . . . 634
Что дальше? . . . 641
Словари Python: краткое резюме . . . 641

4.4 Работа со словарями **643**
Безопасное получение через get() . . . 643

get() со значением по умолчанию . . . 644
Ключи через keys() . . . 645
Значения через values() . . . 646
Пары через items() . . . 647
Перебор словаря через for . . . 648
Перебор ключей и значений . . . 649
Обновление через update() . . . 650
Удаление через pop() . . . 651
Очистка через clear() . . . 653
Изменение значений во время перебора . . . 653
Не меняйте размер dict во время for . . . 654
Пример: дорогие товары . . . 655
Итоговый пример: анализ склада . . . 655
Что использовать . . . 657
Что важно запомнить . . . 658
Практические задания . . . 659
Что дальше? . . . 667
Работа со словарями Python: краткое резюме . . . 668

4.5 Вложенные структуры данных 669
Что такое вложенная структура . . . 670
Список словарей . . . 671
Доступ к одному объекту . . . 673
Читаем выражение слева направо . . . 673
Изменение вложенного значения . . . 675
Добавление поля во вложенный словарь . . . 676
Перебор списка словарей . . . 677
Фильтрация объектов . . . 678
Безопасное получение поля . . . 679
Добавление нового объекта . . . 680
Создание объекта перед append() . . . 681
Создание объекта из input() . . . 681
Словарь со списком . . . 682
Доступ к элементу списка внутри словаря . . . 683
Изменение списка внутри словаря . . . 684
Перебор списка внутри словаря . . . 684
Несколько списков внутри словаря . . . 685
Проверяйте тип структуры на каждом уровне . . . 686
Словарь внутри словаря . . . 687
Изменение вложенного словаря . . . 687
Пример: заказы . . . 688
Пример: сумма заказов . . . 689
Поиск объекта . . . 689
Добавление данных в найденный объект . . . 690
Итоговый пример: команда . . . 690

Не делайте вложенность глубже без необходимости . . . 691
Как выбрать структуру . . . 691
Что важно запомнить . . . 693
Практические задания . . . 694
Что дальше? . . . 701
Вложенные структуры данных Python: краткое резюме . . . 702

4.6 List comprehensions **706**
Первый list comprehension . . . 707
Обычный цикл и list comprehension . . . 708
Как читать list comprehension . . . 710
Новый список создаётся отдельно . . . 711
Преобразование чисел . . . 711
Преобразование строк . . . 712
Получение длины строк . . . 712
Фильтрация с if . . . 713
Как работает условие . . . 714
Чётные числа . . . 715
Фильтрация строк . . . 716
Преобразование и фильтрация вместе . . . 716
List comprehension со списком словарей . . . 718
Фильтрация списка словарей . . . 719
Получение одного поля после фильтрации . . . 720
Можно использовать range() . . . 721
Несколько простых выборок из заказов . . . 722
Когда list comprehension удобен . . . 722
Когда обычный for лучше . . . 723
Однострочная или многострочная запись . . . 724
Типичные ошибки . . . 724
Забыли квадратные скобки . . . 724
Перепутали порядок . . . 724
Слишком много логики . . . 725
Что выбрать: for или list comprehension . . . 725
Что важно запомнить . . . 726
Практические задания . . . 727
Что дальше? . . . 734
List comprehensions в Python: краткое резюме . . . 735

4.7 Обработка ошибок **736**
Ошибки и исключения . . . 737
Знакомые исключения Python . . . 737
Первая обработка ошибки: try и except . . . 738
Как работает try/except . . . 739
Что происходит после исключения . . . 740
Обработка деления на ноль . . . 741

Несколько блоков except . . . 741
Получение текста ошибки через as . . . 742
Обработка нескольких типов одним except . . . 743
Блок else . . . 743
Блок finally . . . 744
Полная конструкция try/except/else/finally . . . 744
Самостоятельное создание исключения: raise . . . 745
raise вместе с try/except . . . 746
Пример: проверка суммы перевода . . . 746
Почему не стоит использовать except без типа . . . 747
Когда try/except не нужен . . . 748
Типичные ошибки начинающих . . . 749
Слишком большой блок try . . . 749
Неправильный тип исключения . . . 749
Использование переменной после неудачного преобразования . . . 749
Что важно запомнить . . . 750
Практические задания . . . 751
Задание 1. Номер этажа . . . 751
Задание 2. Расход топлива . . . 751
Задание 3. Выбор дня недели . . . 752
Задание 4. Получение настроек . . . 753
Задание 5. Проверка уровня заряда . . . 754
Задание 6. Стоимость доставки . . . 754
Задание 7. Распределение мест . . . 755
Задание 8. Мини-проект: терминал покупки билетов . . . 757
Что дальше? . . . 758
Обработка ошибок Python: краткое резюме . . . 759

1 Путь Python-разработчика: книга по Python для начинающих

2 Путь Python-разработчика

От полного новичка до первой работы

Практическая книга по Python и backend-разработке: от первых шагов до уверенной подготовки к первой работе.

Эта книга строится вокруг простого принципа: сначала мышление разработчика, затем синтаксис, инструменты и реальные проекты.

Вы будете постепенно разбирать задачи, превращать их в алгоритмы, писать понятный Python-код и собирать профессиональную рабочую среду.

> **Как читать книгу**
>
> Двигайтесь по главам последовательно. В начале важнее понимать ход мысли, чем запоминать команды.

Часть I

2 Том I. Основы программирования

В этом томе собраны базовые главы, которые помогают понять, как компьютер работает с инструкциями, данными и первыми программами.

Быстрые ссылки

- 2.1 Как думает компьютер
- 2.2 Почему компьютер понимает только 0 и 1
- 2.3 Как компьютер хранит данные
- 2.4 Как компьютер выполняет программы
- 2.5 Почему Python
- 2.6 Рабочее пространство Python
- 2.7 Первая Python-программа

> **Как читать этот том**
>
> Идите по главам последовательно. Сначала разберитесь, как компьютер воспринимает инструкции и данные, затем переходите к рабочему пространству и первой программе на Python.

2 2.1 Как думает компьютер

Прежде чем изучать переменные, условия, циклы и функции, полезно разобраться с более фундаментальным вопросом:

что вообще происходит, когда человек пишет программу?

На первый взгляд кажется, что программирование начинается с изучения языка. Например, с Python.

Но язык — это только инструмент.

Настоящее программирование начинается раньше: с умения сформулировать задачу и разбить её на последовательность понятных действий.

В этой главе мы разберёмся, что такое программа, почему компьютер не умеет догадываться, зачем существуют языки программирования и какую роль во всём этом играет Python.

После этой главы вы сможете

После изучения главы вы сможете:

- объяснить своими словами, что такое программа;
- понимать разницу между задачей, алгоритмом и кодом;
- объяснить, почему компьютеру нужны точные инструкции;
- понимать, зачем существуют языки программирования;
- объяснить на базовом уровне, что происходит после запуска Python-программы;
- разбивать простую бытовую задачу на последовательность шагов.

> **Главная цель главы**
>
> Пока не нужно запоминать команды Python.
> Сначала необходимо понять принцип: **компьютер выполняет инструкции, а разработчик эти инструкции проектирует.**

Вы уже сталкивались с программированием

Представим простую задачу:

приготовить чашку чая.

Для человека этой фразы обычно достаточно. Мы автоматически понимаем, что нужно сделать.

Налить воду. Включить чайник. Подготовить кружку. Положить чай. Дождаться кипения воды. Налить воду в кружку.

Большую часть этих действий мы даже не проговариваем.

Наш мозг самостоятельно заполняет пропущенные шаги.

Но теперь представим устройство, которое умеет выполнять команды, но совершенно не умеет догадываться.

Мы говорим ему:

> Приготовь чай.

Для человека инструкция понятна.

Для машины — практически бесполезна.

Нужно описать задачу гораздо точнее:

1. Найди чайник.
2. Проверь, есть ли в нём вода.
3. Если воды недостаточно — добавь её.
4. Подключи чайник к электропитанию.
5. Включи чайник.
6. Возьми кружку.
7. Положи в неё чай.
8. Дождись кипения воды.
9. Налей горячую воду в кружку.
10. Подожди несколько минут.

Теперь большая задача превратилась в последовательность небольших действий.

Именно с этого начинается программирование.

> ❗ Запомните
>
> **Программирование — это прежде всего умение разбивать задачу на понятные и точные шаги.**
> Язык программирования нужен уже после того, как мы понимаем, какие действия должен выполнить компьютер.

Задача, алгоритм и программа

На этом этапе важно различать три понятия:

задача — результат, который мы хотим получить;

алгоритм — последовательность действий для достижения результата;

программа — алгоритм, записанный в форме, которую компьютер способен выполнить.

Возьмём простой пример.

Наша задача:

> узнать остаток денег после ежемесячных расходов.

Алгоритм можно описать обычным языком:

1. Узнать месячный доход.
2. Узнать месячные расходы.
3. Вычесть расходы из дохода.
4. Показать результат.

А позже мы сможем записать этот алгоритм на Python:

```
monthly_income = 5000
monthly_expenses = 3200

balance = monthly_income - monthly_expenses

print(balance)
```

Сейчас не нужно разбираться в каждой строке.

Важно увидеть связь:

```
Задача
   ↓
Алгоритм
   ↓
Код
   ↓
Выполнение
   ↓
Результат
```

Код не появляется из ниоткуда.

Перед кодом почти всегда должно существовать понимание того, **что именно должна делать программа**.

Компьютер не умеет догадываться

Одна из важнейших мыслей для начинающего разработчика:

компьютер не понимает намерения программиста.

Он выполняет то, что написано.

Не то, что мы хотели написать.

Не то, что имели в виду.

И не то, что кажется очевидным человеку.

Представим инструкцию:

> Если пользователь взрослый, разрешить регистрацию.

Человек понимает общий смысл.

Но компьютеру этого недостаточно.

Что значит «взрослый»?

18 лет?

21 год?

Возраст должен быть больше 18 или больше либо равен 18?

Что делать, если возраст вообще не указан?

Что делать, если вместо возраста пользователь ввёл слово?

Каждый подобный вопрос разработчику приходится превращать в конкретные правила.

Например:

```
Если возраст пользователя больше или равен 18:
    разрешить регистрацию.
Иначе:
    отказать в регистрации.
```

Позже это превратится в настоящий Python-код.

> Как думает разработчик
>
> Начинающий программист часто спрашивает:
>
> > Какую команду Python мне здесь написать?

> Более полезный вопрос:
>
> > Какие точные действия должна выполнить программа?
>
> Сначала решение. Затем код.

Первая практика

Пока не открывайте Python.

Попробуйте выполнить упражнение обычным текстом.

Нужно написать инструкцию для робота, который должен почистить зубы.

Плохой вариант:

```
1. Взять щётку.
2. Почистить зубы.
```

Слишком много действий скрыто внутри второго шага.

Попробуйте описать процесс подробнее.

Например, подумайте:

- откуда взять зубную щётку;
- что сделать с зубной пастой;
- сколько пасты использовать;
- когда открыть воду;
- когда её закрыть;
- сколько времени чистить зубы;
- что сделать со щёткой после использования.

Чем точнее инструкция, тем ближе она к алгоритму.

> ⚠ Типичная ошибка новичка
>
> Не пытайтесь сразу переводить любую задачу в Python.
> Сначала научитесь формулировать решение обычными словами.
> Если алгоритм непонятен на русском языке, код почти наверняка тоже получится запутанным.

Короткий итог

На этом этапе достаточно запомнить четыре вещи:

1. У программы всегда есть задача.

2. Задачу необходимо разбить на последовательность действий.
3. Компьютер выполняет инструкции буквально.
4. Python нужен для записи этих инструкций в форме, которую можно выполнить.

В следующем разделе мы разберёмся, **почему компьютер вообще способен работать только с очень простыми сигналами и откуда появились знаменитые 0 и 1**.

2 2.2 Почему компьютер понимает только 0 и 1

В предыдущей главе мы разобрались, что компьютер выполняет только точные инструкции.

Но возникает следующий вопрос:

почему компьютер вообще работает с нулями и единицами?

Почему нельзя заставить его сразу понимать слова, числа или команды так, как это делает человек?

Чтобы ответить на этот вопрос, нужно немного заглянуть внутрь компьютера.

Не слишком глубоко. Нам не понадобится изучать электронику или физику полупроводников.

Для Python-разработчика важно понять только основной принцип.

Почему компьютер не понимает человеческий язык

Человек умеет использовать контекст.

Если сказать:

> Сделай чай.

другой человек обычно понимает, что нужно сделать.

Он знает:

- где находится чайник;
- сколько примерно налить воды;
- какую кружку взять;
- когда вода закипела;
- что означает слово «чай».

Большая часть информации вообще не проговаривается.

Наш мозг автоматически достраивает недостающие шаги.

Компьютер так не умеет.

Если программа должна выполнить определённое действие, это действие необходимо описать достаточно точно.

Компьютер не понимает намерение разработчика.

Он выполняет только то, что действительно указано в программе.

> ! Главная мысль
>
> Компьютер не обладает здравым смыслом.
> Он не догадывается, что программист хотел сделать.
> Он выполняет конкретные инструкции.



Рисунок 2.1: Человек использует контекст, а компьютеру нужны точные инструкции

Что находится внутри компьютера

Современный компьютер состоит из огромного количества электронных компонентов.

Один из самых важных — **транзистор**.

Транзистор — это очень маленький электронный переключатель.

Для понимания программирования его можно представить как обычный выключатель света.

Выключатель имеет два устойчивых состояния:

```
Включён

Выключен
```

Транзистор работает по похожему принципу.

В упрощённом виде для нас достаточно считать, что он различает два состояния электрического сигнала:

```
Высокий уровень сигнала

Низкий уровень сигнала
```

Или ещё проще:

```
Есть сигнал

Нет сигнала
```

> **Насколько точно это объяснение?**
>
> В реальной электронике всё немного сложнее.
> Компьютер работает не с буквальными состояниями «ток есть» и «тока нет», а с диапазонами напряжений, которые электронные схемы интерпретируют как два логических состояния.
> Но для первого знакомства модель «включено / выключено» достаточно точно передаёт основной принцип.

Современный процессор содержит огромное количество транзисторов.

Каждый из них очень мал, но вместе они способны выполнять миллиарды операций.

Чем больше транзисторов можно разместить внутри микросхемы и чем быстрее они переключаются, тем более сложные вычисления способен выполнять процессор.



Рисунок 2.2: Транзистор как электронный переключатель

Почему именно два состояния

Можно задать вполне разумный вопрос:

> Почему только два?

Почему не три?

Почему не десять?

Теоретически вычислительную систему можно построить и на другом количестве состояний.

Такие идеи действительно существовали и существуют.

Но двоичная система оказалась чрезвычайно удобной для электронной техники.

Причина — **надёжность**.

Представим, что электронная схема должна различать десять уровней напряжения.

Условно:

```
0.5 В
1.0 В
1.5 В
2.0 В
...
```

Тогда даже небольшая помеха может привести к ошибке.

Например, устройство ожидало 1.5 В, а из-за электрического шума получило 1.6 В.

Какое это состояние?

Чем больше состояний должна различать система, тем сложнее надёжно определить каждое из них.

С двумя состояниями гораздо проще.

Электронная схема может работать по принципу:

```
Низкий уровень → одно состояние

Высокий уровень → другое состояние
```

Между ними можно оставить запас.

Если сигнал немного изменится из-за помех, схема всё равно правильно определит его состояние.

Именно поэтому два состояния особенно хорошо подходят для быстрых и надёжных цифровых устройств.

> Простая аналогия
>
> Представьте два выключателя.
> Первый имеет только два положения:
>
> ```
> ВКЛ
> ВЫКЛ
> ```
>
> Второй имеет десять очень близко расположенных положений.
> Каким выключателем проще пользоваться без ошибки?
> Примерно по той же причине электронным схемам удобно работать с двумя состояниями.

Откуда появились 0 и 1

Сами транзисторы не знают никаких цифр.

Внутри компьютера не бегают настоящие нули и единицы.

Цифры **0** и **1** придумали люди как удобные обозначения двух состояний.

Условно можно записать:

```
Низкий уровень сигнала → 0

Высокий уровень сигнала → 1
```

Можно было выбрать и другие обозначения:

```
НЕТ / ДА

ВЫКЛ / ВКЛ

FALSE / TRUE

ЧЁРНЫЙ / БЕЛЫЙ
```

Принцип работы компьютера от этого бы не изменился.

Но обозначения `0` и `1` оказались особенно удобными, потому что позволяют использовать **двоичную систему счисления**.

О ней мы поговорим подробнее позже.

Пока достаточно понимать:

0 и 1 — это математические обозначения двух физических состояний.



Рисунок 2.3: Включённые и выключенные состояния можно записать как последовательность нулей и единиц

Что такое бит

Теперь можно аккуратно познакомиться с первым важным термином.

Бит — это минимальная единица цифровой информации, которая может принимать одно из двух значений:

```
0
```

или

```
1
```

Слово `bit` произошло от английского выражения:

binary digit — двоичная цифра.

Один бит способен хранить только два варианта.

Например:

```
0 → нет

1 → да
```

или:

```
0 → свет выключен

1 → свет включён
```

Но одного бита недостаточно для хранения сложной информации.

Поэтому компьютеры используют большие последовательности битов.

Например:

```
01000001
```

Здесь уже восемь битов.

Группа из восьми битов называется **байтом**.

Подробно с битами, байтами и тем, как из них складываются данные, мы познакомимся в следующей главе.

> **Не нужно зубрить**
>
> Пока достаточно запомнить:
>
> - бит принимает значение `0` или `1`;
> - несколько битов можно объединять;
> - из больших последовательностей битов компьютер строит сложные данные.

Как из двух состояний получается сложная информация

На первый взгляд кажется странным:

как всего два состояния могут описать фотографию, музыку или целую компьютерную игру?

Ответ заключается в количестве комбинаций.

Возьмём один бит:

```
0
1
```

Возможно только **2 комбинации**.

Теперь два бита:

```
00
01
10
11
```

Уже **4 комбинации**.

Три бита:

```
000
001
010
011
100
101
110
111
```

Получаем **8 комбинаций**.

Чем больше битов, тем больше разных состояний можно закодировать.

Для n битов количество комбинаций равно:

```
2ⁿ
```

Например:

```
8 бит → 256 комбинаций
```

Именно поэтому даже небольшая последовательность битов уже может представлять достаточно много разных значений.

Компьютер использует огромные последовательности битов.

Поэтому двух базовых состояний оказывается достаточно для хранения практически любой цифровой информации.

Всё превращается в числа

Компьютер не хранит букву так, как её видит человек.

Возьмём:

```
A
```

Компьютеру нужно договориться, какое число будет обозначать эту букву.

Например, в одной из распространённых систем кодирования латинской букве A соответствует число:

```
65
```

Число 65, в свою очередь, можно представить в двоичной форме:

```
01000001
```

Получается цепочка:

```
A
↓
65
↓
01000001
```

С изображениями идея похожая.

Фотография состоит из пикселей.

Каждый пиксель описывается числами.

Например, числа могут определять интенсивность красного, зелёного и синего цветов.

Дальше эти числа снова представляются последовательностями битов.

```
Фотография
↓
Пиксели
↓
Числа
↓
0 и 1
```

Музыка также может быть представлена числами.

Звуковая волна измеряется много раз в секунду.

Результаты измерений записываются как числа.

```
Звук
↓
Измерения
↓
Числа
↓
0 и 1
```

Видео можно рассматривать как последовательность изображений вместе со звуком.

Программы тоже состоят из данных и инструкций, которые в конечном итоге представлены тем же способом.



Рисунок 2.4: Текст, изображения, звук и программы в конечном итоге превращаются в цифровые данные

> ❗ Главная мысль
>
> Компьютер способен работать с текстом, фотографиями, музыкой и программами не потому, что понимает их смысл.
> Он умеет представлять всё это числами.
> А числа можно представить последовательностями `0` и 1.

Что происходит технически

Теперь немного точнее посмотрим на путь информации.

Когда программа работает с некоторым значением, происходит несколько уровней преобразований.

Упрощённо:

```
Данные программы
        ↓
Числовое представление
        ↓
Биты
        ↓
Электрические сигналы
        ↓
Электронные схемы
```

Например, Python-программа может работать со строкой:

```
message = "Hello"
```

Для разработчика это текст.

Python рассматривает его как объект строки.

Дальше компьютерное оборудование работает уже с машинным представлением этих данных.

В памяти находятся не слова в человеческом смысле, а закодированные значения.

На самом нижнем уровне они физически представлены состояниями электронных элементов.

Важно понимать, что программист почти никогда не работает напрямую с этим уровнем.

Python специально скрывает большую часть сложности.

Мы можем писать:

```
message = "Hello"
```

и не думать о том, какие конкретно биты находятся в оперативной памяти.

Это один из главных принципов программирования:

каждый следующий уровень скрывает сложность предыдущего.

Мы ещё много раз встретим эту идею в книге.

От транзисторов к логическим операциям

Один транзистор сам по себе умеет немного.

Но транзисторы объединяют в электронные схемы.

Из них можно создавать так называемые **логические элементы**.

Они выполняют простые операции над двумя состояниями.

Например:

```
И
ИЛИ
НЕ
```

Позже из таких простых операций строятся более сложные схемы:

```
логические элементы
        ↓
сумматоры
        ↓
регистры
        ↓
память
        ↓
процессор
```

В результате миллиарды очень простых переключателей способны совместно выполнять сложные программы.

> **Почему это важно программисту**
>
> Python-разработчику не нужно уметь проектировать процессоры.
> Но полезно понимать, что любой сложный код в конечном итоге выполняется системой, построенной из огромного количества простых логических операций.
> Это помогает избавиться от ощущения, что компьютер работает благодаря какой-то магии.

Немного истории

Первые электронные компьютеры выглядели совсем не так, как современные ноутбуки.

До появления транзисторов в вычислительной технике широко использовались **электронные лампы**.

Они выполняли похожую функцию — позволяли управлять электрическими сигналами.

Но у них были серьёзные недостатки.

Лампы были:

- большими;
- горячими;
- энергозатратными;
- сравнительно ненадёжными.

Ранние компьютеры могли занимать целые помещения.

Появление транзистора позволило резко уменьшить размеры электронных схем.

Транзисторы были значительно меньше, экономичнее и надёжнее ламп.

Позже инженеры научились размещать множество транзисторов на одной микросхеме.

Количество транзисторов продолжало расти.

Сегодня внутри одного современного процессора могут находиться **миллиарды транзисторов**.

То, для чего когда-то требовалось целое помещение, теперь помещается в небольшом устройстве.



Рисунок 2.5: Упрощённая эволюция вычислительной техники от электронных ламп до современных процессоров

Почему это важно будущему Python-разработчику

Можно задать вопрос:

> Если Python скрывает все эти детали, зачем мне вообще знать про транзисторы и биты?

Чтобы писать первые программы, эти знания действительно не обязательны.

Но они помогают сформировать правильное представление о компьютере.

Когда позже мы начнём говорить о:

- переменных;
- типах данных;

- памяти;
- файлах;
- кодировках;
- базах данных;
- сетях;

вы будете понимать, что всё это разные способы организации одной и той же цифровой информации.

Кроме того, становится понятнее важный принцип:

абстракции позволяют программисту работать со сложной системой, не думая постоянно о её нижних уровнях.

Python — один из таких уровней абстракции.

Практика

Задание 1

Представьте четыре выключателя.

Каждый может быть либо включён, либо выключен.

Попробуйте самостоятельно записать все возможные комбинации из четырёх состояний с помощью `0` и 1.

Подсказка:

```
0000
0001
0010
...
```

Сколько комбинаций получится?

Задание 2

Попробуйте объяснить своими словами:

> Почему два состояния удобнее для электронной системы, чем десять?

Не используйте определение из книги дословно.

Главная задача — сформулировать мысль самостоятельно.

Задание 3

Выберите любой объект:

- фотографию;
- песню;
- текстовый документ;
- видео.

Попробуйте мысленно пройти путь:

```
Объект
↓
Как его можно представить числами?
↓
Как числа можно представить битами?
```

Не нужно знать точные форматы файлов.

Нас интересует только общий принцип.

Ответы к практике

Используйте этот раздел для самопроверки после того, как попробовали выполнить задания самостоятельно.

Ответ к заданию 1

Для четырёх выключателей получится **16 комбинаций**:

```
0000
0001
0010
0011
0100
0101
0110
0111
1000
1001
1010
1011
1100
1101
```

```
1110
1111
```

Почему именно 16:

```
2 × 2 × 2 × 2 = 16
```

У каждого выключателя два состояния, а выключателей четыре.

Ответ к заданию 2

Два состояния удобнее, потому что электронной системе проще быстро и надёжно отличать одно состояние от другого.

Например:

- есть сигнал или нет сигнала;
- высокое напряжение или низкое напряжение;
- включено или выключено.

Если состояний десять, системе пришлось бы гораздо точнее измерять сигнал и чаще решать, к какому состоянию он относится.

Это увеличивает риск ошибок.

Ответ к заданию 3

Пример с текстовым документом:

```
Текстовый документ
↓
Символы получают числовые коды
↓
Числа записываются битами
```

Пример с фотографией:

```
Фотография
↓
Изображение разбивается на пиксели
↓
Цвет каждого пикселя записывается числами
↓
Числа записываются битами
```

Главное: объект не хранится в компьютере «как есть». Он переводится в числовое представление, а числа уже можно записать последовательностями `0` и `1`.

Проверьте себя

Попробуйте ответить без подсказок:

1. Что такое транзистор в упрощённом представлении?
2. Почему электронным схемам удобно использовать два состояния?
3. Действительно ли внутри компьютера находятся цифры `0` и `1`?
4. Что такое бит?
5. Сколько возможных состояний имеет один бит?
6. Почему несколько битов позволяют хранить больше информации?
7. Как текст можно превратить в двоичные данные?
8. Можно ли по одному принципу хранить текст, музыку и фотографии?
9. Зачем Python-разработчику понимать эти основы, если Python скрывает работу электроники?

Если вы можете уверенно объяснить ответы своими словами, значит основная идея главы усвоена.

Ответы для самопроверки

1. В упрощённом представлении транзистор можно считать маленьким электронным выключателем.

Он находится в одном из двух состояний: сигнал есть или сигнала нет.

2. Два состояния удобны, потому что их легко быстро и надёжно отличать друг от друга.

Электронной схеме проще определить «есть сигнал» или «нет сигнала», чем различать много близких состояний.

3. Нет, внутри компьютера нет настоящих цифр `0` и `1`.

Это условные обозначения двух физических состояний.

4. Бит — это минимальная единица информации.

Он может принимать одно из двух значений: `0` или `1`.

5. Один бит имеет два возможных состояния.

```
0
1
```

6. Несколько битов дают больше комбинаций.

Например, два бита дают четыре комбинации:

```
00
01
10
11
```

Чем больше битов, тем больше разных значений можно закодировать.

7. Текст можно превратить в двоичные данные через числовые коды символов.

Например:

```
буква → числовой код → биты
```

8. Да, можно.

Текст, музыка и фотографии отличаются правилами кодирования, но в памяти компьютера всё равно хранятся как последовательности битов.

9. Эти основы помогают понимать, что происходит под уровнем Python.

Когда позже появятся переменные, типы данных, файлы, кодировки и память, будет проще понять, почему они работают именно так.

Главное, что нужно запомнить

1. В основе цифрового компьютера лежат электронные элементы с двумя различимыми состояниями.
2. Эти состояния условно обозначают как `0` и `1`.
3. Один `0` или `1` представляет один бит информации.
4. Последовательности битов позволяют кодировать множество различных значений.
5. Текст, изображения, музыка, видео и программы в конечном итоге представляются цифровыми данными.
6. Python скрывает большую часть нижнего уровня и позволяет работать с понятными человеку объектами.

> ❗ Главный вывод главы
>
> Компьютер не «говорит на языке нулей и единиц».
> Нули и единицы — это удобная математическая модель двух физических состояний электронных схем.
> Комбинируя огромное количество таких простых состояний, компьютер

> способен хранить данные и выполнять сложнейшие программы.

Что дальше

Теперь мы знаем, почему компьютер использует два состояния и откуда появляются `0` и 1.

Но остаётся важный вопрос:

как именно из последовательностей битов получаются числа, буквы, изображения и другие данные?

В следующей главе мы разберёмся, как компьютер хранит информацию и познакомимся с битами, байтами и кодированием данных подробнее.

2 2.3 Как компьютер хранит данные

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Если компьютер знает только 0 и 1, как он хранит текст, числа, фотографии, музыку и программы?**

После предыдущей главы мы уже знаем важную вещь:

> Любая информация внутри компьютера состоит только из нулей и единиц.

Но возникает следующий вопрос.

Когда вы открываете фотографию, запускаете игру или читаете текстовый файл, вы не видите последовательность вроде

```
010011010100101011...
```

Вы видите изображение, буквы, числа и кнопки.

Почему?

Потому что сами биты ничего не означают.

Смысл появляется только тогда, когда существует правило их интерпретации.

---

2 Бит — самая маленькая единица информации

Бит может принимать только два значения.

`0`

или

`1`

Это похоже на обычный выключатель.

`Выключено -> 0`

`Включено -> 1`

Одного бита почти никогда недостаточно.

Поэтому биты объединяют в группы.

---

2 Байт

Самая известная группа — **байт**.

Один байт состоит из восьми битов.

```
10100110
```

или

```
00000000
```

или

```
11111111
```

Всего существует

```
2 × 2 × 2 × 2 × 2 × 2 × 2 × 2 = 256
```

различных комбинаций.

Поэтому один байт способен хранить одно из **256 различных значений**.

Это очень удобно.

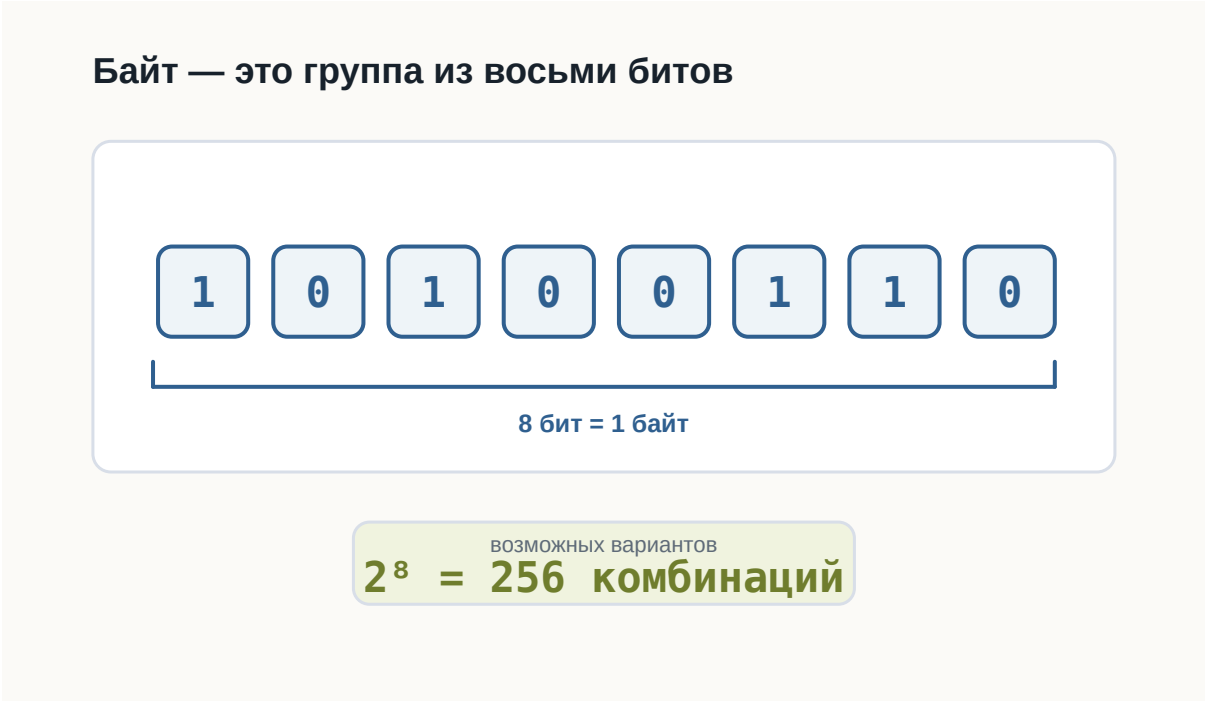


Рисунок 2.1: Восемь битов образуют один байт.

2 Как компьютер понимает, что хранится в байте?

Представьте число

```
65
```

Что это означает?

Без контекста — ничего.

Это может быть:

- возраст;
- скорость;
- номер страницы;
- температура.

То же самое происходит с байтами.

Последовательность

```
01000001
```

сама по себе ничего не означает.

Но если договориться считать её символом ASCII, получится буква

```
A
```

Если считать её числом —

```
65
```

То есть одинаковые биты могут означать разные вещи.

Все зависит от правил.



Рисунок 2.1: Одинаковые биты получают смысл только через правило интерпретации.

2 Как хранится текст

Компьютер не хранит буквы.

Он хранит номера букв.

Например,

Символ	Код
A	65
B	66
C	67

Когда вы набираете

```
ABC
```

в памяти оказывается примерно следующее:

```
65 66 67
```

или в двоичном виде

```
01000001
01000010
01000011
```

Позже программа снова превращает эти числа в буквы.



Рисунок 2.1: Текст хранится как числовые коды, представленные битами.

2 Почему этого оказалось недостаточно

ASCII содержал только 128 символов.

Для английского языка этого хватало.

Но как быть с

```
Привет
□□□□□
□□□□□
□
```

Появился Unicode.

Он назначает номер практически каждому символу, существующему сегодня.

Поэтому современные программы способны одновременно показывать русский, японский, арабский и эмодзи.

---

2 Как хранится число

Компьютер не знает, что такое “сорок два”.

Он хранит двоичное представление.

Например,

```
42
```

становится

```
00101010
```

Для компьютера это просто набор битов.

Если программа знает, что это число, она выполняет арифметику.

Если считает, что это символ, получит совершенно другой результат.

---

2 Как хранится изображение

Фотография — это вовсе не рисунок.

Это огромная таблица пикселей.

Например,

□ □ □
□ □ □

Каждый пиксель имеет цвет.

Цвет тоже можно записать числами.

Например,

```
Красный
R = 255
G = 0
B = 0
```

или

```
Зелёный

R = 0
G = 255
B = 0
```

Таким образом фотография превращается в огромное количество чисел.

А числа уже легко представить битами.

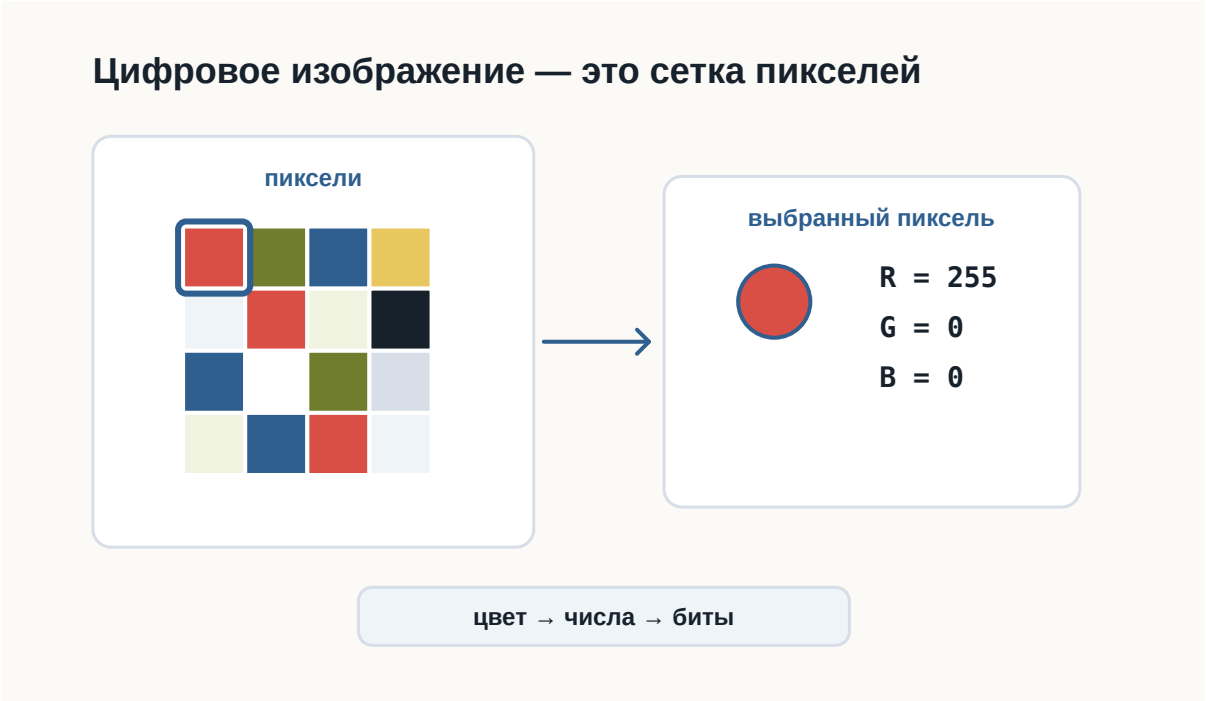


Рисунок 2.1: Изображение хранится как сетка пикселей, а цвет каждого пикселя задаётся числами RGB.

2 Как хранится музыка

Музыка — это изменение давления воздуха.

Микрофон много тысяч раз в секунду измеряет это давление.

Получается последовательность чисел.

Например,

```
125
127
130
126
120
...
```

Компьютер хранит именно эти числа.

При воспроизведении динамик выполняет процесс в обратную сторону.

---

2 Как хранится программа

Самое интересное — программы тоже являются данными.

Файл Python

```
print("Hello")
```

хранится как обычный текст.

Но после запуска интерпретатор читает этот текст и начинает выполнять инструкции.

То есть один и тот же набор битов может быть:

- текстом;
- программой;
- изображением;
- музыкой.

Все определяется тем, **кто именно читает эти данные**.



Рисунок 2.1: Текст, числа, изображения, звук и программы сводятся к числам и битам.

2 Главное, что нужно запомнить

Компьютер хранит только последовательности битов.

Никаких букв, фотографий или музыки внутри памяти нет.

Есть лишь числа.

Именно программы знают, как превратить эти числа обратно в текст, изображения, звук или команды.

Поэтому можно сказать так:

> Компьютер хранит не сами данные, а их двоичное представление.

---

2 Что дальше?

Теперь мы понимаем:

- почему всё сводится к битам;
- как появляются байты;
- почему существуют кодировки;
- как текст, изображения и числа становятся последовательностями битов.

Но остается следующий вопрос.

Даже если программа уже хранится в памяти как набор битов, **как процессор начинает её выполнять?**

Именно об этом будет следующая глава:

«Как компьютер выполняет программы».

2 2.4 Как компьютер выполняет программы

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Если программа хранится в памяти как обычные данные, как процессор начинает её выполнять?**

Когда вы запускаете программу, компьютер не «читает намерения».

Он выполняет конкретную последовательность действий: находит файл, загружает нужные данные в оперативную память и передаёт инструкции процессору.

После этой главы вы сможете

- объяснить, что происходит после запуска программы;
- понимать разницу между файлом программы и работающим процессом;
- объяснить, зачем нужна оперативная память;
- понимать общий цикл работы процессора;
- объяснить, зачем Python-коду нужен интерпретатор.

Программа — это инструкция для компьютера

Программа может храниться как обычный файл.

Например:

```
hello.py
```

Внутри файла находится текст программы.

Сам файл ещё не выполняется. Он просто лежит на накопителе как данные.

Чтобы программа начала работать, операционная система должна подготовить её к выполнению.

Где находится программа до запуска

До запуска программа обычно находится на SSD или другом накопителе.

Накопитель подходит для долговременного хранения:

- файлы не исчезают после выключения компьютера;
- данные можно хранить долго;
- можно открыть файл позже.

Но процессор не выполняет программу прямо с накопителя.

Перед выполнением нужные данные загружаются в оперативную память.

Что происходит после запуска

После запуска операционная система создаёт процесс.

Процесс — это работающая программа вместе с ресурсами, которые ей выделены.

Например, процесс может использовать:

- оперативную память;
- файлы;
- ввод с клавиатуры;
- вывод в терминал.



Рисунок 2.1: Путь программы от накопителя к процессору

Главная идея проста: файл программы хранится на накопителе, но во время работы программа находится в оперативной памяти, а процессор выполняет её инструкции.

Как процессор выполняет инструкции

Процессор работает пошагово.

Он не выполняет всю программу сразу.

В упрощённом виде его работа выглядит как повторяющийся цикл:

1. получить очередную инструкцию;
2. понять, что она означает;
3. выполнить действие.



Рисунок 2.2: Цикл работы процессора: получить, декодировать, выполнить

Этот цикл повторяется очень быстро.

Именно поэтому программа может выполнять тысячи, миллионы или миллиарды операций за короткое время.

Откуда процессор знает, какую инструкцию выполнять

Процессор должен знать, какую инструкцию выполнить следующей.

Для этого он хранит указатель на текущую или следующую инструкцию.

Обычно после одной инструкции процессор переходит к следующей.

Но не всегда.

Условия, циклы и вызовы функций могут изменить порядок выполнения.



Рисунок 2.3: Указатель текущей инструкции и переходы

Это одна из причин, почему программа может не просто идти сверху вниз, а принимать решения и повторять действия.

А где здесь Python?

Python-программа обычно хранится в файле с расширением `.py`.

Например:

```
print("Привет")
```

Процессор не выполняет такой текст напрямую.

Python-код читает специальная программа — интерпретатор Python.

Когда вы запускаете команду:

```
python hello.py
```

вы просите интерпретатор Python прочитать файл `hello.py` и выполнить инструкции внутри него.



Рисунок 2.4: Роль интерпретатора Python между файлом и процессором

На этом этапе достаточно понимать общую идею: Python-код обрабатывает интерпретатор, а процессор выполняет инструкции более низкого уровня.

Короткий итог

Программа хранится как файл.

После запуска операционная система создаёт процесс и загружает нужные данные в оперативную память.

Процессор получает инструкции, декодирует их и выполняет.

Python-код не выполняется процессором напрямую: между файлом `.py` и процессором находится интерпретатор Python.

Практика

Объясните своими словами, чем отличается файл `hello.py` от запущенной программы.

2 2.5 Почему Python

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Если существуют десятки языков программирования, почему мы начинаем именно с Python?**

В предыдущих главах мы разобрались, как компьютер хранит данные и как процессор выполняет инструкции.

Теперь возникает следующий вопрос.

Если процессор в конечном счёте работает с машинными инструкциями, зачем человеку вообще нужен язык программирования?

И если языков программирования много, почему для этой книги выбран именно Python?

Чтобы ответить на этот вопрос, сначала нужно понять, зачем языки программирования появились вообще.

Можно ли писать программы сразу на машинном языке

Технически — да.

Процессор выполняет машинные инструкции.

В самом низком уровне программа действительно превращается в последовательности чисел и битов.

Условно можно представить это так:

```
10110000
01100001
11001010
00010111
```

Для процессора такой формат нормален.

Для человека — почти нет.

Попробуйте определить по этим битам:

- что делает программа;
- где начинается одна инструкция;
- где заканчивается другая;
- какое значение используется;
- что нужно изменить, чтобы программа работала иначе.

Даже небольшая программа быстро превратилась бы в огромную последовательность чисел.

Писать, читать и исправлять такой код было бы чрезвычайно сложно.



Рисунок 2.1: Человеку удобнее писать программу на языке высокого уровня

Язык программирования нужен человеку

Язык программирования позволяет описывать действия компьютера в форме, которую человеку намного проще читать.

Например:

```
print("Привет")
```

Эта строка гораздо понятнее, чем набор машинных инструкций.

Мы можем довольно быстро догадаться, что программа должна вывести текст.

Но процессор не понимает Python напрямую.

Поэтому между кодом программиста и процессором появляется ещё один слой.



Рисунок 2.2: Язык программирования как слой между человеком и процессором

Перевод выполняют специальные программы.

В зависимости от языка это могут быть:

- компиляторы;
- интерпретаторы;
- виртуальные машины;
- комбинации нескольких подходов.

Пока нам не нужно подробно разбираться в этих механизмах.

Важно понять главное:

> Язык программирования существует прежде всего для человека, а не для процессора.

Почему языков программирования так много

Если один язык удобнее машинного кода, возникает естественный вопрос:

Почему не существует одного языка для всех программ?

Потому что разные задачи предъявляют разные требования.

Иногда важнее:

- максимальная скорость;
- полный контроль над памятью;
- безопасность;
- простота разработки;
- работа в браузере;
- удобство обработки данных;
- быстрая автоматизация.

Поэтому разные языки делают разные компромиссы.

Например:

- C позволяет работать близко к устройству компьютера;
- JavaScript стал основным языком программирования в браузере;
- Java широко используется в крупных приложениях;
- Python делает большой акцент на читаемости и скорости разработки.

Это не означает, что один язык лучше остальных.

У каждого инструмента есть область, в которой он особенно удобен.

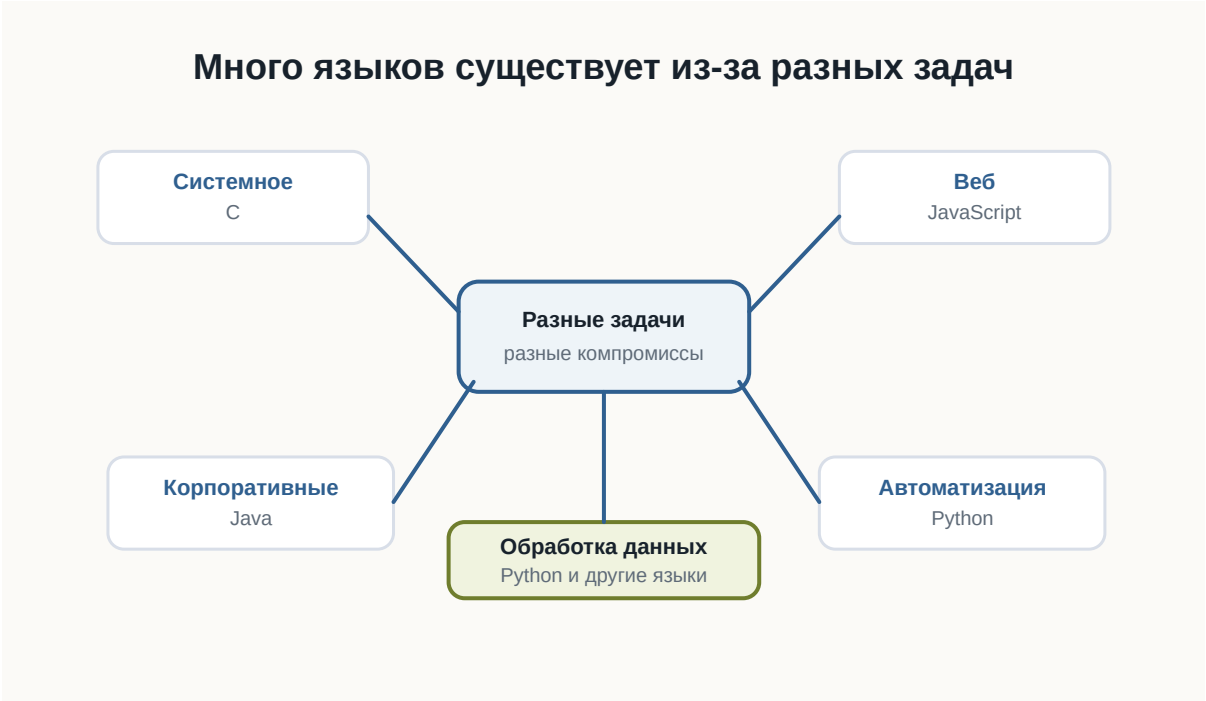


Рисунок 2.3: Разные языки делают разные компромиссы и удобны для разных задач

Почему для обучения выбран Python

Python хорошо подходит для знакомства с программированием по нескольким причинам.



Рисунок 2.4: Python выбран как удобный первый язык и профессиональный инструмент

Первая — **читаемость**.

Сравните простой фрагмент:

```
age = 20

if age >= 18:
    print("Доступ разрешён")
```

Даже не зная Python, можно приблизительно понять смысл программы.

Синтаксис не скрывает основную идею за большим количеством служебных конструкций.

Вторая причина — **небольшой порог входа**.

Чтобы написать простую программу, обычно не требуется сначала изучать:

- сложную систему типов;
- управление памятью;
- структуру проекта;
- процесс компиляции;
- большое количество обязательного синтаксиса.

Это позволяет быстрее перейти к самому программированию:

- переменным;
- условиям;
- циклам;
- функциям;
- структурам данных;
- алгоритмам.

Третья причина — **Python остаётся профессиональным инструментом**.

Это важно.

Мы выбираем Python не только потому, что на нём удобно учиться.

Он используется в реальных проектах.

На Python пишут:

- серверные приложения;
- инструменты автоматизации;
- тесты;
- системы обработки данных;
- научные программы;
- инструменты DevOps;
- приложения машинного обучения;
- внутренние сервисы компаний.

Поэтому знания, полученные во время обучения, не становятся бесполезными после первых глав.

Python не лучший язык для всего

Важно не создавать неправильное впечатление.

Python не является универсально лучшим языком.

Например, он может быть не лучшим выбором, когда требуется:

- писать драйвер устройства;
- создавать часть операционной системы;
- получать максимальную производительность на низком уровне;
- программировать микроконтроллер с очень ограниченной памятью;
- писать код, который должен напрямую выполняться в браузере.

В таких задачах могут лучше подходить другие языки.

Поэтому профессиональный разработчик выбирает язык не потому, что он ему нравится больше остальных.

Он выбирает инструмент под задачу.

В чём сила Python

У Python есть ещё одно важное свойство.

Сам язык относительно компактный, но вокруг него существует огромная экосистема.

Это означает, что многие сложные задачи уже имеют готовые инструменты.

Например, вместо того чтобы самостоятельно писать всё с нуля, разработчик может использовать существующие библиотеки.

Условно:

```
Python
  ↓
Библиотеки
  ↓
Готовые инструменты
  ↓
Решение задачи
```

Это позволяет сосредоточиться не на низкоуровневых деталях, а на самой задаче.

Именно поэтому Python часто используют для автоматизации и быстрого создания новых программ.

Где используется Python

Python применяется в очень разных областях.



Рисунок 2.5: Основные области применения Python

Веб-разработка

На Python можно создавать серверную часть сайтов и API.

Например, программа может:

- принимать запрос;
- обращаться к базе данных;
- выполнять вычисления;
- возвращать результат пользователю.

Автоматизация

Python часто используют для небольших программ и скриптов.

Например:

- переименовать тысячи файлов;
- обработать таблицы;
- собрать данные из нескольких источников;
- автоматически создать отчёт;
- проверить состояние серверов.

Анализ данных

Python широко используется для работы с большими наборами данных.

Программы могут:

- читать данные;
- очищать их;
- выполнять вычисления;
- строить графики;
- искать закономерности.

Машинное обучение

Большая часть современной экосистемы машинного обучения доступна через Python.

При этом сложные вычисления часто выполняются не самим Python.

Python используется как удобный интерфейс к высокопроизводительным библиотекам.

Тестирование

Python также подходит для автоматического тестирования программ.

Вместо того чтобы каждый раз проверять программу вручную, можно написать другой код, который выполнит проверку автоматически.

Простота не означает примитивность

Иногда простой синтаксис создаёт впечатление, что Python — это только учебный язык.

Это не так.

Простой язык и простая задача — не одно и то же.

На Python можно написать:

```
print("Привет")
```

а можно построить сложную систему из:

- десятков сервисов;
- баз данных;
- очередей сообщений;
- фоновых процессов;
- API;
- автоматических тестов.

Сложность реальной разработки появляется не только в синтаксисе языка.

Она появляется в архитектуре, данных, взаимодействии компонентов и требованиях к системе.

Что именно мы будем изучать

Наша цель — не просто выучить команды Python.

Важно научиться мыслить как разработчик.

Поэтому дальше мы будем постепенно разбирать:

- как хранить данные в программе;
- как принимать решения;
- как повторять действия;
- как разбивать программу на функции;
- как работать со структурами данных;
- как организовывать код;
- как находить и исправлять ошибки;
- как тестировать программы.

Python будет нашим инструментом для изучения этих идей.

Что важно запомнить

Процессору не нужен Python.

Процессор выполняет машинные инструкции.

Python нужен нам, потому что писать программу на языке высокого уровня намного удобнее, чем непосредственно на машинном коде.

При этом Python:

- относительно легко читать;
- имеет небольшой порог входа;
- позволяет быстро писать программы;
- используется в реальной разработке;
- имеет большую экосистему библиотек;
- подходит для множества разных задач.

Но самое важное:

> Мы изучаем не только Python. Мы изучаем программирование с помощью Python.

Что дальше?

Теперь понятно, почему для этой книги выбран Python.

Следующий вопрос уже практический.

Что нужно установить и настроить, чтобы начать писать программы на Python?

Этому посвящена следующая глава:

«Рабочее пространство Python-разработчика».

2 2.6 Рабочее пространство Python

Перед тем как писать программы, нужно подготовить рабочее пространство.

Рабочее пространство - это набор инструментов и папок, в которых вы будете писать, запускать и хранить код.

После этой главы вы сможете

- понимать, зачем нужен редактор кода;
- понимать роль терминала;
- создать папку проекта;
- объяснить, зачем нужно виртуальное окружение.

Основные инструменты

Для начала нужны:

- установленный Python;
- Visual Studio Code или другой редактор кода;
- терминал;
- браузер;
- папка проекта.

Этого достаточно, чтобы написать и запустить первую программу.



Рисунок 2.1: Основные инструменты рабочего места Python-разработчика

Где скачать Python

Python скачивают с официального сайта проекта.

Важно понимать: Python устанавливается на компьютер как отдельная программа.

После установки его можно запускать из терминала.



Рисунок 2.2: Процесс установки Python

Python — это интерпретатор

Когда вы запускаете файл `.py`, его читает интерпретатор Python.

Именно интерпретатор понимает Python-код и выполняет программу.



Рисунок 2.3: Роль интерпретатора Python

Что такое редактор кода

Редактор кода помогает писать программы, открывать файлы проекта и запускать команды.

На первом этапе достаточно понимать основные части редактора: дерево файлов, область кода и терминал.

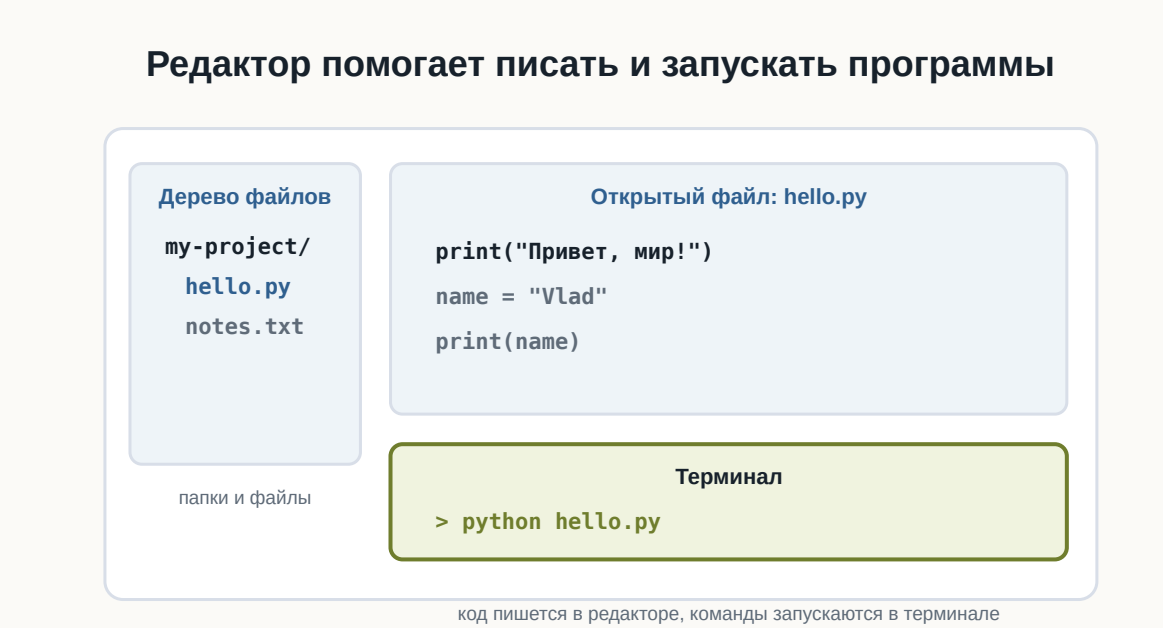


Рисунок 2.4: Условное окно редактора кода

Структура рабочего проекта

Каждый проект лучше хранить в отдельной папке.

Например:

```
python-start/
```

Внутри будут лежать файлы программы, заметки, настройки и позже зависимости.



Рисунок 2.5: Простая структура Python-проекта

Терминал

Терминал позволяет запускать команды.

Например:

```
python --version
```

Эта команда показывает установленную версию Python.

Виртуальное окружение

Позже у разных проектов могут появиться разные зависимости.

Чтобы они не мешали друг другу, используют виртуальные окружения.

> **Пока достаточно понимать идею**
>
> На первом шаге важно знать, что виртуальное окружение изолирует зависимости проекта. Подробно мы разберем это позже.

Короткий итог

Рабочее пространство помогает держать код в порядке.

Редактор нужен для написания кода, терминал - для запуска команд, папка проекта - для организации файлов.

Практика

Создайте папку `python-start` и проверьте в терминале версию Python командой `python --version`.

2 2.7 Первая программа на Python

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как написать, запустить и понять свою первую программу на Python?**

Мы уже подготовили рабочее пространство.

Теперь можно перейти от объяснений к настоящему коду.

В этой главе мы создадим простой Python-файл, запустим его из терминала и разберём, что именно происходит на каждом этапе.

Наша первая программа будет очень маленькой.

Но путь её выполнения почти такой же, как у гораздо более сложных Python-приложений.

Создаём файл программы

Создайте рабочую папку для первых экспериментов.

Например:

```
python-learning/
```

Внутри неё создайте файл:

```
hello.py
```

Расширение `.py` означает, что файл содержит исходный код на Python.

В результате структура папки будет выглядеть так:

```
python-learning/
└── hello.py
```

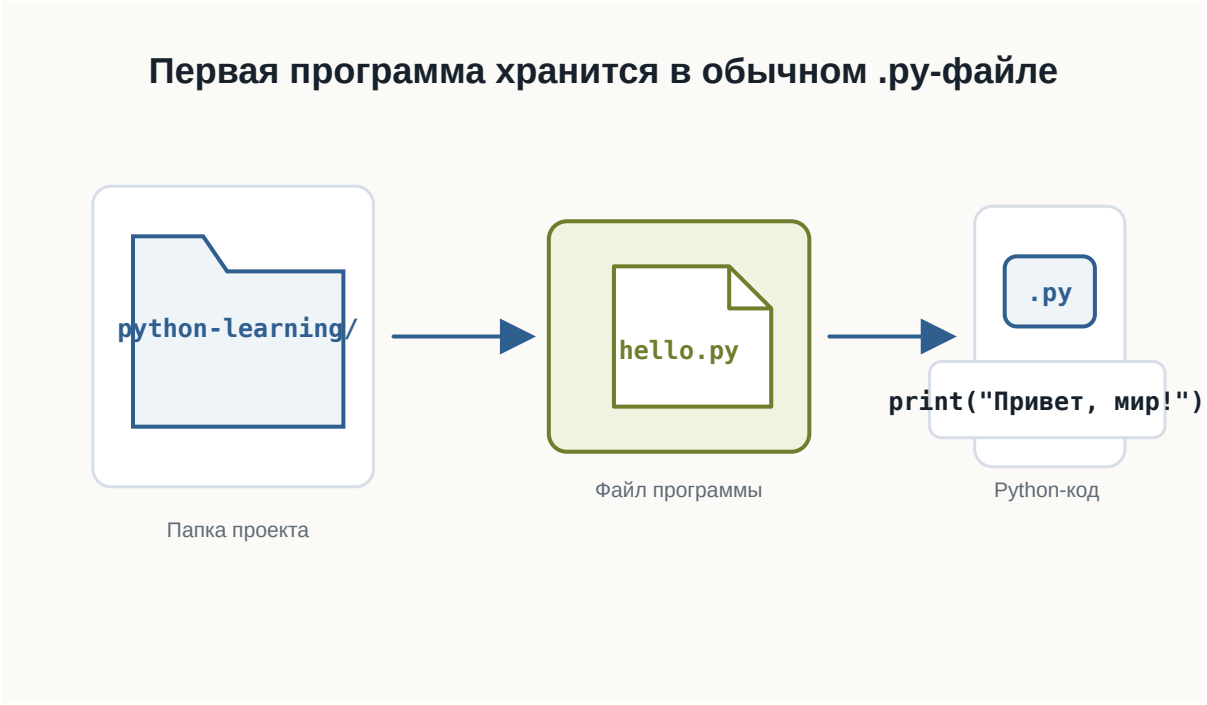


Рисунок 2.1: Первая Python-программа хранится в файле внутри проекта

Теперь откройте `hello.py` в редакторе кода.

Пишем первую строку Python

Добавьте в файл:

```
print("Привет, мир!")
```

> **Попробуйте сами**
>
> Измените текст внутри кавычек, добавьте второй `print()` и запустите пример снова.

Сохраните изменения.

Это уже полноценная программа на Python.

Она содержит одну инструкцию.

Её задача — вывести текст на экран.

Что делает print

В выражении

```
print("Привет, мир!")
```

можно выделить несколько частей.

`print` — это имя функции.

Круглые скобки:

```
()
```

используются для передачи функции данных.

А текст:

```
"Привет, мир!"
```

находится внутри кавычек.

Так Python понимает, что перед ним текстовое значение.

Пока не нужно подробно изучать функции и строки.

Мы разберём их позже.

Сейчас достаточно понимать:

- `print(...)` просит Python вывести переданное значение.



Рисунок 2.2: Из чего состоит инструкция print

Запускаем программу

Откройте терминал в папке проекта.

Убедитесь, что терминал находится в каталоге, где лежит файл:

```
hello.py
```

Теперь выполните:

```
python hello.py
```

В некоторых системах команда может называться:

```
python3 hello.py
```

Если всё работает правильно, терминал выведет:

```
Привет, мир!
```

Первая программа выполнена.

Что произошло после команды запуска

Команда

```
python hello.py
```

содержит две важные части.

Первая:

```
python
```

указывает, какую программу нужно запустить.

Это интерпретатор Python.

Вторая:

```
hello.py
```

указывает, какой файл нужно передать интерпретатору.

Поэтому команду можно мысленно прочитать так:

```
Запусти Python и попроси его выполнить hello.py
```

После этого происходит примерно следующая последовательность:

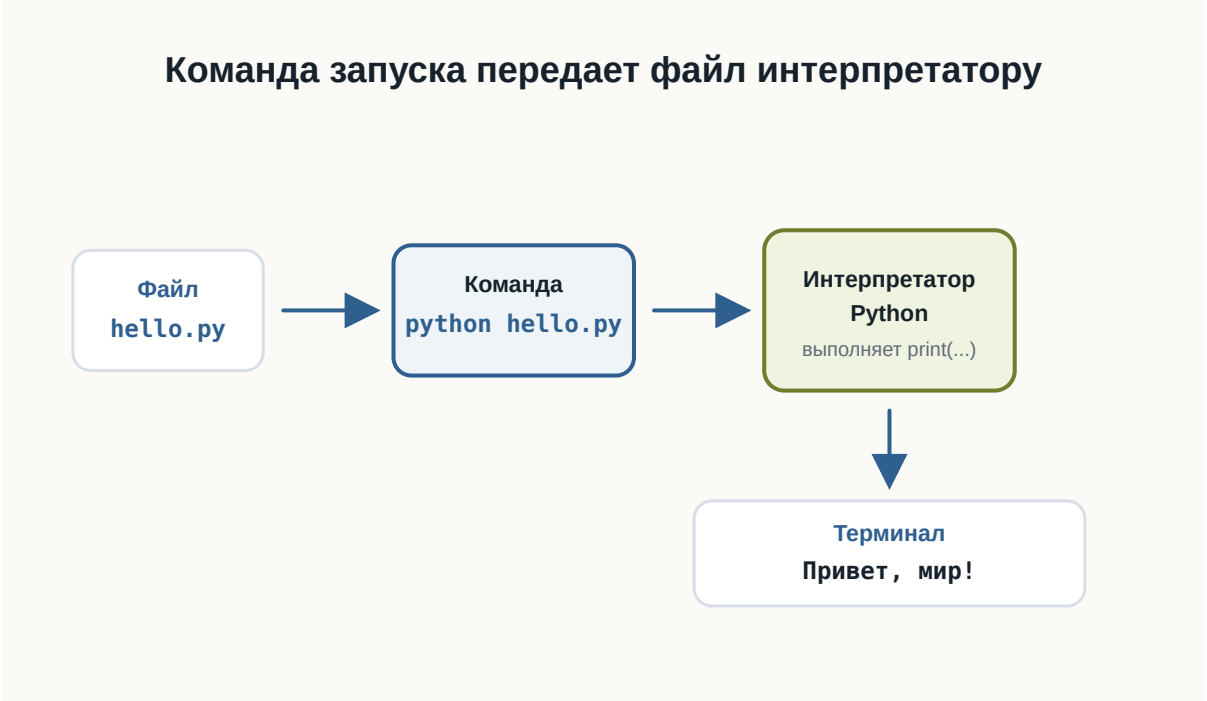


Рисунок 2.3: Путь первой Python-программы от файла до результата

Код и результат — не одно и то же

Это важное различие.

Файл:

```
print("Привет, мир!")
```

содержит **код программы**.

А строка:

```
Привет, мир!
```

является **результатом выполнения программы**.

Код описывает, что компьютер должен сделать.

Результат появляется после выполнения кода.

Позже одна программа сможет:

- принимать данные;
- выполнять вычисления;
- работать с файлами;
- обращаться к сети;
- принимать решения;
- возвращать разные результаты.

Но принцип останется тем же.

Добавим ещё одну инструкцию

Изменим программу:

```
print("Привет, мир!")
print("Я изучаю Python")
```

Теперь снова запустим:

```
python hello.py
```

Результат:

```
Привет, мир!
Я изучаю Python
```

Python выполнил инструкции сверху вниз.

Сначала первую:

```
print("Привет, мир!")
```

затем вторую:

```
print("Я изучаю Python")
```

Это один из фундаментальных принципов выполнения программ.

По умолчанию инструкции выполняются последовательно.

Порядок инструкций имеет значение

Рассмотрим программу:

```
print("Первый шаг")
print("Второй шаг")
print("Третий шаг")
```

Результат будет:

```
Первый шаг
Второй шаг
Третий шаг
```

Если изменить порядок строк:

```
print("Третий шаг")
print("Первый шаг")
print("Второй шаг")
```

изменится и результат:

```
Третий шаг
Первый шаг
Второй шаг
```

Компьютер не пытается догадаться, какой порядок имел в виду программист.

Он выполняет инструкции в том порядке, который указан в программе.



Рисунок 2.4: Последовательное выполнение инструкций сверху вниз

> **Попробуйте сами**
>
> Поменяйте строки местами, запустите пример снова и сравните порядок вывода.

Программа должна быть записана точно

Попробуйте убрать одну кавычку:

```
# Ошибка сделана намеренно для учебного примера.
print("Привет, мир!)
```

> **Попробуйте сами**
>
> Запустите пример с ошибкой, затем исправьте кавычку и запустите ещё раз.

Python не сможет выполнить такую программу.

Он сообщит об ошибке.

Это происходит потому, что исходный код должен соответствовать правилам языка.

Эти правила называются **синтаксисом**.

Как и в естественном языке, форма записи имеет значение.

Но компьютер гораздо строже человека.

Он не пытается догадаться, что вы хотели написать.

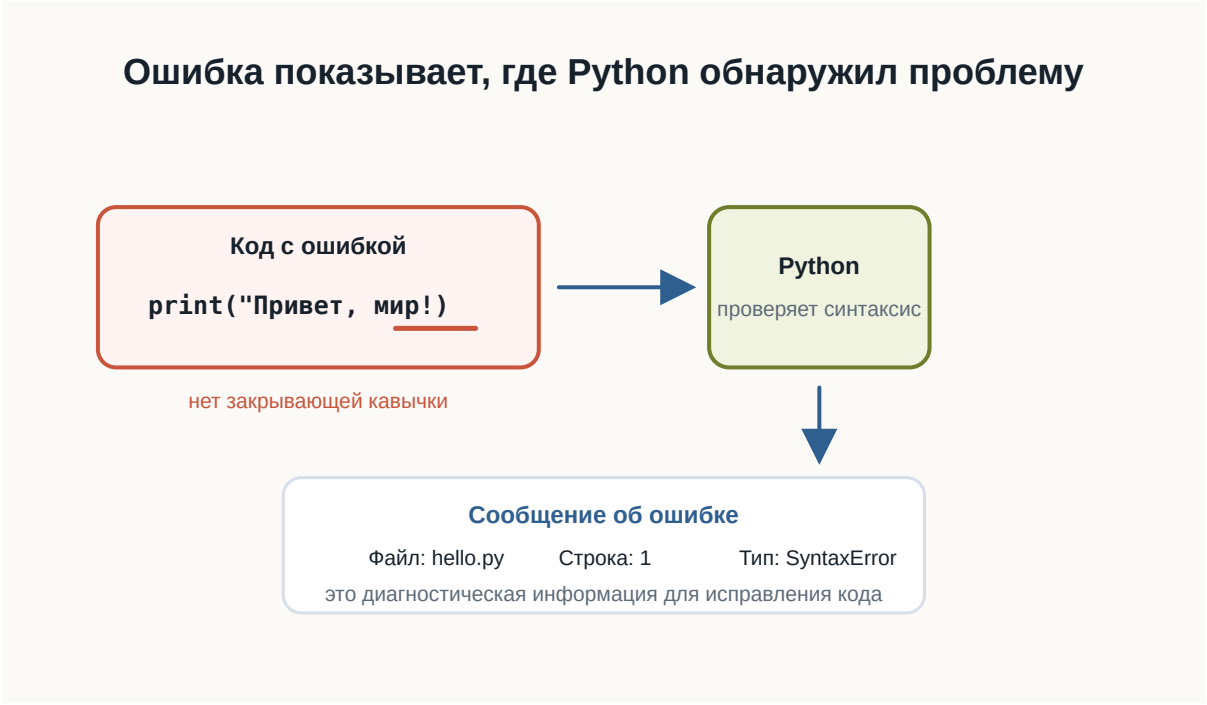


Рисунок 2.5: Синтаксическая ошибка показывает место проблемы

Ошибка — часть программирования

Сообщение об ошибке не означает, что произошло что-то необычное.

Ошибки постоянно возникают во время разработки.

Например:

```
print("Привет"
```

Python может сообщить о синтаксической ошибке.

Это полезная информация.

Интерпретатор показывает, что программу невозможно разобрать согласно правилам языка.

Работа разработчика часто выглядит так:

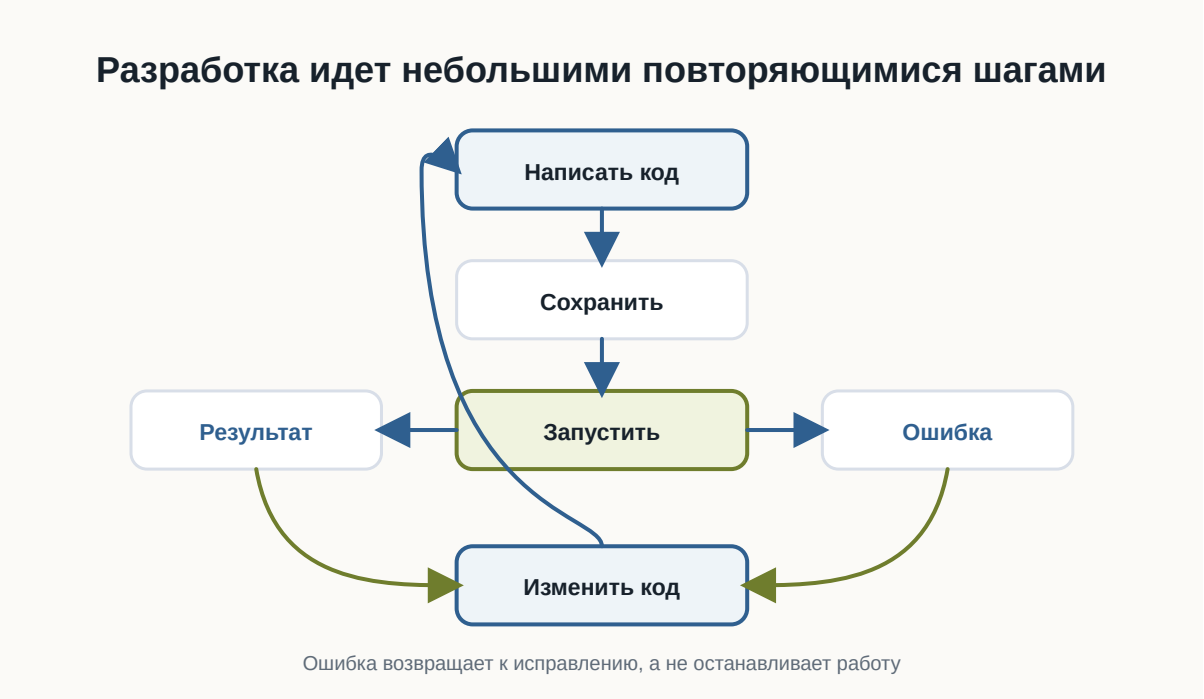


Рисунок 2.6: Цикл небольших изменений, запусков и проверки результата

Этот цикл будет повторяться на протяжении всей книги.

И в профессиональной разработке происходит то же самое.

Читайте сообщения об ошибках

Начинающие разработчики иногда видят красный текст в терминале и сразу считают его проблемой.

На самом деле сообщение об ошибке — это подсказка.

Например, оно может показать:

- тип ошибки;

- файл, в котором она возникла;
- номер строки;
- дополнительное описание.

Позже мы научимся читать такие сообщения подробно.

Пока важно привыкнуть к простой мысли:

> Ошибка — это информация о том, почему программа не смогла выполнить ожидаемое действие.

Первый маленький эксперимент

Измените текст внутри программы самостоятельно.

Например:

```
print("Меня зовут Алекс")
print("Сегодня я написал первую программу")
print("Следующая цель — понять переменные")
```

Запустите файл ещё раз.

Затем:

1. поменяйте порядок строк;
2. добавьте ещё один `print`;
3. удалите одну строку;
4. измените текст;
5. специально допустите ошибку в кавычках;
6. прочитайте сообщение Python;
7. исправьте ошибку и снова запустите программу.

Это простое упражнение важнее, чем может показаться.

Вы уже проходите основной цикл разработки:

```
написать → запустить → посмотреть → изменить → запустить снова
```

Практика: программа-визитка

Создайте новый файл:

```
about_me.py
```

Напишите программу, которая выводит несколько строк о вас.

Например:

```
print("Имя: Анна")
print("Город: Казань")
print("Изучаю: Python")
print("Цель: стать разработчиком")
```

Запустите:

```
python about_me.py
```

Проверьте результат.

Затем измените программу так, чтобы она содержала не менее пяти строк вывода.

> Совет
>
> Не копируйте пример полностью.
> Введите собственные значения и несколько раз измените программу.
> Сейчас важнее привыкнуть самостоятельно редактировать, сохранять и запускать `.py`-файлы.

Не бойтесь экспериментировать

Одно из преимуществ программирования — большинство экспериментов легко отменить.

Можно:

- изменить строку;
- запустить программу;
- посмотреть результат;
- вернуть код обратно;
- попробовать другой вариант.

Именно через такие небольшие эксперименты появляется понимание поведения языка.

Не ограничивайтесь только примерами из книги.

Если возникает вопрос:

> А что произойдёт, если я напишу вот так?

попробуйте.

Что мы уже умеем

После этой главы вы уже способны пройти полный путь небольшой программы:

```
Идея
  ↓
Файл .py
  ↓
Python-код
  ↓
Запуск через интерпретатор
  ↓
Результат в терминале
```

Вы также знаете, что:

- Python-программа хранится в `.py`-файле;
- `print()` позволяет вывести значение;
- интерпретатор выполняет программу;
- инструкции обычно выполняются сверху вниз;
- синтаксис должен быть записан точно;
- ошибки являются нормальной частью разработки;
- программу удобно создавать небольшими итерациями.

Что важно запомнить

Первая программа:

```
print("Привет, мир!")
```

очень маленькая.

Но в ней уже присутствуют основные элементы процесса разработки:

1. исходный код;

2. файл программы;
3. интерпретатор;
4. запуск;
5. выполнение инструкции;
6. результат;
7. возможные ошибки;
8. исправление и повторный запуск.

Именно из таких простых действий постепенно строятся большие программы.

Что дальше?

Мы научились запускать код.

Но пока наши программы умеют только выводить заранее записанный текст.

Следующий важный вопрос:

Как программе запоминать значения и работать с ними?

Для этого нам понадобится одна из основных идей программирования:

переменные.

Часть II

3 Том II. Базовые конструкции Python

В этом томе собраны первые средства Python, которые нужны для написания собственных небольших программ после первой запущенной программы.

Каждая глава отвечает на один главный вопрос и вводит только одну важную идею.

Быстрые ссылки

- 3.1 Переменные
- 3.2 Типы данных
- 3.3 Ввод данных
- 3.4 Преобразование типов
- 3.5 Арифметические операции
- 3.6 Сравнение значений
- 3.7 Условия
- 3.8 Логические операции
- 3.9 Цикл while
- 3.10 Цикл for
- 3.11 Строки
- 3.12 Списки
- 3.13 Функции

> **Логика тома**
>
> Идите по главам последовательно: от сохранения одного значения к типам данных, вводу, преобразованиям, вычислениям, сравнениям, условиям, циклам, строкам, спискам и функциям.

2 3.1 Переменные

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как программа запоминает данные?**

В первой программе мы уже передавали Python готовые значения:

```
print("Привет!")
print(25)
```

Но настоящие программы редко работают только с данными, которые заранее написаны прямо в коде.

Программе часто нужно помнить имя пользователя, город, текущий счёт, выбранный язык или другое значение.

Для этого в Python используются **переменные**.

Значению можно дать имя

Представим, что в программе несколько раз используется имя пользователя:

```
print("Анна")
print("Привет, Анна!")
print("Анна вошла в систему")
```

Такой код работает, но значение `"Анна"` повторяется несколько раз.

Если имя понадобится изменить, придётся искать и исправлять каждое место.

Вместо этого значению можно дать имя:

```
name = "Анна"
```

Теперь это имя можно использовать в программе:

```
name = "Анна"

print(name)
print("Привет,", name)
```

Результат:

```
Анна
Привет, Анна
```

`name` — это **переменная**.

Переменная — это имя, связанное со значением.

В нашем примере:

```
name -> "Анна"
```

Когда Python встречает `name`, он использует значение, связанное с этим именем.



Рисунок 2.1: Имя переменной связано со значением

Что означает знак =

Посмотрим ещё раз:

```
name = "Анна"
```

Знак = здесь не означает математическое равенство.

В Python он используется для **присваивания**.

Эту строку можно прочитать так:

> связать имя name со значением "Анна".

Слева от = находится имя переменной.

Справа от = находится значение.

Python берёт значение справа и связывает его с именем слева.

Например:

```
age = 25
temperature = 21.5
city = "Вена"
```

После выполнения этих строк можно представить связи так:

```
age         -> 25
temperature -> 21.5
city        -> "Вена"
```



Рисунок 2.2: Как работает присваивание

> **Уведомление**
>
> Для начала достаточно помнить простое правило:
> **слева от = находится имя переменной, справа — значение.**

Переменную можно использовать много раз

Главная польза переменной становится заметна, когда одно значение используется в нескольких местах.

```
name = "Миша"

print("Привет,", name)
print("Рады тебя видеть,", name)
print("До встречи,", name)
```

Результат:

```
Привет, Миша
Рады тебя видеть, Миша
До встречи, Миша
```

Если теперь нужно изменить имя, достаточно исправить одну строку:

```
name = "Оля"
```

Остальной код менять не придётся.

Переменные позволяют программе работать с данными через понятные имена.

Значение переменной можно изменить

Имя может быть связано с одним значением, а позже — с другим.

```
score = 10
print(score)

score = 15
print(score)
```

Результат:

```
10
15
```

Сначала:

```
score -> 10
```

После новой строки:

```
score = 15
```

связь становится другой:

```
score -> 15
```



Рисунок 2.3: Изменение значения переменной

Это называется **повторным присваиванием**.

Python использует текущее значение

Python выполняет программу строка за строкой и использует то значение, которое связано с именем в данный момент.

```
message = "Начало"
print(message)

message = "Конец"
print(message)
```

Результат:

```
Начало
Конец
```

Первый `print()` использует значение `"Начало"`.

Второй `print()` использует уже новое значение `"Конец"`.

Имена переменных должны быть понятными

Сравните:

```
x = 25
```

и:

```
age = 25
```

Обе строки работают.

Но во втором случае сразу понятнее, что означает число 25.

Поэтому лучше использовать имена, которые объясняют смысл значения:

```
user_name = "Анна"
product_price = 120
player_score = 50
```

а не:

```
a = "Анна"
b = 120
c = 50
```

Чем больше программа, тем важнее понятные имена.

Как можно называть переменные

Простые имена выглядят так:

```
name = "Анна"
age = 25
score = 100
```

Если имя состоит из нескольких слов, обычно используют символ _:

```
user_name = "Анна"
player_score = 100
product_price = 49
```

Такой стиль называется `snake_case`.

Для начала достаточно соблюдать несколько правил:

- использовать буквы, цифры и _;
- не начинать имя с цифры;
- не использовать пробелы;
- выбирать имя, которое объясняет смысл значения.

Например:

```
user_name = "Маша"
```

работает.

А такие варианты использовать нельзя:

```
2name = "Маша"
user name = "Маша"
```

Python не сможет правильно разобрать такой код.

> Совет
>
> Не пытайтесь делать имена максимально короткими.
> Лучше написать:
>
> ```
> player_score = 100
> ```
>
> чем:

```
ps = 100
```

если длинное имя делает код понятнее.

Попробуйте сами

Создайте несколько переменных:

```
name = "Алекс"
city = "Москва"
age = 20

print(name)
print(city)
print(age)
```

Запустите программу.

Затем:

1. измените имя;
2. измените город;
3. измените возраст;
4. запустите программу ещё раз.

Обратите внимание: команды `print()` менять не приходится.

Теперь добавьте ещё одну переменную:

```
language = "Python"
```

и выведите её значение.

Практика: карточка разработчика

Создайте программу, которая хранит информацию о начинающем Python-разработчике.

```
name = "Анна"
city = "Казань"
language = "Python"
experience_months = 2
```

```
print("Имя:", name)
print("Город:", city)
print("Язык:", language)
print("Месяцев обучения:", experience_months)
```

Измените значения и запустите программу ещё раз.

Практика: изменение счёта

Создайте переменную `score`, несколько раз присвойте ей новое значение и после каждого изменения выводите результат.

```
score = 0
print(score)

score = 10
print(score)

score = 25
print(score)

score = 50
print(score)
```

Каждое новое присваивание связывает имя `score` с новым значением.

Что важно запомнить

- переменная — это имя, связанное со значением;
- = используется для присваивания;
- слева от = находится имя переменной;
- справа находится значение;
- значение, связанное с именем, можно изменить;
- Python выполняет код сверху вниз;
- понятные имена делают программу легче для чтения;
- имена из нескольких слов обычно записывают в стиле `snake_case`.

Главная идея главы:

```
имя -> значение
```

Например:

```
name  -> "Анна"
score -> 100
age   -> 25
```

Что дальше?

Теперь мы умеем давать данным имена.

Но наши переменные уже содержали разные значения:

```
name = "Анна"
age = 25
temperature = 21.5
```

`"Анна"`, `25` и `21.5` выглядят по-разному — и Python работает с ними по-разному.

В следующей главе разберёмся:

Какие данные может хранить программа?

2 3.2 Типы данных

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как Python понимает, что значение является числом, текстом или логическим значением — и почему это важно?**

В предыдущей главе мы познакомились с переменными.

Мы уже можем сохранить значение:

```
name = "Анна"
age = 25
```

Но эти два значения отличаются друг от друга.

`"Анна"` — текст.

25 — число.

Для человека разница очевидна.

Но Python тоже должен её понимать.

Почему?

Потому что с разными данными можно выполнять разные действия.

Например, числа можно складывать:

```
10 + 5
```

а текст можно объединять:

```
"Py" + "thon"
```

В обоих случаях используется знак +, но результат получается разным.

Чтобы понимать, как работать со значением, Python хранит информацию о его **типе**.

Что такое тип данных

Тип данных описывает, **что представляет собой значение и какие операции с ним допустимы**.

Рассмотрим несколько значений:

```
42
3.14
"Python"
True
```

Для человека это:

- целое число;
- дробное число;
- текст;
- логическое значение.

Python различает их примерно так же.

У каждого значения есть свой тип.

```
42          → int
3.14        → float
"Python"    → str
True        → bool
```

Названия `int`, `float`, `str` и `bool` мы скоро разберём подробнее.

Пока важна сама идея:

> Тип сообщает Python, что это за значение и как с ним можно работать.



Рисунок 2.1: Каждое значение Python имеет тип

Тип принадлежит значению

Здесь есть важный момент.

Посмотрим на код:

```
age = 25
```

Иногда говорят:

> `age` — переменная типа `int`.

Для первого знакомства это допустимое упрощение.

Но точнее будет сказать так:

> Имя `age` сейчас ссылается на значение `25`, а значение `25` имеет тип `int`.

Почему это важно?

Потому что позже мы можем написать:

```
age = 25
age = "двадцать пять"
```

Сначала имя age связано с целым числом.

Затем — с текстом.

Само имя не навсегда привязано к одному типу.

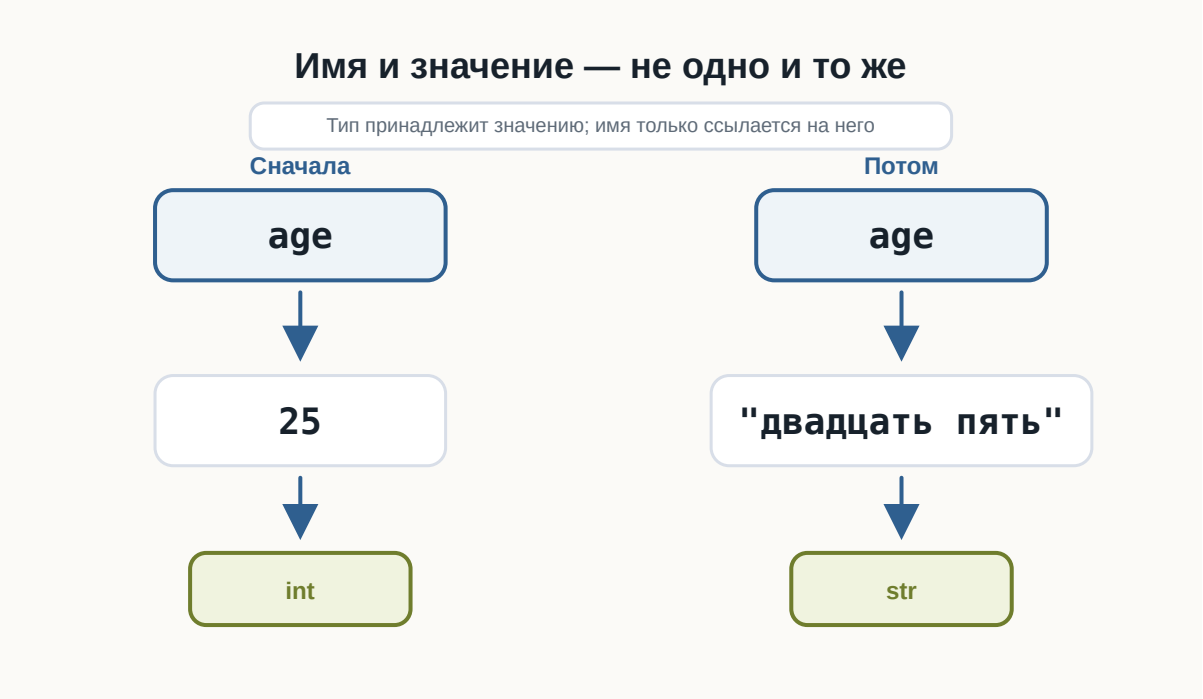


Рисунок 2.2: Имя может ссылаться на разные значения

Python определяет тип автоматически

В некоторых языках программист заранее указывает тип переменной.

Например, концептуально это может выглядеть так:

```
целое число age = 25
```

В Python обычно достаточно написать:

```
age = 25
```

Python видит значение 25 и сам определяет его тип.

То же самое происходит здесь:

```
price = 19.99
language = "Python"
is_learning = True
```

Python определит:

```
19.99      → float
"Python"   → str
True        → bool
```

Нам не нужно отдельно сообщать ему тип каждого значения.

Такой подход делает первые программы компактнее.

Как узнать тип значения

Python предоставляет встроенную функцию `type()`.

Например:

```
print(type(42))
```

Результат будет примерно таким:

```
<class 'int'>
```

Для текста:

```
print(type("Python"))
```

получим:

```
<class 'str'>
```

Можно проверить и значение, связанное с переменной:

```
age = 25

print(type(age))
```

Результат:

```
<class 'int'>
```

Пока слово `class` можно не разбирать.

Мы вернёмся к классам значительно позже.

Сейчас достаточно смотреть на часть:

```
int
```

или:

```
str
```

Она сообщает тип значения.

> Попробуйте сами
>
> Создайте несколько переменных:
>
> ```
> age = 25
> height = 1.82
> name = "Анна"
> is_student = True
> ```
>
> Используйте `type()` и посмотрите тип каждого значения.
> Попробуйте заранее предположить результат до запуска программы.

Целые числа: `int`

Целые числа в Python имеют тип:

```
int
```

Название происходит от английского слова *integer* — целое число.

Примеры:

```
0
1
42
-10
1000000
```

Все они имеют тип int.

```
print(type(42))
print(type(-10))
```

Целые числа можно использовать в арифметических операциях:

```
a = 10
b = 3

print(a + b)
print(a - b)
print(a * b)
```

Результат:

```
13
7
30
```

Размер числа при этом не определяет другой тип.

Например:

```
small = 10
large = 10000000000000000000000000000000
```

Оба значения являются int.

Дробные числа: float

Для чисел с дробной частью Python использует тип:

```
float
```

Например:

```
3.14
0.5
-10.25
```

Важно использовать **точку**, а не запятую:

```
price = 19.99
```

Проверим тип:

```
print(type(price))
```

Результат:

```
<class 'float'>
```

`int` и `float` — разные типы, но оба представляют числа.

Поэтому Python позволяет использовать их вместе:

```
total = 10 + 2.5

print(total)
```

Результат:

```
12.5
```

Об особенностях вычислений с дробными числами мы поговорим отдельно, когда начнём глубже работать с числами.

Текст: `str`

Текстовые значения имеют тип:

```
str
```

Название происходит от слова *string* — строка.

Строка записывается в кавычках:

```
"Python"
```

или:

```
'Python'
```

Например:

```
name = "Анна"
language = "Python"

print(type(name))
print(type(language))
```

Оба значения имеют тип `str`.

Очень важно понимать разницу между:

```
42
```

и:

```
"42"
```

Выглядят они похоже.

Но первое значение — число:

```
int
```

а второе — текст:

```
str
```

Для Python это принципиально разные данные.

Одинаково выглядит — разный смысл

Рассмотрим:

```
number = 42
text = "42"
```

Если вывести значения:

```
print(number)
print(text)
```

мы увидим:

```
42
42
```

На экране они выглядят одинаково.

Но проверим тип:

```
print(type(number))
print(type(text))
```

Получим:

```
<class 'int'>
<class 'str'>
```

Именно тип определяет, какие действия Python будет выполнять с этими значениями.

Один оператор может работать по-разному

Посмотрим на числа:

```
print(10 + 5)
```

Результат:

```
15
```

Python выполняет сложение.

Теперь используем строки:

```
print("Py" + "thon")
```

Результат:

```
Python
```

Знак + остался тем же.

Но операция изменилась.

Для чисел:

```
10 + 5 → сложение
```

Для строк:

```
"Py" + "thon" → объединение
```

Python может выбрать правильное действие именно потому, что знает тип каждого значения.

```
# Для int знак + выполняет арифметическое сложение.
print(10 + 5)

# Для str знак + объединяет строки.
print("Py" + "thon")
```



Рисунок 2.3: Один оператор может работать по-разному

Что произойдёт, если типы несовместимы

Попробуем выполнить:

```
print(10 + "5")
```

Человек может подумать:

> Наверное, получится 15.

Но Python не делает таких предположений.

`10` имеет тип `int`.

`"5"` имеет тип `str`.

Python не знает, хотим ли мы получить:

```
15
```

или:

```
105
```

Поэтому программа завершится ошибкой.

Пример сообщения:

```
TypeError
```

Название уже подсказывает причину:

> ошибка связана с типами данных.

Это полезное поведение.

Python не пытается угадывать намерения программиста там, где результат может быть неоднозначным.

```
number = 10
text = "5"

# Ошибка сделана намеренно для учебного примера.
print(number + text)
```



Рисунок 2.4: Python сообщает TypeError

Логические значения: bool

Иногда программе нужно хранить не число и не текст, а ответ на простой вопрос:

> Да или нет?

Для этого существует тип:

```
bool
```

У него всего два значения:

```
True
False
```

Например:

```
is_student = True
is_admin = False
```

Проверим:

```
print(type(is_student))
```

Результат:

```
<class 'bool'>
```

Обратите внимание: `True` и `False` пишутся с большой буквы.

Это специальные значения Python.

Нельзя писать:

```
true
```

или:

```
false
```

если мы хотим получить логические значения Python.

Зачем нужны True и False

Логические значения особенно важны, когда программа должна принимать решения.

Например:

```
is_logged_in = True
```

Позже программа сможет проверить это значение и решить:

```
показать страницу
```

или:

```
попросить пользователя войти
```

Пока мы только научимся хранить такие значения.

Условия и принятие решений разберём в отдельной главе.

Отсутствие значения: None

Иногда нужно явно показать:

> Здесь пока нет значения.

Для этого Python предоставляет специальное значение:

```
None
```

Например:

```
result = None
```

Проверим его тип:

```
print(type(result))
```

Получим:

```
<class 'NoneType'>
```

None — это не:

```
0
```

не:

```
""
```

и не:

```
False
```

Это отдельное специальное значение.

Оно означает отсутствие обычного значения.

Позже мы встретим None при работе с функциями, поиском данных и многими библиотеками Python.

Не путайте значение и его представление

Рассмотрим ещё один пример:

```
age = 25
```

Здесь 25 — число.

Но после:

```
print(age)
```

терминал показывает символы:

```
25
```

Это не означает, что значение внутри программы стало строкой.

`print()` просто превращает значение в форму, которую можно показать человеку.

Поэтому внешний вид данных на экране не всегда говорит об их типе внутри программы.

Если сомневаетесь — используйте:

```
type()
```

Тип может измениться при новом присваивании

Рассмотрим:

```
value = 10

print(type(value))
```

Получим:

```
<class 'int'>
```

Теперь присвоим этому имени другое значение:

```
value = "Python"

print(type(value))
```

Получим:

```
<class 'str'>
```

Это нормальное поведение Python.

Имя `value` сначала ссылалось на число, затем — на строку.

Такое свойство связано с **динамической типизацией** Python.

Термин полезно знать, но пока достаточно понимать сам принцип:

> Тип определяется значением во время выполнения программы.

```
value = 10
print(value, type(value))
```

```
# Имя value теперь ссылается на другое значение.
value = "Python"
print(value, type(value))
```

Динамическая типизация не означает отсутствие типов

Иногда можно услышать:

> В Python нет типов.

Это неправильно.

Типы в Python есть, и они играют очень важную роль.

Например:

```
10 + "5"
```

не работает именно потому, что Python различает `int` и `str`.

Особенность Python в другом:

> Обычно программисту не нужно заранее объявлять тип переменной.

Python определяет тип значения во время выполнения.

Основные типы, которые мы уже встретили

На этом этапе достаточно знать пять типов:

Тип	Что хранит	Пример
`int`	целые числа	`42`
`float`	дробные числа	`3.14`
`str`	текст	`"Python"`
`bool`	истина или ложь	`True`
`NoneType`	отсутствие значения	`None`

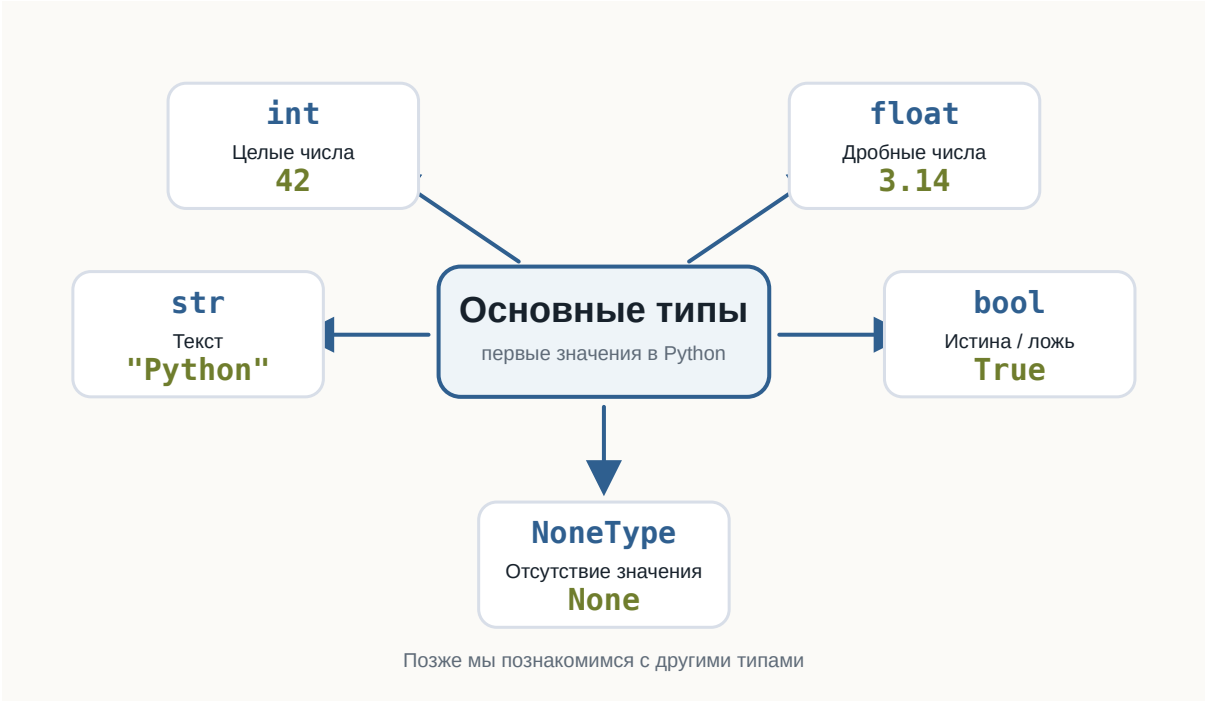


Рисунок 2.5: Основные типы Python

Это далеко не все типы Python.

Позже появятся:

```
list
tuple
dict
set
```

а ещё позже мы научимся создавать собственные типы с помощью классов.

Но сейчас этого набора достаточно для первых программ.

Небольшой эксперимент

Создайте файл:

```
data_types.py
```

и добавьте:

```
age = 25
height = 1.82
name = "Анна"
is_learning = True
result = None

print(age, type(age))
print(height, type(height))
print(name, type(name))
print(is_learning, type(is_learning))
print(result, type(result))
```

Запустите программу.

Перед запуском попробуйте самостоятельно предсказать тип каждого значения.

Затем измените значения.

Например:

```
age = "25"
```

или:

```
height = 182
```

Снова запустите программу и посмотрите, что изменилось.

```
age = 25
height = 1.82
name = "Анна"
is_learning = True
result = None

print(age, type(age))
print(height, type(height))
print(name, type(name))
print(is_learning, type(is_learning))
print(result, type(result))
```

Попробуйте сами

Создайте пять собственных переменных так, чтобы среди них были:

- целое число;

- дробное число;
- строка;
- логическое значение;
- `None`.

Для каждой переменной выведите значение и результат `type()`.
После этого измените тип хотя бы двух значений и запустите программу повторно.

Найдите ошибку

Рассмотрим программу:

```
price = 100
discount = "20"

print(price - discount)
```

Попробуйте перед запуском определить:

1. выполнится ли программа;
2. если нет — почему;
3. какие типы имеют `price` и `discount`.

Запустите код и изучите сообщение Python.

Пока не нужно исправлять программу преобразованием типов.

Мы специально оставим эту проблему для следующей темы.

Что важно запомнить

Каждое значение в Python имеет тип.

Тип определяет:

- что представляет собой значение;
- какие операции с ним можно выполнять;
- как Python должен его обрабатывать.

Мы познакомились с основными типами:

```
int
float
str
bool
NoneType
```

Python обычно определяет тип автоматически.

При этом имя переменной не обязано навсегда быть связано с одним типом:

```
value = 10
value = "Python"
```

Поэтому точнее думать так:

> Переменная — это имя, которое ссылается на значение, а тип принадлежит самому значению.

И ещё одна важная мысль:

> Данные могут выглядеть одинаково для человека, но иметь совершенно разный смысл для программы.

Например:

```
42
```

и:

```
"42"
```

— разные значения, потому что у них разные типы.

В следующей главе разберём:

> **Ввод данных**

2 3.3 Ввод данных

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как программа получает данные от пользователя?**

До сих пор наши программы работали только с данными, которые мы заранее записывали в коде.

Например:

```
name = "Анна"
age = 25

print(name)
print(age)
```

Такая программа всегда использует одни и те же значения.

Но реальные программы часто должны взаимодействовать с человеком.

Например:

- спросить имя;
- получить возраст;
- узнать город;
- попросить ввести поисковый запрос;
- получить ответ на вопрос.

Для этого программе нужен способ **получать данные во время выполнения**.

В Python для этого есть функция `input()`.

Первая интерактивная программа

Создадим программу:

```
name = input("Как вас зовут? ")

print("Привет,", name)
```

После запуска программа остановится и будет ждать ввода.

В терминале появится:

```
Как вас зовут?
```

Если пользователь введёт:

```
Анна
```

программа продолжит работу:

```
Привет, Анна
```

Теперь результат зависит не только от кода.

Он зависит ещё и от данных, которые ввёл пользователь.

Это наша первая по-настоящему **интерактивная программа**.

Как работает `input()`

Рассмотрим строку:

```
name = input("Как вас зовут? ")
```

Здесь происходит несколько действий.

Сначала Python выполняет:

```
input("Как вас зовут? ")
```

Функция показывает текст:

```
Как вас зовут?
```

и ждёт ответа пользователя.

Пользователь вводит, например:

```
Анна
```

После этого input() возвращает введённое значение.

Получается примерно такая последовательность:

```
Программа задаёт вопрос
        ↓
Пользователь вводит данные
        ↓
input() получает данные
        ↓
Возвращает значение
        ↓
Значение сохраняется в переменной
```

В нашем случае:

```
name = input("Как вас зовут? ")
```

после ввода можно мысленно представить как:

```
name = "Анна"
```

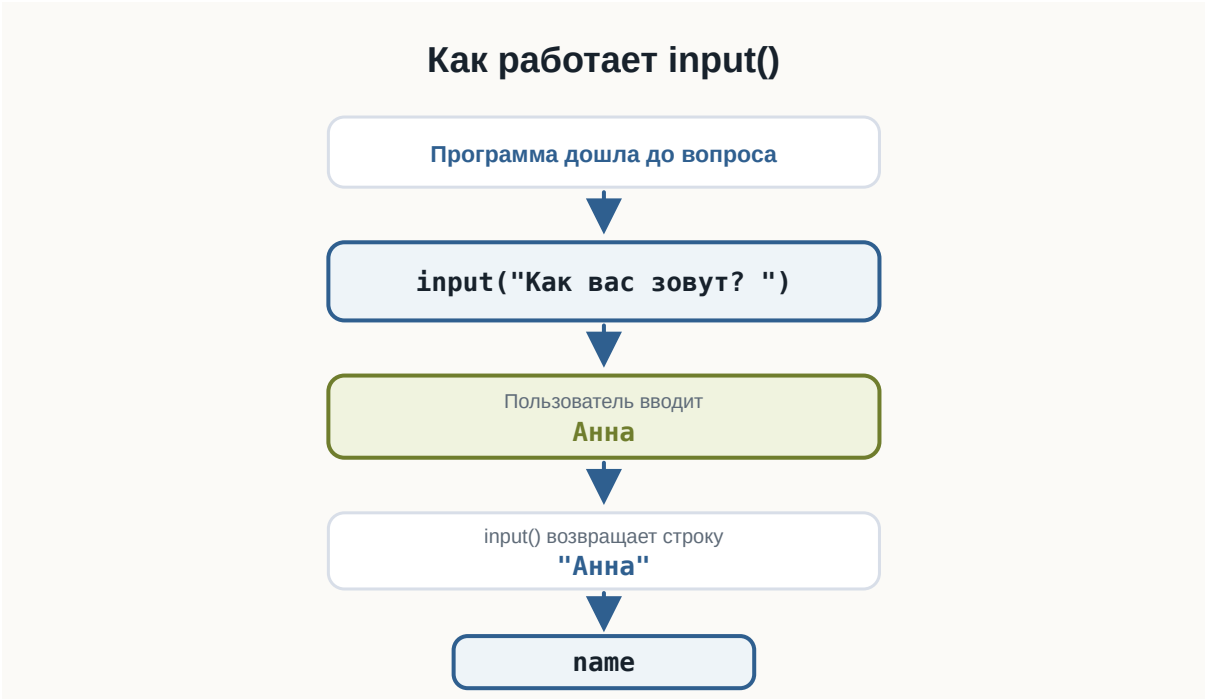


Рисунок 2.1: Путь значения от input() к переменной

Текст внутри input()

Текст:

```
"Как вас зовут? "
```

называется **приглашением ко вводу**.

Он объясняет пользователю, какие данные ожидает программа.

Например:

```
city = input("В каком городе вы живёте? ")
```

или:

```
language = input("Какой язык программирования вы изучаете? ")
```

Без приглашения тоже можно написать:

```
name = input()
```

Программа будет работать.

Но пользователь не поймёт, что именно нужно вводить.

Поэтому приглашение обычно делает программу понятнее.

Значение нужно сохранить

Результат input() можно сохранить в переменной:

```
name = input("Введите имя: ")
```

Теперь введённое значение связано с именем:

```
name
```

и его можно использовать дальше:

```
print("Привет,", name)
```

Например, пользователь вводит:

```
Максим
```

и программа выводит:

```
Привет, Максим
```

Переменная позволяет использовать полученные данные столько раз, сколько потребуется.

```
name = input("Введите имя: ")

print("Привет,", name)
print("Рад познакомиться,", name)
```

Попробуйте сами

Попробуйте сами

Создайте файл:

```
greeting.py
```

Напишите:

```
name = input("Как вас зовут? ")

print("Привет,", name)
```

Запустите программу несколько раз и каждый раз вводите другое имя.
Посмотрите, как меняется результат.
Затем измените приветствие самостоятельно.

Программа может задавать несколько вопросов

`input()` можно использовать несколько раз.

Например:

```
name = input("Как вас зовут? ")
city = input("Где вы живёте? ")

print("Имя:", name)
print("Город:", city)
```

Программа сначала остановится на первом `input()`.

После ответа перейдёт ко второму.

Пример работы:

```
Как вас зовут? Анна
Где вы живёте? Казань

Имя: Анна
Город: Казань
```

Инструкции по-прежнему выполняются сверху вниз.

Если программа встретила `input()`, она ждёт пользователя и только после ввода продолжает выполнение.

`input()` возвращает значение

В предыдущих главах мы уже использовали функции.

Например:

```
print("Привет")
```

Но между `print()` и `input()` есть важное различие.

`print()` показывает данные пользователю.

```
программа → пользователь
```

А `input()` получает данные от пользователя.

```
пользователь → программа
```

Можно представить их как два направления взаимодействия:

```
              print()
Программа ──────────→ Пользователь

Программа ←────────── Пользователь
              input()
```

Это две базовые операции общения программы с человеком:

- вывести информацию;
- получить информацию.

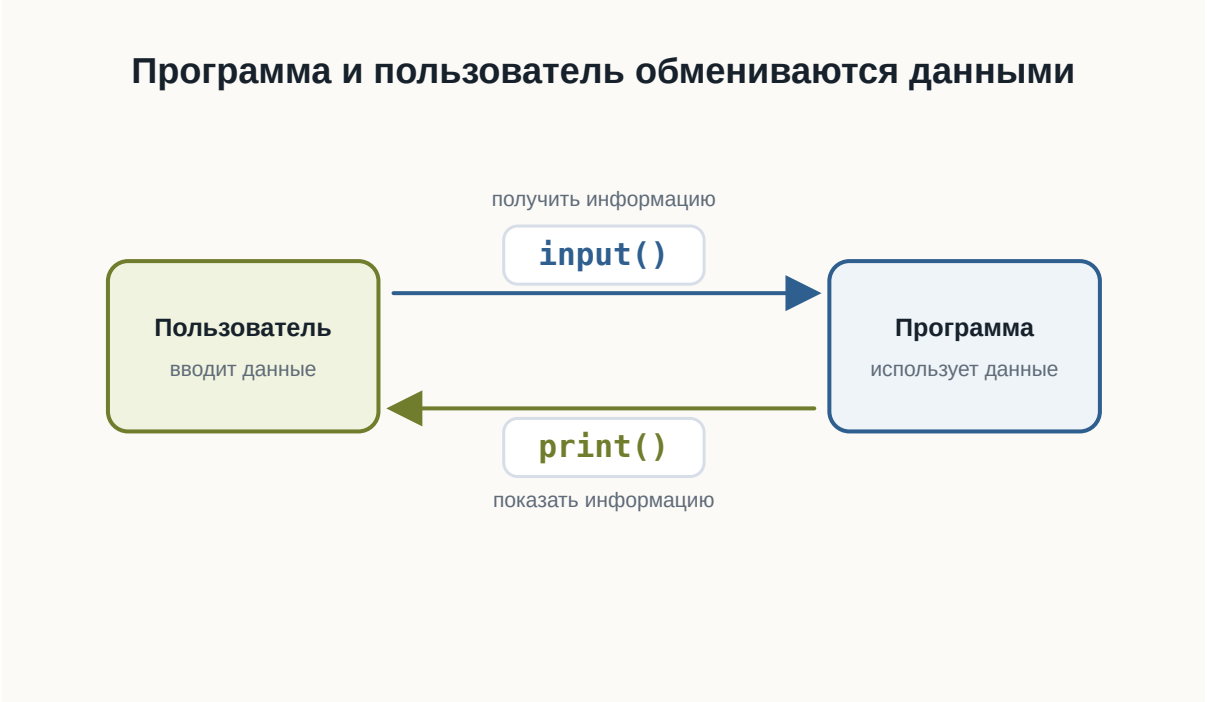


Рисунок 2.2: Диалог программы и пользователя

Какой тип возвращает input()

Теперь вспомним предыдущую главу.

Мы знаем, что каждое значение Python имеет тип.

Попробуем проверить результат input():

```
name = input("Введите имя: ")

print(type(name))
```

Если пользователь введёт:

```
Анна
```

получим:

```
<class 'str'>
```

Это ожидаемо.

Имя — текст.

Но теперь попробуем другое.

```
age = input("Сколько вам лет? ")

print(type(age))
```

Введите:

```
25
```

Результат:

```
<class 'str'>
```

Это уже интереснее.

Мы ввели число:

```
25
```

но Python получил значение типа:

```
str
```

input() всегда возвращает строку

Это одно из главных правил этой главы:

> **input() всегда возвращает строку.**

Неважно, что ввёл пользователь.

Если он введёт:

```
Анна
```

получим строку.

Если введёт:

```
25
```

тоже получим строку.

Если введёт:

```
3.14
```

снова получим строку.

Условно:

```
Пользователь вводит     Python получает

Анна                     "Анна"
25                       "25"
3.14                     "3.14"
True                     "True"
```

Все эти значения имеют тип:

```
str
```

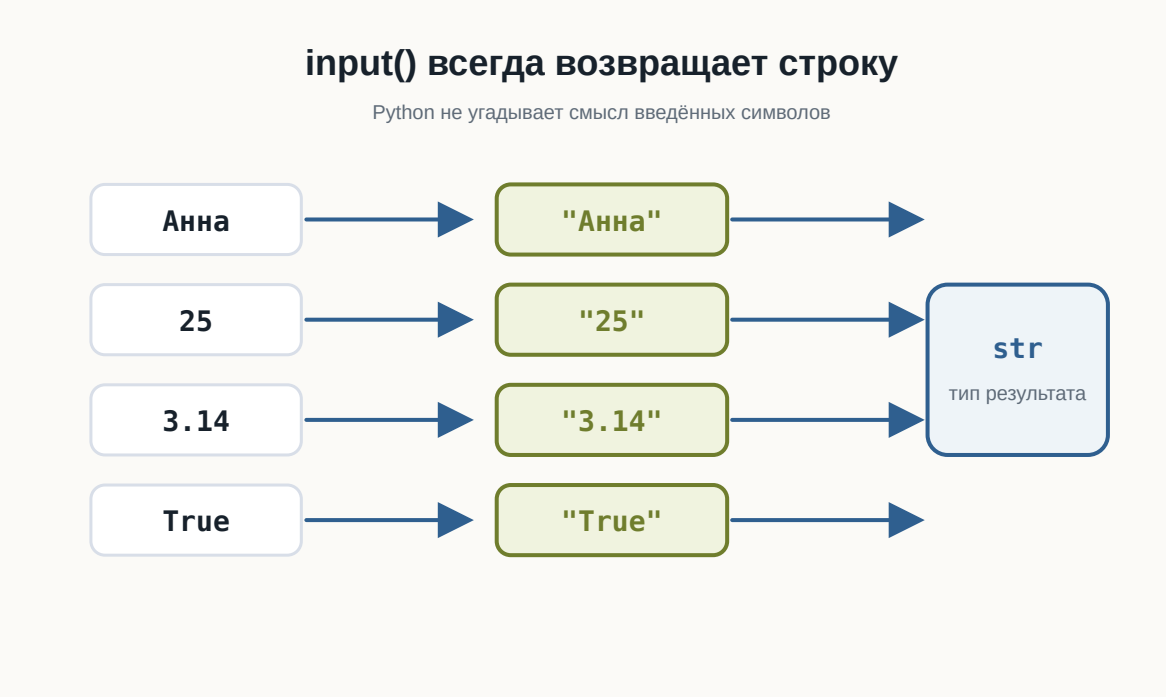


Рисунок 2.3: input всегда возвращает строку

Почему Python делает именно так

Когда пользователь печатает символы на клавиатуре, Python не пытается угадывать их смысл.

Например:

```
007
```

Что это?

Число семь?

Код пользователя?

Часть номера телефона?

Текстовый идентификатор?

Python не знает.

Поэтому `input()` использует простое и предсказуемое правило:

> Всё, что ввёл пользователь, сначала считается текстом.

А уже программист решает, что с этими данными делать дальше.

Проверим разные значения

Создадим программу:

```
value = input("Введите что-нибудь: ")

print("Значение:", value)
print("Тип:", type(value))
```

Запустите её несколько раз.

Введите:

```
Python
```

затем:

```
42
```

затем:

```
3.14
```

затем:

```
True
```

Во всех случаях `type()` покажет:

```
<class 'str'>
```

> **Эксперимент**
>
> До запуска программы попробуйте предположить тип результата.
> Введите по очереди:

```
100
-5
0
3.14
False
None
Python
```

После каждого запуска проверяйте результат `type()`.
Что общего у всех результатов?

Строка "25" — не число 25

В предыдущей главе мы уже встречали:

```
25
```

и:

```
"25"
```

Первое значение имеет тип:

```
int
```

Второе:

```
str
```

После:

```
age = input("Сколько вам лет? ")
```

если пользователь введёт:

```
25
```

в переменной age окажется не число:

```
25
```

а строка:

```
"25"
```

На экране они выглядят одинаково.

Но для Python это разные значения.

Почему это важно

Пока мы просто выводим полученное значение, проблемы нет:

```
age = input("Сколько вам лет? ")

print("Ваш возраст:", age)
```

Это работает.

Но попробуем выполнить вычисление.

```
age = input("Сколько вам лет? ")

# Ошибка сделана намеренно для учебного примера.
print(age + 1)
```

Предположим, пользователь ввёл:

```
25
```

Можно ожидать:

```
26
```

Но программа завершится ошибкой.

Почему?

Потому что фактически Python пытается выполнить:

```
"25" + 1
```

Слева находится:

```
str
```

справа:

```
int
```

Эти типы нельзя таким образом сложить.

Ошибка снова помогает нам

Мы можем увидеть ошибку:

```
TypeError
```

Мы уже встречали её в предыдущей главе.

Теперь становится понятно, почему она особенно важна при работе с пользовательским вводом.

Пользователь вводит:

```
25
```

но программа получает:

```
"25"
```

Если нам действительно нужно число, строку придётся сначала превратить в число.

Но этим мы займёмся в следующей главе.

Не спешите исправлять пример

Возможно, вы уже знаете, что можно написать что-то вроде:

```
int(...)
```

Пока не будем этого делать.

Сейчас важно хорошо понять саму проблему.

Рассмотрим:

```
age = input("Сколько вам лет? ")
```

После ввода 25:

```
age → "25" → str
```

А для арифметики нам нужно:

```
age → 25 → int
```

Между этими двумя состояниями должно произойти **преобразование данных**.

Именно этому будет посвящена следующая глава.

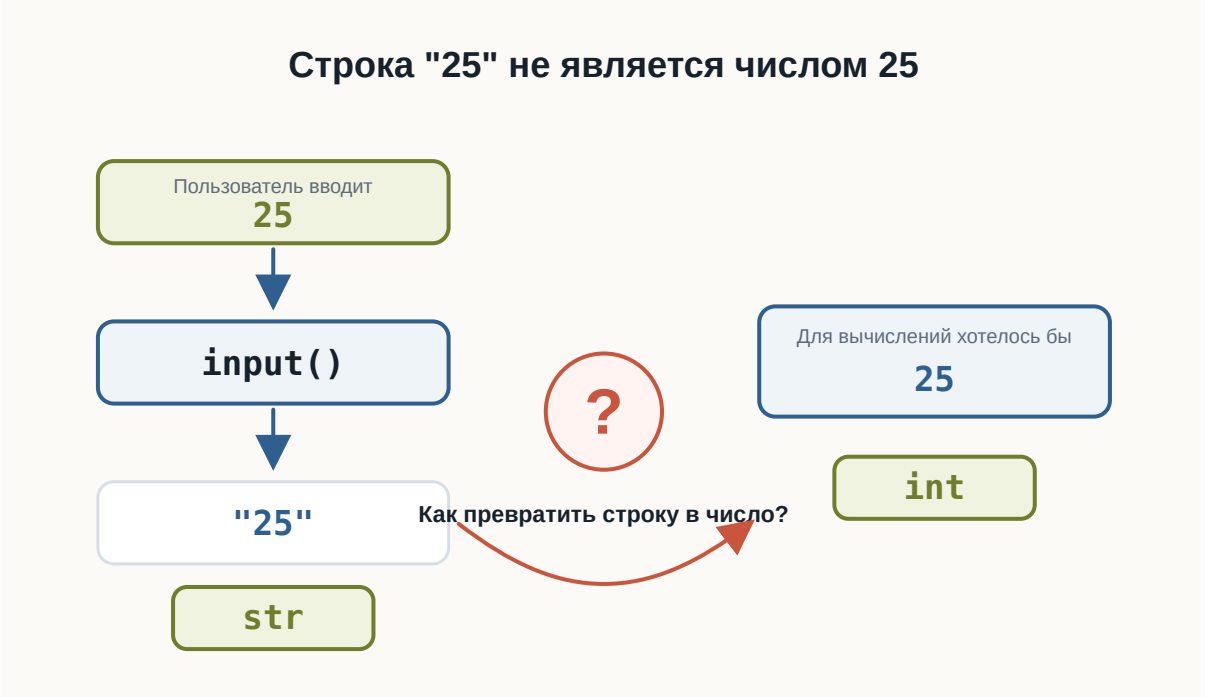


Рисунок 2.4: Строка из input и число для вычислений

Ввод и вывод вместе

Теперь мы можем написать небольшую программу, которая действительно взаимодействует с пользователем.

```
name = input("Как вас зовут? ")
city = input("Ваш город? ")
profession = input("Ваша профессия? ")
language = input("Какой язык вы изучаете? ")

print()
print("Профиль")
print("Имя:", name)
print("Город:", city)
print("Профессия:", profession)
print("Изучает:", language)
```

Пример выполнения:

```
Как вас зовут? Анна
Ваш город? Казань
Ваша профессия? Разработчик
Какой язык вы изучаете? Python

Профиль
Имя: Анна
Город: Казань
Профессия: Разработчик
Изучает: Python
```

Обратите внимание:

сама программа одна и та же.

Но результат может быть разным при каждом запуске.

Именно ввод данных делает программу интерактивной.

Практика: небольшая анкета

Создайте файл:

```
profile.py
```

Программа должна спросить:

- имя;
- город;
- профессию;
- язык программирования, который изучает пользователь.

После этого вывести небольшой профиль.

Например:

```
Как вас зовут? Алексей
Ваш город? Москва
Ваша профессия? Дизайнер
Какой язык вы изучаете? Python

Профиль
Имя: Алексей
Город: Москва
Профессия: Дизайнер
Изучает: Python
```

Не копируйте готовый результат буквально.

Попробуйте самостоятельно придумать:

- тексты вопросов;
- названия переменных;
- формат результата.

Практика: предскажите результат

Рассмотрите программу:

```
value = input("Введите значение: ")

print(value)
print(type(value))
```

Пользователь вводит:

```
100
```

Что выведет:

```
type(value)
```

Выберите ответ:

```
int
float
str
bool
```

> Ответ
>
> Правильный ответ:
>
> ```
> str
> ```
>
> `input()` всегда возвращает строку.
> Поэтому введённое `100` внутри программы становится:
>
> ```
> "100"
> ```

Практика: найдите проблему

Что произойдёт здесь?

```
year = input("Введите текущий год: ")

print(year + 1)
```

Предположим, пользователь вводит:

```
2026
```

Подумайте:

1. какого типа будет `year`;
2. выполнится ли `year + 1`;
3. если нет — почему.

Ответ

`year` будет иметь тип:

```
str
```

Фактически программа попытается выполнить:

```
"2026" + 1
```

Python не может сложить `str` и `int`, поэтому возникнет:

```
TypeError
```

Чтобы выполнить арифметику, сначала потребуется преобразовать введённую строку в число.

Практика: что хранится в переменных

Рассмотрите:

```
first_name = input("Имя: ")
age = input("Возраст: ")
height = input("Рост: ")
```

Пользователь вводит:

```
Анна
25
1.68
```

Определите тип:

```
first_name
age
height
```

до запуска программы.

Ответ

Все три значения имеют тип:

```
str
```

Получается:

```
first_name → "Анна" → str
age        → "25"   → str
height     → "1.68" → str
```

`input()` не пытается определить, является введённый текст числом.

Задача: персональное приветствие

Напишите программу, которая спрашивает:

```
Имя
Город
```

и выводит сообщение примерно такого вида:

```
Привет, Анна!
Казань — отличный город.
```

Попробуйте решить задачу самостоятельно.

Возможное решение

Один из вариантов:

```
name = input("Как вас зовут? ")
city = input("В каком городе вы живёте? ")

print("Привет,", name)
print(city, "— отличный город.")
```

Это не единственное правильное решение.
Текст сообщений и имена переменных могут отличаться.

Задача: мини-интервью

Напишите программу, которая задаёт пользователю минимум четыре вопроса.

Например:

- имя;
- профессия;
- любимая технология;
- цель обучения.

После ввода программа должна вывести все ответы в удобном виде.

Сначала попробуйте решить задачу самостоятельно.

> **Возможное решение**
>
> Например:
>
> ```
> name = input("Как вас зовут? ")
> profession = input("Ваша профессия? ")
> technology = input("Любимая технология? ")
> goal = input("Что вы хотите изучить? ")
>
> print()
> print("Мини-интервью")
> print("Имя:", name)
> print("Профессия:", profession)
> print("Любимая технология:", technology)
> print("Цель:", goal)
> ```
>
> Ваша программа может выглядеть иначе.
> Главное — самостоятельно использовать несколько вызовов `input()` и сохранить полученные значения в переменных.

Задача со звёздочкой

Пользователь вводит:

```
10
```

Программа:

```
value = input("Введите число: ")

print(value + value)
```

Какой результат появится?

Подумайте до запуска.

> **Ответ**
>
> Результат:
>
> ```
> 1010
> ```
>
> Почему?
> После `input()` значение имеет вид:
>
> ```
> "10"
> ```
>
> Поэтому Python выполняет:
>
> ```
> "10" + "10"
> ```
>
> Для строк оператор + означает объединение:
>
> ```
> "10" + "10" → "1010"
> ```
>
> Это ещё раз показывает, почему тип данных имеет значение.

Что важно запомнить

`input()` позволяет программе получать данные от пользователя.

Например:

```
name = input("Введите имя: ")
```

Основная схема выглядит так:

```
Пользователь
     ↓
```

```
вводит данные
      ↓
input()
      ↓
возвращает строку
      ↓
переменная
      ↓
программа использует значение
```

Главные правила этой главы:

1. `input()` останавливает программу и ждёт пользователя.
2. Текст внутри `input()` объясняет, что нужно ввести.
3. Результат `input()` можно сохранить в переменной.
4. `input()` всегда возвращает `str`.
5. Даже введённые числа сначала становятся строками.
6. Поэтому перед вычислениями иногда понадобится изменить тип данных.

И самое важное:

> **Ввод данных превращает программу из заранее заданной последовательности действий в программу, которая может реагировать на пользователя.**

Что дальше?

Теперь наша программа умеет получать данные.

Но появилась новая проблема.

Пользователь вводит:

```
25
```

а Python получает:

```
"25"
```

Что делать, если нам действительно нужно число?

Следующий главный вопрос:

Как превратить значение одного типа в значение другого типа?

Этому посвящена следующая глава:

«Преобразование типов».

2 3.4 Преобразование типов

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как превратить значение одного типа в значение другого типа?**

В предыдущей главе мы столкнулись с важной проблемой.

Пользователь вводит:

```
25
```

Но `input()` возвращает:

```
"25"
```

То есть не число:

```
25 → int
```

а строку:

```
"25" → str
```

Пока мы просто выводим значение, это не мешает:

```
age = input("Сколько вам лет? ")

print("Ваш возраст:", age)
```

Но как только появляется арифметика, возникает проблема:

```
age = input("Сколько вам лет? ")

print(age + 1)
```

Если пользователь введёт:

```
25
```

Python фактически попытается выполнить:

```
"25" + 1
```

Слева находится `str`, справа — `int`.

Поэтому программа завершится с ошибкой `TypeError`.

Чтобы выполнить вычисление, нам нужно превратить строку:

```
"25"
```

в число:

```
25
```

Именно для этого в Python используется **преобразование типов**.

Что такое преобразование типов

Преобразование типов позволяет получить значение одного типа на основе значения другого типа.

Например:

```
"25" → 25
 str     int
```

или:

```
"3.14" → 3.14
  str      float
```

или наоборот:

```
25 → "25"
int    str
```

Для этого Python предоставляет специальные функции.

Основные из них:

```
int()
float()
str()
bool()
```

Каждая функция пытается создать значение определённого типа.

Например:

```
int("25")
```

возвращает:

```
25
```

Теперь это уже настоящее целое число.

Первое преобразование

Вернёмся к нашей программе:

```
age = input("Сколько вам лет? ")
```

После ввода 25:

```
age → "25" → str
```

Попробуем преобразовать значение:

```
age = int(age)
```

Теперь:

```
age → 25 → int
```

Проверим:

```
age = input("Сколько вам лет? ")

print(type(age))

age = int(age)

print(type(age))
print("Через год вам будет:", age + 1)
```

Если пользователь введёт:

```
25
```

получим:

```
<class 'str'>
<class 'int'>
```

Тип значения действительно изменился.

> **Обратите внимание**
>
> Можно читать:
>
> ```
> int(age)
> ```
>
> примерно так:
>
> «Получить целое число из значения age».

Функция int()

Функция int() используется для получения целого числа.

Например:

```
value = "25"

print(value)
print(type(value))

number = int(value)

print(number)
print(type(number))
```

Результат:

```
42
<class 'int'>
```

До преобразования:

```
"42" → str
```

После:

```
42 → int
```

На экране значения выглядят почти одинаково.

Но для Python это разные типы данных.

И теперь с числом можно выполнять арифметические операции:

```
number = int("42")

print(number + 8)
```

Результат:

```
50
```

Исправляем программу с возрастом

Теперь мы можем решить проблему из предыдущей главы.

Было:

```
age = input("Сколько вам лет? ")

print(age + 1)
```

Эта программа не работает, потому что age содержит строку.

Добавим преобразование:

```
age = input("Сколько вам лет? ")
age = int(age)

print(age + 1)
```

Пример работы:

```
Сколько вам лет? 25
26
```

Теперь последовательность выглядит так:

```
Пользователь вводит
        ↓
       25
        ↓
     input()
        ↓
      "25"
        ↓
       str
        ↓
     int()
        ↓
       25
        ↓
       int
        ↓
    вычисление
```

Это одна из самых распространённых схем при работе с пользовательским вводом.

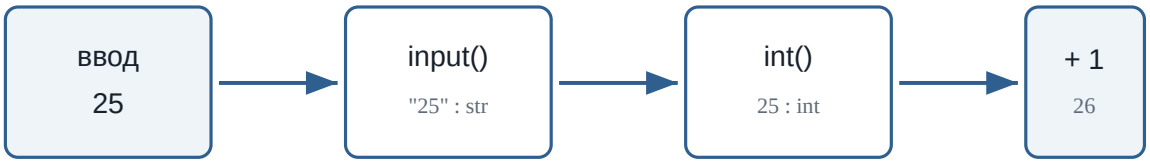


Рисунок 2.1: Путь значения от input() к вычислению

Преобразование можно выполнить сразу

Мы написали:

```
age = input("Сколько вам лет? ")
age = int(age)
```

Но эти действия можно объединить:

```
age = int(input("Сколько вам лет? "))
```

Разберём выражение изнутри.

Сначала выполняется:

```
input("Сколько вам лет? ")
```

Пользователь вводит:

```
25
```

`input()` возвращает:

```
"25"
```

Затем выполняется:

```
int("25")
```

и получается:

```
25
```

И только после этого значение сохраняется:

```
age → 25 → int
```

> **Два варианта**
>
> Эти программы делают одно и то же:
>
> ```
> age = input("Сколько вам лет? ")
> age = int(age)
> ```
>
> и:
>
> ```
> age = int(input("Сколько вам лет? "))
> ```
>
> Второй вариант короче.
> Первый иногда удобнее для начинающих, потому что хорошо показывает каждый шаг.

Попробуйте сами

> **Попробуйте сами**
>
> Создайте программу:
>
> ```
> number = int(input("Введите число: "))
>
> print("Вы ввели:", number)
> print("Тип:", type(number))
> print("Следующее число:", number + 1)
> ```
>
> Запустите её несколько раз.
> Введите:

```
10
```

затем:

```
100
```

затем:

```
-5
```

Перед каждым запуском попробуйте предсказать результат.

---

int() работает не с любой строкой

Рассмотрим:

```
number = int("25")
```

Это работает.

Python понимает, как строку:

```
"25"
```

превратить в целое число:

```
25
```

Но попробуем:

```
number = int("Python")
```

Что должно получиться?

Python не может превратить слово:

```
Python
```

в целое число.

Поэтому возникнет ошибка:

```
ValueError
```

То же самое произойдёт, если пользователь введёт текст там, где программа ожидает число:

```
age = int(input("Сколько вам лет? "))
```

и пользователь введёт:

```
двадцать пять
```

Python попытается выполнить:

```
int("двадцать пять")
```

и не сможет выполнить преобразование.

⚠ Преобразование должно иметь смысл

Функция `int()` не превращает любой текст в число.
Работает:

```
int("25")
int("-10")
int("0")
```

Не работает:

```
int("Python")
int("пять")
int("hello")
```

Дробные числа и `float()`

До сих пор мы работали с целыми числами.

Но пользователь может ввести:

```
3.14
```

После input() мы снова получим строку:

```
"3.14"
```

Попробуем:

```
number = int("3.14")
```

Такое преобразование не сработает.

3.14 — не целое число.

Для дробных чисел используется функция:

```
float()
```

Например:

```
number = float("3.14")

print(number)
print(type(number))
```

Результат:

```
3.14
<class 'float'>
```

Получается:

```
"3.14" → 3.14
  str     float
```

---

float() вместе с input()

Предположим, программа спрашивает рост пользователя:

```
height = input("Введите рост в метрах: ")
```

Пользователь вводит:

```
1.75
```

Но программа получает:

```
"1.75"
```

Для вычислений нужно преобразовать значение:

```
height = float(height)
```

Или сразу:

```
height = float(input("Введите рост в метрах: "))
```

Теперь:

```
height → 1.75 → float
```

Например:

```
height = float(input("Введите рост в метрах: "))

print("Ваш рост:", height)
print(type(height))
```

Когда использовать `int()`, а когда `float()`

Если программа ожидает целое число, обычно используется:

```
int()
```

Например:

```
age = int(input("Возраст: "))
year = int(input("Год: "))
count = int(input("Количество: "))
```

Если значение может содержать дробную часть:

```
float()
```

Например:

```
height = float(input("Рост: "))
weight = float(input("Вес: "))
price = float(input("Цена: "))
temperature = float(input("Температура: "))
```

Можно представить так:

```
целое значение
      ↓
    int()

дробное значение
      ↓
   float()
```

Как выбрать тип

Спросите себя:
Может ли у этого значения быть дробная часть?
Если нет:

```
int()
```

Если да:

```
float()
```

Пример: стоимость покупки

Напишем программу, которая спрашивает цену товара и количество.

```
price = float(input("Цена товара: "))
count = int(input("Количество: "))

total = price * count

print("Стоимость:", total)
```

Пример работы:

```
Цена товара: 12.5
Количество: 4
Стоимость: 50.0
```

Здесь используются два разных преобразования:

```
"12.5" → 12.5 → float
"4"    → 4    → int
```

После преобразования Python может выполнить умножение.

Преобразование float в int

`int()` умеет работать не только со строками.

Например:

```
number = 9.99

print(number)
print(type(number))

result = int(number)

print(result)
print(type(result))

print(int(3.9))
print(int(7.2))
print(int(-4.8))
```

Результат:

```
5
```

Дробная часть исчезла.

Ещё примеры:

```
print(int(3.9))
print(int(7.2))
print(int(-4.8))
```

Результат:

```
3
7
-4
```

> ⚠ Это не округление
>
> `int()` не округляет число.
> Например:
>
> ```
> int(9.99)
> ```
>
> даёт:
>
> ```
> 9
> ```
>
> а не:
>
> ```
> 10
> ```
>
> При преобразовании `float` в `int` дробная часть отбрасывается.

Преобразование `int` в `float`

Можно выполнить и обратное преобразование:

```
number = 10

result = float(number)
```

```
print(result)
print(type(result))
```

Результат:

```
10.0
<class 'float'>
```

Получается:

```
10 → 10.0
int   float
```

Значение по-прежнему представляет десять, но его тип изменился.

Функция str()

До сих пор мы превращали строки в числа.

Но можно сделать наоборот.

Функция:

```
str()
```

преобразует значение в строку.

Например:

```
age = 25

text = str(age)

print(text)
print(type(text))
```

Результат:

```
25
<class 'str'>
```

Получается:

```
25 → "25"
int    str
```

Зачем превращать число в строку

Рассмотрим:

```
age = 25

message = "Возраст: " + age
```

Такой код вызовет ошибку.

Python видит:

```
str + int
```

Но если преобразовать число:

```
age = 25

message = "Возраст: " + str(age)

print(message)
```

получим:

```
Возраст: 25
```

Теперь Python фактически выполняет:

```
"Возраст: " + "25"
```

То есть:

```
str + str
```

Это допустимая операция.

> **i** С print() обычно проще
>
> Если вы просто выводите значения через запятую:
>
> ```
> age = 25
>
> print("Возраст:", age)
> ```
>
> вызывать `str()` не требуется.
> `print()` умеет выводить значения разных типов.

Функция bool()

Ещё одна функция преобразования:

```
bool()
```

Она создаёт логическое значение:

```
True
```

или:

```
False
```

Например:

```
print(bool(1))
print(bool(0))
```

Результат:

```
True
False
```

Для чисел правило можно начать понимать так:

```
0        → False
не 0     → True
```

Например:

```
print(bool(0))
print(bool(5))
print(bool(-10))
print(bool(3.14))
```

Результат:

```
False
True
True
True
```

Строки и bool()

Пустая строка преобразуется в:

```
False
```

Например:

```
print(bool(""))
```

Результат:

```
False
```

Непустая строка:

```
print(bool("Python"))
```

даёт:

```
True
```

То есть:

```
""        → False
"Python"  → True
```

Но здесь есть важная особенность.

Рассмотрим:

```
print(bool("False"))
```

Какой будет результат?

```
True
```

Почему?

Потому что строка:

```
"False"
```

не пустая.

Python в данном случае не пытается понять значение написанного слова.

> ⚠ Не путайте текст и логическое значение
>
> Это разные значения:
>
> ```
> False
> ```
>
> и:
>
> ```
> "False"
> ```
>
> Первое имеет тип:
>
> ```
> bool
> ```
>
> Второе:
>
> ```
> str
> ```

Поэтому:

```
bool(False)
```

даёт `False`, а:

```
bool("False")
```

даёт `True`.

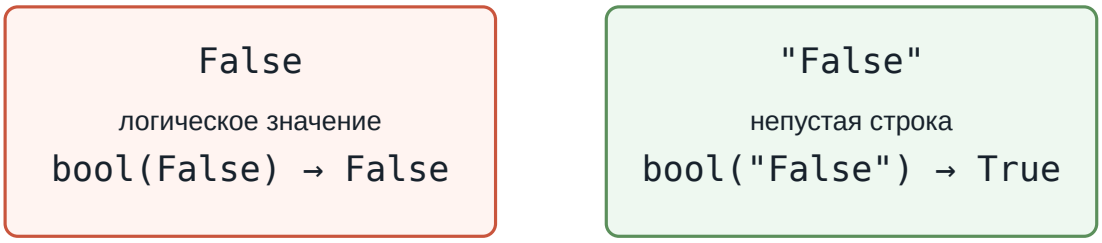


Рисунок 2.2: Почему bool(“False”) даёт True

Тип до и после преобразования

Функция `type()` помогает увидеть результат преобразования.

Например:

```
value = "100"

print(type(value))

value = int(value)

print(type(value))
```

Результат:

```
<class 'str'>
<class 'int'>
```

Можно экспериментировать:

```
value = "25"

number = int(value)
decimal = float(number)
text = str(decimal)

print(number, type(number))
print(decimal, type(decimal))
print(text, type(text))

print(bool(0))
print(bool(1))
print(bool(""))
print(bool("Python"))
print(bool("False"))
```

Так хорошо видно, как значение последовательно получает разные типы.

Преобразование не всегда возможно

Мы уже видели:

```
int("Python")
```

Такое преобразование невозможно.

То же относится к `float()`:

```
float("hello")
```

Python не знает, какое число должно соответствовать слову `"hello"`.

Поэтому возникает `ValueError`.

Запустите пример и посмотрите на сообщение об ошибке:

```
value = "Python"

# Ошибка сделана намеренно для учебного примера.
number = int(value)

print(number)
```

Попробуйте заменить `"Python"` на `"12.8"`.

Это тоже приведёт к `ValueError`, потому что строка `"12.8"` описывает дробное число, а не целое.

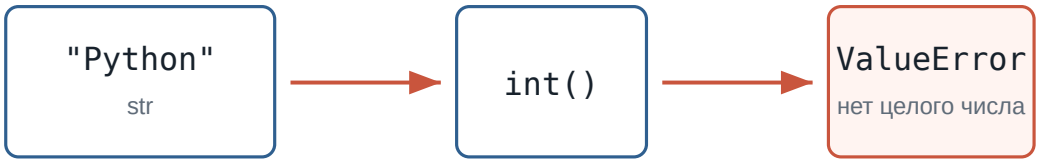


Рисунок 2.3: Когда int() не может создать целое число

При работе с пользовательским вводом это особенно важно.

Программа может ожидать:

```
Введите возраст: 25
```

но пользователь способен написать:

```
Введите возраст: двадцать пять
```

На этом этапе нашего курса достаточно помнить:

> Преобразование работает только тогда, когда исходное значение можно представить в нужном типе.

Позже мы научимся обрабатывать неправильный пользовательский ввод.

Попробуйте предсказать результат

Не запускайте код сразу.

Сначала попробуйте определить результат самостоятельно.

```
a = int("10")
b = float("2.5")

print(a)
```

```
print(b)
print(a + b)
```

Ответ

Получим:

```
10
2.5
12.5
```

a имеет тип `int`, а `b` — `float`.
Python может выполнять арифметические операции с этими числовыми типами.

Что важно запомнить

Преобразование типов позволяет получить значение одного типа из значения другого типа.

Основные функции:

```
int()    → целое число
float()  → дробное число
str()    → строка
bool()   → логическое значение
```

Особенно часто преобразование используется после `input()`:

```
age = int(input("Возраст: "))
```

или:

```
price = float(input("Цена: "))
```

Главная схема:

```
Пользователь
     ↓
   input()
```

```
      ↓
    str
      ↓
преобразование
      ↓
нужный тип
      ↓
вычисления
```

И самое важное:

> **Тип значения должен соответствовать тому, что программа собирается с этим значением делать.**

Если нужно считать — обычно требуется число.

Если нужно работать с текстом — строка.

Если значение пришло через `input()`, сначала помните:

```
input() → str
```

и только затем решайте, требуется ли преобразование.

Что дальше?

Теперь мы умеем получать данные пользователя:

```
input()
```

и превращать их в нужный тип:

```
int()
float()
str()
bool()
```

Теперь программа может не только получать данные, но и готовить их к настоящим вычислениям.

Например:

```
price = float(input("Цена: "))
count = int(input("Количество: "))

print("Итого:", price * count)
```

Следующий главный вопрос:

Как Python выполняет арифметические операции?

Этому посвящена следующая глава:

«Арифметические операции».

Практические задания

Задание 1. Возраст через год

Напишите программу, которая спрашивает возраст пользователя и выводит, сколько ему будет через год.

Пример:

```
Сколько вам лет? 25
Через год вам будет: 26
```

> Ответ

```
age = int(input("Сколько вам лет? "))

print("Через год вам будет:", age + 1)
```

Задание 2. Два числа

Пользователь вводит два целых числа.

Программа должна вывести их сумму.

Пример:

```
Первое число: 10
Второе число: 7
Сумма: 17
```

Ответ

```
first = int(input("Первое число: "))
second = int(input("Второе число: "))

print("Сумма:", first + second)
```

Задание 3. Цена покупки

Пользователь вводит цену одного товара и количество товаров.

Цена может быть дробной.

Вычислите общую стоимость.

Пример:

```
Цена: 12.5
Количество: 4
Итого: 50.0
```

Ответ

```
price = float(input("Цена: "))
count = int(input("Количество: "))

total = price * count

print("Итого:", total)
```

Задание 4. Предскажите типы

Не запускайте программу сразу.

```
a = int("15")
b = float("15")
c = str(15)
d = bool(15)
```

Определите тип каждой переменной.

Ответ

```
a → int
b → float
c → str
d → bool
```

Значения:

```
a == 15
b == 15.0
c == "15"
d == True
```

Задание 5. Найдите ошибку

Почему эта программа не работает?

```
temperature = input("Температура: ")

print(temperature + 5)
```

Исправьте её так, чтобы пользователь мог вводить дробную температуру.

Ответ

`input()` возвращает строку.
Поэтому Python пытается выполнить операцию примерно такого вида:

```
"20.5" + 5
```

Нужно преобразовать ввод в `float`:

```
temperature = float(input("Температура: "))

print(temperature + 5)
```

Задание 6. Задача со звёздочкой

Что произойдёт при выполнении программы?

```
value = "12.8"

number = int(value)

print(number)
```

Сначала попробуйте ответить без запуска.

Ответ

Программа завершится с `ValueError`.
Строку:

```
"12.8"
```

нельзя напрямую преобразовать через:

```
int("12.8")
```

потому что она содержит запись дробного числа.
Можно сначала получить `float`:

```
value = "12.8"

number = int(float(value))

print(number)
```

Результат:

```
12
```

При преобразовании `12.8` из `float` в `int` дробная часть отбрасывается.

2 3.5 Арифметические операции

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как Python выполняет вычисления с числами?**

В предыдущих главах мы научились хранить числа в переменных:

```
price = 120
quantity = 3
```

получать их от пользователя:

```
age = input("Сколько вам лет? ")
```

и превращать строки в числа:

```
age = int(age)
```

Теперь можно использовать эти значения для вычислений.

Например:

```
price = 120
quantity = 3

total = price * quantity

print(total)
```

Результат:

```
360
```

Здесь Python выполнил **арифметическую операцию** — умножение.

Но умножение — только одна из доступных операций.

Python умеет складывать, вычитать, умножать, делить, находить остаток от деления, возводить числа в степень и выполнять другие вычисления.

Основные арифметические операторы

Для вычислений Python использует специальные символы — **арифметические операторы**.

Основные из них:

Оператор	Операция	Пример	Результат
`+`	сложение	`10 + 3`	`13`
`-`	вычитание	`10 - 3`	`7`
`*`	умножение	`10 * 3`	`30`
`/`	обычное деление	`10 / 3`	`3.333...`
`//`	целочисленное деление	`10 // 3`	`3`
`%`	остаток от деления	`10 % 3`	`1`
`**`	возведение в степень	`10 ** 3`	`1000`

Начнём с самых привычных операций.



Рисунок 2.1: Карта основных арифметических операторов

Сложение

Оператор + складывает числа:

```
print(10 + 5)
```

Результат:

```
15
```

Можно складывать значения переменных:

```
apples = 5
oranges = 3

fruits = apples + oranges

print(fruits)
```

Результат:

```
8
```

Или сохранить результат вычисления в новую переменную:

```
salary = 80000
bonus = 15000

total = salary + bonus

print(total)
```

Результат:

```
95000
```

Общая схема:

```
значение + значение
         ↓
     результат
```

Вычитание

Оператор - выполняет вычитание:

```
print(10 - 3)
```

Результат:

```
7
```

Например, можно вычислить оставшуюся сумму:

```
money = 1000
spent = 350

remaining = money - spent

print(remaining)
```

Результат:

```
650
```

Умножение

Для умножения используется оператор *:

```
print(6 * 4)
```

Результат:

```
24
```

Например, стоимость нескольких одинаковых товаров:

```
price = 150
quantity = 4

total = price * quantity

print(total)
```

Результат:

```
600
```

Теперь мы можем соединить это с пользовательским вводом:

```
price = float(input("Цена товара: "))
quantity = int(input("Количество: "))

total = price * quantity

print("Итого:", total)
```

Пример работы:

```
Цена товара: 49.5
Количество: 3
Итого: 148.5
```

Здесь используются сразу несколько уже знакомых элементов:

```
input()
   ↓
преобразование типов
   ↓
числа
   ↓
умножение
   ↓
результат
```

Обычное деление

Для деления используется оператор `/`:

```
print(10 / 2)
```

Результат:

```
5.0
```

Обратите внимание: результат имеет дробный тип `float`.

Проверим:

```
result = 10 / 2

print(result)
print(type(result))
```

Результат:

```
5.0
<class 'float'>
```

Даже если числа делятся без остатка, оператор `/` возвращает `float`.

Например:

```
print(8 / 4)
print(20 / 5)
```

Результат:

```
2.0
4.0
```

> **Важно**
>
> Оператор `/` выполняет обычное деление и возвращает результат типа `float`.
>
> ```
> 10 / 2
> ```
>
> даёт:
>
> ```
> 5.0
> ```
>
> а не:
>
> ```
> 5
> ```

Деление может давать дробный результат

Если одно число не делится на другое без остатка:

```
print(10 / 4)
```

получим:

```
2.5
```

Например, можно разделить общий счёт между несколькими людьми:

```
total = 1500
people = 4

per_person = total / people

print(per_person)
```

Результат:

```
375.0
```

Деление на ноль

Попробуем:

```
print(10 / 0)
```

Такое вычисление невозможно.

Python остановит программу и сообщит об ошибке:

```
ZeroDivisionError
```

Проверьте это в playground: ошибка сделана намеренно. Замените `0` на другое число и запустите код снова.

```
number = 10

# Ошибка сделана намеренно для учебного примера.
result = number / 0

print(result)
```

> ⚠ На ноль делить нельзя
>
> Это относится не только к `/`.
> Следующие выражения также приводят к `ZeroDivisionError`:

```
10 / 0
10 // 0
10 % 0
```

Позже мы научимся проверять данные до выполнения подобных операций.

Целочисленное деление

Кроме обычного `/`, в Python существует оператор:

```
//
```

Он выполняет **целочисленное деление**.

Сравним:

```
print(10 / 3)
print(10 // 3)
```

Результат:

```
3.3333333333333335
3
```

При обычном делении:

```
10 / 3 → 3.333...
```

При целочисленном:

```
10 // 3 → 3
```

Можно представить это так:

```
10 = 3 × 3 + 1
     └──┘

   целая часть
```

Оператор `//` показывает, сколько **полных раз** делитель помещается в делимом.

Например:

```
print(17 // 5)
```

Результат:

```
3
```

Потому что в 17 помещаются три полные пятёрки:

```
5 + 5 + 5 = 15
```

и ещё остаётся 2.

⚠ Не путайте // с int()

Для положительных чисел результаты иногда выглядят одинаково:

```
int(10 / 3)   # 3
10 // 3       # 3
```

Но это разные операции.
Особенно это заметно на отрицательных числах:

```
print(int(-10 / 3))
print(-10 // 3)
```

Результат:

```
-3
-4
```

`int()` отбрасывает дробную часть в направлении нуля.
Оператор `//` выполняет деление с округлением вниз.

Остаток от деления

Оператор % возвращает **остаток от деления**.

Например:

```
print(10 % 3)
```

Результат:

```
1
```

Почему?

Число `10` можно представить так:

```
10 = 3 × 3 + 1
             ↑
          остаток
```

Ещё несколько примеров:

```
print(15 % 4)
print(20 % 6)
print(8 % 2)
```

Результат:

```
3
2
0
```

Если число делится без остатка, `%` возвращает `0`.

Сравните `/`, `//` и `%` на одних и тех же числах:

```
a = 17
b = 5

print(a / b)
print(a // b)
print(a % b)

result = 10 / 2

print(result)
print(type(result))
```



Рисунок 2.2: Три способа деления

Зачем нужен остаток от деления

Оператор % часто используется, когда нужно узнать, делится ли число на другое число без остатка.

Например:

```
print(10 % 2)
```

Результат:

```
0
```

А:

```
print(11 % 2)
```

даёт:

```
1
```

Получается:

```
10 % 2 → 0
11 % 2 → 1
```

Позже это позволит проверять, является ли число чётным.

Но % полезен не только для этого.

Например, переведём минуты в часы и оставшиеся минуты.

Пусть есть:

```
minutes = 135
```

Полные часы:

```
hours = minutes // 60
```

Оставшиеся минуты:

```
remaining_minutes = minutes % 60
```

Полная программа:

```
minutes = 135

hours = minutes // 60
remaining_minutes = minutes % 60

print("Часы:", hours)
print("Минуты:", remaining_minutes)
```

Результат:

```
Часы: 2
Минуты: 15
```

```
minutes = 135

hours = minutes // 60
remaining_minutes = minutes % 60

print("Часы:", hours)
print("Минуты:", remaining_minutes)
```

Здесь // и % удобно работают вместе:

```
135 минут
    ↓
135 // 60 → 2 часа
135 % 60  → 15 минут
```



Рисунок 2.3: Перевод минут в часы и минуты

Возведение в степень

Оператор ** используется для возведения числа в степень.

Например:

```
print(2 ** 3)
```

Результат:

```
8
```

Потому что:

```
2³ = 2 × 2 × 2 = 8
```

Ещё примеры:

```
print(5 ** 2)
print(10 ** 3)
```

Результат:

```
25
1000
```

Квадрат числа можно вычислить так:

```
number = 7

square = number ** 2

print(square)
```

Результат:

```
49
```

Несколько операций в одном выражении

Python позволяет использовать несколько арифметических операций сразу:

```
result = 2 + 3 * 4

print(result)
```

Какой результат получится?

Можно предположить:

```
20
```

если сначала выполнить:

```
2 + 3
```

а затем умножить на 4.

Но Python выведет:

```
14
```

Почему?

Потому что операции имеют **приоритет**.

Сначала выполняется умножение:

```
3 * 4 → 12
```

затем сложение:

```
2 + 12 → 14
```

Приоритет операций

В выражениях Python не всегда выполняет операции просто слева направо.

У разных операций есть разный приоритет.

Для операций этой главы удобно помнить следующий порядок:

```
()
↓
**
↓
*, /, //, %
↓
+, -
```

Сначала выполняются выражения в скобках.

Затем — возведение в степень.

После этого:

```
*
/
//
%
```

И только потом:

```
+
-
```

Например:

```
print(2 + 3 * 4)
```

Результат:

```
14
```

А здесь:

```
print((2 + 3) * 4)
```

результат уже другой:

```
20
```

Скобки заставили Python сначала выполнить:

```
2 + 3
```

Попробуйте изменить выражения и посмотреть, как меняется результат:

```
print(2 + 3 * 4)
print((2 + 3) * 4)

print(2 ** 3 ** 2)
print((2 ** 3) ** 2)
```



Рисунок 2.4: Приоритет арифметических операций

Скобки делают вычисления понятнее

Скобки полезны не только для изменения порядка операций.

Они помогают человеку быстрее понять выражение.

Например:

```
total = price * quantity + delivery
```

можно написать:

```
total = (price * quantity) + delivery
```

Результат будет тем же.

Но второй вариант явно показывает:

1. сначала вычисляется стоимость товаров;
2. затем добавляется доставка.

> **Используйте скобки для ясности**
>
> Не обязательно запоминать все правила приоритета наизусть.
> Если порядок вычислений может быть непонятен, используйте скобки:
>
> ```
> average = (first + second + third) / 3
> ```
>
> Так код легче читать и проверять.

Операции одинакового приоритета

Большинство операций одного уровня выполняются слева направо.

Например:

```
print(20 / 5 * 2)
```

Сначала:

```
20 / 5 → 4.0
```

затем:

```
4.0 * 2 → 8.0
```

Результат:

```
8.0
```

Ещё пример:

```
print(20 - 5 + 2)
```

Получаем:

```
17
```

То есть:

```
20 - 5 → 15
15 + 2 → 17
```

Особенность оператора **

Возведение в степень имеет важную особенность.

Рассмотрим:

```
print(2 ** 3 ** 2)
```

Python интерпретирует это как:

```
2 ** (3 ** 2)
```

Сначала:

```
3 ** 2 → 9
```

затем:

```
2 ** 9 → 512
```

Результат:

```
512
```

> **Для сложных выражений лучше использовать скобки**
>
> Даже если вы знаете правила приоритета, явная запись часто понятнее:
>
> ```
> 2 ** (3 ** 2)
> ```
>
> или:
>
> ```
> (2 ** 3) ** 2
> ```
>
> Эти выражения дают разные результаты, и скобки сразу показывают намерение программиста.

Вычисления с int и float

В одном выражении могут участвовать `int` и `float`.

Например:

```
result = 10 + 2.5

print(result)
print(type(result))
```

Результат:

```
12.5
<class 'float'>
```

Ещё пример:

```
price = 12.5
quantity = 4

total = price * quantity

print(total)
print(type(total))
```

Результат:

```
50.0
<class 'float'>
```

Если в вычислении участвует дробное число, результат часто также становится `float`.

Вычисления с пользовательским вводом

Теперь объединим несколько изученных тем.

Напишем простой калькулятор стоимости покупки:

```
price = float(input("Цена товара: "))
quantity = int(input("Количество: "))
delivery = float(input("Стоимость доставки: "))

total = price * quantity + delivery

print("Итого:", total)
```

Пример работы:

```
Цена товара: 250
Количество: 3
Стоимость доставки: 150
Итого: 900.0
```

В этой небольшой программе уже есть:

```
input()
    ↓
преобразование типов
    ↓
переменные
    ↓
арифметические операции
    ↓
результат
```

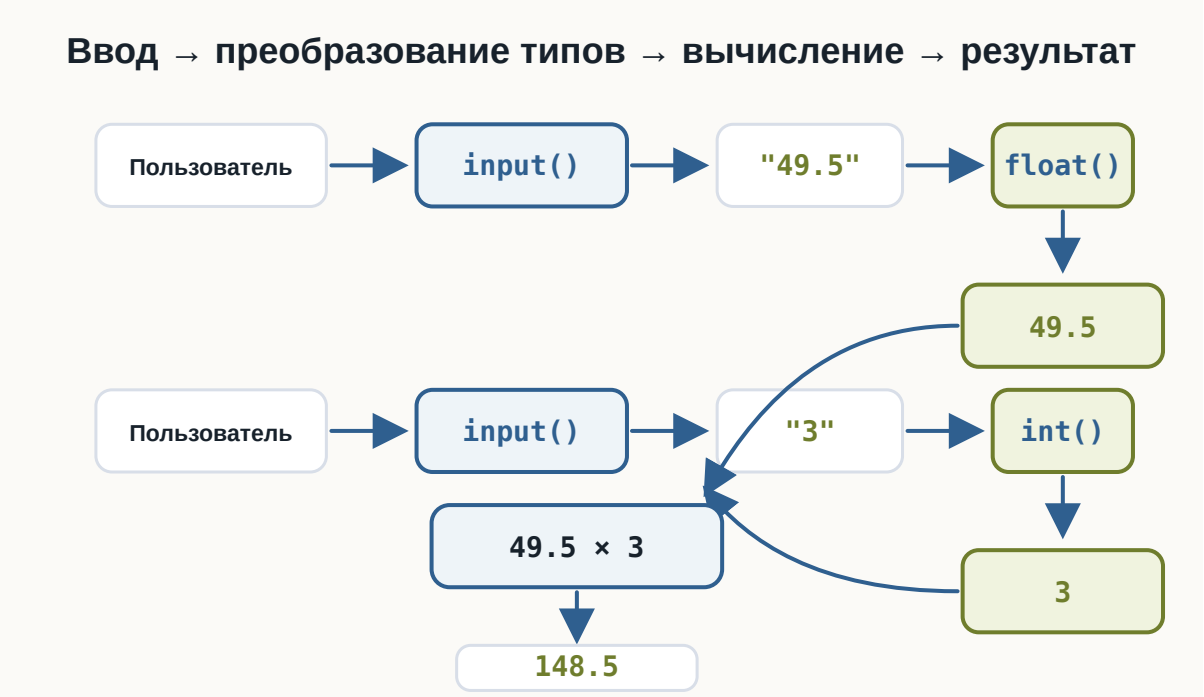


Рисунок 2.5: Связь input, преобразования типов и арифметики

Мы постепенно начинаем соединять отдельные конструкции Python в полноценные программы.

Попробуйте сами

Эксперимент

Создайте программу:

```
a = 17
b = 5

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(a ** b)
```

Перед запуском попробуйте самостоятельно предсказать каждый результат. Затем измените:

```
a = 17
b = 5
```

на другие числа и посмотрите, как изменятся результаты.
Особенно сравните:

```
a / b
a // b
a % b
```

Что важно запомнить

Python поддерживает основные арифметические операции:

```
+   сложение
-   вычитание
*   умножение
/   обычное деление
//  целочисленное деление
%   остаток от деления
**  возведение в степень
```

Особенно важно различать три операции:

```
10 / 3  → 3.333...
10 // 3 → 3
10 % 3  → 1
```

Они отвечают на разные вопросы:

```
/   → какой результат деления?

//  → сколько полных частей помещается?

%   → что осталось?
```

При нескольких операциях Python учитывает их приоритет:

```
()
↓
**
↓
*, /, //, %
↓
+, -
```

Скобки позволяют явно задать нужный порядок:

```
(2 + 3) * 4
```

И самое важное:

> **Арифметическое выражение — это способ получить новое значение из уже известных числовых значений.**

Практические задания

Задание 1. Четыре операции

Пользователь вводит два числа.

Выведите:

- их сумму;
- разность;
- произведение;
- результат деления.

Пример:

```
Первое число: 10
Второе число: 4

Сумма: 14.0
Разность: 6.0
Произведение: 40.0
Деление: 2.5
```

Ответ

```
a = float(input("Первое число: "))
b = float(input("Второе число: "))

print("Сумма:", a + b)
print("Разность:", a - b)
print("Произведение:", a * b)
print("Деление:", a / b)
```

Задание 2. Площадь прямоугольника

Пользователь вводит длину и ширину прямоугольника.

Вычислите его площадь.

Пример:

```
Длина: 5
Ширина: 3
Площадь: 15.0
```

Ответ

```
length = float(input("Длина: "))
width = float(input("Ширина: "))

area = length * width

print("Площадь:", area)
```

Задание 3. Среднее значение

Пользователь вводит три числа.

Вычислите их среднее арифметическое.

Пример:

```
Первое число: 10
Второе число: 20
Третье число: 30
Среднее: 20.0
```

Ответ

```
a = float(input("Первое число: "))
b = float(input("Второе число: "))
c = float(input("Третье число: "))

average = (a + b + c) / 3

print("Среднее:", average)
```

Скобки здесь важны:

```
(a + b + c) / 3
```

Сначала складываются три числа, затем результат делится на 3.

Задание 4. Часы и минуты

Пользователь вводит количество минут.

Переведите его в часы и оставшиеся минуты.

Пример:

```
Количество минут: 135
Часы: 2
Минуты: 15
```

Ответ

```
minutes = int(input("Количество минут: "))

hours = minutes // 60
remaining_minutes = minutes % 60

print("Часы:", hours)
print("Минуты:", remaining_minutes)
```

`//` находит количество полных часов, а `%` — оставшиеся минуты.

Задание 5. Предскажите результат

Не запускайте код сразу.

Определите результаты:

```
print(2 + 3 * 4)
print((2 + 3) * 4)
print(17 // 5)
print(17 % 5)
print(2 ** 4)
```

Ответ

Результат:

```
14
20
3
2
16
```

В первом выражении умножение выполняется раньше сложения:

```
2 + 3 * 4
    ↓
2 + 12
    ↓
14
```

Во втором порядок изменяют скобки:

```
(2 + 3) * 4
    ↓
5 * 4
  ↓
20
```

Задание 6. Найдите проблему

Что произойдёт здесь?

```
number = 10

result = number / 0
```

```
print(result)
```

Почему?

> **Ответ**
>
> Программа завершится с ошибкой:
>
> ```
> ZeroDivisionError
> ```
>
> На ноль делить нельзя.
> Ошибка возникает при выполнении:
>
> ```
> 10 / 0
> ```

Задание 7. Задача со звёздочкой

Пользователь вводит количество секунд.

Переведите его в:

- полные минуты;
- оставшиеся секунды.

Например:

```
Количество секунд: 145
Минуты: 2
Секунды: 25
```

Попробуйте решить задачу с помощью `//` и `%`.

> **Ответ**
>
> ```
> seconds = int(input("Количество секунд: "))
>
> minutes = seconds // 60
> remaining_seconds = seconds % 60
>
> print("Минуты:", minutes)
> print("Секунды:", remaining_seconds)
> ```

> Для 145:
>
> ```
> 145 // 60 → 2
> 145 % 60  → 25
> ```
>
> Поэтому:
>
> ```
> 145 секунд = 2 минуты 25 секунд
> ```

Что дальше?

Теперь программа умеет:

- хранить данные в переменных;
- различать типы данных;
- получать данные через `input()`;
- преобразовывать строки в числа;
- выполнять вычисления.

Например:

```
price = float(input("Цена: "))
quantity = int(input("Количество: "))

total = price * quantity

print("Итого:", total)
```

Но пока программа выполняет одни и те же действия независимо от результата вычислений.

Что если нужно сделать так:

```
если пользователь совершеннолетний → показать одно сообщение
иначе → другое
```

Или:

```
если пароль правильный → продолжить
иначе → сообщить об ошибке
```

Для этого программе нужно научиться **сравнивать значения и принимать решения**.

Следующий главный вопрос:

Как Python сравнивает значения?

Этому посвящена следующая глава:

«Сравнение значений».

2 3.6 Сравнение значений

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как программа сравнивает значения и получает ответ `True` или `False`?**

В предыдущей главе мы научились выполнять арифметические операции:

```
price = 120
quantity = 3

total = price * quantity
```

Теперь программа умеет получать числа, преобразовывать их и выполнять вычисления.

Но часто вычислить значение недостаточно.

Программе нужно уметь отвечать на вопросы:

```
Возраст больше 18?
Цена меньше 1000?
Пароли одинаковые?
Количество не равно нулю?
```

Для этого используются **операторы сравнения**.

Результат любого сравнения — логическое значение:

```
True
```

или:

```
False
```

Первое сравнение

Рассмотрим:

```
print(10 > 5)
```

Результат:

```
True
```

Python проверил:

```
10 больше 5?
```

Ответ:

```
да → True
```

Теперь изменим выражение:

```
print(10 < 5)
```

Результат:

```
False
```

То есть сравнение можно представить так:

```
значение
   ↓
сравнение
   ↓
True или False
```

Операторы сравнения

В Python используются следующие основные операторы:

Оператор	Значение	Пример	Результат
`==`	равно	`5 == 5`	`True`
`!=`	не равно	`5 != 3`	`True`
`>`	больше	`10 > 5`	`True`
`<`	меньше	`10 < 5`	`False`
`>=`	больше или равно	`10 >= 10`	`True`
`<=`	меньше или равно	`5 <= 10`	`True`

Главная идея:

```
сравнение
   ↓
bool
   ↓
True / False
```

Попробуйте изменить значения `a` и `b` и посмотреть, как меняются ответы:

```
a = 10
b = 5

print(a == b)
print(a != b)
print(a > b)
print(a < b)
print(a >= b)
print(a <= b)
```

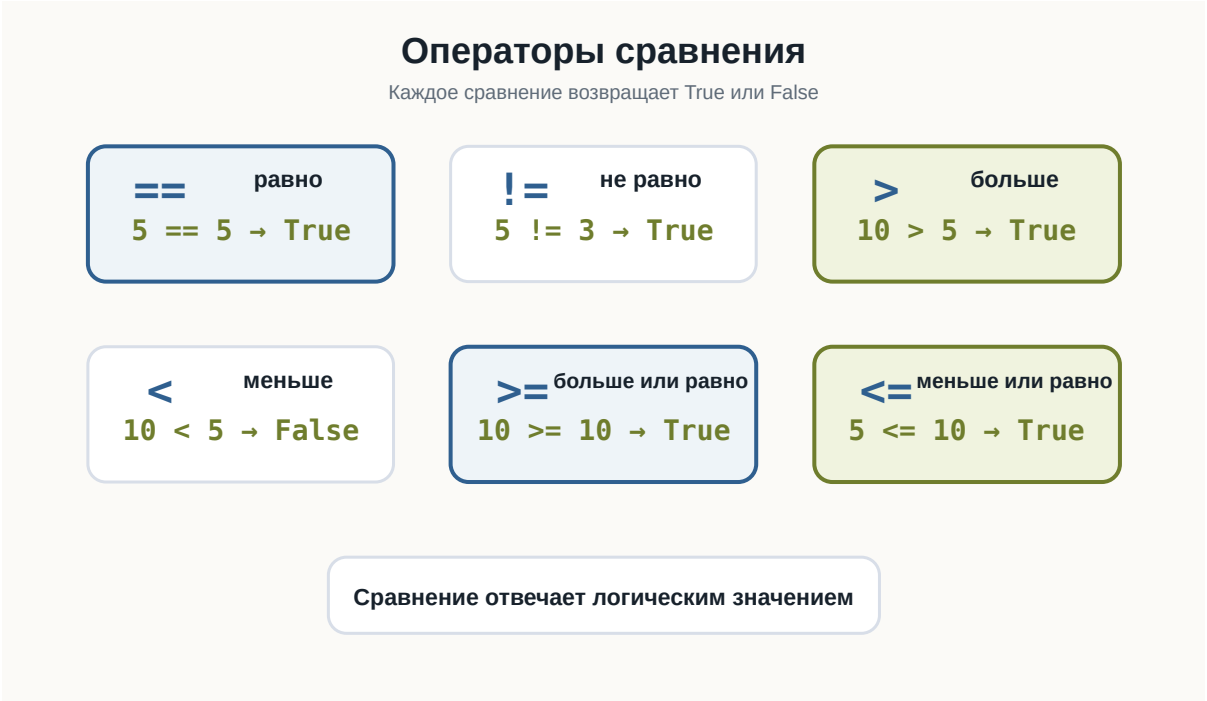


Рисунок 2.1: Карта операторов сравнения

Равенство

Для проверки равенства используется оператор:

```
==
```

Например:

```
print(5 == 5)
```

Результат:

```
True
```

А:

```
print(5 == 7)
```

даёт:

```
False
```

Можно сравнивать значения переменных:

```
first = 10
second = 10

print(first == second)
```

Результат:

```
True
```

= и == — разные вещи

Это одна из самых важных деталей.

Один знак:

```
=
```

используется для присваивания:

```
age = 25
```

Два знака:

```
==
```

используются для сравнения:

```
age == 25
```

То есть:

```
=   → сохранить значение

==  → сравнить значения
```

⚠ Не путайте = и ==

Это разные операции.

```
age = 25
```

связывает имя age со значением 25.
А:

```
age == 25
```

проверяет, равно ли значение age числу 25.
Результатом второго выражения будет:

```
True
```

или:

```
False
```

В этом примере видно, что присваивание сохраняет значение, а сравнение задаёт вопрос:

```
# = присваивает значение переменной
age = 25

print(age)

# == сравнивает значения
print(age == 25)
print(age == 18)
```

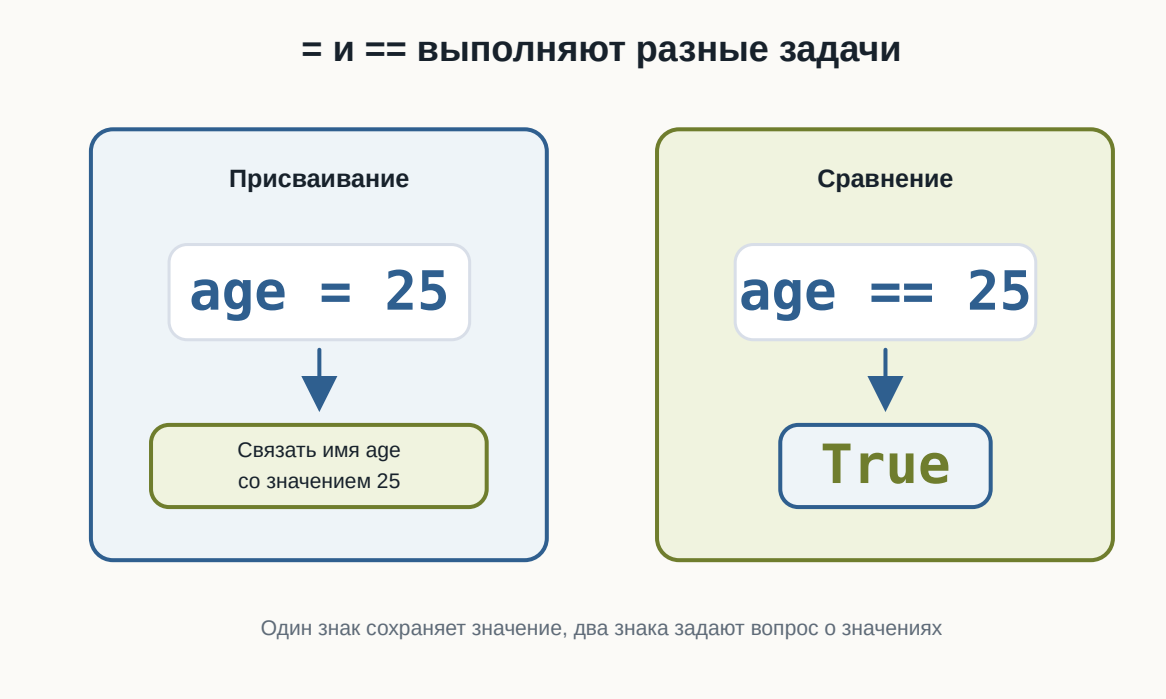


Рисунок 2.2: Разница между присваиванием и сравнением

Неравенство

Для проверки того, что значения **не равны**, используется:

```
!=
```

Например:

```
print(10 != 5)
```

Результат:

```
True
```

Потому что `10` действительно не равно `5`.

Но:

```
print(10 != 10)
```

даёт:

```
False
```

Больше и меньше

Для сравнения чисел используются:

```
>
<
```

Например:

```
print(10 > 3)
print(10 < 3)
```

Результат:

```
True
False
```

Можно читать буквально:

```
10 > 3
```

как:

> 10 больше 3?

А:

```
10 < 3
```

как:

> 10 меньше 3?

Больше или равно

Оператор:

```
>=
```

проверяет сразу два варианта:

```
больше
или
равно
```

Например:

```
print(10 >= 5)
print(10 >= 10)
print(10 >= 15)
```

Результат:

```
True
True
False
```

То есть выражение:

```
10 >= 10
```

возвращает `True`, потому что значения равны.

Меньше или равно

Оператор:

```
<=
```

работает похожим образом.

Например:

```
print(5 <= 10)
print(5 <= 5)
print(5 <= 3)
```

Результат:

```
True
True
False
```

Результат сравнения имеет тип bool

Сравнение не просто выводит слова `True` и `False`.

Оно действительно создаёт значение типа `bool`.

Проверим:

```
result = 10 > 5

print(result)
print(type(result))
```

Результат:

```
True
<class 'bool'>
```

Получается:

```
10 > 5
  ↓
True
  ↓
bool
```

Это важно, потому что позже программа сможет использовать такой результат для принятия решений.

Проверьте тип результата сравнения:

```
result = 10 > 5

print(result)
print(type(result))

print(type(10 == 10))
print(type(5 != 3))
```

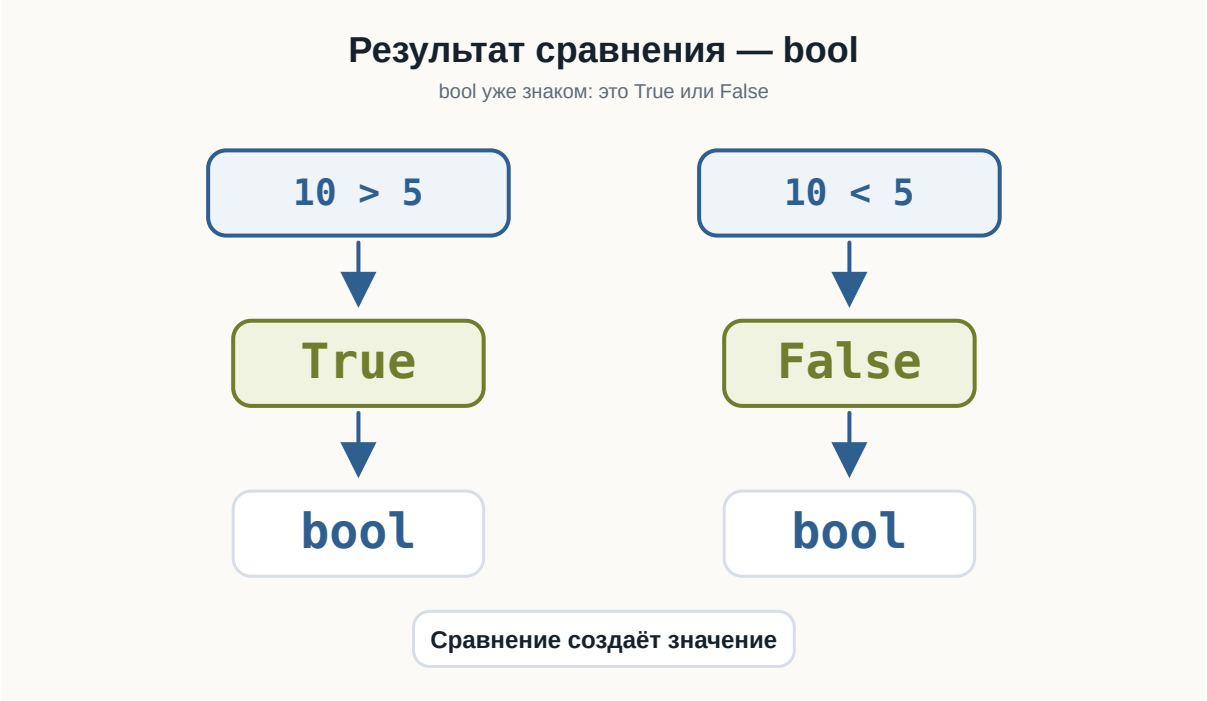


Рисунок 2.3: Результат сравнения имеет тип bool

Сравнение переменных

Сравнивать можно не только числа, записанные прямо в коде.

Например:

```
age = 25
limit = 18

print(age > limit)
```

Результат:

```
True
```

Или:

```
price = 1200
budget = 1000

print(price <= budget)
```

Результат:

```
False
```

Такие выражения уже похожи на реальные вопросы программы:

```
Возраст подходит?
Цена помещается в бюджет?
Количество достигло лимита?
```

Сравнение после input()

Теперь объединим тему с пользовательским вводом.

Например:

```
age = int(input("Введите возраст: "))

print(age >= 18)
```

Если пользователь введёт:

```
20
```

результат:

```
True
```

Если:

```
15
```

результат:

```
False
```

Здесь происходит знакомая цепочка:

```
пользователь вводит "20"
        ↓
      input()
        ↓
      "20"
        ↓
      int()
        ↓
       20
        ↓
    20 >= 18
        ↓
      True
```

Запустите пример несколько раз и введите 15, 18 и 20:

```
age = int(input("Введите возраст: "))

print(age >= 18)
```

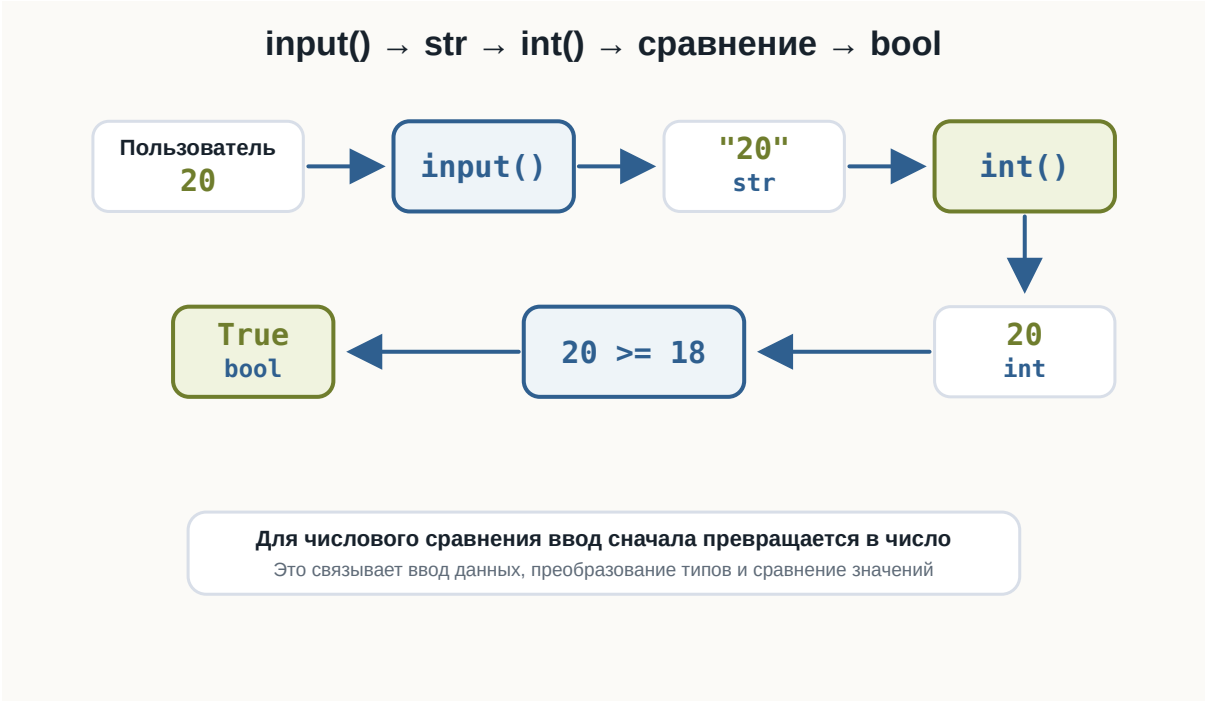


Рисунок 2.4: Путь от input к числовому сравнению

Сравнение строк

Сравнивать можно не только числа.

Например:

```
print("Python" == "Python")
```

Результат:

```
True
```

А:

```
print("Python" == "python")
```

даёт:

```
False
```

Для Python это разные строки.

Почему?

Потому что отличается регистр букв:

```
Python
python
```

Регистр имеет значение

Рассмотрим:

```
password = "Python123"

print(password == "Python123")
print(password == "python123")
```

Результат:

```
True
False
```

> **Строки сравниваются точно**
>
> При сравнении строк учитываются символы и их регистр.
>
> ```
> "Python" == "python"
> ```
>
> даёт:
>
> ```
> False
> ```
>
> То же относится к пробелам:
>
> ```
> "Python" == "Python "
> ```
>
> тоже:
>
> ```
> False
> ```

Проверьте сравнение строк с разным регистром:

```
print("Python" == "Python")
print("Python" == "python")
print("Python" != "Java")

password = "Python123"

print(password == "Python123")
print(password == "python123")
```

Сравнение строк с помощью > и <

Python позволяет использовать с некоторыми строками и операторы:

```
>
<
```

Например:

```
print("apple" < "banana")
```

Результат:

```
True
```

Строки сравниваются последовательно по символам.

Но на этом этапе курса не нужно запоминать внутренние правила такого сравнения.

Для начала достаточно уверенно использовать со строками:

```
==
!=
```

Именно они чаще всего нужны для проверки пользовательских значений.

Сравнение разных числовых типов

`int` и `float` можно сравнивать между собой.

Например:

```
print(10 == 10.0)
```

Результат:

```
True
```

Хотя типы разные:

```
print(type(10))
print(type(10.0))
```

Результат:

```
<class 'int'>
<class 'float'>
```

Python сравнивает числовые значения:

```
10
10.0
```

Они представляют одно и то же число.

Значение и тип — не одно и то же

Рассмотрим:

```
10 == 10.0
```

Результат:

```
True
```

Но:

```
type(10) == type(10.0)
```

Результат:

```
False
```

Почему?

Потому что:

```
10    → int
10.0  → float
```

Значения равны по числовому смыслу, но типы отличаются.

> Сравнение отвечает на конкретный вопрос
>
> Выражение:
>
> ```
> 10 == 10.0
> ```
>
> спрашивает:
>
> равны ли значения?
>
> А:
>
> ```
> type(10) == type(10.0)
> ```
>
> спрашивает:
>
> одинаковы ли их типы?
>
> Это разные вопросы.

Сначала вычисление, потом сравнение

В сравнении можно использовать арифметические выражения.

Например:

```
print(2 + 3 == 5)
```

Результат:

```
True
```

Сначала Python вычисляет:

```
2 + 3
```

Получается:

```
5
```

Затем выполняет:

```
5 == 5
```

и получает:

```
True
```

Ещё пример:

```
print(10 * 2 > 15)
```

Сначала:

```
10 * 2 → 20
```

Затем:

```
20 > 15
```

Результат:

```
True
```

Приоритет арифметики и сравнения

Арифметические операции выполняются раньше сравнений.

Например:

```
print(5 + 5 > 8)
```

Python сначала вычисляет:

```
5 + 5 → 10
```

а затем:

```
10 > 8
```

Результат:

```
True
```

Можно записать явно:

```
print((5 + 5) > 8)
```

Результат тот же.

Скобки здесь необязательны, но могут сделать выражение понятнее.

Проверьте несколько выражений, где сначала выполняется арифметика:

```
print(2 + 3 == 5)
print(10 * 2 > 15)
print(5 + 5 > 8)
print(20 / 2 == 10)
```

Можно сохранить результат сравнения

Результат сравнения можно сохранить в переменной:

```
age = 20

is_adult = age >= 18

print(is_adult)
```

Результат:

```
True
```

Обратите внимание на имя:

```
is_adult
```

Оно хорошо описывает логическое значение.

Такие имена часто начинаются с:

```
is_
has_
can_
```

Например:

```
is_adult = age >= 18
has_access = password == "Python123"
can_buy = money >= price
```

Пока это просто переменные типа `bool`.

В следующей теме мы начнём использовать их для управления поведением программы.

Несколько полезных примеров

Проверка возраста:

```
age = 21

is_adult = age >= 18

print(is_adult)
```

Проверка бюджета:

```
price = 800
budget = 1000

can_buy = price <= budget

print(can_buy)
```

Проверка пароля:

```
password = "python"

is_correct = password == "python"

print(is_correct)
```

Проверка количества:

```
count = 0

is_empty = count == 0

print(is_empty)
```

Все эти примеры имеют одну общую схему:

```
данные
  ↓
сравнение
  ↓
True / False
```

Попробуйте сами

Эксперимент

Создайте программу:

```
a = 10
b = 5

print(a == b)
print(a != b)
print(a > b)
print(a < b)
print(a >= b)
print(a <= b)
```

Перед запуском попробуйте предсказать каждый результат.
Затем измените:

```
a = 10
b = 5
```

например на:

```
a = 5
b = 5
```

Посмотрите, какие результаты изменятся.

Типичная ошибка: сравнение строки и числа

Вспомним `input()`:

```
age = input("Введите возраст: ")
```

Если пользователь введёт:

```
18
```

переменная будет содержать:

```
"18"
```

То есть `str`.

Поэтому перед числовым сравнением обычно нужно выполнить преобразование:

```
age = int(input("Введите возраст: "))

print(age >= 18)
```

Это корректно.

> ⚠ Сначала нужный тип, потом сравнение
>
> Если данные должны быть числом, сначала преобразуйте результат `input()`:
>
> ```
> age = int(input("Введите возраст: "))
> ```
>
> и только затем сравнивайте:
>
> ```
> age >= 18
> ```
>
> Не забывайте:
>
> ```
> input() → str
> ```

Цепочки сравнений

В Python можно записать несколько сравнений в одном выражении.

Например:

```
age = 25

print(18 <= age <= 65)
```

Результат:

```
True
```

Такое выражение можно прочитать:

```
18 <= age <= 65
```

как:

> age находится между 18 и 65 включительно.

Другой пример:

```
temperature = 20

print(0 <= temperature <= 30)
```

Результат:

```
True
```

> **Цепочки сравнений**
>
> Python позволяет писать:
>
> ```
> 0 <= value <= 100
> ```
>
> Это компактный способ проверить, находится ли значение внутри диапазона. Пока достаточно понимать саму запись. Позже такие выражения будут особенно полезны в условиях.

Измените `age` или `temperature` и посмотрите, когда результат станет `False`:

```
age = 25
temperature = 20

print(18 <= age <= 65)
print(0 <= temperature <= 30)
```

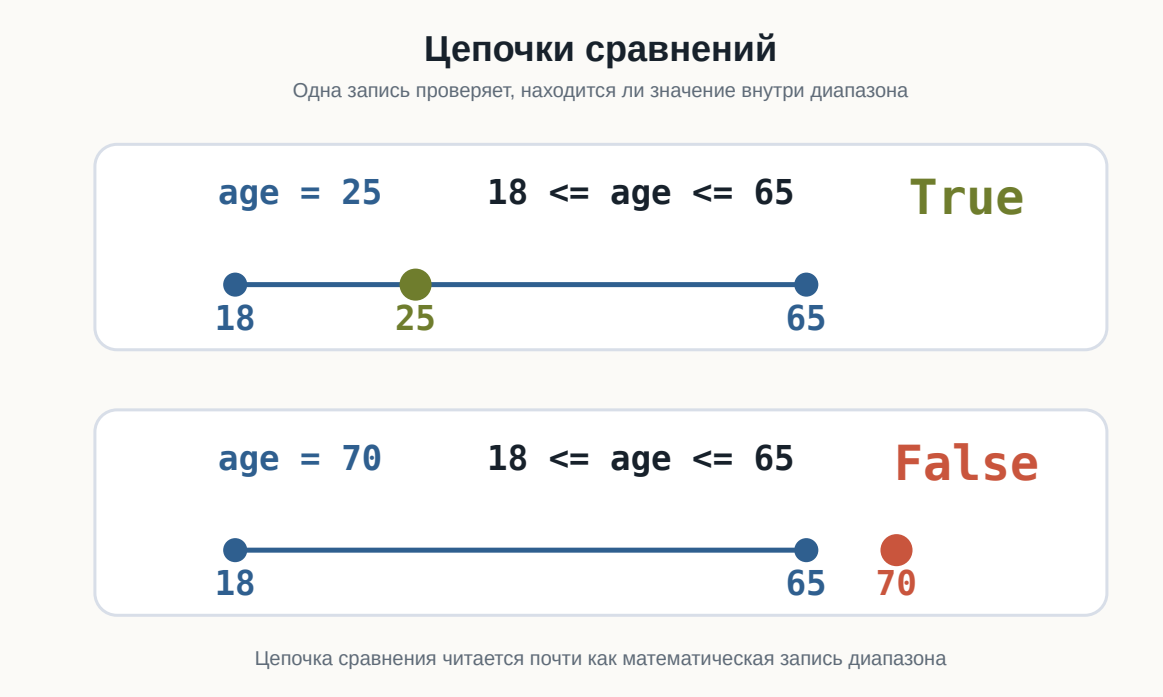


Рисунок 2.5: Цепочка сравнений проверяет диапазон

Что важно запомнить

Основные операторы сравнения:

```
==  равно
!=  не равно
>   больше
<   меньше
>=  больше или равно
<=  меньше или равно
```

Любое сравнение возвращает:

```
True
```

или:

```
False
```

То есть результат имеет тип:

```
bool
```

Важно различать:

```
=   присваивание
==  сравнение
```

Сравнивать можно:

- числа;
- строки;
- значения переменных;
- результаты вычислений.

Например:

```
age >= 18
price <= budget
password == "python"
count != 0
```

И самое важное:

> **Сравнение превращает вопрос о значениях в логический ответ `True` или `False`.**

Практические задания

Задание 1. Больше ли число?

Пользователь вводит число.

Проверьте, больше ли оно `10`.

Пример:

```
Введите число: 15
True
```

Ответ

```
number = float(input("Введите число: "))

print(number > 10)
```

Задание 2. Равны ли числа?

Пользователь вводит два числа.

Выведите результат проверки, равны ли они.

Пример:

```
Первое число: 10
Второе число: 10
True
```

Ответ

```
first = float(input("Первое число: "))
second = float(input("Второе число: "))

print(first == second)
```

Задание 3. Проверка возраста

Пользователь вводит возраст.

Проверьте, достиг ли он 18 лет.

Пример:

```
Возраст: 20
True
```

Ответ

```
age = int(input("Возраст: "))

print(age >= 18)
```

Задание 4. Проверка пароля

Дан правильный пароль:

```
correct_password = "python123"
```

Попросите пользователя ввести пароль и выведите результат сравнения.

Пример:

```
Введите пароль: python123
True
```

Ответ

```
correct_password = "python123"

password = input("Введите пароль: ")

print(password == correct_password)
```

Задание 5. Помещается ли покупка в бюджет?

Пользователь вводит:

- стоимость товара;
- свой бюджет.

Проверьте, хватает ли бюджета на покупку.

Пример:

```
Стоимость товара: 750
Бюджет: 1000
True
```

Ответ

```
price = float(input("Стоимость товара: "))
budget = float(input("Бюджет: "))

print(price <= budget)
```

Задание 6. Предскажите результат

Не запускайте код сразу.

Определите результаты:

```
print(10 == 10)
print(10 != 10)
print(5 > 3)
print(5 <= 5)
print(10 == 10.0)
print("Python" == "python")
```

Ответ

Результаты:

```
True
False
True
True
True
False
```

`10 == 10.0` возвращает `True`, потому что числовые значения равны.
А:

```
"Python" == "python"
```

возвращает `False`, потому что строки отличаются регистром.

Задание 7. Сначала вычисление, потом сравнение

Какой результат будет у каждого выражения?

```
print(2 + 3 == 5)
print(10 * 2 > 25)
print((4 + 6) <= 10)
print(20 / 2 == 10)
```

Ответ

Результат:

```
True
False
True
True
```

Сначала Python выполняет арифметическую операцию, затем сравнивает полученное значение.

Задание 8. Задача со звёздочкой

Пользователь вводит число.

Проверьте, находится ли оно в диапазоне от `0` до `100` включительно.

Пример:

```
Введите число: 75
True
```

Используйте цепочку сравнений.

Ответ

```
number = float(input("Введите число: "))

print(0 <= number <= 100)
```

Например:

```
75 → True
0 → True
100 → True
120 → False
-5 → False
```

Что дальше?

Теперь программа умеет не только вычислять значения, но и задавать вопросы:

```
age >= 18
```

```
password == "python"
```

```
price <= budget
```

Пока Python только получает результат:

```
True
```

или:

```
False
```

Но следующий шаг намного интереснее.

Мы хотим, чтобы программа могла сделать так:

```
если результат True
    выполнить одно действие

если False
    выполнить другое
```

То есть программа должна научиться **принимать решения**.

Для этого в Python используются условные конструкции.

Следующая глава: **Условия**.

2 3.7 Условия

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как сделать так, чтобы программа выполняла разные действия в зависимости от ситуации?**

В предыдущей главе мы научились сравнивать значения:

```
age >= 18
```

```
price <= budget
```

```
password == "python"
```

Каждое такое выражение возвращает:

```
True
```

или:

```
False
```

Но пока программа только получает этот результат.

Например:

```
age = 20

print(age >= 18)
```

Результат:

```
True
```

Гораздо интереснее сделать так:

```
если возраст не меньше 18
    вывести "Доступ разрешён"
```

Именно для этого в Python используются **условия**.

Первая условная конструкция

Начнём с простого примера:

```
age = 20

if age >= 18:
    print("Доступ разрешён")
```

```
age = 20

if age >= 18:
    print("Доступ разрешён")
```

Результат:

```
Доступ разрешён
```

Здесь появилась новая конструкция:

```
if
```

Слово `if` можно перевести как:

> **если**

Поэтому код:

```
if age >= 18:
    print("Доступ разрешён")
```

можно прочитать почти как обычное предложение:

> Если age больше или равно 18, вывести «Доступ разрешён».

Как работает if

Рассмотрим:

```
age = 20

if age >= 18:
    print("Доступ разрешён")
```

Python сначала вычисляет условие:

```
age >= 18
```

То есть:

```
20 >= 18
```

Результат:

```
True
```

Так как условие истинно, Python выполняет:

```
print("Доступ разрешён")
```

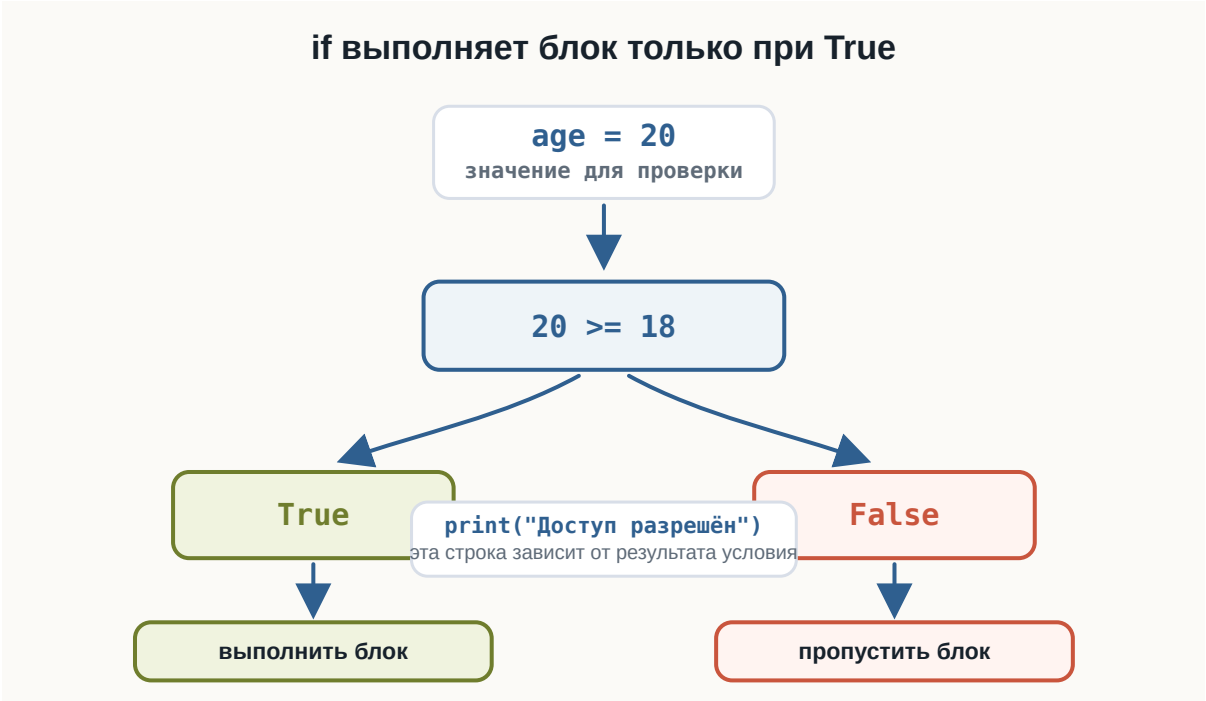


Рисунок 2.1: Как работает первый if

Что произойдёт при False

Изменим возраст:

```
age = 15

if age >= 18:
    print("Доступ разрешён")
```

Теперь условие:

```
age >= 18
```

превращается в:

```
15 >= 18
```

Результат:

```
False
```

Поэтому строка:

```
print("Доступ разрешён")
```

не выполняется.

Программа просто идёт дальше.

Например:

```
age = 15

print("Проверяем возраст")

if age >= 18:
    print("Доступ разрешён")

print("Проверка завершена")
```

Результат:

```
Проверяем возраст
Проверка завершена
```

Строка внутри `if` была пропущена.

Синтаксис if

Условная конструкция начинается с:

```
if
```

Затем записывается условие:

```
if age >= 18
```

После условия обязательно ставится двоеточие:

```
if age >= 18:
```

Следующая строка записывается с отступом:

```
if age >= 18:
    print("Доступ разрешён")
```

Общая форма:

```
if условие:
    действие
```

> ⚠ Не забывайте двоеточие
>
> После условия ставится:
>
> ```
> :
> ```
>
> Правильно:
>
> ```
> if age >= 18:
>     print("Доступ разрешён")
> ```
>
> Неправильно:
>
> ```
> if age >= 18
>     print("Доступ разрешён")
> ```
>
> Без двоеточия Python сообщит о синтаксической ошибке.

Отступ — часть синтаксиса Python

Посмотрите ещё раз:

```
if age >= 18:
    print("Доступ разрешён")
```

Перед `print()` есть отступ.

Он показывает, что эта команда относится к `if`.

Без отступа:

```
if age >= 18:
print("Доступ разрешён")
```

код записан неправильно.

В Python отступы — не просто оформление.

Они определяют структуру программы.

> ! Отступы имеют значение
>
> Внутри `if` обычно используется отступ в **4 пробела**:
>
> ```
> if condition:
>     print("Эта строка находится внутри if")
> ```
>
> Код без отступа уже находится за пределами этого блока:
>
> ```
> if condition:
>     print("Внутри if")
>
> print("После if")
> ```



Рисунок 2.2: Отступ определяет блок кода

Блок кода

Внутри условия может находиться не одна команда, а несколько.

Например:

```
age = 20

if age >= 18:
    print("Возраст подтверждён")
    print("Доступ разрешён")
    print("Добро пожаловать!")

print("Программа завершена")
```

Если условие `True`, выполняются все три строки с одинаковым отступом.

Можно представить:

```
if условие:
    ├── команда 1
    ├── команда 2
    └── команда 3

следующая команда
```

Несколько строк с одинаковым отступом образуют **блок кода**.

Условие после input()

Теперь объединим `if` с пользовательским вводом.

```
age = int(input("Введите возраст: "))

if age >= 18:
    print("Доступ разрешён")
```

Если пользователь введёт:

```
20
```

получим:

```
Введите возраст: 20
Доступ разрешён
```

Если пользователь введёт:

```
15
```

сообщение не появится.

Здесь соединяются уже знакомые конструкции:

```
пользователь
    ↓
 input()
    ↓
   str
    ↓
  int()
    ↓
сравнение
    ↓
True / False
    ↓
   if
    ↓
действие
```

Теперь результат сравнения действительно начинает влиять на поведение программы.

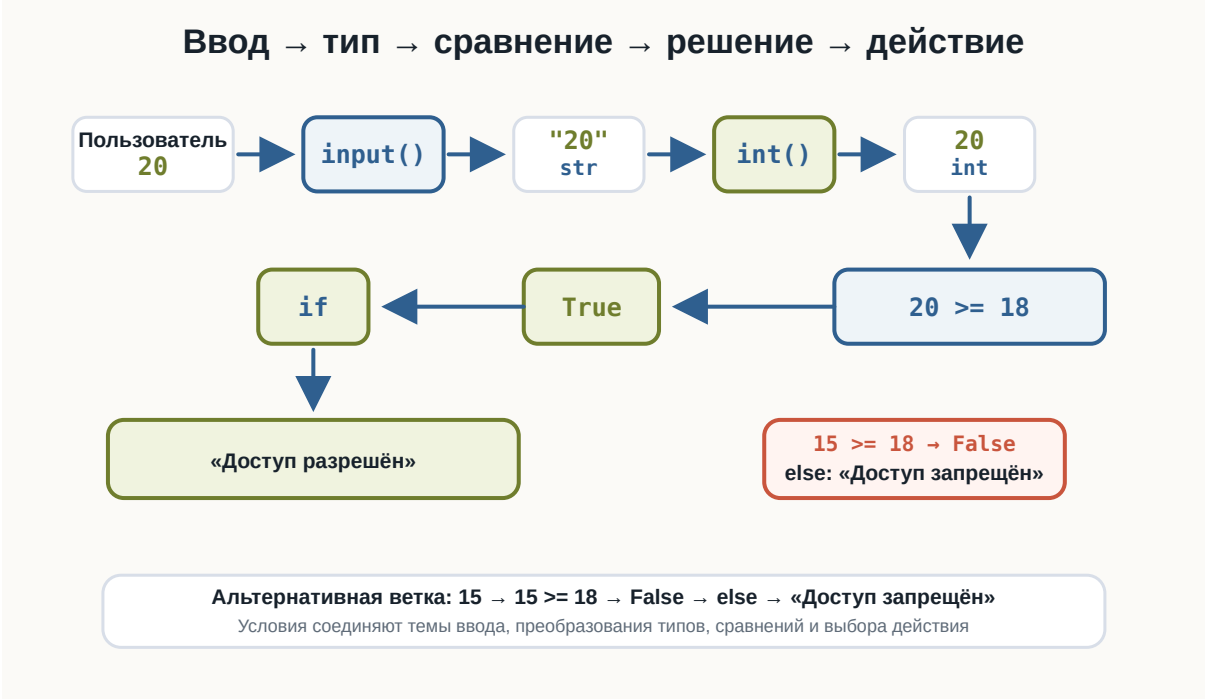


Рисунок 2.3: Ввод, преобразование типа, сравнение и решение

Ветка else

Часто одного `if` недостаточно.

Например, мы хотим:

```
если возраст >= 18
    вывести "Доступ разрешён"

иначе
    вывести "Доступ запрещён"
```

Для второй ветки используется:

```
else
```

Пример:

```
age = int(input("Введите возраст: "))

if age >= 18:
    print("Доступ разрешён")
```

```
else:
    print("Доступ запрещён")
```

```
age = int(input("Введите возраст: "))

if age >= 18:
    print("Доступ разрешён")
else:
    print("Доступ запрещён")
```

Если пользователь введёт:

```
20
```

результат:

```
Доступ разрешён
```

Если:

```
15
```

результат:

```
Доступ запрещён
```

Как работает if...else

Конструкцию можно представить так:

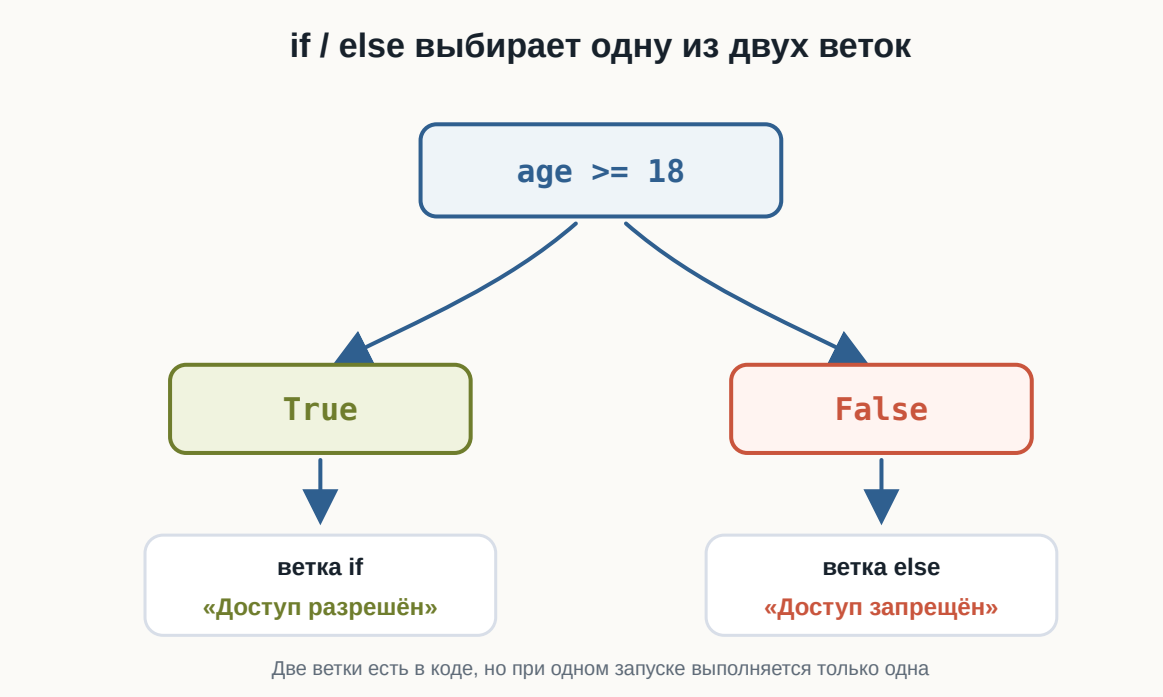


Рисунок 2.4: Развилка if и else

В коде:

```
if condition:
    действие_если_true
else:
    действие_если_false
```

Важно:

> Из двух веток выполняется только одна.

Если условие `True` — выполняется `if`.

Если `False` — выполняется `else`.

Пример с паролем

Рассмотрим простой пример:

```
correct_password = "python123"
password = input("Введите пароль: ")

if password == correct_password:
```

```
    print("Пароль верный")
else:
    print("Пароль неверный")
```

```
correct_password = "python123"
password = input("Введите пароль: ")

if password == correct_password:
    print("Добро пожаловать")
else:
    print("Неверный пароль")
```

Если пользователь введёт:

```
python123
```

сравнение:

```
password == correct_password
```

даст:

```
True
```

и программа выведет:

```
Пароль верный
```

Другой пароль даст `False`, поэтому выполнится ветка `else`.

Пример с бюджетом

Условия могут использовать результаты числовых сравнений.

```
price = float(input("Цена товара: "))
budget = float(input("Ваш бюджет: "))

if budget >= price:
    print("Покупку можно совершить")
else:
    print("Недостаточно денег")
```

Пример:

```
Цена товара: 750
Ваш бюджет: 1000
Покупку можно совершить
```

А при:

```
Цена товара: 1200
Ваш бюджет: 1000
Недостаточно денег
```

В этой же схеме можно не только выбрать сообщение, но и вычислить остаток:

```
price = float(input("Цена товара: "))
money = float(input("Ваши деньги: "))

if money >= price:
    remaining = money - price

    print("Покупка возможна")
    print("Остаток:", remaining)
else:
    print("Недостаточно денег")
```

Несколько вариантов: elif

Иногда двух вариантов недостаточно.

Например, программа оценивает температуру:

```
выше 25     → Жарко
выше 10     → Тепло
иначе       → Холодно
```

Для дополнительных условий используется:

```
elif
```

Пример:

```
temperature = 18

if temperature > 25:
    print("Жарко")
elif temperature > 10:
    print("Тепло")
else:
    print("Холодно")
```

```
temperature = float(input("Введите температуру: "))

if temperature > 25:
    print("Жарко")
elif temperature > 10:
    print("Тепло")
else:
    print("Холодно")
```

Результат:

```
Тепло
```

Как работает elif

Python проверяет условия сверху вниз.

Для программы:

```
temperature = 18

if temperature > 25:
    print("Жарко")
elif temperature > 10:
    print("Тепло")
else:
    print("Холодно")
```

сначала проверяется:

```
temperature > 25
```

То есть:

```
18 > 25 → False
```

Python переходит к следующему условию:

```
temperature > 10
```

Получается:

```
18 > 10 → True
```

Поэтому выполняется:

```
print("Тепло")
```

После этого остальные ветки этой конструкции уже не проверяются.

Общая схема:

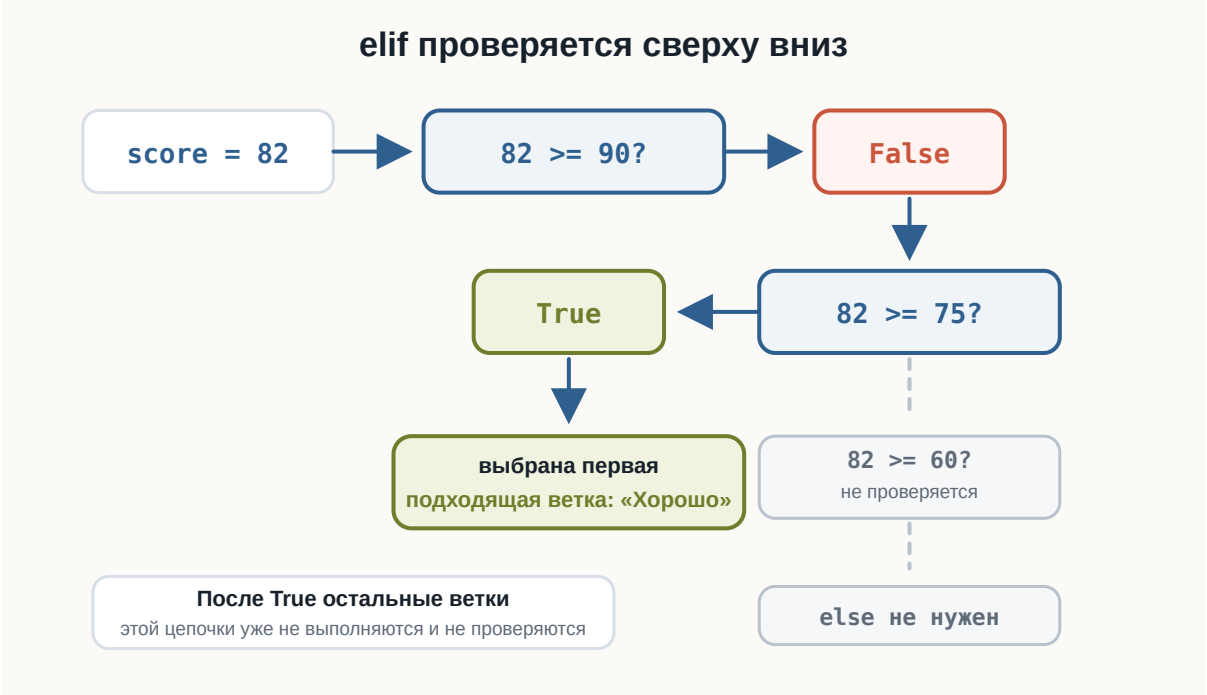


Рисунок 2.5: Проверка elif сверху вниз

Порядок условий имеет значение

Рассмотрим:

```
score = 95

if score >= 60:
    print("Зачёт")
elif score >= 90:
    print("Отличный результат")
```

Какой текст появится?

```
Зачёт
```

Почему не:

```
Отличный результат
```

Потому что первое условие:

```
score >= 60
```

уже равно `True`.

После выполнения этой ветки Python не проверяет следующий `elif`.

Если нам нужны категории, более строгую проверку следует поставить раньше:

```
score = 95

if score >= 90:
    print("Отличный результат")
elif score >= 60:
    print("Зачёт")
else:
    print("Незачёт")
```

Теперь результат:

```
Отличный результат
```

⚠ Проверки идут сверху вниз

В цепочке:

```
if ...
elif ...
elif ...
else ...
```

Python выбирает **первую подходящую ветку**.
Поэтому порядок условий может изменить результат программы.

---

elif может быть несколько

В одной конструкции может быть несколько `elif`.

Например:

```
score = int(input("Введите балл: "))

if score >= 90:
    print("Отлично")
elif score >= 75:
    print("Хорошо")
elif score >= 60:
    print("Зачёт")
else:
    print("Незачёт")
```


```
score = int(input("Введите балл: "))

if score >= 90:
    print("Отлично")
elif score >= 75:
    print("Хорошо")
elif score >= 60:
    print("Зачёт")
else:
    print("Незачёт")
```


Например:

```
Введите балл: 82
Хорошо
```

Python проверяет:

```
82 >= 90 → False
82 >= 75 → True
```

После этого нужная ветка найдена.

---

else не содержит условия

Обратите внимание:

```
else:
```

не содержит никакого сравнения.

Мы не пишем:

```
else age < 18:
```

Правильно:

```
if age >= 18:
    print("Совершеннолетний")
else:
    print("Несовершеннолетний")
```

`else` означает:

> если ни одна предыдущая подходящая проверка не сработала.

---

elif и else не всегда обязательны

Можно использовать только `if`:

```
if temperature < 0:
    print("На улице мороз")
```

Можно использовать:

```
if ...
else ...
```

Например:

```
if age >= 18:
    print("Доступ разрешён")
else:
    print("Доступ запрещён")
```

Или:

```
if ...
elif ...
else ...
```

Например:

```
if score >= 90:
    print("Отлично")
elif score >= 60:
    print("Зачёт")
else:
    print("Незачёт")
```

Выбор зависит от количества вариантов поведения программы.

Условием уже может быть bool

После `if` не обязательно писать сравнение непосредственно.

Можно заранее сохранить результат:

```
age = 20

is_adult = age >= 18

if is_adult:
    print("Доступ разрешён")
```

Переменная:

```
is_adult
```

содержит:

```
True
```

Поэтому Python выполняет блок.

Это связывает текущую тему с типом `bool`, который мы изучали раньше.

```
age >= 18
    ↓
  True
    ↓
is_adult
    ↓
   if
```

Код после условия

Код без отступа выполняется после всей условной конструкции.

Например:

```
age = 15

if age >= 18:
    print("Доступ разрешён")
else:
    print("Доступ запрещён")

print("Проверка завершена")
```

Результат:

```
Доступ запрещён
Проверка завершена
```

Последний `print()` не относится ни к `if`, ни к `else`.

Поэтому он выполняется в любом случае.

Сравните:

```
if condition:
    print("Внутри if")

print("После if")
```

Отступ визуально показывает структуру программы.

Вложенные условия

Внутри одного `if` можно разместить другой `if`.

Например:

```
age = 20
has_ticket = True

if age >= 18:
    print("Возраст подходит")

    if has_ticket:
        print("Билет есть")
        print("Можно войти")
```

Второй `if` выполняется только после того, как программа попала внутрь первого блока.

Структура выглядит так:

```
if первое условие:
    действие

    if второе условие:
        действие
```

Каждый новый уровень вложенности получает дополнительный отступ.

> **Не усложняйте вложенность без необходимости**
>
> Вложенные условия полезны, когда вторая проверка действительно зависит от первой.
> Но большое количество вложенных `if` быстро делает код трудным для чтения.
> Пока используйте вложенность только в простых и очевидных случаях.

Пример: билет в кино

Напишем небольшую интерактивную программу:

```
age = int(input("Введите возраст: "))

if age < 6:
    price = 0
elif age < 18:
    price = 300
else:
    price = 500

print("Стоимость билета:", price)
```


```
age = int(input("Введите возраст: "))

if age < 6:
    price = 0
elif age < 18:
    price = 300
else:
    price = 500

print("Стоимость билета:", price)
```


Пример:

```
Введите возраст: 12
Стоимость билета: 300
```

Здесь условие не только выбирает сообщение.

Оно определяет значение переменной:

```
price
```

Это важный шаг: разные ветки программы могут выполнять разные вычисления.

Пример: положительное, отрицательное или ноль

Условия позволяют классифицировать значение.

```
number = float(input("Введите число: "))

if number > 0:
    print("Положительное число")
elif number < 0:
    print("Отрицательное число")
else:
    print("Ноль")
```


```
number = float(input("Введите число: "))

if number > 0:
    print("Положительное число")
elif number < 0:
    print("Отрицательное число")
else:
    print("Ноль")
```


Для:

```
Введите число: -5
```

результат:

```
Отрицательное число
```

Для:

```
Введите число: 0
```

результат:

```
Ноль
```

Здесь три возможных пути:

```
number > 0
    ↓
положительное

number < 0
    ↓
отрицательное

иначе
    ↓
ноль
```

Попробуйте сами

Эксперимент

Запустите программу с разными значениями `score`:

```python
score = 75

if score >= 90:
    print("Отлично")
elif score >= 75:
    print("Хорошо")
elif score >= 60:
    print("Зачёт")
else:
    print("Незачёт")
```

Попробуйте:

```
100
90
89
75
74
60
59
0
```

Перед каждым запуском сначала предскажите, какая ветка выполнится. Особенно обратите внимание на граничные значения:

```
90
75
60
```

Типичные ошибки

При работе с условиями начинающие часто сталкиваются с несколькими ошибками.

Пропущено двоеточие

Неправильно:

```
if age >= 18
    print("Доступ разрешён")
```

Правильно:

```
if age >= 18:
    print("Доступ разрешён")
```

Нет отступа

Неправильно:

```
if age >= 18:
print("Доступ разрешён")
```

Правильно:

```
if age >= 18:
    print("Доступ разрешён")
```

Перепутаны = и ==

Для сравнения:

```
password == "python"
```

а не:

```
password = "python"
```

Неправильный порядок elif

Если более широкое условие поставить раньше более строгого:

```
if score >= 60:
    print("Зачёт")
elif score >= 90:
    print("Отлично")
```

ветка `score >= 90` никогда не будет выбрана для значения 95.

Правильнее:

```
if score >= 90:
    print("Отлично")
elif score >= 60:
    print("Зачёт")
```

Что важно запомнить

Условные конструкции позволяют программе выбирать, какой код выполнять.

Основная форма:

```
if условие:
    действие
```

Два варианта:

```
if условие:
    действие_1
else:
    действие_2
```

Несколько вариантов:

```
if условие_1:
    действие_1
elif условие_2:
    действие_2
else:
    действие_3
```

Главная цепочка:

```
данные
  ↓
сравнение
  ↓
True / False
  ↓
if
  ↓
выбор действия
```

Важно помнить:

```
if    → если

elif  → иначе если

else  → иначе
```

Также:

- после `if`, `elif` и `else` ставится `:`;

- блоки кода обозначаются отступами;
- Python проверяет ветки сверху вниз;
- в цепочке `if / elif / else` выполняется первая подходящая ветка;
- `else` выполняется, если предыдущие условия не подошли.

И самое важное:

> **Условия превращают программу из последовательности команд в программу, которая может выбирать своё поведение.**

Практические задания

Задание 1. Положительное число

Пользователь вводит число.

Если оно больше `0`, выведите:

```
Число положительное
```

Если число равно `0` или меньше, ничего выводить не нужно.

Ответ

```
number = float(input("Введите число: "))

if number > 0:
    print("Число положительное")
```

Задание 2. Совершеннолетие

Пользователь вводит возраст.

Если возраст не меньше 18, выведите:

```
Доступ разрешён
```

Иначе:

```
Доступ запрещён
```

Ответ

```
age = int(input("Введите возраст: "))

if age >= 18:
    print("Доступ разрешён")
else:
    print("Доступ запрещён")
```

Задание 3. Проверка пароля

Правильный пароль:

```
correct_password = "python123"
```

Попросите пользователя ввести пароль.

Если он совпадает с правильным, выведите:

```
Добро пожаловать
```

Иначе:

```
Неверный пароль
```

Ответ

```
correct_password = "python123"
password = input("Введите пароль: ")

if password == correct_password:
    print("Добро пожаловать")
else:
    print("Неверный пароль")
```

Задание 4. Больше из двух чисел

Пользователь вводит два разных числа.

Выведите большее из них.

Пример:

```
Первое число: 10
Второе число: 25
Большее число: 25.0
```

Ответ

```
first = float(input("Первое число: "))
second = float(input("Второе число: "))

if first > second:
    print("Большее число:", first)
else:
    print("Большее число:", second)
```

Задание 5. Температура

Пользователь вводит температуру.

Выведите:

```
Жарко
```

если температура выше 25.

Выведите:

```
Тепло
```

если температура выше 10, но не выше 25.

В остальных случаях:

```
Холодно
```

Ответ

```
temperature = float(input("Введите температуру: "))

if temperature > 25:
    print("Жарко")
elif temperature > 10:
    print("Тепло")
else:
    print("Холодно")
```

Задание 6. Оценка результата

Пользователь вводит количество баллов от `0` до `100`.

Выведите:

```
Отлично
```

если балл не меньше `90`.

```
Хорошо
```

если балл не меньше `75`.

```
Зачёт
```

если балл не меньше `60`.

В остальных случаях:

```
Незачёт
```

Ответ

```
score = int(input("Введите балл: "))

if score >= 90:
    print("Отлично")
elif score >= 75:
    print("Хорошо")
elif score >= 60:
    print("Зачёт")
else:
    print("Незачёт")
```

Порядок условий здесь важен.
Сначала проверяются самые высокие границы:

```
90
↓
75
↓
60
```

Задание 7. Найдите ошибку

Почему программа работает неправильно для `score = 95`?

```
score = 95

if score >= 60:
    print("Зачёт")
elif score >= 90:
    print("Отлично")
else:
    print("Незачёт")
```

Исправьте её.

Ответ

Python проверяет условия сверху вниз.
Для:

```
score = 95
```

первое условие:

```
score >= 60
```

уже равно `True`.
Поэтому Python выполняет эту ветку и больше не проверяет `elif`.
Нужно поменять условия местами:

```
score = 95

if score >= 90:
    print("Отлично")
elif score >= 60:
    print("Зачёт")
else:
    print("Незачёт")
```

Теперь результат:

```
Отлично
```

Задание 8. Положительное, отрицательное или ноль

Пользователь вводит число.

Программа должна вывести одно из трёх сообщений:

```
Положительное число
Отрицательное число
Ноль
```

Ответ

```
number = float(input("Введите число: "))

if number > 0:
    print("Положительное число")
elif number < 0:
    print("Отрицательное число")
else:
    print("Ноль")
```

Задание 9. Стоимость билета

Пользователь вводит возраст.

Стоимость билета:

```
до 6 лет        → 0
от 6 до 17      → 300
18 и старше     → 500
```

Определите стоимость и выведите её.

Ответ

```
age = int(input("Введите возраст: "))

if age < 6:
    price = 0
elif age < 18:
    price = 300
else:
    price = 500

print("Стоимость билета:", price)
```

Обратите внимание: после того как первое условие:

```
age < 6
```

оказалось `False`, во второй ветке уже известно, что возраст не меньше 6. Поэтому достаточно проверить:

```
age < 18
```

Задание 10. Задача со звёздочкой

Напишите программу проверки покупки.

Пользователь вводит:

- цену товара;
- количество денег.

Если денег больше или столько же, сколько стоит товар:

1. вывести:

```
Покупка возможна
```

2. вычислить остаток денег;
3. вывести остаток.

Иначе вывести:

```
Недостаточно денег
```

Пример:

```
Цена товара: 750
Ваши деньги: 1000
Покупка возможна
Остаток: 250.0
```

Ответ

```
price = float(input("Цена товара: "))
money = float(input("Ваши деньги: "))

if money >= price:
    remaining = money - price

    print("Покупка возможна")
    print("Остаток:", remaining)
else:
    print("Недостаточно денег")
```

Здесь объединяются сразу несколько изученных тем:

```
input()
↓
float()
↓
сравнение
↓
if / else
↓
арифметика
↓
результат
```

Что дальше?

Теперь наши программы уже могут реагировать на данные:

```
age = int(input("Возраст: "))

if age >= 18:
    print("Доступ разрешён")
else:
    print("Доступ запрещён")
```

Но реальные решения часто зависят сразу от нескольких требований.

Например:

```
возраст не меньше 18
И
билет куплен
```

или:

```
пользователь администратор
ИЛИ
пользователь владелец
```

Чтобы объединять несколько условий, понадобятся **логические операторы**.

Они позволят строить более сложные проверки из уже знакомых значений `True` и `False`.

Следующая глава: **Логические операции**.

2 3.8 Логические операции

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как объединять несколько условий в одно выражение?**

В предыдущих главах мы научились получать логический ответ:

```
age >= 18
password == "python123"
total >= 3000
```

Каждое сравнение возвращает `True` или `False`.

Но реальные решения часто зависят не от одного вопроса, а от нескольких сразу:

```
возраст подходит
и
билет есть
```

или:

```
сумма заказа большая
или
есть подписка
```

Для таких случаев в Python используются **логические операторы**.

Три логических оператора

В этой главе нужны три слова:

```
and
or
not
```

Их можно читать почти как обычные русские слова:

```
and -> и
or  -> или
not -> не
```

Они работают с логическими значениями `True` и `False` и помогают получить один итоговый ответ.

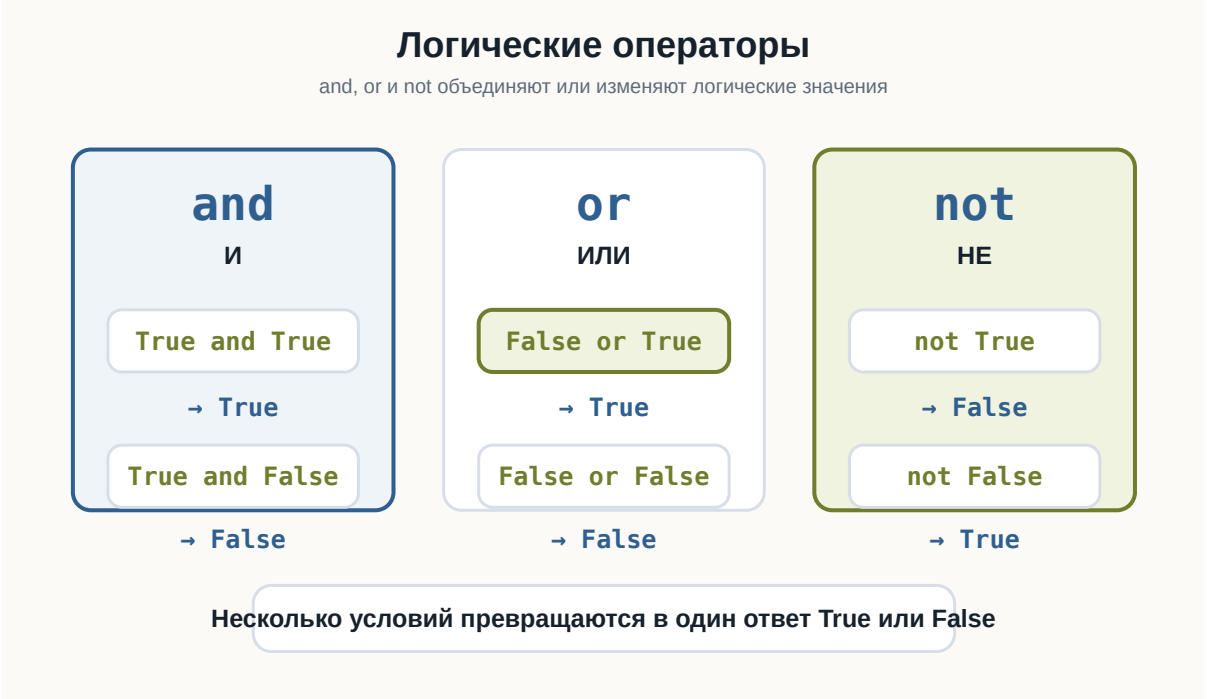


Рисунок 2.1: Карта логических операторов and, or и not

Проверьте базовые сочетания `and`, `or` и `not` в playground ниже. В HTML можно менять значения и запускать код снова.

Как работает and

Оператор and означает:

оба условия должны быть истинными.

Например:

```
age >= 18 and has_ticket
```

Такое выражение можно прочитать:

> возраст не меньше 18 **и** билет есть.

Если хотя бы одно условие равно `False`, весь результат тоже будет `False`.

Список 2.1 `and-basics.py`

```
print(True and True)
print(True and False)
print(False and True)
print(False and False)

age = 20
has_ticket = True

print(age >= 18 and has_ticket)
```

Результаты для and:

```
True and True   -> True
True and False  -> False
False and True  -> False
False and False -> False
```

Главная идея:

> and возвращает `True` только тогда, когда оба значения `True`.

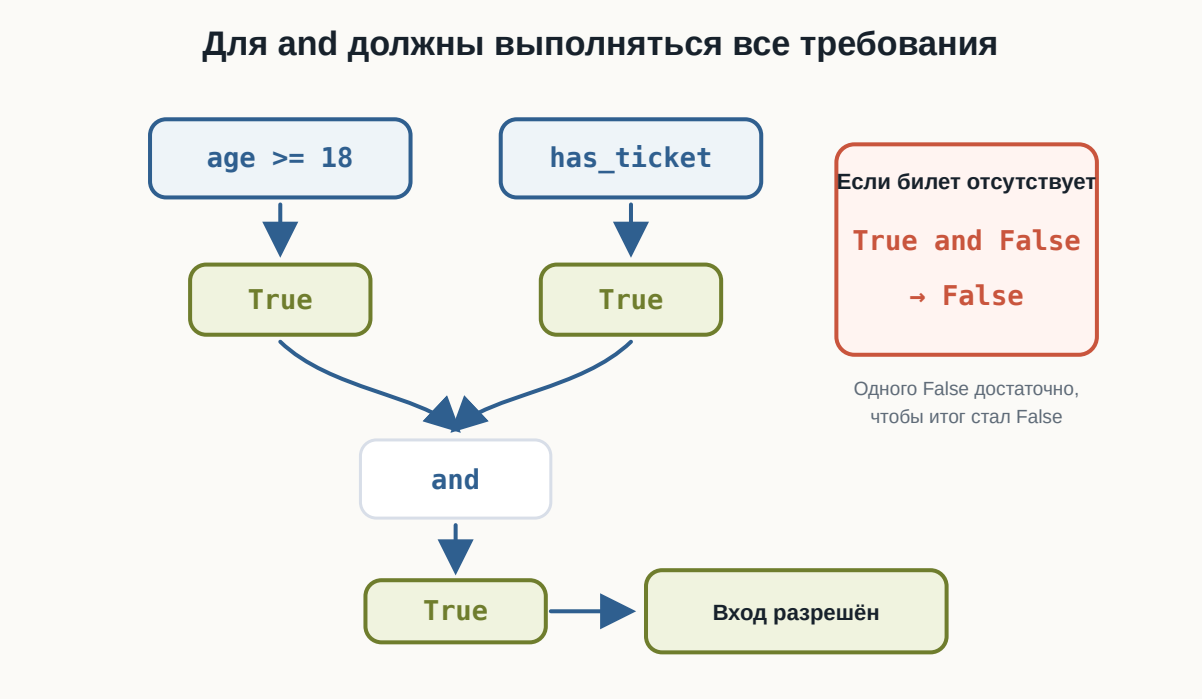


Рисунок 2.2: Поток решения для оператора and

---

and внутри условия

Теперь соединим and с `if`.

Пользователь вводит возраст и отвечает, есть ли билет:

Список 2.2 `event-access.py`

```
age = int(input("Возраст: "))
has_ticket = input("Есть билет? yes/no: ") == "yes"

if age >= 18 and has_ticket:
    print("Вход разрешён")
else:
    print("Вход запрещён")
```

Проверьте несколько вариантов:

```
20 / yes -> Вход разрешён
20 / no  -> Вход запрещён
16 / yes -> Вход запрещён
16 / no  -> Вход запрещён
```

Условие:

```
age >= 18 and has_ticket
```

становится True только в первом случае.

Как работает or

Оператор or означает:

> достаточно, чтобы истинным было хотя бы одно условие.

Например:

```
is_admin or is_owner
```

Можно прочитать так:

> пользователь администратор **или** пользователь владелец.

Список 2.3 or-basics.py

```
print(True or True)
print(True or False)
print(False or True)
print(False or False)

is_admin = False
is_owner = True

print(is_admin or is_owner)
```

Результаты для or:

```
True or True    -> True
True or False   -> True
False or True   -> True
False or False  -> False
```

Главная идея:

> or возвращает True, если хотя бы одно значение True.

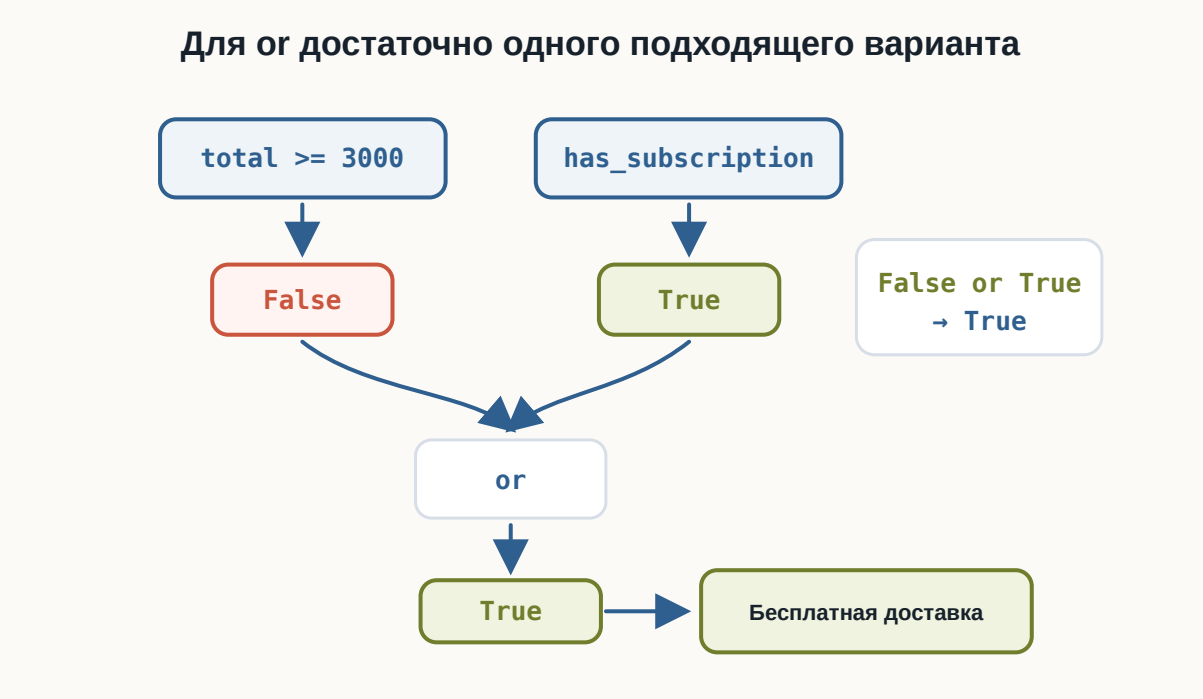


Рисунок 2.3: Поток решения для оператора or

---

or внутри условия

Представим интернет-магазин.

Доставка бесплатная, если:

```
сумма заказа не меньше 3000
или
есть подписка
```

Список 2.4 `free-delivery.py`

```
total = float(input("Сумма заказа: "))
has_subscription = input("Есть подписка? yes/no: ") == "yes"

if total >= 3000 or has_subscription:
    print("Доставка бесплатная")
else:
    print("Доставка платная")
```

Проверьте:

```
3500 / no  -> Доставка бесплатная
1500 / yes -> Доставка бесплатная
1500 / no  -> Доставка платная
```

В первых двух случаях хотя бы одно условие истинно.

Как работает not

Оператор not меняет логическое значение на противоположное:

```
not True
not False
```

Список 2.5 not-basics.py

```
print(not True)
print(not False)

is_blocked = False

print(not is_blocked)
```

Результаты:

```
not True  -> False
not False -> True
```

Это удобно, когда имя переменной описывает запрет или блокировку:

```
is_blocked = False

not is_blocked
```

Такое выражение можно прочитать:

> пользователь не заблокирован.

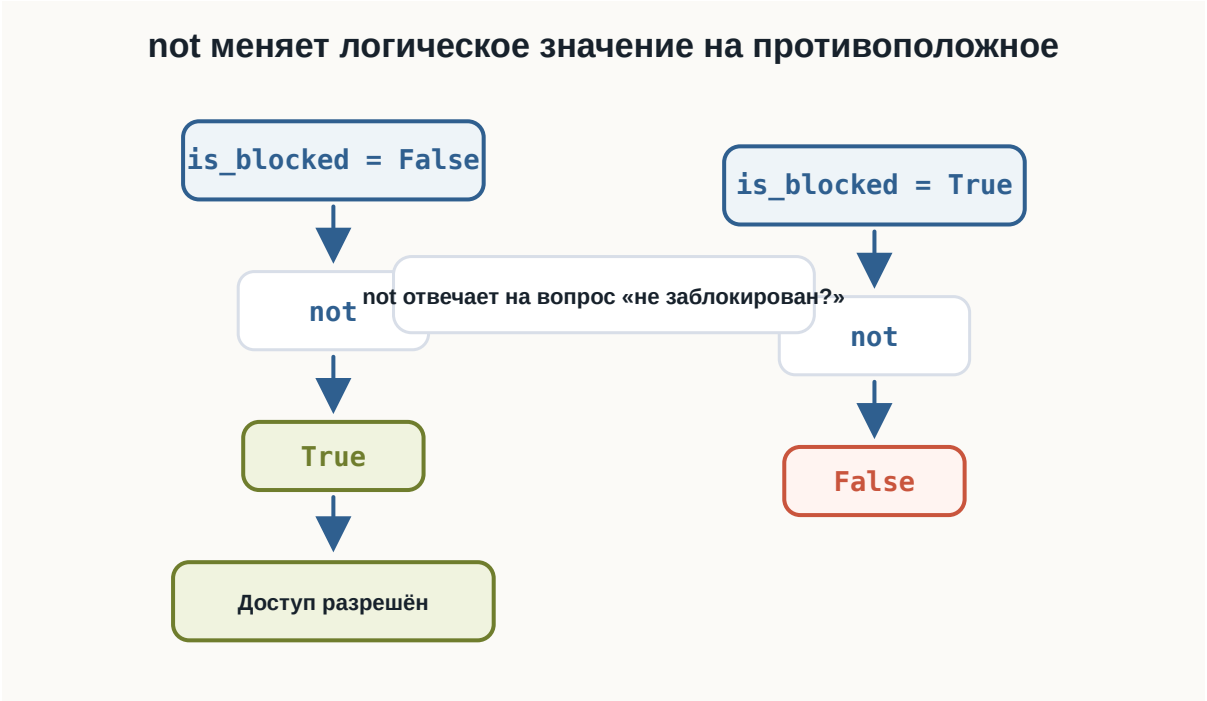


Рисунок 2.4: Поток решения для оператора not

Сравнение, and, not и if

Проверим пароль и блокировку пользователя.

Для входа нужно:

```
пароль правильный
и
пользователь не заблокирован
```

Список 2.6 `login-check.py`

```
correct_password = "python123"

password = input("Введите пароль: ")
is_blocked = False

if password == correct_password and not is_blocked:
    print("Вход выполнен")
else:
    print("Вход запрещён")
```

Условие:

```
password == correct_password and not is_blocked
```

состоит из трёх знакомых частей:

- сравнение `password == correct_password`;
- объединение через `and`;
- обращение значения через `not`.

Если пароль правильный и `is_blocked` равен `False`, вход выполнится.

Приоритет логических операций

У логических операторов есть порядок вычисления:

```
not
and
or
```

Сначала выполняется `not`, затем `and`, затем `or`.

Список 2.7 `logical-priority.py`

```
# Сначала выполняется and, потом or.
print(True or False and False)

# Скобки меняют порядок вычислений.
print((True or False) and False)

# not выполняется раньше and.
print(not False and True)
```

Разберём результаты:

```
True or False and False
```

сначала считается `False and False`, поэтому итог:

```
True
```

А здесь скобки меняют смысл:

```
(True or False) and False
```

Сначала считается `True or False`, затем результат объединяется с `False`, поэтому итог:

```
False
```

Главное правило:

> Если выражение можно понять по-разному, используйте скобки.

Сложное условие лучше разделять

Логическое выражение может быстро стать длинным.

Например:

```
is_admin or (is_owner and not is_blocked)
```

Его можно прочитать так:

> доступ есть, если пользователь администратор или если он владелец и не заблокирован.

Список 2.8 `complex-access.py`

```
is_admin = False
is_owner = True
is_blocked = False

can_access = is_admin or (is_owner and not is_blocked)

print(can_access)
```

Здесь скобки помогают явно показать, что `not is_blocked` относится к владельцу:

```
is_owner and not is_blocked
```

А имя `can_access` сохраняет итог:

```
can_access = is_admin or (is_owner and not is_blocked)
```

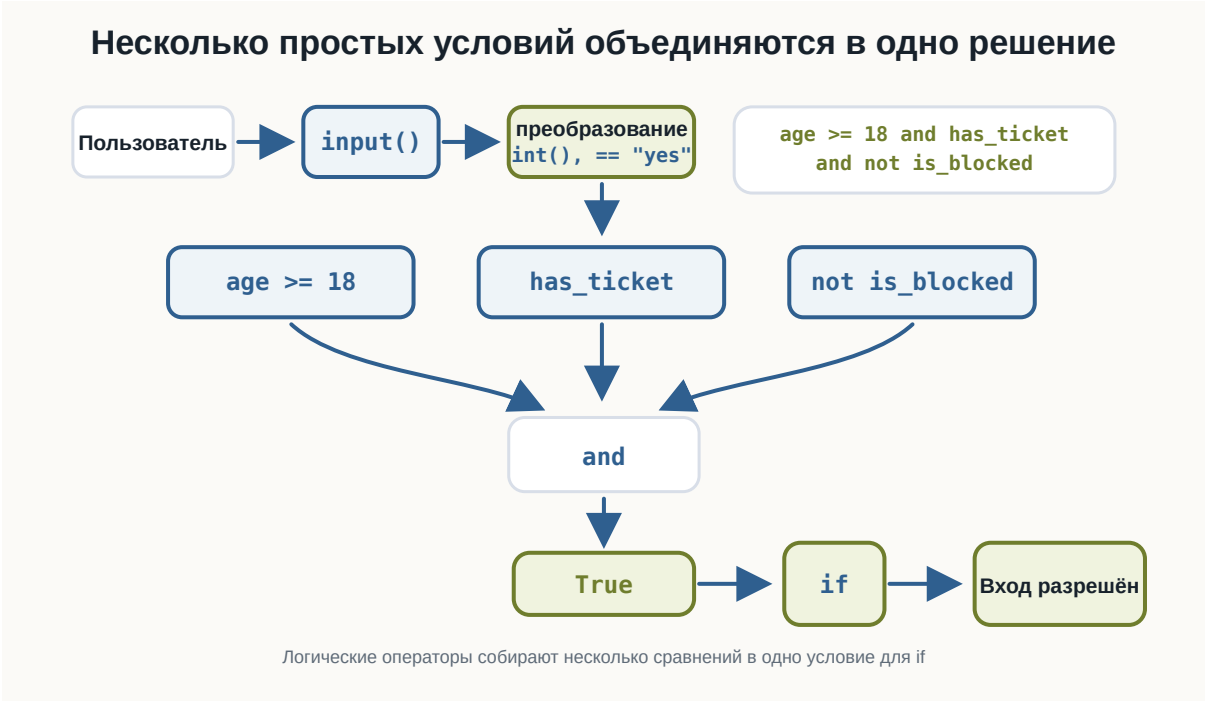


Рисунок 2.5: Общий поток составного логического решения

Короткое вычисление

Python не всегда вычисляет всё выражение до конца.

Для and:

```
False and ...
```

итог уже известен: он будет `False`.

Для or:

```
True or ...
```

итог уже известен: он будет `True`.

На этом этапе достаточно понимать саму идею: Python может остановиться, когда результат уже ясен.

Что важно запомнить

and требует, чтобы все условия были истинными:

```
age >= 18 and has_ticket
```

or требует, чтобы истинным было хотя бы одно условие:

```
total >= 3000 or has_subscription
```

not меняет логическое значение на противоположное:

```
not is_blocked
```

Приоритет логических операций:

```
not
and
or
```

Скобки помогают читать сложные выражения:

```
is_admin or (is_owner and not is_blocked)
```

Главная цепочка этой главы:

```
условие 1
условие 2
  ↓
and / or / not
  ↓
одно логическое решение
  ↓
if
  ↓
действие
```

Практические задания

Задание 1. Вход на мероприятие

Пользователь вводит возраст и отвечает, есть ли билет.

Разрешите вход только если возраст не меньше 18 и билет есть.

Ответ

```
age = int(input("Возраст: "))
has_ticket = input("Есть билет? yes/no: ") == "yes"

if age >= 18 and has_ticket:
    print("Вход разрешён")
else:
    print("Вход запрещён")
```

Задание 2. Бесплатная доставка

Пользователь вводит сумму заказа и отвечает, есть ли подписка.

Доставка бесплатная, если сумма не меньше `3000` или есть подписка.

Ответ

```
total = float(input("Сумма заказа: "))
has_subscription = input("Есть подписка? yes/no: ") == "yes"

if total >= 3000 or has_subscription:
    print("Доставка бесплатная")
else:
    print("Доставка платная")
```

Задание 3. Пользователь не заблокирован

Дана переменная:

```
is_blocked = False
```

Выведите `True`, если пользователь не заблокирован.

Ответ

```
is_blocked = False

print(not is_blocked)
```

Задание 4. Проверка входа

Пользователь вводит пароль.

Вход разрешён, если пароль равен `"python123"` и пользователь не заблокирован.

Ответ

```
correct_password = "python123"

password = input("Введите пароль: ")
is_blocked = False

if password == correct_password and not is_blocked:
    print("Вход выполнен")
else:
    print("Вход запрещён")
```

Задание 5. Предскажите результат

Не запускайте код сразу.

Определите результаты:

```
print(True and False)
print(False or True)
print(not True)
print(True or False and False)
print((True or False) and False)
```

Ответ

Результат:

```
False
True
False
True
False
```

В выражении:

```
True or False and False
```

сначала выполняется and, поэтому итог остаётся True.

Задание 6. Доступ к файлу

Даны переменные:

```
is_admin = False
is_owner = True
is_blocked = False
```

Создайте переменную can_access.

Доступ есть, если пользователь администратор или если он владелец и не заблокирован.

Ответ

```
is_admin = False
is_owner = True
is_blocked = False

can_access = is_admin or (is_owner and not is_blocked)

print(can_access)
```

Если изменить:

```
is_blocked = True
```

при is_admin = False, результат станет False.

Что дальше?

Теперь программа умеет объединять несколько условий в одно решение.

Следующий шаг:

```
повторять действие,
пока условие остаётся истинным
```

Для этого используются циклы. Они будут отдельной темой.

2 3.9 Цикл while

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как заставить программу повторять действия, пока выполняется условие?**

В предыдущих главах мы научились задавать программе вопросы:

```
age >= 18
password == "python123"
number > 0
```

Каждое такое выражение даёт логический результат:

```
True
```

или:

```
False
```

Условия позволяют выбрать действие один раз. Но иногда одного выбора недостаточно.

Например:

```
пока пароль неверный
    спрашивать пароль снова
```

или:

```
пока число не дошло до 5
    вывести число
    увеличить число
```

Для таких ситуаций в Python используется цикл `while`.

Зачем программе повторять действия

Представьте программу, которая должна вывести приветствие три раза.

Без цикла можно написать:

```
print("Привет")
print("Привет")
print("Привет")
```

Такой код работает, но в нём есть проблема: количество повторений зашито вручную.

Если нужно не три, а тридцать раз, код быстро станет неудобным.

Цикл позволяет описать повторение как правило:

```
пока условие истинно
    выполняй блок команд
```

В Python это записывается так:

```
while условие:
    действие
```

Слово `while` можно читать как:

> **пока**

Первый цикл while

Начнём с простого примера:

```
count = 1

while count <= 3:
    print("Привет")
    count += 1
```

Список 2.1 `first-while.py`

```
count = 1

while count <= 3:
    print("Привет")
    count += 1
```

Результат:

```
Привет
Привет
Привет
```

Здесь есть три важные части:

- `count = 1` задаёт начальное значение;
- `while count <= 3:` проверяет условие;
- `count += 1` изменяет значение после каждой итерации.

Итерация — это одно выполнение тела цикла.

В этом примере тело цикла выполняется три раза.

Попробуйте в HTML-версии изменить:

- начальное значение `count`;
- границу 3;
- шаг `count += 1`.

Как работает while

Цикл `while` проверяет условие **до** выполнения тела цикла.

Общая схема такая:

```
проверить условие
если True:
    выполнить тело цикла
    вернуться к условию
если False:
    выйти из цикла
```

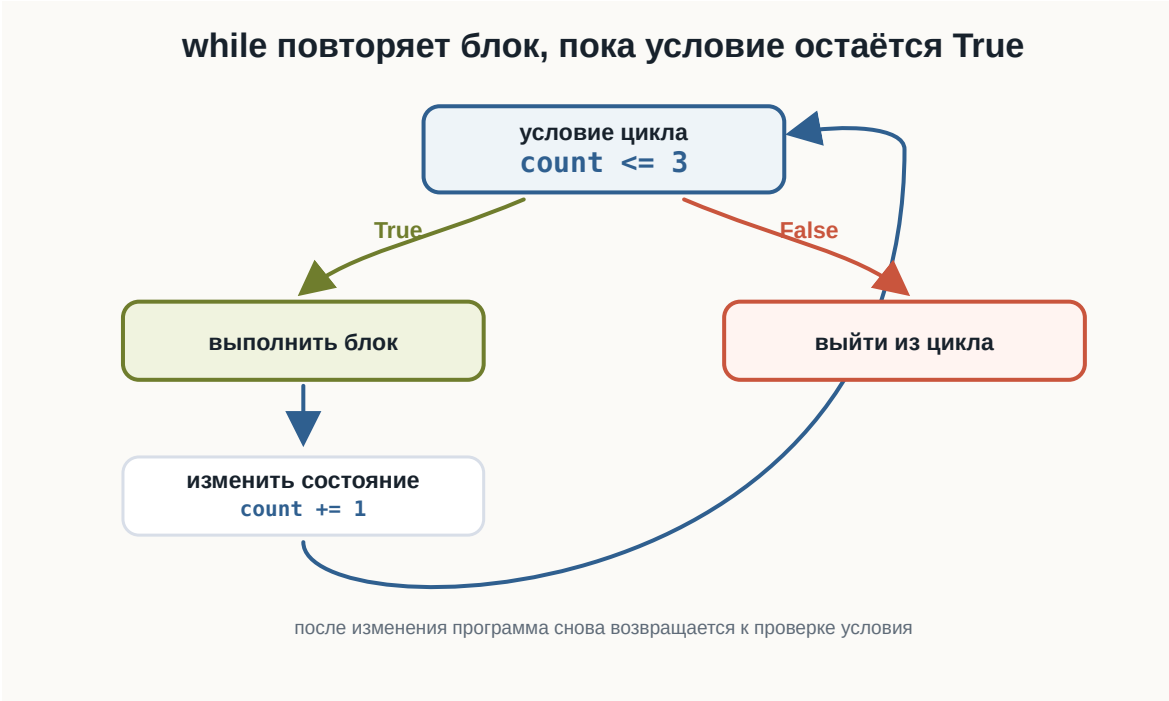


Рисунок 2.1: Схема работы цикла while

Важно:

> `while` повторяет блок, пока условие остаётся `True`.

Если условие сразу равно `False`, тело цикла не выполнится ни разу.

Например:

```
count = 10

while count < 5:
    print(count)
    count += 1
```

Здесь `10 < 5` сразу даёт `False`, поэтому `print(count)` не выполнится.

Счётчик в цикле

Часто внутри `while` используется счётчик.

Счётчик — это переменная, которая помогает управлять количеством повторений.

```
number = 1

while number <= 5:
    print(number)
    number += 1

print("После цикла number =", number)
```

Список 2.2 `count-up.py`

```
number = 1

while number <= 5:
    print(number)
    number += 1

print("После цикла number =", number)
```

Результат:

```
1
2
3
4
5
После цикла number = 6
```

Сначала `number` равен 1.

После каждой итерации выполняется строка:

```
number += 1
```

Она увеличивает `number` на 1.

Когда `number` становится равен 6, условие:

```
number <= 5
```

становится `False`, и цикл завершается.

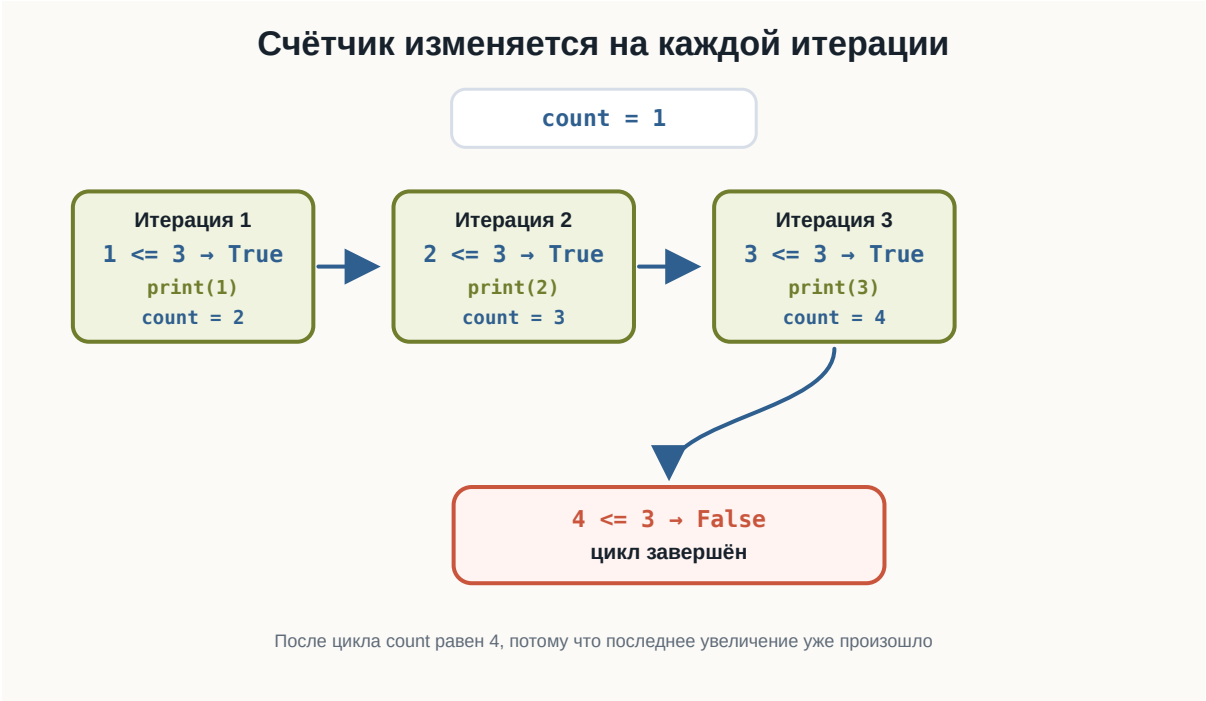


Рисунок 2.2: Пошаговое изменение счётчика в while

Граница включается или не включается

Сравнение в условии определяет, попадёт ли граничное значение в цикл.

Если написать:

```
while count < 5:
```

цикл выполняется, пока `count` меньше 5. Само значение 5 уже не подходит.

Если написать:

```
while count <= 5:
```

значение 5 подходит, потому что условие означает «меньше или равно».

Разница между < и <= особенно важна в циклах, потому что она меняет количество итераций.

Обратный счёт

Счётчик не обязан увеличиваться.

Он может двигаться в обратную сторону:

```
count = 5

while count >= 1:
    print(count)
    count -= 1

print("Старт!")
```

Список 2.3 `countdown.py`

```
count = 5

while count >= 1:
    print(count)
    count -= 1

print("Старт!")
```

Результат:

```
5
4
3
2
1
Старт!
```

Здесь условие:

```
count >= 1
```

остаётся истинным для 5, 4, 3, 2 и 1.

После каждой итерации выполняется:

```
count -= 1
```

Когда `count` становится равен `0`, условие `0 >= 1` даёт `False`.

Бесконечный цикл

У цикла `while` есть важная опасность: условие может никогда не стать `False`.

Например:

```
count = 1

# ВНИМАНИЕ: этот цикл бесконечный, потому что count не изменяется.
while count <= 5:
    print(count)
```

Список 2.4 `infinite-loop.py`

```
count = 1

# ВНИМАНИЕ: этот цикл бесконечный, потому что count не изменяется.
while count <= 5:
    print(count)
```

В этом коде `count` всегда остаётся равен 1.

Проверка:

```
count <= 5
```

каждый раз превращается в:

```
1 <= 5
```

Результат снова `True`, поэтому цикл продолжает выполняться.

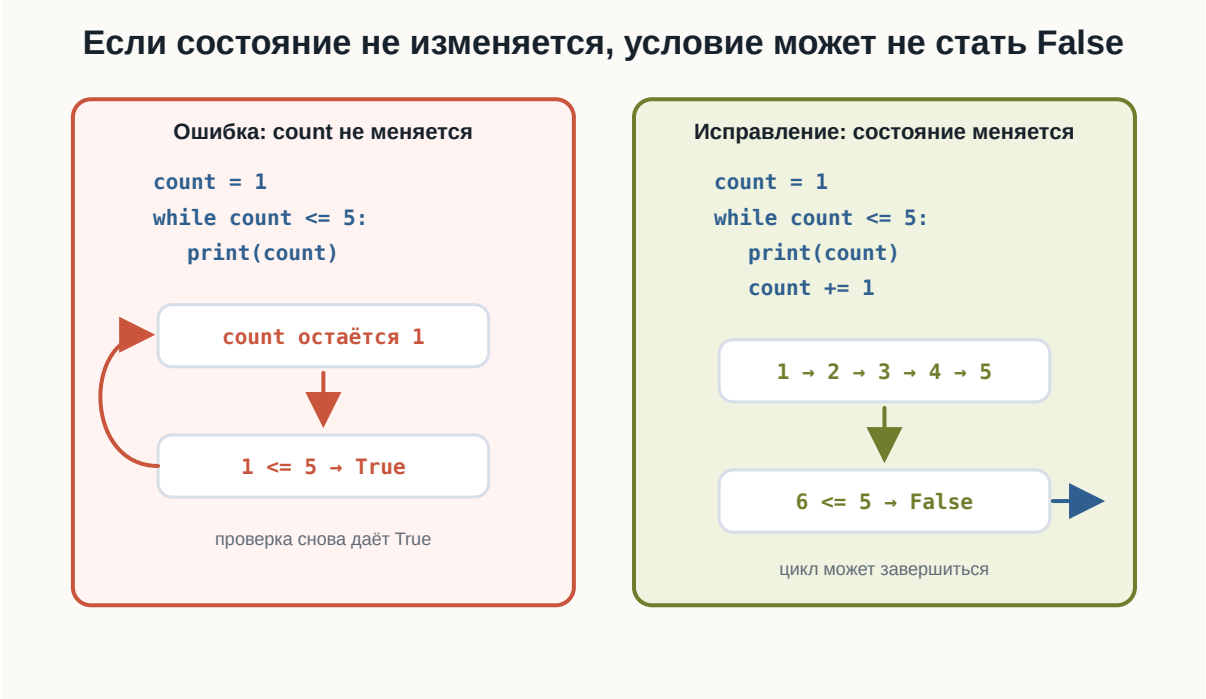


Рисунок 2.3: Бесконечный цикл и исправленный вариант

Чтобы цикл завершился, состояние должно изменяться так, чтобы условие когда-нибудь стало `False`.

Правильный вариант:

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

В HTML-версии бесконечный пример оставлен статическим. Текущий playground не имеет безопасного прерывания синхронного бесконечного цикла, поэтому такой код нельзя предлагать к запуску.

---

while и пользовательский ввод

`while` полезен, когда программа должна повторять ввод до подходящего значения.

Например, можно спрашивать пароль, пока пользователь не введёт правильный:

```
correct_password = "python123"

password = input("Введите пароль: ")

while password != correct_password:
    print("Неверный пароль")
    password = input("Попробуйте ещё раз: ")

print("Доступ разрешён")
```

Список 2.5 `password-loop.py`

```
correct_password = "python123"

password = input("Введите пароль: ")

while password != correct_password:
    print("Неверный пароль")
    password = input("Попробуйте ещё раз: ")

print("Доступ разрешён")
```

Попробуйте ввести несколько неправильных паролей, а затем:

```
python123
```

Цикл повторяется, пока условие:

```
password != correct_password
```

равно `True`.

Когда пароль совпадает с правильным, условие становится `False`, и программа выходит из цикла.



Рисунок 2.4: Повторный ввод пароля с while

Повторная проверка данных

Та же идея работает не только с паролем.

Например, программа может требовать положительное число:

```
number = float(input("Введите положительное число: "))

while number <= 0:
    print("Нужно число больше нуля")
    number = float(input("Попробуйте ещё раз: "))

print("Принято:", number)
```

Проверьте последовательность:

```
-5
0
10
```

Для `-5` и `0` условие `number <= 0` истинно, поэтому программа просит ввести число снова.

Для `10` условие становится ложным, и цикл завершается.

Список 2.6 `positive-input.py`

```
number = float(input("Введите положительное число: "))

while number <= 0:
    print("Нужно число больше нуля")
    number = float(input("Попробуйте ещё раз: "))

print("Принято:", number)
```

Накопитель

В циклах часто встречается ещё одна роль переменной: накопитель.

Счётчик управляет повторением.

Накопитель собирает результат.

Например, найдём сумму чисел от 1 до 5:

```
number = 1
total = 0

while number <= 5:
    total += number
    number += 1

print("Сумма:", total)
```

Список 2.7 `sum-numbers.py`

```
number = 1
total = 0

while number <= 5:
    total += number
    number += 1

print("Сумма:", total)
```

Результат:

```
Сумма: 15
```

Здесь:

- `number` — счётчик;
- `total` — накопитель.

На каждой итерации выполняется:

```
total += number
```

Это значит: прибавить текущее значение `number` к сумме.

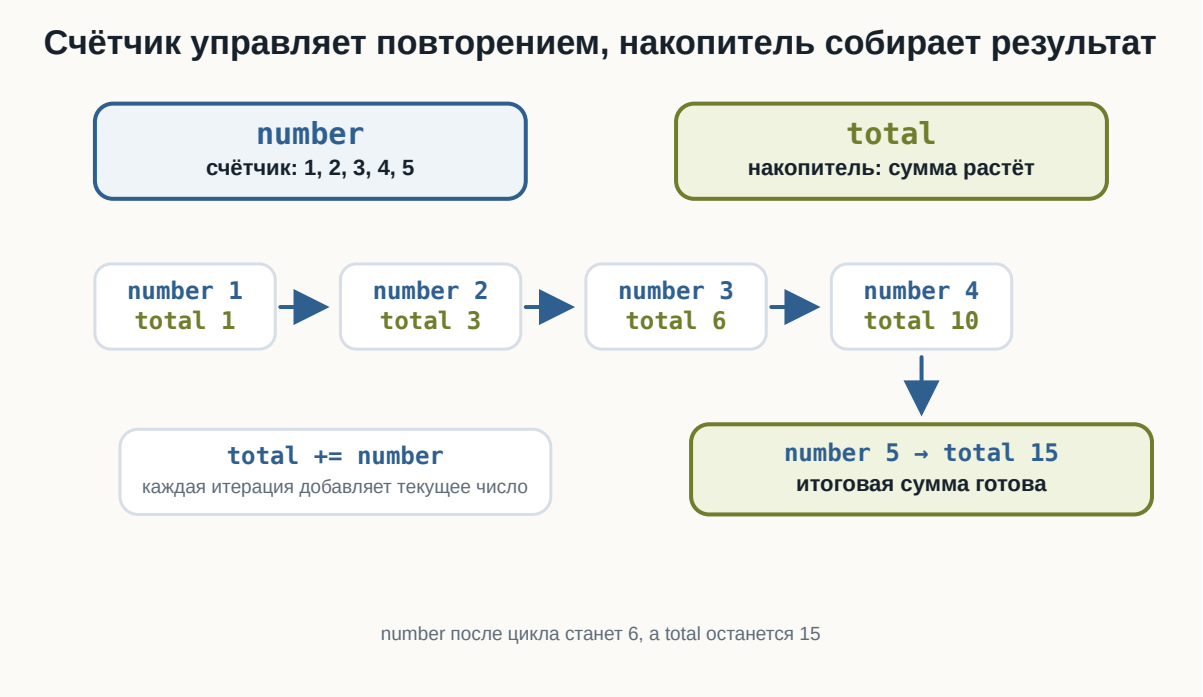


Рисунок 2.5: Счётчик и накопитель в цикле while

После цикла:

```
number == 6
total == 15
```

Повторный ввод и накопитель

Теперь объединим три идеи:

- повторный `input()`;
- счётчик;
- накопитель.

Программа спросит три числа и найдёт их сумму:

```
count = 1
total = 0

while count <= 3:
    number = float(input("Введите число: "))
    total += number
    count += 1

print("Сумма:", total)
```

Список 2.8 `input-sum.py`

```
count = 1
total = 0

while count <= 3:
    number = float(input("Введите число: "))
    total += number
    count += 1

print("Сумма:", total)
```

Проверьте ввод:

```
10
20
5
```

Результат:

```
Сумма: 35.0
```

Шаг счётчика

Счётчик можно изменять не только на 1.

Например:

```
number = 1

while number <= 5:
    print(number)
    number += 2
```

Результат:

```
1
3
5
```

Такой цикл завершается, потому что `number` всё равно движется к значению, при котором условие станет `False`.

Важно проверять направление изменения.

Если условие ждёт, что число станет больше границы, счётчик должен увеличиваться.

Если условие ждёт, что число станет меньше границы, счётчик должен уменьшаться.

Что важно запомнить

`while` повторяет блок, пока условие истинно:

```
while условие:
    тело_цикла
```

Условие проверяется до первой итерации.

Поэтому цикл может не выполниться ни разу.

Тело цикла записывается с отступом.

Внутри цикла обычно должно изменяться состояние программы:

```
count += 1
```

или:

```
count -= 1
```

Если состояние не меняется, условие может никогда не стать `False`.

Счётчик управляет повторением.

Накопитель собирает результат.

Главная цепочка этой главы:

```
условие
  ↓
True
  ↓
выполнить действие
  ↓
изменить состояние
  ↓
вернуться к условию
```

Практические задания

Задание 1. Три приветствия

Напишите цикл `while`, который выводит:

```
Привет
```

три раза.

> Ответ

```
count = 1

while count <= 3:
    print("Привет")
    count += 1
```

Задание 2. Числа от 1 до 5

Выведите числа от 1 до 5 с помощью `while`.

Ответ

```
number = 1

while number <= 5:
    print(number)
    number += 1
```

Задание 3. Обратный счёт

Выведите числа от 5 до 1, а затем строку:

```
Старт!
```

Ответ

```
count = 5

while count >= 1:
    print(count)
    count -= 1

print("Старт!")
```

Задание 4. Положительное число

Попросите пользователя ввести положительное число.

Пока число меньше или равно `0`, выводите:

```
Нужно число больше нуля
```

и спрашивайте снова.

Ответ

```
number = float(input("Введите положительное число: "))

while number <= 0:
    print("Нужно число больше нуля")
    number = float(input("Попробуйте ещё раз: "))

print("Принято:", number)
```

Задание 5. Сумма от 1 до 5

С помощью `while` найдите сумму чисел:

```
1 + 2 + 3 + 4 + 5
```

Ответ

```
number = 1
total = 0

while number <= 5:
    total += number
    number += 1

print("Сумма:", total)
```

Задание 6. Сумма трёх чисел

Попросите пользователя ввести три числа и выведите их сумму.

Используйте `while`, счётчик и накопитель.

Ответ

```
count = 1
total = 0

while count <= 3:
    number = float(input("Введите число: "))
    total += number
    count += 1

print("Сумма:", total)
```

Задание 7. Найдите ошибку

Почему этот цикл не завершается?

```
count = 1

while count <= 5:
    print(count)
```

Исправьте программу.

Ответ

Переменная `count` не изменяется внутри цикла.
Условие:

```
count <= 5
```

всё время остаётся истинным.
Исправленный вариант:

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

Задание 8. Граница цикла

Что выведет программа?

```
count = 1

while count < 5:
    print(count)
    count += 1
```

Почему число 5 не выводится?

> **Ответ**
>
> Результат:
>
> ```
> 1
> 2
> 3
> 4
> ```
>
> Число 5 не выводится, потому что условие:
>
> ```
> count < 5
> ```
>
> истинно только для чисел меньше 5.
> Когда `count` становится равен 5, условие даёт `False`, и цикл завершается.

Что дальше?

Теперь программа умеет повторять действия, пока условие остаётся истинным.

Следующий шаг:

```
повторение по условию
↓
перебор последовательности
```

Для этого используется цикл `for`.

2 3.10 Цикл for

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как повторять действия для каждого элемента или значения последовательности?**

В предыдущей главе мы познакомились с циклом `while`.

Например:

```
number = 1

while number <= 5:
    print(number)
    number += 1
```

Результат:

```
1
2
3
4
5
```

Здесь нам пришлось самостоятельно:

1. создать счётчик;
2. задать условие;
3. изменять счётчик внутри цикла.

Но во многих задачах мы заранее знаем, **какие значения нужно перебрать**.

Например:

```
числа от 1 до 5
каждый символ строки
несколько повторений одного действия
```

Для таких случаев в Python существует цикл:

```
for
```

Первый цикл for

Начнём с простого примера:

```
for number in range(1, 6):
    print(number)
```

Список 2.1 `first-for.py`

```
for number in range(1, 6):
    print(number)
```

Результат:

```
1
2
3
4
5
```

На первый взгляд здесь появились сразу несколько новых элементов:

```
for
```

```
in
```

```
range()
```

Разберём их постепенно.

Как читать цикл for

Код:

```
for number in range(1, 6):
    print(number)
```

можно прочитать примерно так:

> Для каждого значения `number` из последовательности от `1` до `5` выполнить `print(number)`.

Во время работы `number` последовательно получает значения:

```
1
2
3
4
5
```

Для каждого из них выполняется тело цикла.

Как работает for

Рассмотрим:

```
for number in range(1, 4):
    print(number)
```

`range(1, 4)` задаёт значения:

```
1
2
3
```

Первая итерация:

```
number = 1
```

Python выполняет:

```
print(number)
```

Затем:

```
number = 2
```

и блок выполняется снова.

После этого:

```
number = 3
```

и выполняется третья итерация.

Когда значения заканчиваются, цикл завершается.

Общая схема:

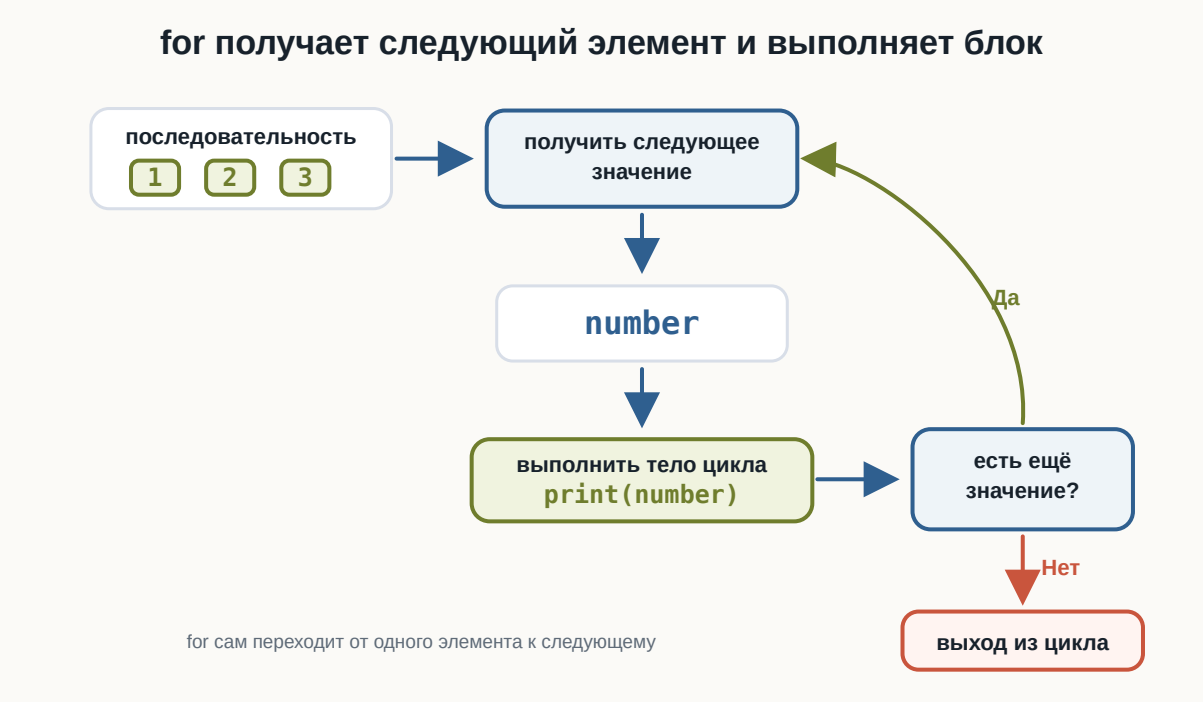


Рисунок 2.1: Схема работы цикла for

Переменная цикла

В конструкции:

```
for number in range(1, 6):
```

`number` называется **переменной цикла**.

На каждой итерации она получает очередное значение.

Например:

```
for number in range(1, 4):
    print("Текущее значение:", number)
```

Результат:

```
Текущее значение: 1
Текущее значение: 2
Текущее значение: 3
```

Можно представить:

```
итерация 1 → number = 1
итерация 2 → number = 2
итерация 3 → number = 3
```

Нам не нужно вручную писать:

```
number += 1
```

Цикл `for` сам переходит к следующему значению.

Что делает range()

В предыдущем примере использовалась функция:

```
range()
```

Она задаёт последовательность целых чисел.

Например:

```
range(1, 6)
```

соответствует значениям:

```
1
2
3
4
5
```

Обратите внимание:

```
6 не входит
```

Второе число задаёт границу, **до которой нужно идти, не включая её**.

> ❗ Правая граница не входит
>
> ```
> range(1, 6)
> ```
>
> даёт:
>
> ```
> 1
> 2
> 3
> 4
> 5
> ```
>
> но не 6.
> Это одно из самых важных правил `range()`.

---

range() с одним аргументом

Можно написать:

```
range(5)
```

Это означает:

```
0
1
2
3
4
```

Поэтому:

```
for number in range(5):
    print(number)
```

выведет:

```
0
1
2
3
4
```

Если начало не указано, Python начинает с:

```
0
```

Получается:

```
range(5)
↓
0, 1, 2, 3, 4
```

---

range() с двумя аргументами

Запись:

```
range(start, stop)
```

задаёт:

```
начало
↓
...
↓
до stop, не включая stop
```

Например:

```
for number in range(3, 8):
    print(number)
```

Результат:

```
3
4
5
6
7
```

То есть:

```
start = 3
stop = 8
```

Проверьте в HTML-версии две базовые формы `range()`: с одним аргументом и с двумя аргументами.

Список 2.2 `range-basics.py`

```
for number in range(5):
    print(number)

print()

for number in range(2, 6):
    print(number)
```

range() с шагом

У `range()` может быть третий аргумент:

```
range(start, stop, step)
```

Он задаёт **шаг**.

Например:

```
for number in range(2, 11, 2):
    print(number)
```

Результат:

```
2
4
6
8
10
```

Здесь:

```
start = 2
stop = 11
step = 2
```

Каждое следующее значение увеличивается на 2.

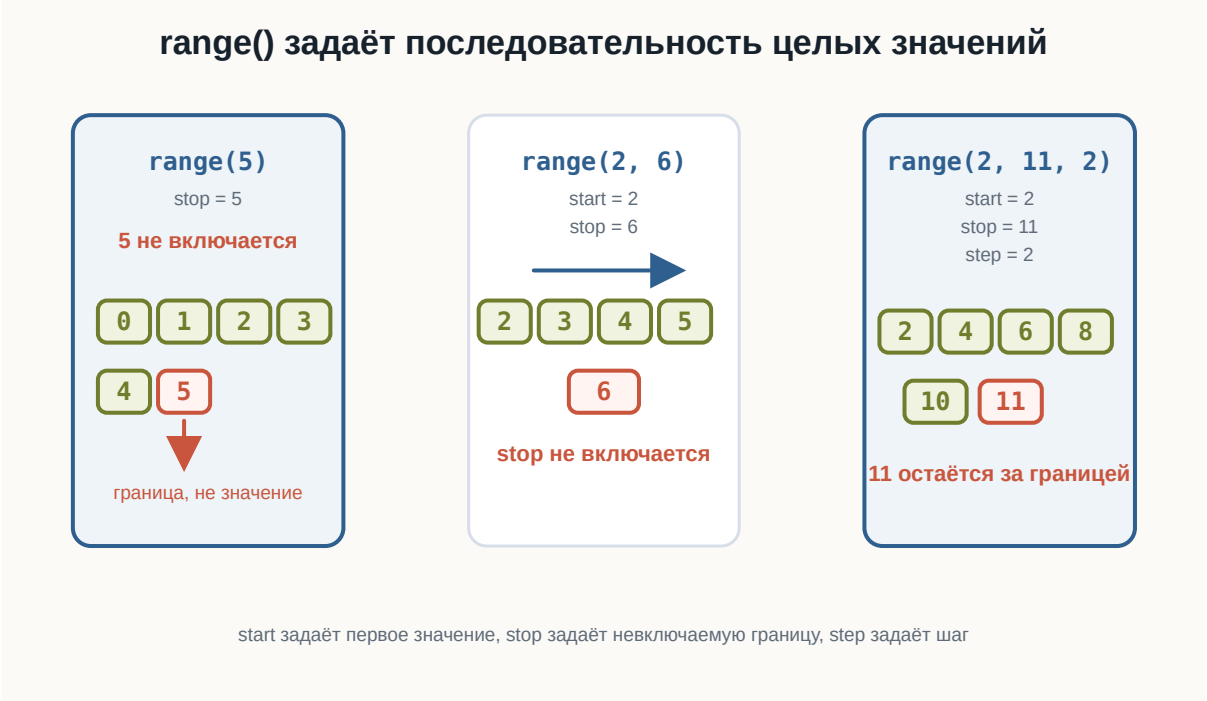


Рисунок 2.2: Три формы range

Шаг может быть больше единицы

Например:

```
for number in range(0, 21, 5):
    print(number)
```

Результат:

```
0
5
10
15
20
```

Можно представить:

```
0
↓ +5
5
↓ +5
10
↓ +5
15
↓ +5
20
```

Поменяйте в HTML-версии третий аргумент `range()` и посмотрите, как меняется шаг.

Список 2.3 `range-step.py`

```
for number in range(2, 11, 2):
    print(number)

print()

for number in range(0, 21, 5):
    print(number)
```

Обратный отсчёт

Шаг может быть отрицательным.

Например:

```
for number in range(5, 0, -1):
    print(number)

print("Старт!")
```

Список 2.4 `countdown-for.py`

```
for number in range(5, 0, -1):
    print(number)

print("Старт!")
```

Результат:

```
5
4
3
2
1
Старт!
```

Здесь:

```
start = 5
stop = 0
step = -1
```

Значение `0` снова не включается.

Важно правильно задать направление

Рассмотрим:

```
for number in range(5, 0):
    print(number)
```

Такой цикл ничего не выведет.

Почему?

По умолчанию шаг равен:

```
+1
```

Но нам нужно двигаться:

```
5 → 4 → 3 → 2 → 1
```

То есть вниз.

Поэтому нужен отрицательный шаг:

```
range(5, 0, -1)
```

⚠ Шаг должен соответствовать направлению

Если начало больше конца:

```
range(5, 0)
```

а шаг положительный, значений не будет.
Для движения назад используйте отрицательный шаг:

```
range(5, 0, -1)
```

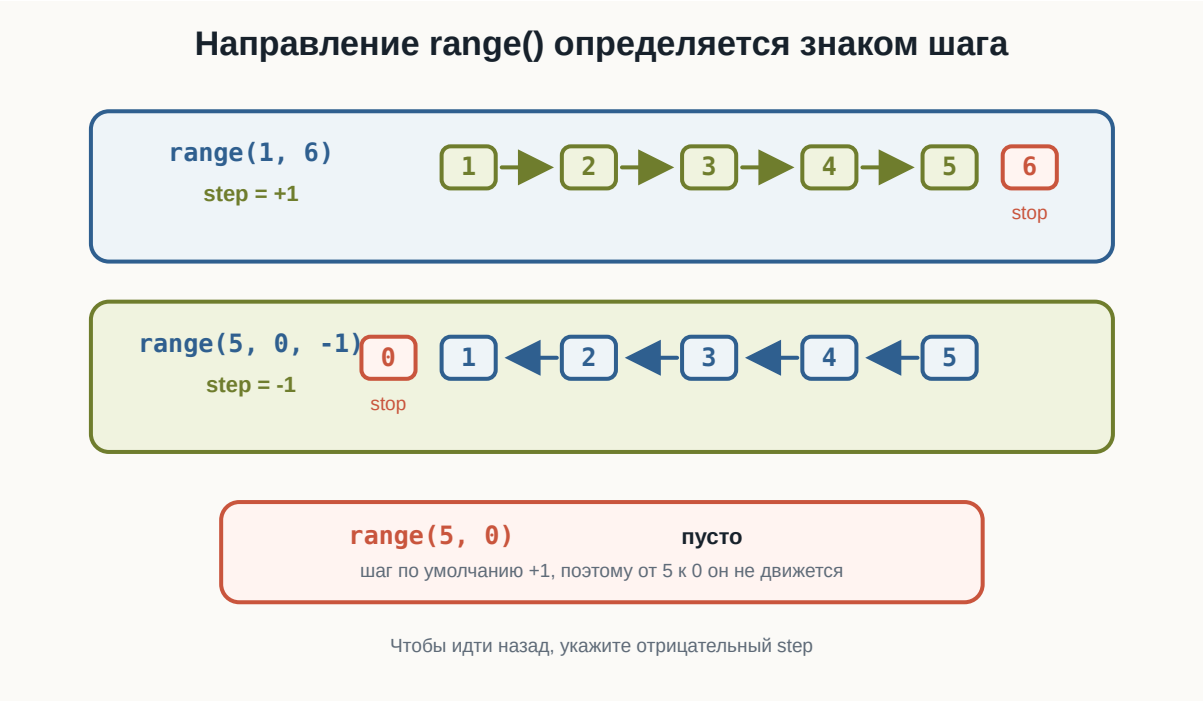


Рисунок 2.3: Направление range зависит от шага

Повторение действия несколько раз

Иногда само значение переменной цикла не так важно.

Например, нужно вывести сообщение пять раз:

```
for number in range(5):
    print("Изучаю Python")
```

Результат:

```
Изучаю Python
Изучаю Python
Изучаю Python
Изучаю Python
Изучаю Python
```

`range(5)` задаёт пять значений для перебора:

```
0
1
2
```

```
3
4
```

Поэтому тело выполняется пять раз.

Отступы работают так же, как в if и while

Тело `for` определяется отступом:

```
for number in range(3):
    print("Итерация")
    print(number)

print("Цикл завершён")
```

Результат:

```
Итерация
0
Итерация
1
Итерация
2
Цикл завершён
```

Две строки:

```
    print("Итерация")
    print(number)
```

принадлежат циклу.

А:

```
print("Цикл завершён")
```

выполняется после цикла.

---

for и while могут решать одну задачу

Рассмотрим `while`:

```
number = 1

while number <= 5:
    print(number)
    number += 1
```

То же самое можно написать через `for`:

```
for number in range(1, 6):
    print(number)
```

Результат в обоих случаях:

```
1
2
3
4
5
```

Но `for` короче.

Когда удобен while

`while` особенно удобен, когда мы не знаем заранее, сколько будет повторений.

Например:

```
password = input("Введите пароль: ")

while password != "python123":
    password = input("Попробуйте ещё раз: ")
```

Мы не знаем:

```
пользователь введёт пароль со второй попытки?
с пятой?
с десятой?
```

Количество повторений зависит от условия.

Когда удобен for

`for` особенно удобен, когда известна последовательность, которую нужно перебрать.

Например:

```
for number in range(1, 6):
    print(number)
```

Здесь заранее известны значения:

```
1
2
3
4
5
```

Можно запомнить:

```
while
→ повторять, пока условие True

for
→ выполнить для каждого значения
```

> while или for?
>
> Используйте `while`, когда главное — **условие продолжения**.
> Используйте `for`, когда главное — **перебор известных значений или элементов**.

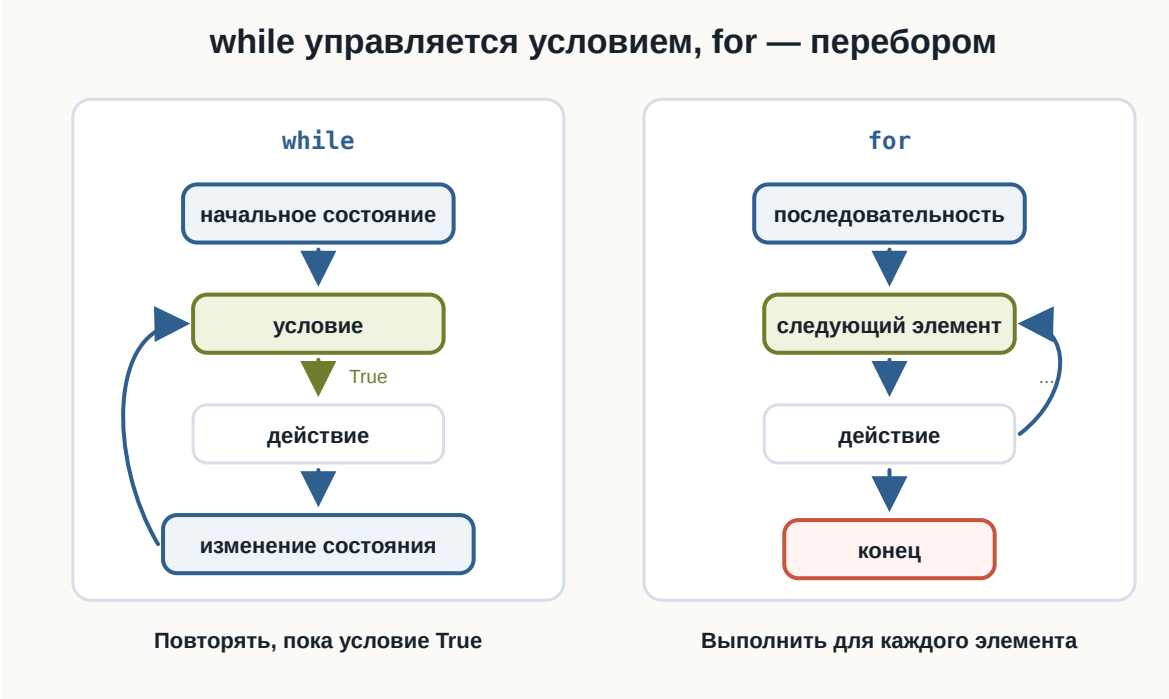


Рисунок 2.4: Сравнение циклов while и for

---

for умеет перебирать строку

`for` работает не только с `range()`.

Строка тоже содержит последовательность элементов — символов.

Например:

```
for letter in "Python":
    print(letter)
```

Результат:

```
P
y
t
h
o
n
```

На каждой итерации переменная:

```
letter
```

получает следующий символ.

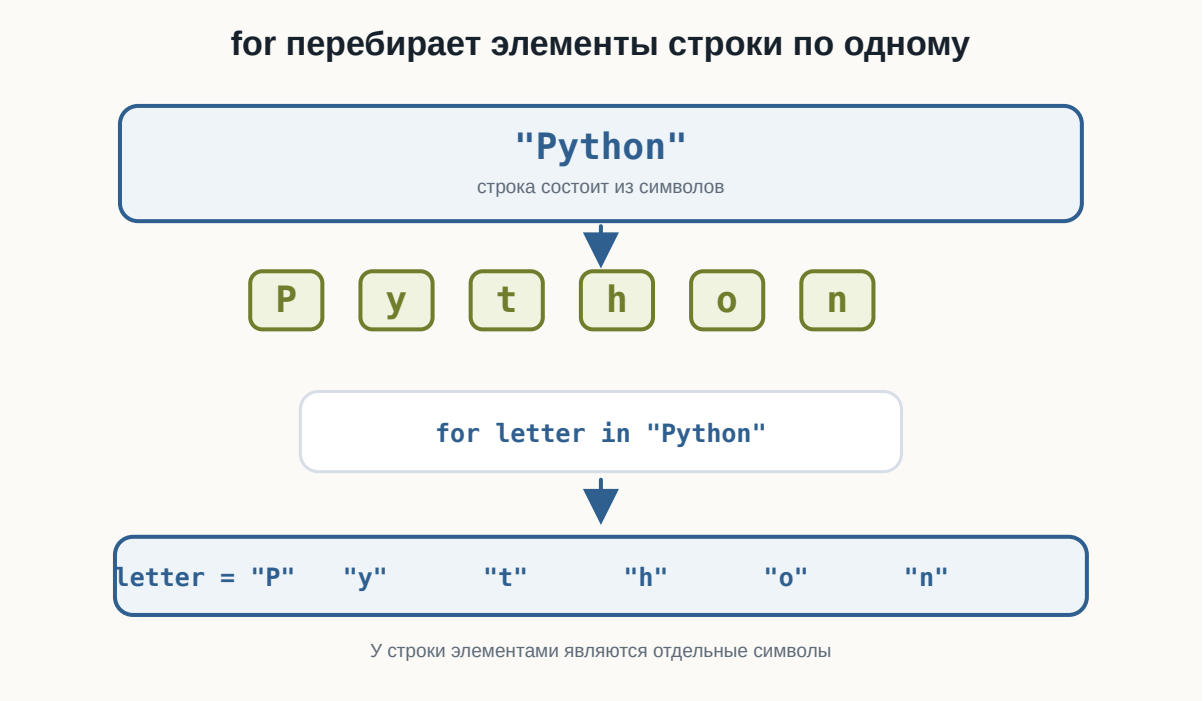


Рисунок 2.5: Перебор строки в for

Как перебирается строка

Для:

```
for letter in "Cat":
    print(letter)
```

происходит:

```
итерация 1 → letter = "C"
итерация 2 → letter = "a"
итерация 3 → letter = "t"
```

После последнего символа цикл завершается.

Это показывает важную особенность `for`:

> `for` перебирает элементы некоторой последовательности по одному.

Имя переменной цикла должно быть понятным

Технически можно написать:

```
for x in "Python":
    print(x)
```

Но чаще понятнее:

```
for letter in "Python":
    print(letter)
```

Для чисел:

```
for number in range(5):
```

Для символов:

```
for letter in "Python":
```

Хорошее имя помогает понять смысл цикла.

Подсчёт количества итераций

Рассмотрим:

```
for number in range(2, 8):
    print(number)
```

Значения:

```
2
3
4
5
6
7
```

То есть цикл выполнит:

```
6 итераций
```

Полезно уметь заранее определять:

- первое значение;
- последнее значение;
- шаг;
- количество итераций.

Типичная ошибка с правой границей

Рассмотрим задачу:

> Вывести числа от 1 до 10.

Можно ошибочно написать:

```
for number in range(1, 10):
    print(number)
```

Результат закончится на:

```
9
```

Потому что 10 не входит в `range(1, 10)`.

Правильно:

```
for number in range(1, 11):
    print(number)
```

⚠ Помните о stop

Чтобы получить:

```
1, 2, 3, ..., 10
```

нужно:

```
range(1, 11)
```

Значение `stop` не включается.

Сумма чисел с for

В предыдущей главе мы использовали накопитель:

```
total = 0
```

С `for` он работает так же.

Например, найдём сумму чисел от 1 до 5:

```
total = 0

for number in range(1, 6):
    total += number

print("Сумма:", total)
```

Список 2.5 `sum-with-for.py`

```
total = 0

for number in range(1, 6):
    total += number

print("Сумма:", total)
```

Результат:

```
Сумма: 15
```

На каждой итерации:

```
number = 1 → total = 1
number = 2 → total = 3
number = 3 → total = 6
number = 4 → total = 10
number = 5 → total = 15
```

Накопитель внутри for

Здесь у переменных разные роли:

```
number
```

получает очередное значение из `range()`.

А:

```
total
```

сохраняет накопленный результат.

Код:

```
total = 0

for number in range(1, 6):
    total += number
```

можно читать:

> Для каждого числа от 1 до 5 прибавить это число к общей сумме.

Вычисления внутри цикла

Внутри `for` можно выполнять любые знакомые операции.

Например, вывести квадраты чисел:

```
for number in range(1, 6):
    square = number ** 2

    print(number, "→", square)
```

Результат:

```
1 → 1
2 → 4
3 → 9
4 → 16
5 → 25
```

Условия внутри for

Внутри цикла можно использовать `if`.

Например:

```
for number in range(1, 6):
    if number == 3:
        print("Нашли тройку")

    print(number)
```

Результат:

```
1
2
Нашли тройку
3
4
5
```

На каждой итерации Python получает новое значение `number`, а затем может проверить его условием.

Общая схема:

```
следующее значение
        ↓
       for
        ↓
       if
        ↓
проверить значение
        ↓
выполнить нужное действие
```

Условие может зависеть от каждого значения

Например, определим, какие числа делятся на 2 без остатка:

```
for number in range(1, 11):
    if number % 2 == 0:
        print(number)
```

Список 2.6 `for-with-if.py`

```
for number in range(1, 11):
    if number % 2 == 0:
        print(number)
```

Результат:

```
2
4
6
8
10
```

Мы уже знаем:

```
number % 2 == 0
```

означает, что число делится на 2 без остатка.

Теперь эта проверка выполняется для каждого числа.

Таблица умножения для одного числа

Цикл удобно использовать для повторяющихся вычислений.

Например:

```
number = int(input("Введите число: "))

for multiplier in range(1, 11):
    result = number * multiplier

    print(number, "*", multiplier, "=", result)
```

Если пользователь введёт:

Список 2.7 `multiplication-table.py`

```
number = int(input("Введите число: "))

for multiplier in range(1, 11):
    result = number * multiplier

    print(number, "*", multiplier, "=", result)
```

```
5
```

результат:

```
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
```

Здесь `for` выполняет одно и то же вычисление с разными значениями `multiplier`.

Повторный input() с for

Если заранее известно количество вводов, `for` тоже может быть удобнее `while`.

Например, пользователь должен ввести три числа:

```
total = 0

for attempt in range(3):
    number = float(input("Введите число: "))
    total += number

print("Сумма:", total)
```

Цикл выполнится ровно:

```
3 раза
```

Переменная цикла после завершения

Рассмотрим:

```
for number in range(1, 4):
    print(number)

print("Последнее значение:", number)
```

После цикла `number` содержит последнее полученное значение:

```
3
```

Но обычно переменная цикла нужна прежде всего внутри самого перебора.

Не стоит строить программу вокруг её значения после завершения цикла без необходимости.

---

range() задаёт числа по правилам

Полезно собрать основные формы `range()` вместе.

Только stop

```
range(5)
```

значения:

```
0, 1, 2, 3, 4
```

start и stop

```
range(2, 6)
```

значения:

```
2, 3, 4, 5
```

start, stop и step

```
range(2, 11, 2)
```

значения:

```
2, 4, 6, 8, 10
```

Отрицательный шаг

```
range(5, 0, -1)
```

значения:

```
5, 4, 3, 2, 1
```

---

range() не включает stop

Это правило работает и для отрицательного шага.

Например:

```
range(5, 0, -1)
```

не включает:

```
0
```

А:

```
range(5, -1, -1)
```

даст:

```
5
4
3
2
1
0
```

Потому что правая граница теперь:

```
-1
```

Пустой range

Некоторые значения `range()` могут не содержать ни одного числа.

Например:

```
for number in range(5, 1):
    print(number)
```

ничего не выведет.

По умолчанию Python пытается двигаться вверх:

```
+1
```

Но начало:

```
5
```

уже больше правой границы:

```
1
```

Поэтому последовательность пуста.

Не нужно изменять переменную цикла вручную

С `while` мы писали:

```
number += 1
```

Но в `for` это обычно не требуется.

Правильно:

```
for number in range(1, 6):
    print(number)
```

Не нужно добавлять:

```
number += 1
```

для перехода к следующей итерации.

`for` сам получает следующее значение из `range()`.

> ⚠ for сам управляет перебором
>
> В цикле:
>
> ```
> for number in range(1, 6):
> ```
>
> переменная `number` автоматически получает очередное значение.
> Не нужно вручную увеличивать её, как счётчик `while`.

---

for — это не просто счётчик

Важно не воспринимать `for` только как сокращённый `while`.

Например:

```
for letter in "Python":
    print(letter)
```

Здесь никакого числового счётчика вообще нет.

`for` получает **следующий элемент** последовательности.

Именно поэтому его удобно читать так:

```
для каждого элемента
```

Сравним while и for

Одна задача через `while`:

```
number = 1

while number <= 5:
    print(number)
    number += 1
```

Через `for`:

```
for number in range(1, 6):
    print(number)
```

В `while` мы управляем:

```
начальным значением
условием
изменением
```

В `for` мы задаём:

```
последовательность значений
```

а перебор выполняется автоматически.

Попробуйте сами

> Эксперимент
>
> Запустите:

```
for number in range(1, 11):
    print(number)
```

Затем измените программу так, чтобы она выводила:

```
0, 1, 2, 3, 4
```

затем:

```
2, 4, 6, 8, 10
```

затем:

```
10, 8, 6, 4, 2
```

Перед запуском определите:

- `start`;
- `stop`;
- `step`.

Типичные ошибки

Неправильная правая граница

Задача:

```
вывести 1..10
```

Ошибка:

```
range(1, 10)
```

Правильно:

```
range(1, 11)
```

Неправильный шаг

Ошибка:

```
range(5, 0, 1)
```

Для движения вниз нужен:

```
range(5, 0, -1)
```

Лишнее изменение переменной

Не нужно писать:

```
for number in range(5):
    number += 1
```

чтобы цикл перешёл к следующему значению.

`for` делает это самостоятельно.

Неправильный отступ

Неправильно:

```
for number in range(5):
print(number)
```

Правильно:

```
for number in range(5):
    print(number)
```

Пошаговый разбор for

Рассмотрим:

```
total = 0

for number in range(1, 4):
    total += number
```

Пошагово:

Итерация	`number`	`total` до	`total` после
1	1	0	1
2	2	1	3
3	3	3	6

После этого значения в `range(1, 4)` заканчиваются.

Цикл завершается.

Пример: сумма покупок

Допустим, пользователь должен ввести стоимость трёх покупок.

```
total = 0

for purchase in range(3):
    price = float(input("Стоимость покупки: "))
    total += price

print("Общая сумма:", total)
```

Список 2.8 `purchase-sum.py`

```
total = 0

for purchase in range(3):
    price = float(input("Стоимость покупки: "))
    total += price

print("Общая сумма:", total)
```

Пример:

```
Стоимость покупки: 100
Стоимость покупки: 250
Стоимость покупки: 50
Общая сумма: 400.0
```

Здесь нам заранее известно:

```
нужно получить ровно 3 значения
```

Поэтому `for` подходит очень хорошо.

Пример: символы слова

Пользователь вводит слово:

```
word = input("Введите слово: ")

for letter in word:
    print(letter)
```

Список 2.9 `string-iteration.py`

```
word = input("Введите слово: ")

for letter in word:
    print(letter)
```

Например:

```
Введите слово: Code
C
o
d
e
```

Количество итераций зависит от количества символов в строке.

Нам не нужно знать его заранее.

Сам `for` перебирает строку до конца.

Что важно запомнить

Цикл `for` выполняет блок кода для каждого значения или элемента последовательности.

Основная форма:

```
for переменная in последовательность:
    действие
```

Например:

```
for number in range(1, 6):
    print(number)
```

На каждой итерации:

```
следующее значение
↓
переменная цикла
↓
тело цикла
↓
следующее значение
```

Основные формы `range()`:

```
range(stop)

range(start, stop)

range(start, stop, step)
```

Например:

```
range(5)
→ 0, 1, 2, 3, 4

range(2, 6)
→ 2, 3, 4, 5

range(2, 11, 2)
→ 2, 4, 6, 8, 10

range(5, 0, -1)
→ 5, 4, 3, 2, 1
```

Важно:

> **Значение `stop` в `range()` не включается.**

`for` можно использовать не только с числами:

```
for letter in "Python":
    print(letter)
```

Главное различие между циклами:

```
while
→ повторять, пока условие True

for
→ выполнить для каждого элемента или значения
```

И самое важное:

> **`for` позволяет программе последовательно обработать все значения некоторой последовательности без ручного управления счётчиком.**

Практические задания

Задание 1. Числа от 1 до 10

С помощью `for` выведите:

```
1
2
3
4
5
6
7
8
9
10
```

i Ответ

```
for number in range(1, 11):
    print(number)
```

Задание 2. Пять сообщений

Выведите строку:

```
Изучаю Python
```

ровно пять раз.

> Ответ

```
for number in range(5):
    print("Изучаю Python")
```

Цикл выполнится пять раз, потому что:

```
range(5)
→ 0, 1, 2, 3, 4
```

Задание 3. Чётные числа

Выведите:

```
2
4
6
8
10
```

Используйте шаг `range()`.

> Ответ

```
for number in range(2, 11, 2):
    print(number)
```

Задание 4. Обратный отсчёт

Выведите:

```
5
4
3
2
1
Старт!
```

Ответ

```
for number in range(5, 0, -1):
    print(number)

print("Старт!")
```

Задание 5. Квадраты чисел

Выведите квадраты чисел от 1 до 5.

Пример:

```
1 → 1
2 → 4
3 → 9
4 → 16
5 → 25
```

Ответ

```
for number in range(1, 6):
    square = number ** 2

    print(number, "→", square)
```

Задание 6. Сумма от 1 до 10

С помощью `for` вычислите:

```
1 + 2 + 3 + ... + 10
```

Ответ

```
total = 0

for number in range(1, 11):
    total += number

print("Сумма:", total)
```

Результат:

```
Сумма: 55
```

Задание 7. Таблица умножения

Пользователь вводит число.

Выведите таблицу умножения этого числа от 1 до 10.

Например:

```
Введите число: 3

3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
...
3 * 10 = 30
```

Ответ

```
number = int(input("Введите число: "))

for multiplier in range(1, 11):
    result = number * multiplier

    print(number, "*", multiplier, "=", result)
```

Задание 8. Символы строки

Пользователь вводит слово.

Выведите каждый символ на отдельной строке.

Ответ

```
word = input("Введите слово: ")

for letter in word:
    print(letter)
```

Задание 9. Сумма покупок

Пользователь вводит стоимость 4 покупок.

Найдите общую сумму.

Ответ

```
total = 0

for purchase in range(4):
    price = float(input("Стоимость покупки: "))
    total += price

print("Общая сумма:", total)
```

Задание 10. Найдите ошибку

Нужно вывести числа:

```
5
4
3
2
1
```

Но программа:

```
for number in range(5, 0):
    print(number)
```

ничего не выводит.

Почему?

Исправьте её.

Ответ

По умолчанию `range()` использует шаг:

```
+1
```

Но нам нужно идти от 5 вниз к 1.
Поэтому требуется отрицательный шаг:

```
for number in range(5, 0, -1):
    print(number)
```

Результат:

```
5
4
3
2
1
```

Задание 11. Предскажите результат

Не запускайте код сразу.

```
for number in range(2, 9, 3):
    print(number)
```

Ответьте:

1. Какие числа будут выведены?
2. Сколько будет итераций?

Ответ

Значения:

```
2
5
8
```

Потому что:

```
2
↓ +3
5
↓ +3
8
↓ +3
11
```

11 уже находится за границей 9.
Поэтому цикл выполняет:

```
3 итерации
```

Задание 12. Задача со звёздочкой

Пользователь вводит число.

Выведите все числа от 1 до этого числа, которые делятся на 3 без остатка.

Например:

```
Введите число: 15

3
6
9
12
15
```

Используйте `for`, `range()`, `%` и `if`.

Ответ

```
limit = int(input("Введите число: "))

for number in range(1, limit + 1):
    if number % 3 == 0:
        print(number)
```

Здесь:

```
limit + 1
```

используется потому, что правая граница `range()` не включается.
Для:

```
limit = 15
```

нужно задать последовательность:

```
range(1, 16)
```

чтобы число 15 тоже участвовало в переборе.

Что дальше?

Теперь программа умеет повторять действия для каждого значения или элемента последовательности.

Следующий шаг:

```
перебор символов строки
↓
подробная работа со строками
```

Следующая глава: **Строки**.

2 3.11 Строки

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как Python хранит текст и как работать с отдельными символами строки?**

Со строками мы работаем почти с самого начала курса.

Например:

```
name = "Анна"
city = "Москва"
language = "Python"
```

Функция `input()` тоже возвращает строку:

```
name = input("Введите имя: ")
```

Мы уже сравнивали строки:

```
password == "python123"
```

и даже перебирали их через `for`:

```
for letter in "Python":
    print(letter)
```

Теперь разберём строки подробнее.

Что такое строка

Строка — это значение типа:

```
str
```

Она используется для хранения текста.

Например:

```
language = "Python"

print(language)
print(type(language))
```

Результат:

```
Python
<class 'str'>
```

Строка может содержать:

- буквы;
- цифры;
- пробелы;
- знаки препинания;
- специальные символы.

Например:

```
text = "Python 3.14!"
```

Всё значение целиком имеет тип:

```
str
```

Строка состоит из символов

Рассмотрим:

```
Python
```

Строка состоит из отдельных символов:

```
P y t h o n
```

Можно представить её так:

```
"P" "y" "t" "h" "o" "n"
```

Каждый символ занимает определённую позицию.

Именно поэтому Python позволяет получать отдельные символы строки.

Индексы

Позиция символа называется **индексом**.

Но Python начинает считать не с 1, а с:

```
0
```

Для строки:

```
Python
```

индексы выглядят так:

```
символ:   P   y   t   h   o   n
индекс:   0   1   2   3   4   5
```

Поэтому первый символ имеет индекс:

```
0
```

а не:

```
1
```

> ❗ Индексация начинается с нуля
>
> Для строки:

```
word = "Python"
```

первый символ:

```
word[0]
```

второй:

```
word[1]
```

третий:

```
word[2]
```

Это правило встречается в Python очень часто.

Получение символа по индексу

Чтобы получить символ, после имени строки используются квадратные скобки:

```
word = "Python"

print(word[0])
```

Результат:

```
P
```

Другие примеры:

Список 2.1 `string-indexing.py`

```
word = "Python"

print(word[0])
print(word[1])
print(word[2])
print(word[5])
```

Результат:

```
P
y
t
n
```

Индекс может храниться в переменной

Не обязательно писать число прямо внутри скобок.

Например:

```
word = "Python"
index = 3

print(word[index])
```

Результат:

```
h
```

Это особенно полезно, когда позиция вычисляется программой.

Последний символ

Для строки:

```
word = "Python"
```

последний символ имеет индекс:

```
5
```

Можно написать:

```
print(word[5])
```

Но для строки другой длины индекс будет уже другим.

Python предоставляет более удобный способ:

```
print(word[-1])
```

Результат:

```
n
```

Отрицательные индексы считаются с конца строки.

Отрицательные индексы

Для строки:

```
Python
```

можно представить:

```
символ:    P   y   t   h   o   n
индекс:    0   1   2   3   4   5
отриц.:   -6  -5  -4  -3  -2  -1
```

Например:

Список 2.2 `negative-indexing.py`

```
word = "Python"

print(word[-1])
print(word[-2])
print(word[-3])
```

Результат:

```
n
o
h
```

> **Когда удобны отрицательные индексы**
>
> Если нужен последний символ строки, обычно проще написать:
>
> ```
> word[-1]
> ```
>
> чем сначала вычислять длину строки.



Рисунок 2.1: Положительные и отрицательные индексы строки

Ошибка индекса

Что произойдёт здесь?

```
word = "Python"

print(word[10])
```

В строке нет символа с индексом 10.

Поэтому Python сообщит об ошибке:

```
IndexError
```

⚠ Индекс должен существовать

Для строки:

```
"Python"
```

доступны положительные индексы:

```
0..5
```

Попытка обратиться к несуществующей позиции приводит к:

```
IndexError
```

Длина строки

Чтобы узнать количество символов, используется функция:

```
len()
```

Например:

```
word = "Python"

print(len(word))
```

Результат:

```
6
```

Строка содержит шесть символов:

```
P
y
t
h
o
n
```

Пробел тоже является символом

Рассмотрим:

```
text = "Hello Python"
```

Проверим длину:

```
print(len(text))
```

Результат:

```
12
```

Почему не 11?

Потому что пробел между словами тоже является символом.

Можно представить:

```
H e l l o _ P y t h o n
```

Здесь _ условно показывает пробел.

Пустая строка

Строка может вообще не содержать символов:

```
text = ""
```

Её длина:

```
print(len(text))
```

Результат:

```
0
```

Такая строка называется **пустой строкой**.

Мы уже встречали её раньше:

```
bool("")
```

даёт:

```
False
```

Проверьте в HTML-версии три случая: обычное слово, строку с пробелом и пустую строку.

Список 2.3 `string-length.py`

```
word = "Python"
text = "Hello Python"
empty = ""

print(len(word))
print(len(text))
print(len(empty))
```

Последний индекс и длина строки

Если длина строки:

```
len(word)
```

равна 6, последний индекс равен:

```
5
```

То есть:

```
последний индекс = длина - 1
```

Например:

```
word = "Python"

last_index = len(word) - 1

print(last_index)
print(word[last_index])
```

Результат:

```
5
n
```

Хотя для последнего символа проще использовать:

```
word[-1]
```

Перебор строки через for

В предыдущей главе мы уже видели:

```
for letter in "Python":
    print(letter)
```

Теперь понятно, что `for` получает каждый символ строки по очереди.

```
"P"
↓
"y"
↓
"t"
↓
"h"
↓
"o"
↓
"n"
```

Полный пример:

Список 2.4 `string-iteration.py`

```
word = input("Введите слово: ")

for letter in word:
    print(letter)
```

Например:

```
Введите слово: Python
P
y
t
h
o
n
```

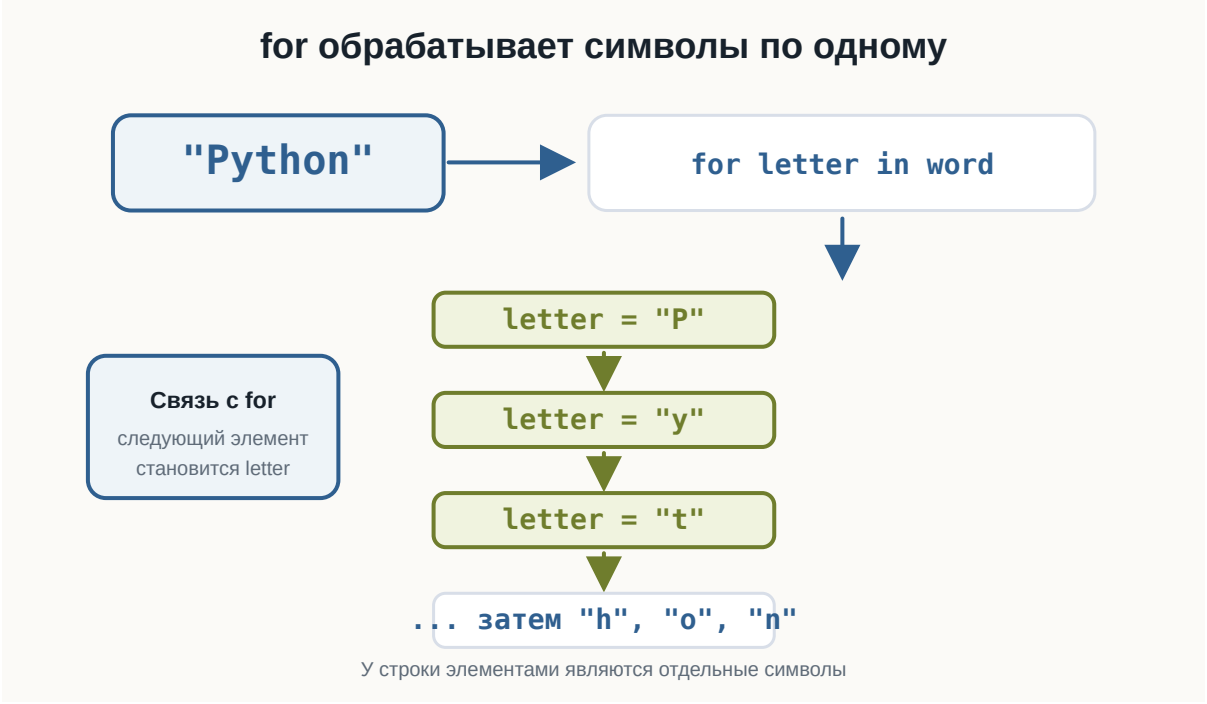


Рисунок 2.2: Перебор строки через цикл for

Символ тоже является строкой

Рассмотрим:

```
word = "Python"
letter = word[0]

print(letter)
print(type(letter))
```

Результат:

```
P
<class 'str'>
```

В Python нет отдельного типа данных для одного символа.

Даже один символ имеет тип:

```
str
```

То есть:

```
"P"
```

— это строка длиной 1.

Проверка длины одного символа

Например:

```
letter = "P"

print(len(letter))
```

Результат:

```
1
```

Получается:

```
""       → длина 0
"P"      → длина 1
"Python" → длина 6
```

Объединение строк

Мы уже знаем, что оператор:

```
+
```

для строк означает объединение.

Например:

```
first_name = "Анна"
last_name = "Иванова"

full_name = first_name + last_name

print(full_name)
```

Результат:

```
АннаИванова
```

Между строками нет пробела.

Его нужно добавить самостоятельно:

```
full_name = first_name + " " + last_name

print(full_name)
```

Результат:

```
Анна Иванова
```

Строки можно умножать

Для строк оператор:

```
*
```

может означать повторение.

Например:

```
print("Python " * 3)
```

Результат:

```
Python Python Python
```

Другой пример:

```
line = "-" * 20

print(line)
```

Результат:

```
--------------------
```

Это удобно для простого текстового оформления.

Строку нельзя изменить по индексу

Рассмотрим:

```
word = "Python"

word[0] = "J"
```

Можно ожидать:

```
Jython
```

Но Python сообщит об ошибке.

Строки в Python **неизменяемые**.

Это означает, что нельзя изменить отдельный символ существующей строки.

> ❗ Строки неизменяемы
>
> Можно получить символ:

```
word[0]
```

но нельзя написать:

```
word[0] = "J"
```

Чтобы получить другую строку, нужно создать новое значение.

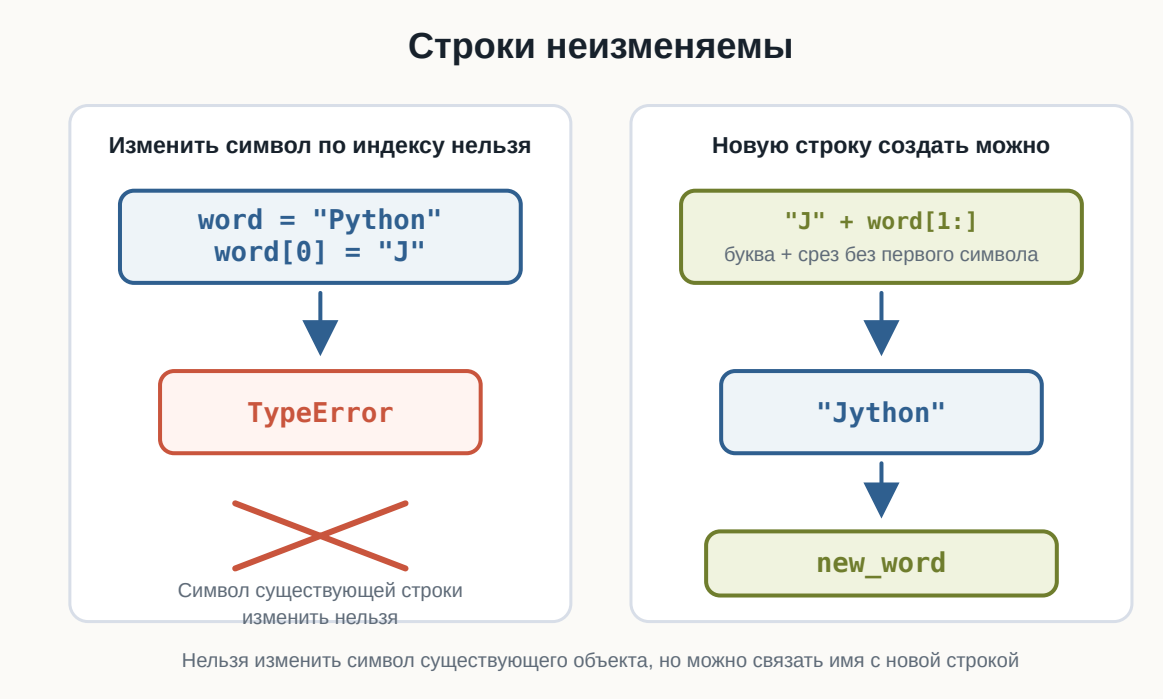


Рисунок 2.3: Неизменяемость строк

Новую строку создать можно

Хотя изменить существующую строку нельзя, можно создать новую:

```
word = "Python"

new_word = "J" + word[1:]

print(new_word)
```

Результат:

```
Jython
```

Здесь появилась новая операция:

```
word[1:]
```

Это **срез строки**.

Разберём его подробнее.

Срез строки

Срез позволяет получить часть строки.

Синтаксис:

```
text[start:stop]
```

Например:

```
word = "Python"

print(word[0:3])
```

Результат:

```
Pyt
```

Python берёт символы с индекса:

```
0
```

до:

```
3
```

но индекс 3 не включается.

Получается:

```
0 → P
1 → y
2 → t
```

Список 2.5 `string-slicing.py`

```
word = "Python"

print(word[0:3])
print(word[:3])
print(word[2:])
print(word[:-1])
print(word[-3:])
print(word[::2])
```

Правая граница среза не включается

Это похоже на `range()`.

Для:

```
word[0:3]
```

получаем символы:

```
0
1
2
```

Индекс:

```
3
```

уже не входит.

> **i** Похоже на range()
>
> В:
>
> ```
> range(1, 5)
> ```

число 5 не включается.
В:

```
text[1:5]
```

символ с индексом 5 тоже не включается.
И в `range()`, и в срезах правая граница `stop` не входит в результат.



Рисунок 2.4: Срез строки с правой границей

Срез с начала строки

Если `start` не указан:

```
word[:3]
```

Python начинает с первого символа.

Например:

```
word = "Python"

print(word[:3])
```

Результат:

```
Pyt
```

Это то же самое, что:

```
word[0:3]
```

Срез до конца строки

Если не указан `stop`:

```
word[2:]
```

Python берёт строку от указанного индекса до конца.

Например:

```
word = "Python"

print(word[2:])
```

Результат:

```
thon
```

Копия всей строки

Можно написать:

```
word[:]
```

Например:

```
word = "Python"

print(word[:])
```

Результат:

```
Python
```

Отрицательные индексы в срезах

Отрицательные индексы можно использовать и в срезах.

Например:

```
word = "Python"

print(word[:-1])
```

Результат:

```
Pytho
```

То есть:

> взять всё, кроме последнего символа.

А:

```
print(word[-3:])
```

даст:

```
hon
```

Срез с шагом

Как и `range()`, срез может иметь шаг:

```
text[start:stop:step]
```

Например:

```
word = "Python"

print(word[::2])
```

Результат:

```
Pto
```

Берутся символы с шагом 2:

```
индекс 0 → P
индекс 2 → t
индекс 4 → o
```

Строка в обратном порядке

Срез с отрицательным шагом позволяет идти справа налево.

Например:

Список 2.6 `reverse-string.py`

```
word = "Python"

print(word[::-1])
```

Результат:

```
nohtyP
```

Здесь шаг:

```
-1
```

означает движение по строке в обратном направлении.

> Полезный приём
>
> Запись:

```
text[::-1]
```

возвращает строку в обратном порядке.
Например:

```
"Python"[::-1]
```

даёт:

```
nohtyP
```

Проверка наличия текста с in

Оператор:

```
in
```

можно использовать для проверки наличия подстроки.

Например:

```
text = "Изучаю Python"

print("Python" in text)
```

Результат:

```
True
```

А:

```
print("Java" in text)
```

даёт:

```
False
```

Проверка отсутствия с not in

Можно проверить и обратное:

```
text = "Изучаю Python"

print("Java" not in text)
```

Результат:

```
True
```

Получается:

```
in
→ содержится?

not in
→ не содержится?
```

Список 2.7 `string-membership.py`

```
text = "Изучаю Python"

print("Python" in text)
print("Java" in text)
print("Java" not in text)
```

---

in внутри if

Например:

```
text = input("Введите сообщение: ")

if "Python" in text:
    print("В сообщении упоминается Python")
else:
    print("Python не найден")
```

Здесь оператор `in` возвращает `True` или `False`, а `if` использует этот результат.

Регистр по-прежнему имеет значение

Рассмотрим:

```
text = "Изучаю Python"

print("Python" in text)
print("python" in text)
```

Результат:

```
True
False
```

Для Python:

```
Python
```

и:

```
python
```

— разные последовательности символов.

Методы строк

Кроме операторов, строки имеют специальные операции, которые вызываются через точку.

Например:

```
text.upper()
```

Такие операции называются **методами**.

Общая форма:

```
строка.метод()
```

Познакомимся с несколькими часто используемыми методами.

Список 2.8 `string-methods.py`

```
text = "  Python Programming  "

print(text.upper())
print(text.lower())
print(text.strip())
print(text.replace("Python", "Java"))

word = "banana"

print(word.count("a"))
print(word.find("na"))

filename = "report.pdf"

print(filename.startswith("report"))
print(filename.endswith(".pdf"))
```

upper()

Метод:

```
upper()
```

создаёт строку в верхнем регистре.

Например:

```
text = "Python"

result = text.upper()

print(result)
```

Результат:

```
PYTHON
```

Исходная строка при этом не изменяется:

```
print(text)
```

по-прежнему:

```
Python
```

Это снова связано с неизменяемостью строк.

---

lower()

Метод:

```
lower()
```

создаёт строку в нижнем регистре:

```
text = "PyThOn"

print(text.lower())
```

Результат:

```
python
```

Это удобно, например, для сравнения пользовательского ввода:

```
answer = input("Продолжить? yes/no: ")

if answer.lower() == "yes":
    print("Продолжаем")
```

Теперь сработают:

```
yes
YES
Yes
YeS
```

---

strip()

Пользователь иногда случайно вводит лишние пробелы:

```
   Python
```

Метод:

```
strip()
```

убирает пробелы по краям строки.

Например:

```
text = "   Python   "

print(text.strip())
```

Результат:

```
Python
```

Пробелы внутри строки не удаляются.

Несколько методов подряд

Методы можно применять последовательно.

Например:

```
answer = input("Продолжить? ")

answer = answer.strip().lower()

print(answer)
```

Если пользователь введёт:

```
   YES
```

результат:

```
yes
```

Это полезно для нормализации пользовательского ввода.

---

replace()

Метод:

```
replace()
```

создаёт новую строку, заменяя один фрагмент другим.

Например:

```
text = "Я изучаю Java"

new_text = text.replace("Java", "Python")

print(new_text)
```

Результат:

```
Я изучаю Python
```

Исходная строка:

```
text
```

при этом не изменяется.

---

count()

Метод:

```
count()
```

позволяет подсчитать количество вхождений.

Например:

```
text = "banana"

print(text.count("a"))
```

Результат:

```
3
```

А:

```
print(text.count("na"))
```

даёт:

```
2
```

---

find()

Метод:

```
find()
```

позволяет найти позицию подстроки.

Например:

```
text = "Python"

print(text.find("t"))
```

Результат:

```
2
```

Потому что:

```
P → 0
y → 1
t → 2
```

Если фрагмент не найден:

```
print(text.find("Java"))
```

результат:

```
-1
```

> find() и in отвечают на разные вопросы
>
> ```
> "Py" in text
> ```
>
> отвечает:
>
> содержится ли фрагмент?
>
> А:
>
> ```
> text.find("Py")
> ```
>
> отвечает:
>
> с какого индекса начинается фрагмент?
>
> Если нужна только проверка наличия, обычно понятнее использовать `in`.

---

startswith()

Метод:

```
startswith()
```

проверяет, начинается ли строка с определённого текста.

Например:

```
filename = "report.pdf"

print(filename.startswith("report"))
```

Результат:

```
True
```

---

endswith()

Метод:

```
endswith()
```

проверяет окончание строки.

Например:

```
filename = "report.pdf"

print(filename.endswith(".pdf"))
```

Результат:

```
True
```

Так можно проверять, например, расширение файла.

Методы возвращают новые значения

Рассмотрим:

```
text = "Python"

text.upper()

print(text)
```

Результат:

```
Python
```

Почему не:

```
PYTHON
```

Потому что:

```
text.upper()
```

создало новое значение, но мы его никуда не сохранили.

Нужно:

```
text = text.upper()

print(text)
```

Теперь:

```
PYTHON
```

⚠ Методы обычно не изменяют исходную строку

Такой вызов:

```
text.upper()
```

сам по себе не изменяет `text`.
Если новый результат нужен дальше, сохраните его:

```
text = text.upper()
```

или:

```
new_text = text.upper()
```

Подсчёт символов через for

Можно посчитать символы самостоятельно:

```
text = "Python"
count = 0

for letter in text:
    count += 1

print(count)
```

Результат:

```
6
```

Но для длины строки уже есть:

```
len(text)
```

Такой цикл полезен не для замены `len()`, а для понимания того, как перебирается строка.

Подсчёт определённого символа

Например:

Список 2.9 `count-character.py`

```
text = input("Введите текст: ")
count = 0

for letter in text:
    if letter == "a":
        count += 1

print("Количество:", count)
```

Например:

```
Введите текст: banana
Количество: 3
```

Это та же идея, что:

```
text.count("a")
```

Но цикл показывает сам алгоритм подсчёта.

Поиск символа внутри цикла

Например:

```
word = "Python"

for letter in word:
    if letter == "t":
        print("Символ найден")
```

Когда `letter` становится:

```
t
```

условие выполняется.

Позже мы научимся управлять циклом более гибко, когда нужный символ уже найден.

Пример: количество гласных

Рассмотрим простой пример:

```
word = input("Введите слово: ")
count = 0

for letter in word.lower():
    if letter in "aeiou":
```

```
        count += 1

print("Количество гласных:", count)
```

Например:

```
Введите слово: Python
Количество гласных: 1
```

Здесь объединяются:

```
input()
↓
строка
↓
lower()
↓
for
↓
in
↓
if
↓
счётчик
```

Пример: проверка начала и конца

Список 2.10 `filename-check.py`

```
filename = input("Введите имя файла: ")

if filename.endswith(".py"):
    print("Python-файл")
else:
    print("Не Python-файл")
```

Например:

```
Введите имя файла: main.py
Python-файл
```

Пример: нормализация ответа

Пользователь может написать:

```
YES
Yes
 yes
   YES
```

Можно привести ввод к единому виду:

Список 2.11 `normalize-input.py`

```
answer = input("Продолжить? yes/no: ")

answer = answer.strip().lower()

if answer == "yes":
    print("Продолжаем")
else:
    print("Останавливаемся")
```

Это один из самых практичных способов работы со строками пользовательского ввода.

Что лучше: индекс или for

Если нужен конкретный символ:

```
word[0]
```

удобно использовать индекс.

Если нужно обработать все символы:

```
for letter in word:
```

обычно удобнее использовать `for`.

Например:

```
print(word[0])
```

отвечает:

> какой первый символ?

А:

```
for letter in word:
```

означает:

> обработать каждый символ.

Что важно запомнить

Строка — это значение типа:

```
str
```

Она состоит из символов.

Индексы начинаются с:

```
0
```

Например:

```
Python

P y t h o n
0 1 2 3 4 5
```

Отрицательные индексы идут с конца:

```
-6 -5 -4 -3 -2 -1
```

Получение символа:

```
word[0]
word[-1]
```

Длина строки:

```
len(word)
```

Срез:

```
word[start:stop]
```

или:

```
word[start:stop:step]
```

Правая граница `stop` не включается.

Строки неизменяемые:

```
word[0] = "J"
```

не работает.

Полезные операции:

```
"text" + "text"
"text" * 3
"Py" in text
"Java" not in text
```

Полезные методы:

```
upper()
lower()
strip()
replace()
count()
find()
startswith()
endswith()
```

И самое важное:

Строка — это последовательность символов, которую можно читать по индексам, получать частями и последовательно обрабатывать.

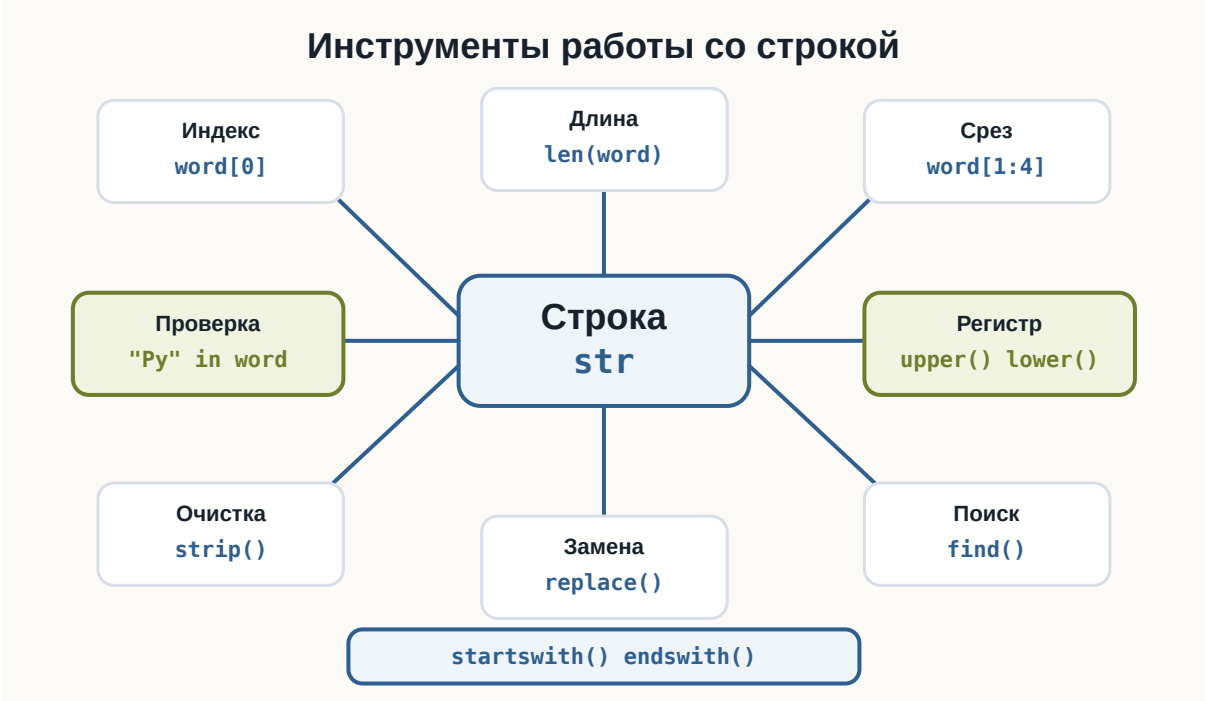


Рисунок 2.5: Карта работы со строками

Практические задания

Задание 1. Первый и последний символ

Пользователь вводит слово.

Выведите:

- первый символ;
- последний символ.

Ответ

```
word = input("Введите слово: ")

print("Первый символ:", word[0])
print("Последний символ:", word[-1])
```

Задание 2. Длина строки

Пользователь вводит текст.

Выведите количество символов.

Ответ

```
text = input("Введите текст: ")

print("Длина:", len(text))
```

Пробелы тоже учитываются как символы.

Задание 3. Первые три символа

Пользователь вводит строку.

Выведите первые три символа с помощью среза.

Ответ

```
text = input("Введите текст: ")

print(text[:3])
```

Задание 4. Строка без первого символа

Пользователь вводит слово.

Выведите всё слово, кроме первого символа.

Ответ

```
word = input("Введите слово: ")

print(word[1:])
```

Задание 5. Строка наоборот

Пользователь вводит слово.

Выведите его в обратном порядке.

Ответ

```
word = input("Введите слово: ")

print(word[::-1])
```

Задание 6. Проверка подстроки

Пользователь вводит сообщение.

Проверьте, содержится ли в нём слово:

```
Python
```

Ответ

```
text = input("Введите сообщение: ")

print("Python" in text)
```

Задание 7. Нормализация ответа

Пользователь отвечает на вопрос:

```
Продолжить? yes/no:
```

Ответ может содержать пробелы и любой регистр.

Сделайте так, чтобы:

```
YES
 yes
Yes
   YES
```

считались ответом `yes`.

Ответ

```
answer = input("Продолжить? yes/no: ")

answer = answer.strip().lower()

if answer == "yes":
    print("Продолжаем")
else:
    print("Останавливаемся")
```

Задание 8. Подсчёт символа

Пользователь вводит строку.

Подсчитайте, сколько раз в ней встречается буква:

```
a
```

Используйте `count()`.

Ответ

```
text = input("Введите текст: ")

print(text.count("a"))
```

Задание 9. Подсчёт символа через for

Решите предыдущую задачу без `count()`.

Используйте `for` и `if`.

Ответ

```
text = input("Введите текст: ")
count = 0

for letter in text:
    if letter == "a":
        count += 1

print(count)
```

Задание 10. Python-файл

Пользователь вводит имя файла.

Проверьте, заканчивается ли оно на:

```
.py
```

Ответ

```
filename = input("Введите имя файла: ")

if filename.endswith(".py"):
    print("Python-файл")
else:
    print("Не Python-файл")
```

Задание 11. Найдите ошибку

Почему этот код не работает?

```
word = "Python"

word[0] = "J"

print(word)
```

Ответ

Строки в Python неизменяемые.
Нельзя изменить отдельный символ:

```
word[0] = "J"
```

Можно создать новую строку:

```
word = "Python"

new_word = "J" + word[1:]

print(new_word)
```

Результат:

```
Jython
```

Задание 12. Предскажите результат

Не запускайте код сразу.

```
word = "Python"

print(word[0])
print(word[-1])
print(word[1:4])
print(word[:2])
print(word[2:])
print(word[::-1])
```

Ответ

Результат:

```
P
n
yth
Py
thon
nohtyP
```

Задание 13. Задача со звёздочкой

Пользователь вводит слово.

Подсчитайте количество гласных букв:

```
a
e
i
o
u
```

Регистр учитывать не нужно.

Например:

```
Введите слово: Education
Количество гласных: 5
```

Ответ

```
word = input("Введите слово: ")

count = 0

for letter in word.lower():
    if letter in "aeiou":
        count += 1

print("Количество гласных:", count)
```

Здесь:

```
word.lower()
```

приводит строку к нижнему регистру.
Затем цикл перебирает каждый символ.
Условие:

```
letter in "aeiou"
```

проверяет, является ли символ одной из гласных.

Что дальше?

Теперь мы умеем подробно работать с текстом:

```
строка
↓
символы
↓
индексы
↓
срезы
↓
for
↓
методы строк
```

В примерах этой главы код уже стал длиннее:

```
answer = answer.strip().lower()
```

и:

```
for letter in text:
    if letter == "a":
        count += 1
```

Следующий шаг:

```
повторяющийся фрагмент кода
↓
отдельная команда со своим именем
↓
функция
```

В следующей главе разберём **функции**.

2 3.12 Списки

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как хранить несколько значений в одной переменной и работать с ними как с единым набором?**

До сих пор одна переменная обычно хранила одно значение.

Например:

```
name = "Анна"
age = 25
price = 199.9
```

Но часто программе нужно работать сразу с несколькими значениями.

Например:

```
несколько имён
несколько цен
несколько результатов
несколько задач
несколько городов
```

Можно было бы создать много отдельных переменных:

```
city1 = "Москва"
city2 = "Казань"
city3 = "Самара"
```

Но такой подход быстро становится неудобным.

В Python для хранения нескольких значений существует **список**.

Первый список

Список создаётся с помощью квадратных скобок:

```
cities = ["Москва", "Казань", "Самара"]
```

Здесь одна переменная:

```
cities
```

содержит сразу три значения.

Выведем список:

```
cities = ["Москва", "Казань", "Самара"]

print(cities)
```

Результат:

```
['Москва', 'Казань', 'Самара']
```

Тип list

Списки имеют тип:

```
list
```

Проверим:

```
cities = ["Москва", "Казань", "Самара"]

print(type(cities))
```

Результат:

```
<class 'list'>
```

Получается:

```
["Москва", "Казань", "Самара"]
              ↓
            list
```

В HTML-версии можно сразу проверить несколько списков: со строками, с числами и пустой список.

Список 2.1 `list-basics.py`

```
languages = ["Python", "Java", "Go"]
numbers = [10, 20, 30]
empty = []

print(languages)
print(numbers)
print(empty)

print(type(languages))
print(len(languages))
print(len(empty))
```

Элементы списка

Значения внутри списка называются **элементами**.

Например:

```
numbers = [10, 20, 30]
```

Здесь три элемента:

```
10
20
30
```

В списке:

```
languages = ["Python", "Java", "Go"]
```

элементами являются строки.

Можно представить:

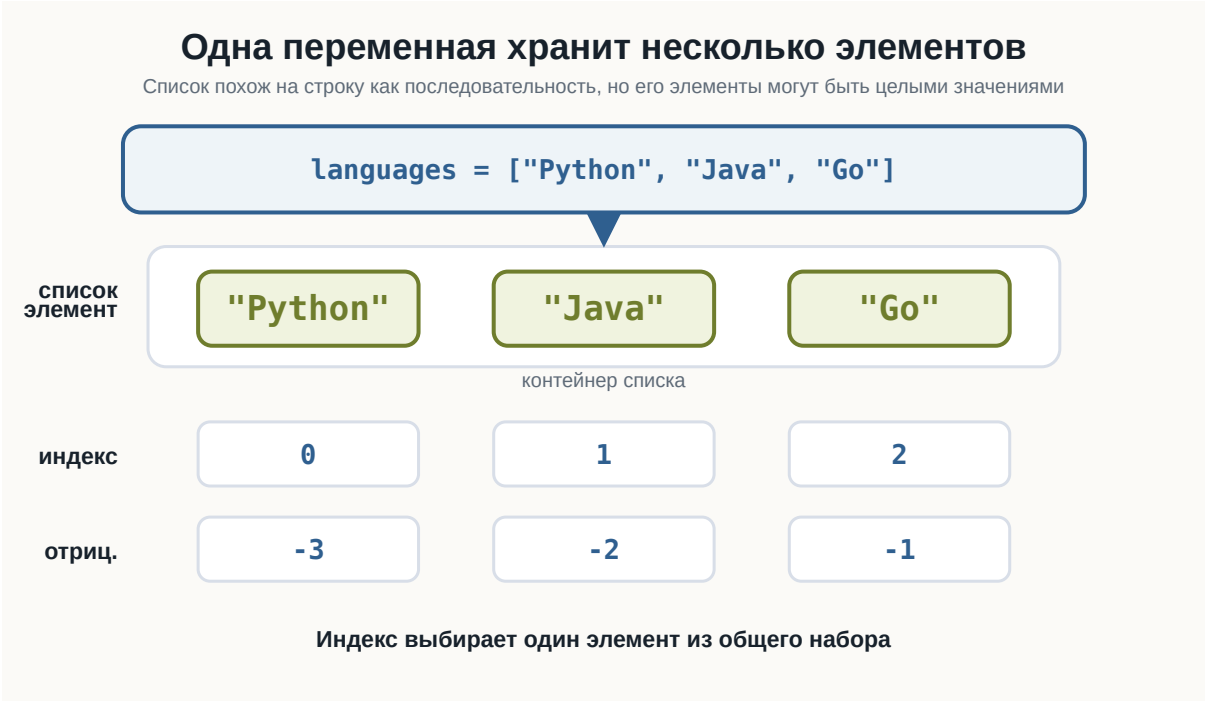


Рисунок 2.1: Устройство списка

В списке могут храниться числа

Например:

```
prices = [100, 250, 79, 500]

print(prices)
```

Результат:

```
[100, 250, 79, 500]
```

Или дробные числа:

```
temperatures = [18.5, 20.1, 17.8]
```

Список может содержать значения разных типов

Python позволяет написать:

```
data = ["Анна", 25, 1.68, True]
```

Здесь находятся:

```
str
int
float
bool
```

Такой список технически допустим.

Но в учебных и реальных программах часто удобнее, когда элементы списка имеют один общий смысл.

Например:

```
prices = [100, 250, 79]
```

или:

```
names = ["Анна", "Максим", "Олег"]
```

> Список лучше использовать для связанных значений
>
> Технически список может содержать разные типы:
>
> ```
> ["Анна", 25, True]
> ```
>
> Но обычно понятнее хранить вместе значения, которые относятся к одной задаче:
>
> ```
> names = ["Анна", "Максим", "Олег"]
> ```
>
> или:
>
> ```
> scores = [90, 75, 82]
> ```

Пустой список

Список может не содержать элементов:

```
items = []
```

Это **пустой список**.

Проверим:

```
items = []

print(items)
print(type(items))
```

Результат:

```
[]
<class 'list'>
```

Пустой список часто используется как начало набора, который будет заполняться позже.

Индексы списка

Как и строки, списки используют индексы.

Рассмотрим:

```
languages = ["Python", "Java", "Go"]
```

Индексы:

```
элемент:   Python   Java   Go
индекс:      0       1     2
```

Индексация снова начинается с:

```
0
```

Получение элемента по индексу

Например:

```
languages = ["Python", "Java", "Go"]

print(languages[0])
```

Результат:

```
Python
```

Другие примеры:

```
print(languages[1])
print(languages[2])
```

Результат:

```
Java
Go
```

Отрицательные индексы

Списки поддерживают отрицательные индексы так же, как строки.

Например:

```
languages = ["Python", "Java", "Go"]

print(languages[-1])
print(languages[-2])
```

Результат:

```
Go
Java
```

Проверьте положительные и отрицательные индексы в playground.

Для списка:

Список 2.2 `list-indexing.py`

```
languages = ["Python", "Java", "Go"]

print(languages[0])
print(languages[1])
print(languages[-1])
print(languages[-2])
```

```
["Python", "Java", "Go"]
```

можно представить:

```
индекс:       0       1      2
            Python  Java    Go
отриц.:      -3      -2     -1
```

Ошибка индекса

Если обратиться к несуществующему элементу:

```
languages = ["Python", "Java", "Go"]

print(languages[10])
```

возникнет:

```
IndexError
```

Причина та же, что и со строками: такого индекса нет.

Длина списка

Функция:

```
len()
```

работает и со списками.

Например:

```
languages = ["Python", "Java", "Go"]

print(len(languages))
```

Результат:

```
3
```

Пустой список:

```
items = []

print(len(items))
```

Результат:

```
0
```

Списки можно изменять

Здесь появляется важное отличие от строк.

Строки неизменяемые:

```
word = "Python"
word[0] = "J"
```

не работает.

Но элемент списка изменить можно:

```
languages = ["Python", "Java", "Go"]

languages[1] = "Rust"

print(languages)
```

Результат:

```
['Python', 'Rust', 'Go']
```

Список 2.3 `list-modification.py`

```
languages = ["Python", "Java", "Go"]

languages[1] = "Rust"

print(languages)
```

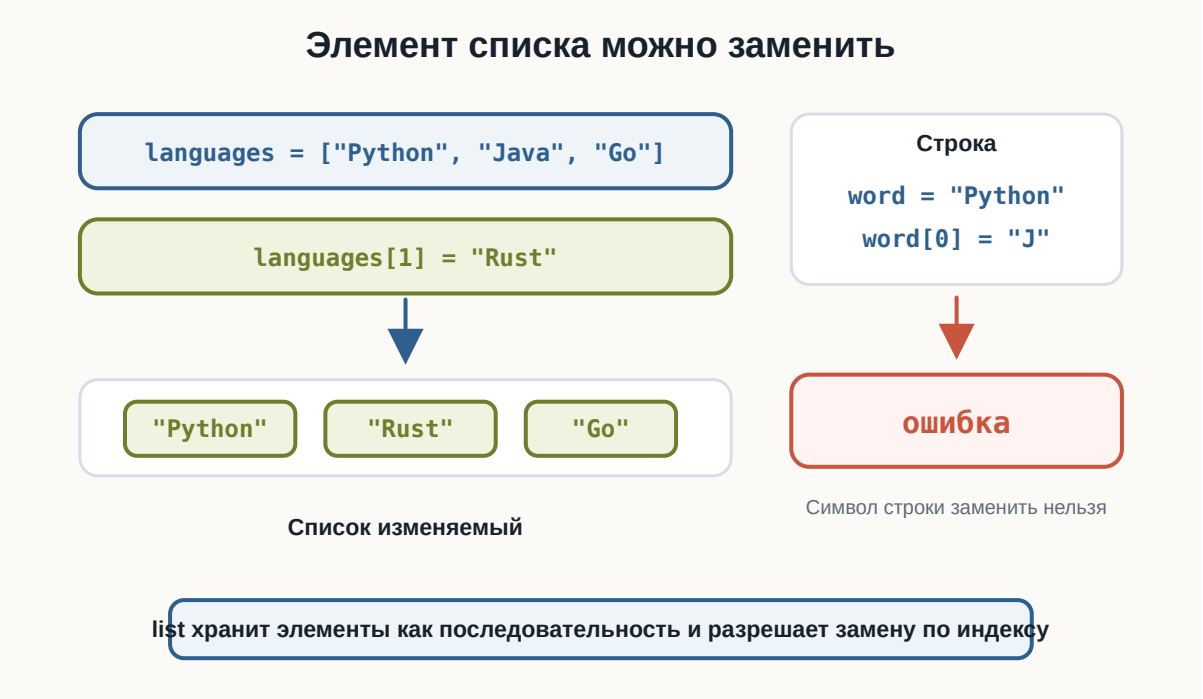


Рисунок 2.2: Изменяемость списка и строки

! Списки изменяемые

Строку нельзя изменить по индексу:

```
word[0] = "J"
```

А список можно:

```
languages[0] = "Rust"
```

Это одно из важных различий между `str` и `list`.

Изменение элемента

Например:

```
prices = [100, 200, 300]

prices[1] = 250

print(prices)
```

Результат:

```
[100, 250, 300]
```

Изменяется только выбранный элемент.

---

append()

Чтобы добавить новый элемент в конец списка, используется метод:

```
append()
```

Например:

```
languages = ["Python", "Java"]

languages.append("Go")

print(languages)
```

Результат:

```
['Python', 'Java', 'Go']
```

Можно представить:

```
["Python", "Java"]
          ↓
   append("Go")
          ↓
["Python", "Java", "Go"]
```

Добавление пользовательского значения

Например:

```
names = []

name = input("Введите имя: ")

names.append(name)

print(names)
```

Если пользователь введёт:

```
Анна
```

получим:

```
['Анна']
```

---

append() внутри цикла

Список особенно полезен вместе с циклами.

Например:

```
numbers = []

for i in range(3):
    number = int(input("Введите число: "))
    numbers.append(number)

print(numbers)
```

Если пользователь введёт:

```
10
20
30
```

результат:

```
[10, 20, 30]
```

Здесь список постепенно заполняется.

Попробуйте изменить добавляемые языки и ввести свои числа.

Список 2.4 `list-append.py`

```
languages = ["Python", "Java"]

languages.append("Go")

print(languages)

numbers = []

for i in range(3):
    number = int(input("Введите число: "))
    numbers.append(number)

print(numbers)
```



Рисунок 2.3: Рост списка через append

---

insert()

Метод:

```
insert()
```

позволяет добавить элемент в определённую позицию.

Например:

```
languages = ["Python", "Go"]

languages.insert(1, "Java")

print(languages)
```

Результат:

```
['Python', 'Java', 'Go']
```

Синтаксис:

```
список.insert(индекс, значение)
```

Разница между append() и insert()

`append()` добавляет в конец:

```
items.append(value)
```

`insert()` добавляет в указанную позицию:

```
items.insert(index, value)
```

Например:

```
numbers = [10, 30]

numbers.append(40)
```

получим:

```
[10, 30, 40]
```

А:

```
numbers.insert(1, 20)
```

даст:

```
[10, 20, 30, 40]
```

Удаление по значению: remove()

Метод:

```
remove()
```

удаляет элемент по его значению.

Например:

```
languages = ["Python", "Java", "Go"]

languages.remove("Java")

print(languages)
```

Результат:

```
['Python', 'Go']
```

---

remove() удаляет первое совпадение

Рассмотрим:

```
numbers = [10, 20, 10, 30]

numbers.remove(10)

print(numbers)
```

Результат:

```
[20, 10, 30]
```

Удалилось только первое значение:

```
10
```

Если значения нет

Например:

```
languages = ["Python", "Java"]

languages.remove("Go")
```

Такого элемента нет.

Python сообщит об ошибке:

```
ValueError
```

Поэтому перед удалением иногда полезно сначала проверить:

```
if "Go" in languages:
    languages.remove("Go")
```

Удаление по индексу: pop()

Метод:

```
pop()
```

может удалить элемент по индексу и вернуть его.

Например:

```
languages = ["Python", "Java", "Go"]

removed = languages.pop(1)

print(removed)
print(languages)
```

Результат:

```
Java
['Python', 'Go']
```

Проверьте удаление по индексу и удаление первого совпадающего значения.

Список 2.5 `list-removal.py`

```
languages = ["Python", "Java", "Go"]

removed = languages.pop(1)

print("Удалено:", removed)
print(languages)

numbers = [10, 20, 10, 30]
numbers.remove(10)

print(numbers)
```

---

pop() без индекса

Если индекс не указан:

```
languages.pop()
```

удаляется последний элемент.

Например:

```
languages = ["Python", "Java", "Go"]

removed = languages.pop()

print(removed)
print(languages)
```

Результат:

```
Go
['Python', 'Java']
```

---

clear()

Чтобы удалить все элементы:

```
clear()
```

Например:

```
numbers = [10, 20, 30]

numbers.clear()

print(numbers)
```

Результат:

```
[]
```

Сам список остаётся существовать, но становится пустым.

Проверка элемента с in

Оператор:

```
in
```

работает и со списками.

Например:

```
languages = ["Python", "Java", "Go"]

print("Python" in languages)
```

Результат:

```
True
```

А:

```
print("Rust" in languages)
```

даёт:

```
False
```

---

not in

Можно проверить отсутствие:

```
languages = ["Python", "Java", "Go"]

print("Rust" not in languages)
```

Результат:

```
True
```

Это удобно вместе с `if`:

```
if "Python" in languages:
    print("Python найден")
```

Перебор списка через for

`for` отлично подходит для списков.

Например:

```
languages = ["Python", "Java", "Go"]

for language in languages:
    print(language)
```

Результат:

```
Python
Java
Go
```

Список 2.6 `list-iteration.py`

```
languages = ["Python", "Java", "Go"]

for language in languages:
    print(language)

for i in range(len(languages)):
    print(i, languages[i])
```

На каждой итерации:

```
language
```

получает очередной элемент списка.

Как for перебирает список

Можно представить:

```
["Python", "Java", "Go"]
        ↓
      for
        ↓
language = "Python"
        ↓
language = "Java"
        ↓
language = "Go"
        ↓
      конец
```

Здесь не нужно использовать индексы.



Рисунок 2.4: Перебор списка через for

Когда индекс не нужен

Если нужно просто обработать каждый элемент:

```
for language in languages:
    print(language)
```

обычно лучше перебирать сами значения.

Необязательно писать:

```
for i in range(len(languages)):
    print(languages[i])
```

Такой вариант тоже возможен, но для простой обработки элементов он сложнее.

> Перебирайте элементы напрямую
>
> Если индекс не нужен, обычно проще:
>
> ```
> for item in items:
>     print(item)
> ```
>
> чем:
>
> ```
> for i in range(len(items)):
>     print(items[i])
> ```

Перебор по индексам

Иногда индекс действительно нужен.

Например:

```
languages = ["Python", "Java", "Go"]

for i in range(len(languages)):
    print(i, languages[i])
```

Результат:

```
0 Python
1 Java
2 Go
```

Здесь:

```
range(len(languages))
```

даёт:

```
0
1
2
```

Изменение элементов по индексам

Если нужно изменить каждый элемент списка, индексы могут быть полезны.

Например:

```
numbers = [1, 2, 3]

for i in range(len(numbers)):
    numbers[i] = numbers[i] * 2

print(numbers)
```

Результат:

```
[2, 4, 6]
```

Здесь изменяется сам список.

Срезы списков

Списки поддерживают срезы так же, как строки.

Например:

```
numbers = [10, 20, 30, 40, 50]

print(numbers[1:4])
```

Результат:

```
[20, 30, 40]
```

Правая граница снова не включается.

Другие срезы

```
numbers = [10, 20, 30, 40, 50]

print(numbers[:3])
print(numbers[2:])
print(numbers[-2:])
print(numbers[::-1])
```

Результат:

```
[10, 20, 30]
[30, 40, 50]
[40, 50]
[50, 40, 30, 20, 10]
```

Поменяйте границы среза и проверьте, что `stop` не включается.

Список 2.7 `list-slicing.py`

```
numbers = [10, 20, 30, 40, 50]

print(numbers[1:4])
print(numbers[:3])
print(numbers[2:])
print(numbers[-2:])
print(numbers[::-1])
```

Правила очень похожи на строки.

Срез создаёт новый список

Например:

```
numbers = [10, 20, 30, 40]

part = numbers[1:3]

print(part)
```

Результат:

```
[20, 30]
```

`part` — отдельный список.

Объединение списков

Оператор:

```
+
```

может объединять списки.

Например:

```
first = [1, 2]
second = [3, 4]

result = first + second

print(result)
```

Результат:

```
[1, 2, 3, 4]
```

Повторение списка

Оператор:

```
*
```

может повторять элементы списка.

Например:

```
numbers = [1, 2]

print(numbers * 3)
```

Результат:

```
[1, 2, 1, 2, 1, 2]
```

Это похоже на повторение строк:

```
"Hi" * 3
```

---

count()

Метод:

```
count()
```

подсчитывает количество элементов с определённым значением.

Например:

```
numbers = [10, 20, 10, 30, 10]

print(numbers.count(10))
```

Результат:

```
3
```

---

index()

Метод:

```
index()
```

возвращает индекс первого совпадения.

Например:

```
languages = ["Python", "Java", "Go"]

print(languages.index("Java"))
```

Результат:

```
1
```

Если значения нет, возникнет:

```
ValueError
```

Сортировка списка

Метод:

```
sort()
```

сортирует сам список.

Например:

```
numbers = [30, 10, 20]

numbers.sort()

print(numbers)
```

Результат:

```
[10, 20, 30]
```

---

sort() изменяет список

Это отличается от многих методов строк.

Например:

```
numbers = [3, 1, 2]

numbers.sort()

print(numbers)
```

Сам `numbers` теперь:

```
[1, 2, 3]
```

> **Списки изменяемые**
>
> Методы списков часто изменяют сам список:
>
> ```
> append()
> remove()
> pop()
> clear()
> sort()
> ```
>
> Это отличается от строк, где методы вроде:
>
> ```
> upper()
> lower()
> replace()
> ```
>
> создают новые строковые значения.

Сортировка строк

Можно сортировать и список строк:

```
names = ["Олег", "Анна", "Максим"]

names.sort()

print(names)
```

Python упорядочит строки согласно своим правилам сравнения.

На этом этапе достаточно понимать сам факт сортировки.

---

reverse()

Метод:

```
reverse()
```

изменяет порядок элементов на обратный.

Например:

```
numbers = [1, 2, 3, 4]

numbers.reverse()

print(numbers)
```

Результат:

```
[4, 3, 2, 1]
```

Соберём несколько методов в одном компактном примере.

Можно также использовать срез:

```
numbers[::-1]
```

Но между этими подходами есть различие:

Список 2.8 `list-methods.py`

```
numbers = [30, 10, 20, 10]

print(numbers.count(10))
print(numbers.index(20))

numbers.sort()
print(numbers)

numbers.reverse()
print(numbers)
```

```
reverse()
```

изменяет существующий список.

Срез создаёт новый список.

---

sum()

Для списка чисел можно использовать:

```
sum()
```

Например:

```
numbers = [10, 20, 30]

print(sum(numbers))
```

Результат:

```
60
```

Это короче, чем писать накопитель вручную.

---

min() и max()

Функция:

```
min()
```

находит минимальное значение.

```
max()
```

— максимальное.

Например:

```
numbers = [12, 5, 30, 8]

print(min(numbers))
print(max(numbers))
```

Результат:

```
5
30
```

Среднее значение

Отдельной встроенной функции среднего в этом базовом наборе нет.

Но можно вычислить:

```
numbers = [10, 20, 30]

average = sum(numbers) / len(numbers)

print(average)
```

Результат:

```
20.0
```

Здесь объединяются:

```
sum()
len()
/
```

Важный случай: пустой список

Рассмотрим:

```
numbers = []

average = sum(numbers) / len(numbers)
```

Получается:

```
0 / 0
```

Это приведёт к:

```
ZeroDivisionError
```

Поэтому перед таким вычислением нужно убедиться, что список не пуст.

Например:

```
numbers = []

if len(numbers) > 0:
    average = sum(numbers) / len(numbers)
    print(average)
else:
    print("Список пуст")
```

---

bool списка

Мы уже знаем:

```
bool("")
```

для пустой строки даёт:

```
False
```

Со списками работает похожее правило.

Пустой список:

```
bool([])
```

даёт:

```
False
```

Непустой:

```
bool([1, 2])
```

даёт:

```
True
```

Поэтому позже можно встретить:

```
if numbers:
    print("В списке есть элементы")
```

На текущем этапе можно читать это как:

> если список не пуст.

Проверка пустого списка

Есть несколько понятных вариантов.

Например:

```
if len(numbers) > 0:
    print("Список не пуст")
```

Или:

```
if numbers:
    print("Список не пуст")
```

Второй вариант очень распространён в Python.

> Пустой список считается False
>
> ```
> []
> ```
>
> в логическом контексте считается `False`.
> Поэтому:
>
> ```
> if items:
>     print("Есть элементы")
> ```
>
> выполнится только для непустого списка.

Создание списка из пользовательского ввода

Например, пользователь вводит три числа:

```
numbers = []

for i in range(3):
    number = float(input("Введите число: "))
    numbers.append(number)

print(numbers)
```

Пример:

```
Введите число: 10
Введите число: 20
Введите число: 30
[10.0, 20.0, 30.0]
```

Теперь все введённые значения сохранены и доступны после завершения цикла.

Список 2.9 `build-list-input.py`

```
numbers = []

for i in range(5):
    number = float(input("Введите число: "))
    numbers.append(number)

print("Список:", numbers)
print("Количество:", len(numbers))
print("Сумма:", sum(numbers))
print("Минимум:", min(numbers))
print("Максимум:", max(numbers))

average = sum(numbers) / len(numbers)

print("Среднее:", average)
```

Почему это лучше одного накопителя

В прошлых главах мы могли написать:

```
total = 0

for i in range(3):
    number = float(input("Введите число: "))
    total += number
```

После цикла у нас есть только итоговая сумма.

Но если использовать список:

```
numbers = []

for i in range(3):
    number = float(input("Введите число: "))
    numbers.append(number)
```

после цикла доступны **все значения**.

Например:

```
print(numbers)
print(sum(numbers))
print(min(numbers))
print(max(numbers))
```

Список сохраняет данные для дальнейшей обработки.

Пример: оценки

```
scores = [85, 90, 72, 100, 88]

print("Количество:", len(scores))
print("Сумма:", sum(scores))
print("Минимум:", min(scores))
print("Максимум:", max(scores))

average = sum(scores) / len(scores)

print("Среднее:", average)
```

Это уже похоже на небольшую обработку набора данных.

Список 2.10 `list-statistics.py`

```
numbers = [12, 5, 30, 8]

print("Количество:", len(numbers))
print("Сумма:", sum(numbers))
print("Минимум:", min(numbers))
print("Максимум:", max(numbers))

average = sum(numbers) / len(numbers)

print("Среднее:", average)
```

Пример: список покупок

```
products = ["Хлеб", "Молоко", "Сыр"]

for product in products:
    print("-", product)
```

Результат:

```
- Хлеб
- Молоко
- Сыр
```

Можно добавить новый товар:

```
products.append("Кофе")
```

или удалить:

```
products.remove("Молоко")
```

Пример: поиск элемента

```
products = ["Хлеб", "Молоко", "Сыр"]

product = input("Что ищем? ")

if product in products:
    print("Товар найден")
else:
    print("Такого товара нет")
```

Здесь объединяются:

```
input()
↓
список
```

Список 2.11 `product-search.py`

```
products = ["Хлеб", "Молоко", "Сыр"]

product = input("Что ищем? ")

if product in products:
    print("Товар найден")
else:
    print("Такого товара нет")
```

```
↓
in
↓
True / False
↓
if
```

Пример: фильтрация значений

Пока не будем создавать отдельный новый список, но можем вывести только подходящие значения.

Например:

```
numbers = [3, 10, 7, 20, 2]

for number in numbers:
    if number >= 10:
        print(number)
```

Результат:

```
10
20
```

Позже мы научимся сохранять такие выбранные значения в отдельный список более системно.

Изменение списка во время перебора

Рассмотрим идею:

```
numbers = [1, 2, 3, 4]
```

Можно захотеть удалять элементы прямо внутри:

```
for number in numbers:
    ...
```

Но изменение размера списка во время его перебора может привести к неожиданному поведению.

На этом этапе лучше придерживаться простого правила:

> ⚠ Не изменяйте размер списка во время перебора
>
> Пока вы изучаете списки, избегайте конструкций вроде:
>
> ```
> for item in items:
>     items.remove(item)
> ```
>
> Перебор и одновременное удаление могут привести к пропуску элементов. Сначала завершите перебор или используйте другой подход.

Копия списка

Рассмотрим:

```
numbers = [1, 2, 3]

copy = numbers[:]

print(copy)
```

Результат:

```
[1, 2, 3]
```

Срез:

```
[:]
```

создаёт новый список с теми же элементами.

Для простых значений это удобный способ сделать копию списка.

Присваивание — не копия

Рассмотрим:

```
first = [1, 2, 3]
second = first

second.append(4)

print(first)
print(second)
```

Результат:

```
[1, 2, 3, 4]
[1, 2, 3, 4]
```

Почему изменились оба?

Потому что:

```
second = first
```

не создаёт новый список.

Оба имени связаны с одним списком.

Можно представить:

```
first ──┐
        ├──→ [1, 2, 3]
second ─┘
```

Настоящая простая копия

Если нужен отдельный список:

```
first = [1, 2, 3]
second = first[:]

second.append(4)

print(first)
print(second)
```

Результат:

```
[1, 2, 3]
[1, 2, 3, 4]
```

Теперь списки разные.

Проверьте обе ситуации рядом: присваивание имени и простую копию через срез.

Список 2.12 `list-copy.py`

```
first = [1, 2, 3]
second = first

second.append(4)

print(first)
print(second)

first = [1, 2, 3]
second = first[:]

second.append(4)

print(first)
print(second)
```

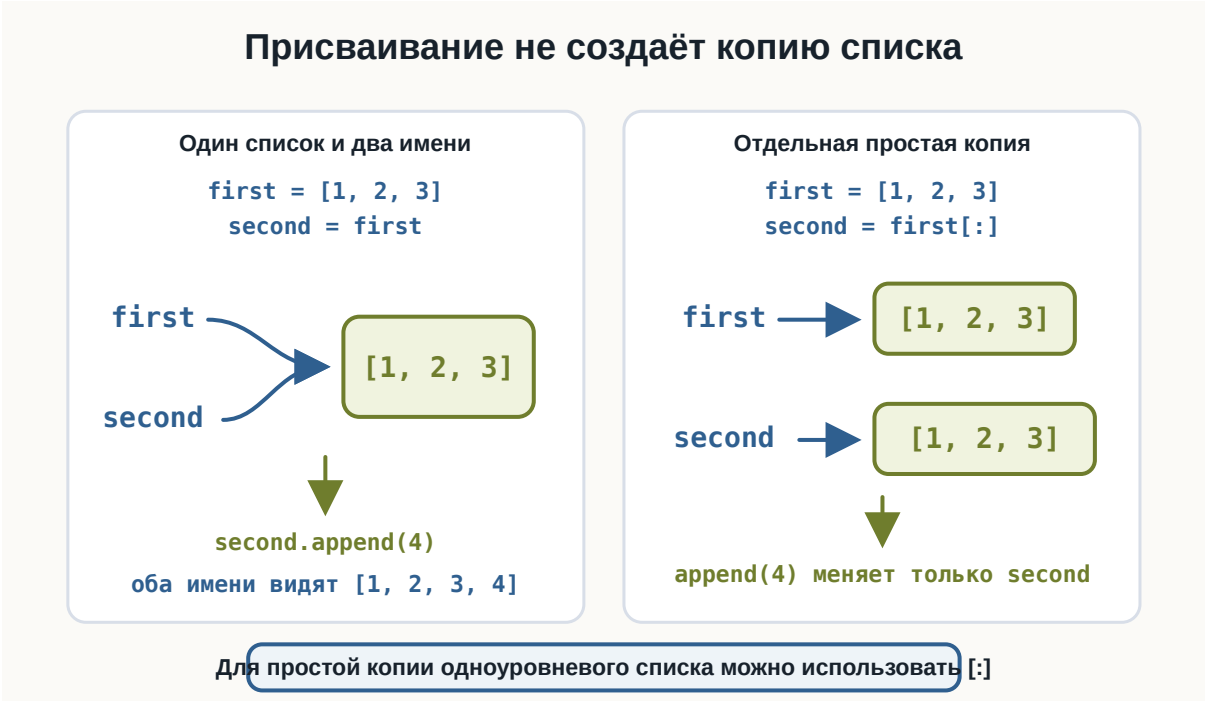


Рисунок 2.5: Присваивание и копия списка

> ❗ Присваивание списка не создаёт копию
>
> ```
> second = first
> ```
>
> означает, что два имени ссылаются на один и тот же список.
> Для простой копии одноуровневого списка можно использовать:
>
> ```
> second = first[:]
> ```
>
> К более сложным случаям копирования вернёмся позже.

Что важно запомнить

Список — это значение типа:

```
list
```

Он позволяет хранить несколько элементов:

```
numbers = [10, 20, 30]
```

Элементы имеют индексы:

```
10   20   30
0    1    2
```

Получение элемента:

```
numbers[0]
numbers[-1]
```

Длина:

```
len(numbers)
```

Списки изменяемые:

```
numbers[0] = 100
```

Основные методы:

```
append()
insert()
remove()
pop()
clear()
count()
index()
sort()
reverse()
```

Полезные функции:

```
len()
sum()
min()
max()
```

Перебор:

```
for item in items:
    print(item)
```

Проверка:

```
item in items
item not in items
```

Срез:

```
items[start:stop:step]
```

И самое важное:

> **Список позволяет сохранить несколько связанных значений и затем обрабатывать их как единый набор данных.**

Практические задания

Задание 1. Первый и последний элемент

Дан список:

```
languages = ["Python", "Java", "Go", "Rust"]
```

Выведите первый и последний элементы.



Ответ

```
languages = ["Python", "Java", "Go", "Rust"]

print(languages[0])
print(languages[-1])
```

Задание 2. Изменение элемента

Дан список:

```
numbers = [10, 20, 30]
```

Замените 20 на 25.

Ответ

```
numbers = [10, 20, 30]

numbers[1] = 25

print(numbers)
```

Результат:

```
[10, 25, 30]
```

Задание 3. Добавление элемента

Создайте список:

```
cities = ["Москва", "Казань"]
```

Добавьте:

```
Самара
```

в конец списка.

Ответ

```
cities = ["Москва", "Казань"]

cities.append("Самара")

print(cities)
```

Задание 4. Удаление элемента

Дан список:

```
languages = ["Python", "Java", "Go"]
```

Удалите `"Java"`.

Ответ

```
languages = ["Python", "Java", "Go"]

languages.remove("Java")

print(languages)
```

Результат:

```
['Python', 'Go']
```

Задание 5. Перебор списка

Дан список:

```
names = ["Анна", "Максим", "Олег"]
```

Выведите каждое имя на отдельной строке.

Ответ

```
names = ["Анна", "Максим", "Олег"]

for name in names:
    print(name)
```

Задание 6. Сумма элементов

Дан список:

```
numbers = [10, 20, 30, 40]
```

Выведите сумму всех элементов.

Ответ

```
numbers = [10, 20, 30, 40]

print(sum(numbers))
```

Результат:

```
100
```

Задание 7. Минимум и максимум

Дан список:

```
numbers = [15, 3, 28, 9, 12]
```

Выведите минимальное и максимальное значения.

Ответ

```
numbers = [15, 3, 28, 9, 12]

print("Минимум:", min(numbers))
print("Максимум:", max(numbers))
```

Задание 8. Среднее значение

Дан список:

```
scores = [80, 90, 70, 100]
```

Вычислите среднее значение.

Ответ

```
scores = [80, 90, 70, 100]

average = sum(scores) / len(scores)

print("Среднее:", average)
```

Результат:

```
Среднее: 85.0
```

Задание 9. Проверка элемента

Дан список:

```
languages = ["Python", "Java", "Go"]
```

Попросите пользователя ввести язык программирования и проверьте, есть ли он в списке.

Ответ

```
languages = ["Python", "Java", "Go"]

language = input("Введите язык: ")

if language in languages:
    print("Язык найден")
else:
    print("Языка нет в списке")
```

Задание 10. Создание списка через input()

Создайте пустой список.

Попросите пользователя ввести три числа и добавьте их в список.

Затем выведите весь список.

Ответ

```
numbers = []

for i in range(3):
    number = float(input("Введите число: "))
    numbers.append(number)

print(numbers)
```

Задание 11. Только большие числа

Дан список:

```
numbers = [4, 15, 2, 30, 8, 21]
```

Выведите только значения больше 10.

Ответ

```
numbers = [4, 15, 2, 30, 8, 21]

for number in numbers:
    if number > 10:
        print(number)
```

Результат:

```
15
30
21
```

Задание 12. Срез списка

Дан список:

```
numbers = [10, 20, 30, 40, 50]
```

Выведите:

```
[20, 30, 40]
```

с помощью среза.

Ответ

```
numbers = [10, 20, 30, 40, 50]

print(numbers[1:4])
```

Задание 13. Разворот списка

Дан список:

```
numbers = [1, 2, 3, 4, 5]
```

Получите новый список в обратном порядке с помощью среза.

Ответ

```
numbers = [1, 2, 3, 4, 5]

reversed_numbers = numbers[::-1]

print(reversed_numbers)
```

Результат:

```
[5, 4, 3, 2, 1]
```

Задание 14. Найдите проблему

Что произойдёт?

```
numbers = [10, 20, 30]

print(numbers[3])
```

Почему?

Ответ

Список содержит три элемента.
Их индексы:

```
10   20   30
0    1    2
```

Индекса:

```
3
```

нет.
Поэтому возникнет:

```
IndexError
```

Задание 15. Присваивание и копия

Какой результат будет здесь?

```
first = [1, 2, 3]
second = first

second.append(4)

print(first)
```

Сначала попробуйте ответить без запуска.

Ответ

Результат:

```
[1, 2, 3, 4]
```

Запись:

```
second = first
```

не создаёт новый список.
Оба имени ссылаются на один список.
Поэтому изменение через `second` видно и через `first`.

Если нужен отдельный список:

```
second = first[:]
```

Задание 16. Задача со звёздочкой

Пользователь вводит пять чисел.

Сохраните их в список.

После ввода выведите:

- весь список;
- количество элементов;
- сумму;
- минимальное число;
- максимальное число;
- среднее значение.

Ответ

```
numbers = []

for i in range(5):
    number = float(input("Введите число: "))
    numbers.append(number)

print("Список:", numbers)
print("Количество:", len(numbers))
print("Сумма:", sum(numbers))
print("Минимум:", min(numbers))
print("Максимум:", max(numbers))

average = sum(numbers) / len(numbers)

print("Среднее:", average)
```

Здесь объединяется сразу несколько изученных тем:

```
input()
↓
float()
↓
for
↓
append()
↓
list
↓
len() / sum() / min() / max()
```

Что дальше?

Теперь программа умеет хранить несколько связанных значений в одном списке:

```
numbers = [10, 20, 30]
```

Но в примерах главы код уже начинает повторяться.

Например, одни и те же действия можно захотеть выполнить в разных местах программы:

```
получить данные
↓
проверить их
↓
посчитать результат
↓
вывести ответ
```

Следующая тема — **функции**.

Функция позволяет дать имя фрагменту программы и вызывать его снова, когда он нужен.

2 3.13 Функции

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как объединить несколько действий в один переиспользуемый блок кода и вызывать его тогда, когда он нужен?**

К этому моменту наши программы уже умеют многое.

Мы можем получать данные:

```
name = input("Введите имя: ")
```

выполнять вычисления:

```
total = price * quantity
```

принимать решения:

```
if age >= 18:
    print("Доступ разрешён")
```

повторять действия:

```
for number in range(1, 6):
    print(number)
```

и работать с наборами данных:

```
numbers = [10, 20, 30]
```

Но по мере роста программы появляется новая проблема.

Один и тот же код может понадобиться несколько раз.

Например:

```
print("--------------------")
print("Добро пожаловать!")
print("--------------------")
```

Если такой блок нужен в трёх местах, можно скопировать его:

```
print("--------------------")
print("Добро пожаловать!")
print("--------------------")

print("Основная часть программы")

print("--------------------")
print("Добро пожаловать!")
print("--------------------")
```

Но копирование кода быстро становится неудобным.

Гораздо лучше один раз дать группе команд имя, а затем вызывать её тогда, когда она нужна.

Для этого в Python существуют **функции**.

Что такое функция

Функция — это именованный блок кода, который можно выполнить по вызову.

Например:

```
def greet():
    print("Привет!")
```

Здесь мы создали функцию:

```
greet
```

Но пока она ещё не выполнялась.

Чтобы запустить код внутри функции, её нужно **вызвать**:

```
greet()
```

Полная программа:

```
def greet():
    print("Привет!")


greet()
```

Результат:

```
Привет!
```

В HTML-версии можно проверить, что само определение функции ничего не выводит, а каждый вызов запускает тело заново.

Список 2.1 `first-function.py`

```
def greet():
    print("Привет!")


greet()
greet()
```

Общая идея:

```
создать функцию
      ↓
дать ей имя
      ↓
описать действия
      ↓
вызвать функцию
      ↓
действия выполняются
```

Создание функции с def

Для создания собственной функции используется слово:

```
def
```

Например:

```
def greet():
    print("Привет!")
```

Эту запись можно разобрать так:

```
def       → объявляем функцию

greet     → имя функции

()        → круглые скобки

:         → начало блока

    ...   → тело функции
```

Общая форма:

```
def имя_функции():
    действие
```

> def означает определение функции
>
> С помощью:
>
> ```
> def
> ```
>
> мы **определяем** функцию.
> Например:
>
> ```
> def show_message():
>     print("Python")
> ```
>
> Но код внутри неё выполняется только после вызова:
>
> ```
> show_message()
> ```

Вызов функции

После определения функции её можно вызвать:

```
def greet():
    print("Привет!")


greet()
```

Вызов выглядит так:

```
greet()
```

Круглые скобки здесь важны.

Сравните:

```
greet
```

и:

```
greet()
```

На текущем этапе достаточно запомнить:

> Чтобы выполнить функцию, после её имени ставятся круглые скобки.

Функцию можно вызвать несколько раз

Главное преимущество функции — возможность повторного использования.

Например:

```
def greet():
    print("Добро пожаловать!")


greet()
greet()
greet()
```

Результат:

```
Добро пожаловать!
Добро пожаловать!
Добро пожаловать!
```

Код функции написан один раз, но выполнен три раза.

Несколько команд внутри функции

Функция может содержать несколько строк:

```
def show_header():
    print("--------------------")
    print("Путь Python-разработчика")
    print("--------------------")
```

Вызов:

```
show_header()
```

Результат:

```
--------------------
Путь Python-разработчика
--------------------
```

Все строки с одинаковым отступом относятся к функции.

Отступы внутри функции

Как и в `if`, `for` и `while`, Python использует отступы для определения блока.

Правильно:

```
def greet():
    print("Привет!")
    print("Добро пожаловать!")
```

Неправильно:

```
def greet():
print("Привет!")
```

⚠ Тело функции должно иметь отступ

После:

```
def greet():
```

команды функции записываются с отступом:

```
def greet():
    print("Привет!")
```

Обычно используется 4 пробела.

Код после функции

Рассмотрим:

```
def greet():
    print("Привет!")


print("Начало программы")

greet()

print("Конец программы")
```

Результат:

```
Начало программы
Привет!
Конец программы
```

Python не выполняет функцию в момент её определения.

Сначала он узнаёт, что функция существует:

```
def greet():
```

А код внутри выполняется только при:

```
greet()
```

Зачем нужны функции

Функции помогают решать сразу несколько задач.

Они позволяют:

```
не повторять одинаковый код
↓
разделять программу на понятные части
↓
давать действиям понятные имена
↓
повторно использовать готовую логику
↓
упрощать чтение и изменение программы
```

Например, вместо:

```
print("--------------------")
print("Меню")
print("--------------------")
```

в нескольких местах можно написать:

```
def show_menu_header():
    print("--------------------")
    print("Меню")
    print("--------------------")
```

а затем:

```
show_menu_header()
```

Понятные имена функций

Имя функции должно описывать действие.

Хорошие примеры:

```
show_menu()
print_report()
calculate_total()
check_password()
greet_user()
```

Менее понятные:

```
f()
do()
func1()
x()
```

Функции обычно называют глаголами или фразами, описывающими действие.

> Имя должно отвечать на вопрос «что делает функция?»
>
> Например:
>
> ```
> calculate_total()
> ```
>
> сразу говорит:
>
> вычислить итог.
>
> А:
>
> ```
> show_message()
> ```
>
> говорит:
>
> показать сообщение.

Параметры функции

Пока наша функция всегда делает одно и то же:

```
def greet():
    print("Привет!")
```

Но часто функции нужны данные.

Например, мы хотим приветствовать разных пользователей:

```
Привет, Анна!
Привет, Максим!
Привет, Олег!
```

Для этого используется **параметр**.

```
def greet(name):
    print("Привет,", name)
```

Теперь при вызове нужно передать значение:

```
greet("Анна")
```

Результат:

```
Привет, Анна
```

На схеме видно, где находится параметр в определении и где находится аргумент при вызове.

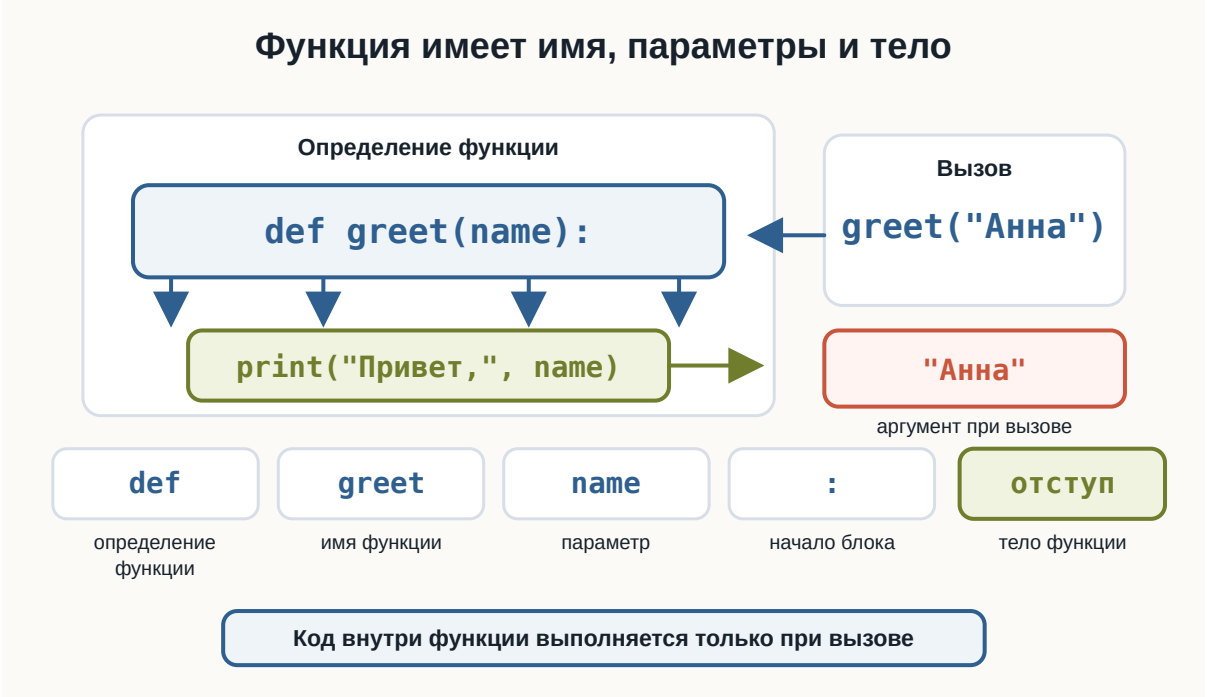


Рисунок 2.1: Устройство функции

Как работает параметр

Рассмотрим:

```
def greet(name):
    print("Привет,", name)


greet("Анна")
```

Во время вызова:

```
greet("Анна")
```

значение:

```
"Анна"
```

передаётся параметру:

```
name
```

Внутри функции получается:

```
name → "Анна"
```

После этого выполняется:

```
print("Привет,", name)
```

Можно представить:

```
greet("Анна")
      ↓
   "Анна"
      ↓
    name
      ↓
тело функции
      ↓
Привет, Анна
```

Эту передачу значения можно представить как переход из основной программы в функцию и обратно.

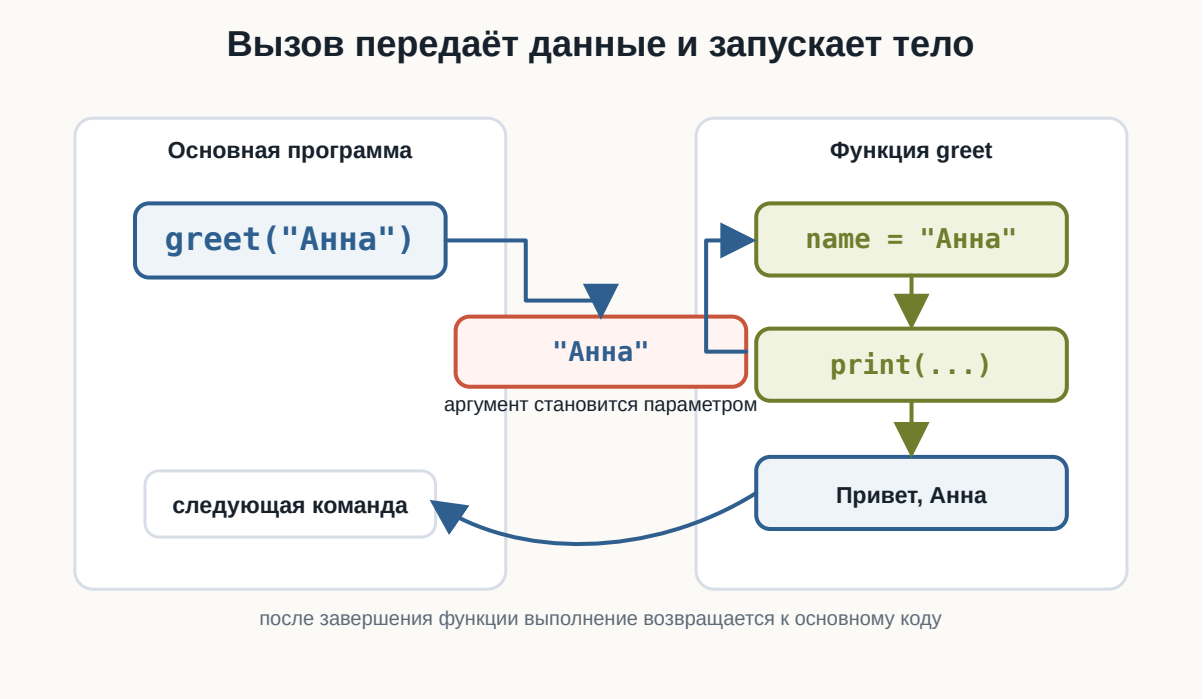


Рисунок 2.2: Поток вызова функции

Параметр и аргумент

Здесь важно различать два понятия.

В определении:

```
def greet(name):
```

`name` — **параметр**.

А при вызове:

```
greet("Анна")
```

`"Анна"` — **аргумент**.

То есть:

```
параметр
→ имя внутри определения функции
```

```
аргумент
→ конкретное значение при вызове
```

> **Параметр и аргумент**
>
> ```
> def greet(name):
> ```
>
> `name` — параметр.
>
> ```
> greet("Анна")
> ```
>
> `"Анна"` — аргумент.
> Аргумент передаёт конкретное значение параметру.

Одна функция — разные данные

Теперь одну функцию можно использовать с разными значениями:

```
def greet(name):
    print("Привет,", name)


greet("Анна")
greet("Максим")
greet("Олег")
```

Результат:

```
Привет, Анна
Привет, Максим
Привет, Олег
```

Поменяйте аргументы в HTML-версии и проверьте, что тело функции остаётся тем же.

Именно поэтому функции настолько полезны.

Логика написана один раз:

Список 2.2 `function-parameters.py`

```
def greet(name):
    # name - параметр функции.
    print("Привет,", name)


# "Анна", "Максим" и "Олег" - аргументы при вызове.
greet("Анна")
greet("Максим")
greet("Олег")
```

```
print("Привет,", name)
```

а данные могут быть разными.

Несколько параметров

Функция может принимать несколько параметров.

Например:

```
def show_user(name, age):
    print("Имя:", name)
    print("Возраст:", age)
```

Вызов:

```
show_user("Анна", 25)
```

Результат:

```
Имя: Анна
Возраст: 25
```

В playground можно сравнить позиционный вызов и короткий пример именованных аргументов.

Значения передаются по порядку:

Список 2.3 `multiple-parameters.py`

```
def show_user(name, age):
    print("Имя:", name)
    print("Возраст:", age)


show_user("Анна", 25)
show_user(name="Анна", age=25)
```

```
"Анна" → name
25     → age
```

Порядок аргументов имеет значение

Рассмотрим:

```
def show_user(name, age):
    print("Имя:", name)
    print("Возраст:", age)
```

Правильный вызов:

```
show_user("Анна", 25)
```

Если написать:

```
show_user(25, "Анна")
```

Python технически передаст:

```
name → 25
age  → "Анна"
```

и результат окажется неправильным по смыслу.

⚠ Следите за порядком аргументов

Для:

```
def show_user(name, age):
```

вызов:

```
show_user("Анна", 25)
```

передаёт:

```
"Анна" → name
25     → age
```

Позиция аргумента важна.

Функция для вычисления

Функция может не только выводить текст.

Например:

```
def show_sum(a, b):
    result = a + b

    print(result)
```

Вызов:

```
show_sum(10, 5)
```

Результат:

```
15
```

Другой вызов:

```
show_sum(100, 25)
```

Результат:

```
125
```

Функция и input()

Данные можно получить до вызова функции:

```
def greet(name):
    print("Привет,", name)


user_name = input("Введите имя: ")

greet(user_name)
```

Например:

```
Введите имя: Анна
Привет, Анна
```

Получается:

```
input()
   ↓
значение
   ↓
аргумент
   ↓
параметр функции
   ↓
действие
```

Возвращаемое значение

До сих пор функции сами выводили результат:

```
def show_sum(a, b):
    print(a + b)
```

Но часто результат нужен **не для немедленного вывода**, а для дальнейшей работы.

Например:

```
вычислить стоимость
↓
сохранить результат
↓
прибавить доставку
↓
сравнить с бюджетом
```

Для этого функция может **возвращать значение**.

Используется:

```
return
```

Первый return

Например:

```
def add(a, b):
    return a + b
```

Теперь можно написать:

```
result = add(10, 5)

print(result)
```

Результат:

```
15
```

Функция:

```
add(10, 5)
```

создаёт результат:

```
15
```

и возвращает его в место вызова.

Как работает return

Рассмотрим:

```
def add(a, b):
    return a + b


result = add(10, 5)
```

Пошагово:

```
add(10, 5)
    ↓
a = 10
b = 5
    ↓
10 + 5
    ↓
15
    ↓
return
    ↓
result = 15
```

После этого значение можно использовать как обычное число:

```
print(result * 2)
```

Результат:

```
30
```

Схема показывает весь путь возвращаемого значения: от аргументов до переменной `result`.

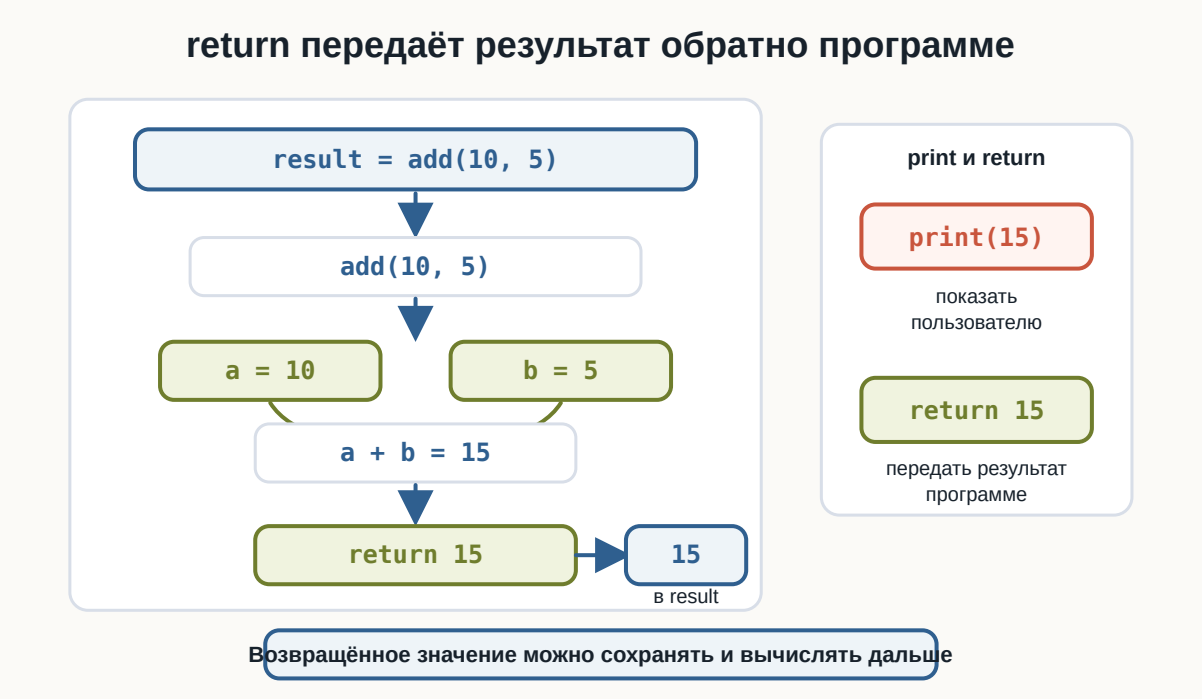


Рисунок 2.3: Поток return

Проверьте в HTML-версии, что возвращённое значение можно сохранить и использовать в следующем вычислении.

Список 2.4 `return-value.py`

```
def add(a, b):
    return a + b


result = add(10, 5)

print(result)
print(result * 2)
```

---

print() и return — не одно и то же

Это очень важное различие.

Функция:

```
def add(a, b):
    print(a + b)
```

сама выводит значение на экран.

А:

```
def add(a, b):
    return a + b
```

возвращает значение вызывающему коду.

Сравним.

print()

```
def add(a, b):
    print(a + b)


add(10, 5)
```

выведет:

```
15
```

return

```
def add(a, b):
    return a + b


result = add(10, 5)

print(result)
```

тоже выведет:

```
15
```

Но во втором случае число:

```
15
```

сохранено в `result` и его можно использовать дальше.

> ❗ print() показывает, return возвращает
>
> `print()`
>
> выводит значение пользователю.
>
> `return`
>
> передаёт результат функции обратно в программу.
> Например:
>
> `result = add(10, 5)`
>
> имеет смысл только тогда, когда функция возвращает значение.

В этом примере удобно увидеть разницу между выводом, возвращаемым значением и None.

Возвращённое значение можно использовать сразу

Например:

```
def add(a, b):
    return a + b


print(add(10, 5))
```

Результат:

Список 2.5 `print-vs-return.py`

```
def show_sum(a, b):
    print(a + b)


def calculate_sum(a, b):
    return a + b


show_sum(10, 5)

result = calculate_sum(10, 5)

print(result)


def no_return():
    print("Привет")


result = no_return()
print(result)
```

```
15
```

Или:

```
print(add(10, 5) * 2)
```

Результат:

```
30
```

Функция становится частью выражения.

Пример: стоимость покупки

Напишем функцию:

```
def calculate_total(price, quantity):
    return price * quantity
```

Использование:

```
total = calculate_total(250, 3)

print(total)
```

Результат:

```
750
```

Теперь можно добавить доставку:

```
total = calculate_total(250, 3)

final_price = total + 150

print(final_price)
```

Результат:

```
900
```

Функция может возвращать строку

`return` работает не только с числами.

Например:

```
def make_greeting(name):
    return "Привет, " + name
```

Использование:

```
message = make_greeting("Анна")

print(message)
```

Результат:

```
Привет, Анна
```

Функция может возвращать bool

Например:

```
def is_adult(age):
    return age >= 18
```

Проверим:

```
print(is_adult(20))
print(is_adult(15))
```

Результат:

```
True
False
```

Теперь результат функции можно использовать в `if`:

```
age = int(input("Введите возраст: "))

if is_adult(age):
    print("Доступ разрешён")
else:
    print("Доступ запрещён")
```

Здесь соединяются сразу несколько предыдущих тем.

Поменяйте возраст и условие внутри функции, чтобы увидеть, как `bool`-результат работает прямо в `if`.

---

return завершает выполнение функции

Рассмотрим:

Список 2.6 `boolean-function.py`

```
def is_adult(age):
    return age >= 18


print(is_adult(20))
print(is_adult(15))

age = int(input("Введите возраст: "))

if is_adult(age):
    print("Доступ разрешён")
else:
    print("Доступ запрещён")
```

```
def check_number(number):
    if number > 0:
        return "Положительное"

    return "Ноль или отрицательное"
```

Если:

```
number > 0
```

истинно, выполняется:

```
return "Положительное"
```

После `return` функция завершается.

Код ниже этого `return` в текущем вызове уже не выполняется.

Код после return

Например:

```
def example():
    return 10
    print("Эта строка не выполнится")
```

После:

```
return 10
```

функция заканчивает работу.

Поэтому следующий `print()` недостижим.

> ⚠ return завершает функцию
>
> Как только Python выполняет:
>
> ```
> return
> ```
>
> текущий вызов функции завершается.
> Команды ниже этого `return` в той же ветке уже не выполняются.

Несколько return

В одной функции может быть несколько `return`, если они находятся в разных ветках.

Например:

```
def describe_number(number):
    if number > 0:
        return "Положительное"
    elif number < 0:
        return "Отрицательное"
    else:
        return "Ноль"
```

Использование:

```
print(describe_number(10))
print(describe_number(-5))
print(describe_number(0))
```

Результат:

```
Положительное
Отрицательное
Ноль
```

Для каждого вызова выполняется только один `return`.

Что возвращает функция без return

Рассмотрим:

```
def greet():
    print("Привет!")
```

Попробуем сохранить результат:

```
result = greet()

print(result)
```

Результат:

```
Привет!
None
```

Почему?

Функция не содержит явного:

```
return
```

Поэтому Python возвращает:

```
None
```

None как результат функции

Мы уже встречали:

```
None
```

Это специальное значение, означающее отсутствие обычного результата.

Например:

```
def show_message():
    print("Python")


result = show_message()

print(type(result))
```

После выполнения функции `result` содержит None.

> **i** Функция всегда возвращает значение
>
> Если вы не написали явный:
>
> ```
> return ...
> ```
>
> функция возвращает:
>
> ```
> None
> ```
>
> Поэтому функции, предназначенные только для действий вроде `print()`, обычно не используют их результат дальше.

Параметры можно использовать внутри вычислений

Например:

```
def rectangle_area(width, height):
    area = width * height

    return area
```

Вызов:

```
result = rectangle_area(5, 3)

print(result)
```

Результат:

```
15
```

Функция скрывает детали вычисления за понятным именем:

```
rectangle_area(...)
```

Функция может вызывать другую функцию

Функции можно комбинировать.

Например:

```
def calculate_total(price, quantity):
    return price * quantity


def show_total(price, quantity):
    total = calculate_total(price, quantity)

    print("Итого:", total)
```

Вызов:

```
show_total(100, 3)
```

Результат:

```
Итого: 300
```

Можно представить:

```
show_total()
     ↓
calculate_total()
     ↓
   результат
     ↓
show_total()
     ↓
   print()
```

Разделение большой задачи

Представим программу:

```
получить данные
↓
посчитать сумму
↓
проверить скидку
↓
показать результат
```

Каждую часть можно вынести в отдельную функцию.

Например:

```
def calculate_total(price, quantity):
    return price * quantity


def has_discount(total):
    return total >= 3000


def show_result(total):
    print("Итого:", total)
```

Основная программа становится понятнее:

```
total = calculate_total(1200, 3)

show_result(total)

print(has_discount(total))
```

Функции помогают разделять программу на небольшие смысловые части.

Локальные переменные

Рассмотрим:

```
def calculate_total(price, quantity):
    total = price * quantity

    return total
```

Переменная:

```
total
```

создана внутри функции.

Такая переменная называется **локальной**.

Она предназначена для работы внутри функции.

Переменная внутри функции

Например:

```
def example():
    message = "Привет"

    print(message)


example()
```

Это работает.

Но после функции:

```
print(message)
```

обычно приведёт к ошибке:

```
NameError
```

Потому что `message` существует только внутри локальной области функции.

> ❗ Переменные функции имеют свою область
>
> Переменная, созданная внутри функции:
>
> ```
> def example():
>     value = 10
> ```
>
> является локальной.
> За пределами функции к ней напрямую обращаться не следует.

Параметры тоже локальные

Параметр:

```
name
```

в:

```
def greet(name):
    print(name)
```

существует внутри конкретного вызова функции.

Например:

```
greet("Анна")
greet("Максим")
```

В первом вызове:

```
name → "Анна"
```

Во втором:

```
name → "Максим"
```

Каждый вызов получает собственные значения параметров.

Внешняя и локальная переменная могут иметь одинаковое имя

Например:

```
name = "Анна"


def greet(name):
    print("Привет,", name)


greet("Максим")

print(name)
```

Результат:

```
Привет, Максим
Анна
```

Параметр name внутри функции — отдельное локальное имя.

Он не изменяет внешнюю переменную name.

На этом этапе достаточно понимать именно эту идею.

Схема разделяет внешний код и локальную область функции.

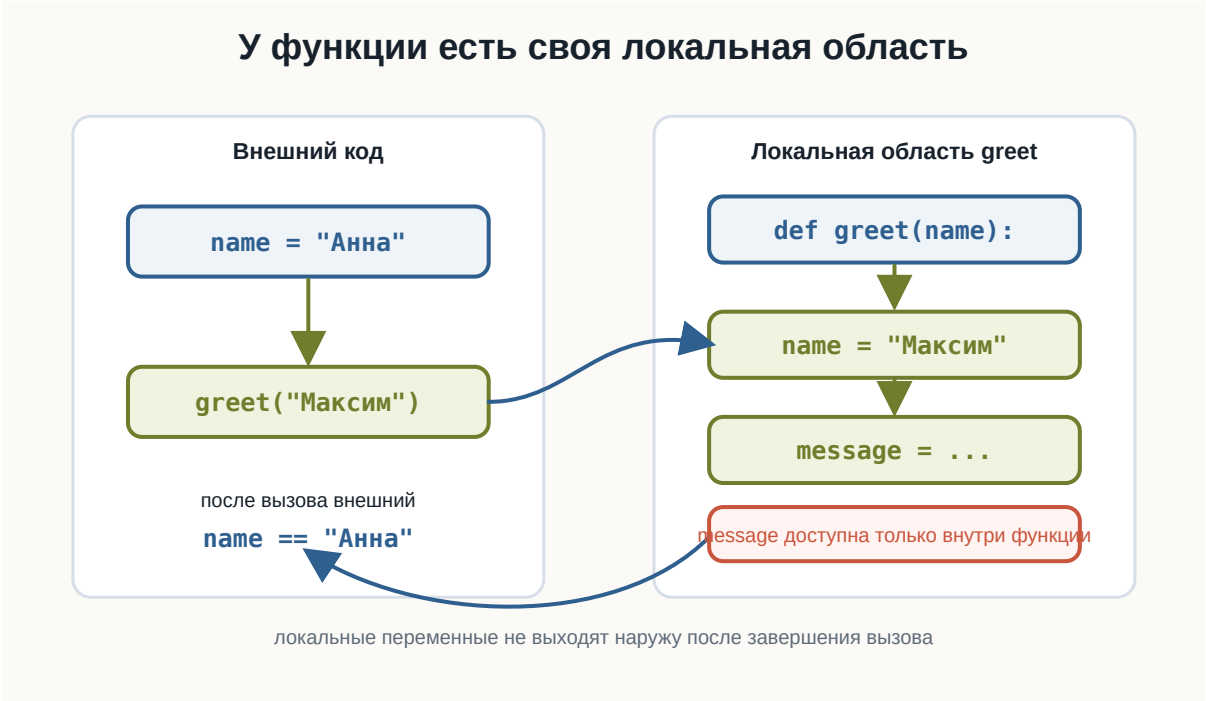


Рисунок 2.4: Локальная область функции

В HTML-версии можно убедиться, что внешний `name` не меняется после вызова функции.

Список 2.7 `local-scope.py`

```
name = "Анна"


def greet(name):
    message = "Привет, " + name
    print(message)


greet("Максим")

print(name)
```

Не полагайтесь на глобальные данные без необходимости

Можно написать:

```
name = "Анна"


def greet():
    print("Привет,", name)
```

Такой код может работать.

Но функция становится зависимой от переменной, созданной где-то снаружи.

Часто понятнее передать нужное значение явно:

```
def greet(name):
    print("Привет,", name)


greet("Анна")
```

Так сразу видно, какие данные нужны функции.

> Передавайте функции нужные данные явно
>
> Понятнее:
>
> ```
> def calculate_total(price, quantity):
>     return price * quantity
> ```
>
> чем функция, которая незаметно берёт `price` и `quantity` из внешней части программы.
> Параметры делают зависимости функции видимыми.

Аргументы можно брать из переменных

Необязательно передавать значения напрямую:

```
calculate_total(100, 3)
```

Можно использовать переменные:

```
price = 100
quantity = 3
```

```
total = calculate_total(price, quantity)

print(total)
```

Это один из самых распространённых способов работы с функциями.

Именованные аргументы

До сих пор значения передавались по позиции:

```
show_user("Анна", 25)
```

Python также позволяет явно указать имя параметра:

```
show_user(name="Анна", age=25)
```

Такой вызов называется использованием **именованных аргументов**.

Например:

```
def show_user(name, age):
    print("Имя:", name)
    print("Возраст:", age)


show_user(age=25, name="Анна")
```

Результат остаётся корректным:

```
Имя: Анна
Возраст: 25
```

Потому что теперь значения связаны с параметрами по именам.

Значения параметров по умолчанию

Иногда один параметр обычно имеет одно и то же значение.

Например:

```
def greet(name, greeting="Привет"):
    print(greeting + ",", name)
```

Теперь можно вызвать:

```
greet("Анна")
```

Результат:

```
Привет, Анна
```

Параметр:

```
greeting
```

получил значение:

```
"Привет"
```

по умолчанию.

Проверьте два вызова: без второго аргумента и с заменой значения по умолчанию.

Список 2.8 `default-parameter.py`

```
def greet(name, greeting="Привет"):
    print(greeting + ",", name)


greet("Анна")
greet("Максим", "Здравствуйте")
```

Значение по умолчанию можно заменить

Можно передать другой аргумент:

```
greet("Анна", "Здравствуйте")
```

Результат:

```
Здравствуйте, Анна
```

То есть:

```
аргумент не передан
→ используется значение по умолчанию

аргумент передан
→ используется переданное значение
```

Обязательные параметры идут раньше

Например:

```
def greet(name, greeting="Привет"):
```

записано корректно.

`name` нужно передать обязательно.

`greeting` имеет значение по умолчанию.

На текущем этапе придерживайтесь простого правила:

> Сначала записывайте обязательные параметры, затем параметры со значениями по умолчанию.

Функция и список

Функция может получать список:

```
def show_items(items):
    for item in items:
        print(item)
```

Использование:

```
languages = ["Python", "Java", "Go"]

show_items(languages)
```

Результат:

```
Python
Java
Go
```

Функция для суммы списка

Например:

```
def calculate_sum(numbers):
    total = 0

    for number in numbers:
        total += number

    return total
```

Вызов:

```
values = [10, 20, 30]

result = calculate_sum(values)

print(result)
```

Результат:

```
60
```

Передайте функции другой список и посмотрите, как меняется результат.

Список 2.9 `function-with-list.py`

```
def calculate_sum(numbers):
    total = 0

    for number in numbers:
        total += number

    return total


values = [10, 20, 30]

print(calculate_sum(values))
```

Конечно, Python уже предоставляет:

```
sum()
```

Но собственная функция помогает увидеть, как можно скрыть алгоритм за понятным именем.

Функция может возвращать список

Например:

```
def double_numbers(numbers):
    result = []

    for number in numbers:
        result.append(number * 2)

    return result
```

Вызов:

```
numbers = [1, 2, 3]

doubled = double_numbers(numbers)

print(doubled)
```

Результат:

```
[2, 4, 6]
```

В этом примере функция возвращает новый список, а исходный список можно оставить без изменений.

Список 2.10 `double-numbers.py`

```
def double_numbers(numbers):
    result = []

    for number in numbers:
        result.append(number * 2)

    return result


numbers = [1, 2, 3, 4]

print(double_numbers(numbers))
```

Здесь функция:

```
получает список
↓
создаёт новый список
↓
заполняет его
↓
возвращает результат
```

Не смешивайте слишком много задач в одной функции

Рассмотрим функцию, которая:

```
спрашивает данные
вычисляет
проверяет
печатает
сохраняет
```

Чем больше разных обязанностей у функции, тем сложнее её понимать.

Например, функция:

```
def calculate_total(price, quantity):
    return price * quantity
```

делает одну понятную вещь.

Это хороший ориентир:

> одна функция — одна понятная задача.

Маленькие функции легче читать

Например:

```
def is_adult(age):
    return age >= 18
```

Она короткая, но полезная.

В основном коде:

```
if is_adult(age):
    print("Доступ разрешён")
```

намерение программы читается почти как обычный текст.

Функции уменьшают повторение кода

Без функции:

```
total1 = 100 * 3
total2 = 250 * 2
total3 = 75 * 5
```

С функцией:

```
def calculate_total(price, quantity):
    return price * quantity


total1 = calculate_total(100, 3)
total2 = calculate_total(250, 2)
total3 = calculate_total(75, 5)
```

Формула хранится в одном месте.

Если логика изменится, её проще изменить внутри функции.

Функции делают основную программу понятнее

Сравните:

```
price = 120
quantity = 3
total = price * quantity

if total >= 300:
    discount = total * 0.1
else:
    discount = 0

final_price = total - discount

print(final_price)
```

И вариант с функциями:

```
def calculate_total(price, quantity):
    return price * quantity


```

```
def calculate_discount(total):
    if total >= 300:
        return total * 0.1

    return 0


total = calculate_total(120, 3)
discount = calculate_discount(total)

final_price = total - discount

print(final_price)
```

Теперь отдельные действия имеют имена.

Это становится особенно важно по мере роста программы.

Типичные ошибки

Функция определена, но не вызвана

Например:

```
def greet():
    print("Привет!")
```

На экране ничего не появится.

Нужно:

```
greet()
```

Забыт аргумент

Функция:

```
def greet(name):
    print("Привет,", name)
```

Требует значение для `name`.

Вызов:

```
greet()
```

приведёт к:

```
TypeError
```

Нужно:

```
greet("Анна")
```

Передано слишком много аргументов

Например:

```
def greet(name):
    print("Привет,", name)


greet("Анна", 25)
```

Функция ожидает один аргумент, а получила два.

Python сообщит о `TypeError`.

Перепутаны print() и return

Например:

```
def add(a, b):
    print(a + b)
```

```
result = add(10, 5)

print(result)
```

Получим:

```
15
None
```

Функция вывела 15, но не вернула его.

Если результат нужен дальше:

```
def add(a, b):
    return a + b
```

Код после return

```
def example():
    return 10
    print("Не выполнится")
```

После `return` функция уже завершена.

Пошаговый разбор вызова

Рассмотрим:

```
def calculate_area(width, height):
    area = width * height

    return area


result = calculate_area(5, 4)

print(result)
```

Пошагово:

```
1. Python встречает def
   ↓
   запоминает функцию calculate_area

2. выполняется:
   calculate_area(5, 4)

3. параметры получают значения:
   width  = 5
   height = 4

4. выполняется:
   area = 5 * 4

5. получается:
   area = 20

6. выполняется:
   return 20

7. значение возвращается:
   result = 20

8. print(result)
   ↓
   20
```

Попробуйте сами

Эксперимент

Создайте:

```
def multiply(a, b):
    return a * b
```

Попробуйте вызовы:

```
print(multiply(2, 3))
print(multiply(10, 5))
print(multiply(1.5, 4))
```

Затем измените функцию так, чтобы она:

- складывала числа;
- вычитала второе число из первого;
- возвращала квадрат одного числа;
- проверяла, положительное ли число.

Перед каждым запуском попробуйте предсказать результат.

Практический пример: итог покупки

Соединим несколько тем курса.

```
def calculate_total(price, quantity):
    return price * quantity


def calculate_discount(total):
    if total >= 3000:
        return total * 0.1

    return 0


price = float(input("Цена товара: "))
quantity = int(input("Количество: "))

total = calculate_total(price, quantity)
discount = calculate_discount(total)
final_price = total - discount

print("Сумма:", total)
print("Скидка:", discount)
print("К оплате:", final_price)
```

В HTML-версии можно менять цену и количество, чтобы увидеть, как основная программа использует две функции.

Список 2.11 `purchase-functions.py`

```
def calculate_total(price, quantity):
    return price * quantity


def calculate_discount(total):
    if total >= 3000:
        return total * 0.1

    return 0


price = float(input("Цена товара: "))
quantity = int(input("Количество: "))

total = calculate_total(price, quantity)
discount = calculate_discount(total)
final_price = total - discount

print("Сумма:", total)
print("Скидка:", discount)
print("К оплате:", final_price)
```

Здесь используются:

```
input()
↓
преобразование типов
↓
функции
↓
параметры
↓
вычисления
↓
условие
↓
return
↓
итоговый результат
```

Это уже пример программы, разбитой на отдельные смысловые части.

Практический пример: обработка списка

Создадим функцию для поиска максимального значения без использования `max()`:

```
def find_max(numbers):
    maximum = numbers[0]

    for number in numbers:
        if number > maximum:
            maximum = number

    return maximum
```

Использование:

```
values = [10, 50, 20, 35]

result = find_max(values)

print(result)
```

Результат:

```
50
```

Здесь функция объединяет:

```
список
↓
for
↓
if
↓
локальную переменную
↓
return
```

> **i** Предполагается непустой список
>
> В этом примере используется:

```
numbers[0]
```

Поэтому функция рассчитана на непустой список.
Проверку пустого списка можно добавить отдельно, но сейчас главная цель — увидеть, как уже изученные конструкции объединяются внутри функции.

Функции объединяют изученные конструкции

Внутри функции можно использовать практически всё, что мы уже изучили:

```
переменные
input()
арифметику
сравнения
if / elif / else
and / or / not
while
for
строки
списки
```

Например:

```
def count_positive(numbers):
    count = 0

    for number in numbers:
        if number > 0:
            count += 1

    return count
```

Вызов:

```
values = [-2, 5, 0, 10, -1]

print(count_positive(values))
```

Результат:

```
2
```

Функция становится способом объединить уже знакомые конструкции в отдельный инструмент.

Что важно запомнить

Функция — это именованный блок кода.

Создание функции:

```
def greet():
    print("Привет!")
```

Вызов:

```
greet()
```

Функция может принимать параметры:

```
def greet(name):
    print("Привет,", name)
```

и аргументы передаются при вызове:

```
greet("Анна")
```

Несколько параметров:

```
def add(a, b):
    ...
```

Возврат результата:

```
def add(a, b):
    return a + b
```

Использование результата:

```
result = add(10, 5)
```

Важно различать:

```
print()
→ показывает значение

return
→ возвращает значение
```

Если явного `return` нет, функция возвращает:

```
None
```

Переменные, созданные внутри функции, обычно локальные.

Функции помогают:

```
убирать повторение
↓
разделять большую программу
↓
давать действиям понятные имена
↓
повторно использовать код
↓
делать программу понятнее
```

И самое важное:

> **Функция превращает набор команд в отдельный переиспользуемый инструмент программы.**

Завершение тома II

На этом заканчивается **Том II — «Базовые конструкции Python»**.

Мы начали с простых команд и постепенно дошли до программ, которые умеют получать данные, обрабатывать их, принимать решения и разделять логику на функции.

За этот том мы познакомились с основными строительными блоками Python:

```
переменные
↓
типы данных
↓
input()
↓
преобразование типов
↓
арифметика
↓
сравнения
↓
условия
↓
логические операции
↓
while
↓
for
↓
строки
↓
списки
↓
функции
```

Финальная карта показывает, как функции объединяют эти блоки в переиспользуемые части программы.

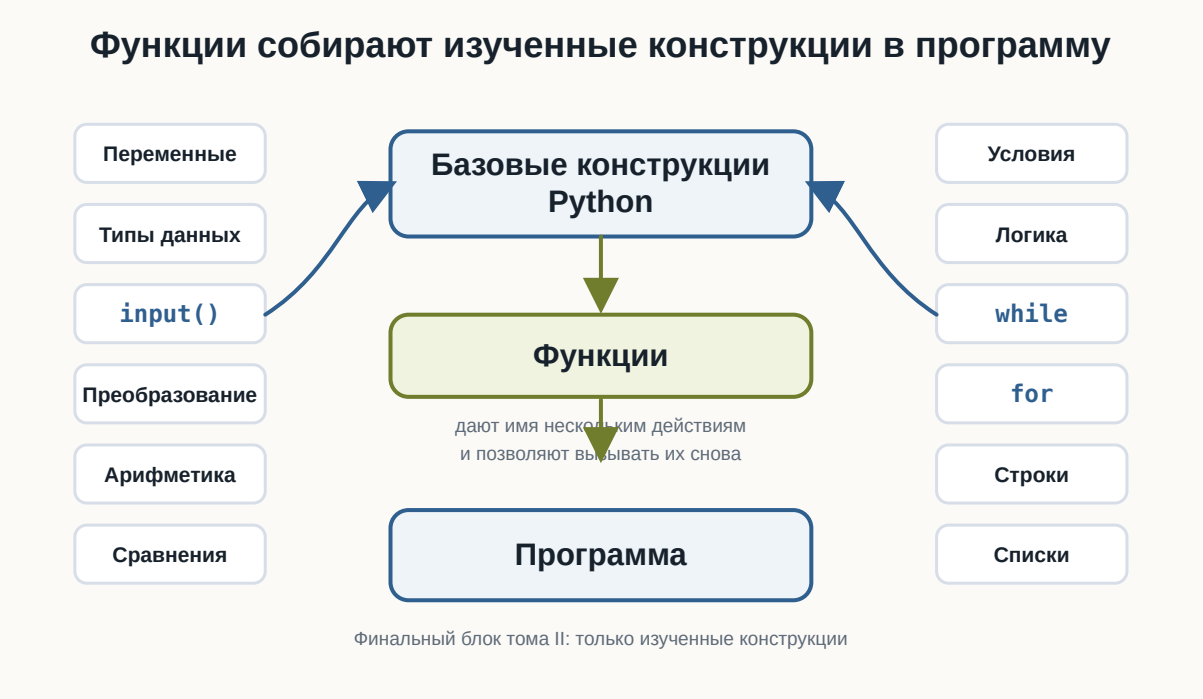


Рисунок 2.5: Базовые конструкции Python

Теперь код уже может выглядеть так:

В этой небольшой программе соединяется почти всё, что мы изучали в томе.

И это важная точка.

Теперь отдельные конструкции Python начинают складываться в программы, состоящие из понятных частей.

Дальше можно двигаться к более крупным идеям языка и разработки, опираясь на этот фундамент.

> **Итог тома**
>
> Главная цель этого тома была не просто запомнить синтаксис.
> Важно научиться видеть программу как сочетание нескольких базовых идей:

Список 2.12 `final-volume-project.py`

```
def calculate_average(numbers):
    return sum(numbers) / len(numbers)


def count_positive(numbers):
    count = 0

    for number in numbers:
        if number > 0:
            count += 1

    return count


numbers = []

for i in range(5):
    number = float(input("Введите число: "))
    numbers.append(number)


average = calculate_average(numbers)
positive_count = count_positive(numbers)

print("Список:", numbers)
print("Среднее:", average)
print("Положительных:", positive_count)
print("Минимум:", min(numbers))
print("Максимум:", max(numbers))
```

```
данные
↓
вычисления
↓
решения
↓
повторения
↓
структуры данных
↓
функции
```

Именно из этих элементов начинают складываться более серьёзные программы.

Практические задания

Задание 1. Первая функция

Создайте функцию:

```
say_hello()
```

которая выводит:

```
Привет, Python!
```

Вызовите её два раза.

> Ответ

```
def say_hello():
    print("Привет, Python!")


say_hello()
say_hello()
```

Задание 2. Приветствие пользователя

Создайте функцию:

```
greet(name)
```

которая выводит приветствие.

Пример:

```
greet("Анна")
```

Результат:

```
Привет, Анна
```

Ответ

```
def greet(name):
    print("Привет,", name)


greet("Анна")
```

Задание 3. Сумма двух чисел

Создайте функцию:

```
add(a, b)
```

которая возвращает сумму двух чисел.

Ответ

```
def add(a, b):
    return a + b


result = add(10, 5)

print(result)
```

Результат:

```
15
```

Задание 4. Площадь прямоугольника

Создайте функцию:

```
rectangle_area(width, height)
```

которая возвращает площадь прямоугольника.

Ответ

```
def rectangle_area(width, height):
    return width * height


area = rectangle_area(5, 3)

print(area)
```

Результат:

```
15
```

Задание 5. Совершеннолетие

Создайте функцию:

```
is_adult(age)
```

которая возвращает:

```
True
```

если возраст не меньше 18, иначе:

```
False
```

Ответ

```
def is_adult(age):
    return age >= 18


print(is_adult(20))
print(is_adult(15))
```

Результат:

```
True
False
```

Задание 6. Положительное, отрицательное или ноль

Создайте функцию:

```
describe_number(number)
```

которая возвращает одну из строк:

```
Положительное
Отрицательное
Ноль
```

Ответ

```
def describe_number(number):
    if number > 0:
        return "Положительное"
    elif number < 0:
        return "Отрицательное"
    else:
        return "Ноль"


print(describe_number(10))
print(describe_number(-5))
print(describe_number(0))
```

Задание 7. Приветствие по умолчанию

Создайте функцию:

```
greet(name, greeting="Привет")
```

Примеры:

```
greet("Анна")
greet("Максим", "Здравствуйте")
```

Ответ

```
def greet(name, greeting="Привет"):
    print(greeting + ",", name)


greet("Анна")
greet("Максим", "Здравствуйте")
```

Результат:

```
Привет, Анна
Здравствуйте, Максим
```

Задание 8. Сумма списка

Создайте собственную функцию:

```
calculate_sum(numbers)
```

которая с помощью `for` возвращает сумму элементов списка.

Не используйте `sum()` внутри функции.

Ответ

```
def calculate_sum(numbers):
    total = 0

    for number in numbers:
        total += number

    return total


values = [10, 20, 30]

print(calculate_sum(values))
```

Результат:

```
60
```

Задание 9. Только положительные значения

Создайте функцию:

```
count_positive(numbers)
```

которая возвращает количество положительных чисел в списке.

Ответ

```
def count_positive(numbers):
    count = 0

    for number in numbers:
        if number > 0:
            count += 1

    return count


values = [-5, 10, 0, 3, -2, 7]

print(count_positive(values))
```

Результат:

```
3
```

Задание 10. Удвоение списка

Создайте функцию:

```
double_numbers(numbers)
```

которая возвращает новый список, где каждое число умножено на 2.

Ответ

```
def double_numbers(numbers):
    result = []

    for number in numbers:
        result.append(number * 2)

    return result


numbers = [1, 2, 3, 4]

print(double_numbers(numbers))
```

Результат:

```
[2, 4, 6, 8]
```

Задание 11. Найдите ошибку

Что не так с этой программой?

```
def add(a, b):
    print(a + b)


result = add(10, 5)

print(result)
```

Какой будет вывод?

Исправьте функцию так, чтобы результат можно было сохранить.

Ответ

Функция выводит:

```
15
```

но ничего явно не возвращает.
Поэтому:

```
result
```

получает:

```
None
```

Полный вывод:

```
15
None
```

Нужно использовать `return`:

```
def add(a, b):
    return a + b


result = add(10, 5)

print(result)
```

Теперь:

```
15
```

Задание 12. Что выполнится после return?

Не запускайте код сразу:

```
def test():
    print("Начало")
    return 10
    print("Конец")


result = test()

print(result)
```

Определите вывод.

Ответ

Результат:

```
Начало
10
```

Строка:

```
print("Конец")
```

не выполнится.
После:

```
return 10
```

функция завершается.

Задание 13. Локальная переменная

Что произойдёт?

```
def example():
    message = "Python"

    print(message)


example()

print(message)
```

Ответ

Внутри функции будет выведено:

```
Python
```

Но внешний:

```
print(message)
```

приведёт к:

```
NameError
```

`message` — локальная переменная функции.

Задание 14. Функция максимума

Создайте функцию:

```
find_max(numbers)
```

которая возвращает максимальное значение непустого списка.

Не используйте встроенную функцию `max()`.

Ответ

```
def find_max(numbers):
    maximum = numbers[0]

    for number in numbers:
        if number > maximum:
            maximum = number

    return maximum


numbers = [10, 5, 30, 12, 25]

print(find_max(numbers))
```

Результат:

```
30
```

Задание 15. Итог покупки

Создайте две функции:

```
calculate_total(price, quantity)
```

которая возвращает стоимость товаров,

и:

```
calculate_discount(total)
```

которая возвращает скидку 10%, если сумма не меньше 3000, иначе 0.

Получите цену и количество через input() и выведите итоговую сумму.

Ответ

```
def calculate_total(price, quantity):
    return price * quantity


def calculate_discount(total):
    if total >= 3000:
        return total * 0.1

    return 0


price = float(input("Цена товара: "))
quantity = int(input("Количество: "))

total = calculate_total(price, quantity)
discount = calculate_discount(total)

final_price = total - discount

print("Сумма:", total)
print("Скидка:", discount)
print("К оплате:", final_price)
```

Задание 16. Финальная задача тома

Напишите программу для обработки результатов пользователя.

Программа должна:

1. попросить пользователя ввести 5 чисел;

2. сохранить их в список;
3. создать функцию:

```
calculate_average(numbers)
```

которая возвращает среднее значение;

4. создать функцию:

```
count_positive(numbers)
```

которая возвращает количество положительных чисел;

5. вывести:
    - список;
    - среднее значение;
    - количество положительных чисел;
    - минимальное значение;
    - максимальное значение.

Ответ

```
def calculate_average(numbers):
    return sum(numbers) / len(numbers)


def count_positive(numbers):
    count = 0

    for number in numbers:
        if number > 0:
            count += 1

    return count


numbers = []

for i in range(5):
    number = float(input("Введите число: "))
    numbers.append(number)


average = calculate_average(numbers)
positive_count = count_positive(numbers)

print("Список:", numbers)
print("Среднее:", average)
print("Положительных:", positive_count)
print("Минимум:", min(numbers))
print("Максимум:", max(numbers))
```

Эта задача объединяет основные конструкции всего тома:

```
input()
↓
преобразование типов
↓
for
↓
список
↓
append()
↓
функции
↓
параметры
↓
условия
↓
return
↓
обработка результата
```

Часть III

Том III. Структуры данных и организация кода

Том III посвящён структурам данных и организации растущих Python-программ.

После переменных, условий, циклов, строк, списков и функций читатель переходит к более выразительным способам хранить данные, безопаснее работать с ошибками и разделять программу на несколько файлов.

Главная линия тома ведёт к небольшому консольному приложению: менеджеру задач.

Быстрые ссылки

- 4.1 Кортежи
- 4.2 Множества
- 4.3 Словари
- 4.4 Работа со словарями
- 4.5 Вложенные структуры данных
- 4.6 List comprehensions
- Обработка ошибок

> **Логика тома**
>
> Идите по главам последовательно: от неизменяемых наборов значений к уникальным элементам, словарям, вложенным структурам, компактному созданию списков, обработке ошибок, модулям, пакетам, файлам и JSON.

2 4.1 Кортежи

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как хранить несколько значений, которые после создания не должны изменяться?**

В предыдущем томе мы познакомились со списками. Список хранит несколько значений и позволяет менять набор элементов:

```
languages = ["Python", "Java", "Go"]

languages.append("Rust")
languages[1] = "C++"
```

Но иногда набор значений должен оставаться фиксированным после создания.

Например:

```
координаты точки
цвет в формате RGB
дата
размер изображения
набор фиксированных настроек
```

Для таких случаев в Python есть **кортеж**.

Первый кортеж

Кортеж обычно записывают в круглых скобках:

```
coordinates = (10, 20)

print(coordinates)
print(type(coordinates))
```

Результат:

```
(10, 20)
<class 'tuple'>
```

Внутри два значения, но вся конструкция целиком — одно значение типа `tuple`.

Список 2.1 `tuple-basics.py`

```
coordinates = (10, 20)
languages = ("Python", "Java", "Go")
empty = ()

print(coordinates)
print(type(coordinates))

print(languages)
print(len(languages))

print(empty)
print(len(empty))
```

Элементы кортежа

Как и список, кортеж содержит элементы в определённом порядке.

Например:

```
point = (10, 20, 30)
```

Здесь три элемента: `10`, `20` и `30`. Все они принадлежат одному значению `point`.



Рисунок 2.1: Устройство кортежа

Кортеж похож на список

Список и кортеж во многом ведут себя одинаково:

```
numbers_list = [10, 20, 30]
numbers_tuple = (10, 20, 30)
```

Обе структуры сохраняют порядок, поддерживают индексы, `len()`, срезы, перебор через `for` и проверку через `in`.

Главное различие:

```
list  → изменяемый

tuple → неизменяемый
```

Кортеж, как и список, может хранить значения разных типов:

```
user = ("Анна", 25, 1.68, True)

print(user)
```

Но чаще кортеж используют для небольшой фиксированной группы связанных значений: координат, RGB-цвета, размера изображения.

Пустой кортеж

Кортеж может быть пустым:

```
items = ()
```

Проверим:

```
items = ()

print(items)
print(type(items))
print(len(items))
```

Результат:

```
()
<class 'tuple'>
0
```

Кортеж из одного элемента

У кортежа из одного элемента есть важная особенность:

```
value = (10)

print(type(value))
```

Результат:

```
<class 'int'>
```

Круглые скобки сами по себе могут просто группировать выражение. Для кортежа из одного элемента нужна запятая:

```
value = (10,)

print(value)
print(type(value))
```

Результат:

```
(10,)
<class 'tuple'>
```

Главная разница:

```
(10)  → int

(10,) → tuple
```

> ⚠ Не забывайте запятую
>
> Кортежи обычно записывают в круглых скобках, а важную роль в синтаксисе кортежа играет запятая.

Скобки помогают читать код

Технически кортеж можно записать и без круглых скобок:

```
point = 10, 20
```

Проверим:

```
point = 10, 20

print(point)
print(type(point))
```

Результат:

```
(10, 20)
<class 'tuple'>
```

Но в учебном коде чаще будем использовать:

```
point = (10, 20)
```

Так структура заметнее, и читателю проще увидеть, что это единая группа значений.

Список 2.2 `single-element-tuple.py`

```
value1 = (10)
value2 = (10,)

print(value1)
print(type(value1))

print(value2)
print(type(value2))

point = 10, 20

print(point)
print(type(point))
```

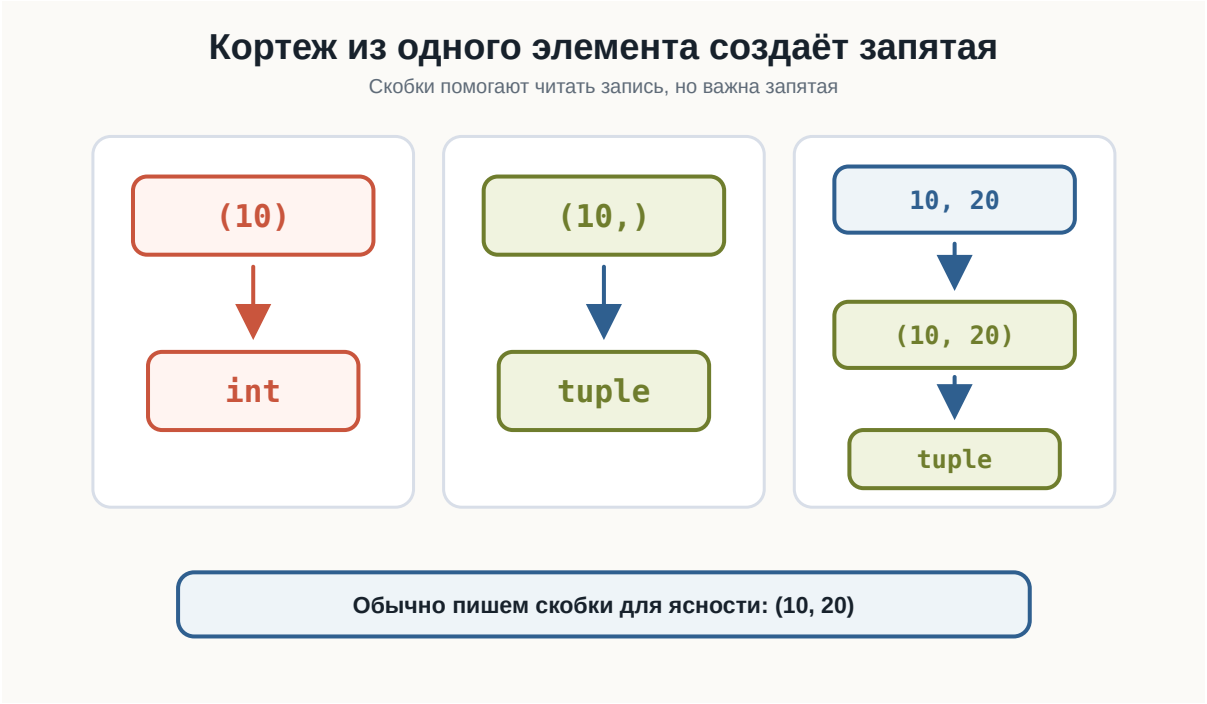


Рисунок 2.2: Кортеж из одного элемента

Индексы кортежа

Индексы кортежа работают так же, как индексы строки и списка.

Рассмотрим:

```
languages = ("Python", "Java", "Go")
```

Положительные индексы начинаются с `0`, отрицательные идут с конца:

```
элемент:   Python   Java   Go
индекс:      0       1     2
отриц.:     -3      -2    -1
```

Пример:

```
languages = ("Python", "Java", "Go")

print(languages[0])
print(languages[1])
print(languages[-1])
print(languages[-2])
```

Результат:

```
Python
Java
Go
Java
```

Список 2.3 `tuple-indexing.py`

```
languages = ("Python", "Java", "Go")

print(languages[0])
print(languages[1])
print(languages[-1])
print(languages[-2])
```

Ошибка индекса

Если обратиться к несуществующей позиции:

```
languages = ("Python", "Java", "Go")

print(languages[10])
```

Python сообщит:

```
IndexError
```

Причина та же, что со строками и списками: элемента с таким индексом нет.

Длина кортежа

Количество элементов возвращает `len()`:

```
languages = ("Python", "Java", "Go")

print(len(languages))
print(len(()))
```

Результат:

```
3
0
```

Срезы кортежей

Кортеж поддерживает срезы в формате `[start:stop:step]`. Граница `stop` не включается:

```
numbers = (10, 20, 30, 40, 50)

print(numbers[1:4])
print(numbers[:3])
```

```
print(numbers[2:])
print(numbers[-2:])
print(numbers[::2])
print(numbers[::-1])
```

Результат:

```
(20, 30, 40)
(10, 20, 30)
(30, 40, 50)
(40, 50)
(10, 30, 50)
(50, 40, 30, 20, 10)
```

Срез кортежа возвращает новый кортеж. Исходный кортеж не изменяется.

Список 2.4 `tuple-slicing.py`

```
numbers = (10, 20, 30, 40, 50)

print(numbers[1:4])
print(numbers[:3])
print(numbers[2:])
print(numbers[-2:])
print(numbers[::2])
print(numbers[::-1])
```

Перебор и проверка

Кортеж можно перебирать через `for` и проверять через `in` / `not in`:

```
languages = ("Python", "Java", "Go")

for language in languages:
    print(language)

print("Python" in languages)
print("Rust" not in languages)
```

Список 2.5 `tuple-iteration.py`

```
languages = ("Python", "Java", "Go")

for language in languages:
    print(language)

print("Python" in languages)
print("Rust" not in languages)
```

Кортеж нельзя изменить

Рассмотрим список:

```
languages = ["Python", "Java", "Go"]

languages[1] = "Rust"

print(languages)
```

Результат:

```
['Python', 'Rust', 'Go']
```

Со списком это работает.

С кортежем та же операция приведёт к ошибке:

```
languages = ("Python", "Java", "Go")

languages[1] = "Rust"
```

Python сообщит об ошибке:

```
TypeError
```

Кортеж **неизменяемый**.

Что означает неизменяемость

После создания кортежа нельзя заменить, добавить или удалить отдельный элемент.

Список 2.6 `tuple-immutability.py`

```
languages_list = ["Python", "Java", "Go"]
languages_list[1] = "Rust"

print(languages_list)

languages_tuple = ("Python", "Java", "Go")

# Ошибка сделана намеренно:
languages_tuple[1] = "Rust"
```

Например, не работает:

```
point[0] = 100
```

! Кортеж неизменяемый

После создания:

```
point = (10, 20)
```

нельзя изменить отдельный элемент:

```
point[0] = 100
```

Если нужны другие значения, создаётся новый кортеж.



Рисунок 2.3: List и tuple

Можно создать новый кортеж

Неизменяемость не означает, что имя переменной навсегда связано с одним значением:

```
point = (10, 20)

point = (100, 200)

print(point)
```

Результат:

```
(100, 200)
```

Здесь старый кортеж не изменился. Переменная `point` просто получила новое значение.

Объединение кортежей

Кортежи можно объединять через +:

```
first = (1, 2)
second = (3, 4)

result = first + second

print(result)
```

Результат:

```
(1, 2, 3, 4)
```

Исходные кортежи не изменяются. Создаётся новый кортеж.

Повторение кортежа

Оператор * повторяет содержимое и тоже создаёт новый кортеж:

```
values = (1, 2)

print(values * 3)
```

Результат:

```
(1, 2, 1, 2, 1, 2)
```

---

count()

Метод `count()` считает количество совпадений:

```
numbers = (10, 20, 10, 30, 10)

print(numbers.count(10))
```

Результат:

```
3
```

---

index()

Метод `index()` возвращает индекс первого совпадения:

```
languages = ("Python", "Java", "Go")

print(languages.index("Java"))
```

Результат:

```
1
```

Если значения нет, возникнет:

```
ValueError
```

Почему у кортежа мало методов

У списка есть методы для изменения: `append()`, `insert()`, `remove()`, `pop()`, `sort()`. У кортежа таких методов нет, потому что он неизменяемый.

Основные методы кортежа всего два:

```
count()
index()
```

List и tuple

Соберём главное различие:

Свойство	`list`	`tuple`
Синтаксис	`[1, 2, 3]`	`(1, 2, 3)`
Индексы	да	да
Отрицательные индексы	да	да
`len()`	да	да
Срезы	да	да
`for`	да	да
`in`	да	да
Изменение элемента	да	нет
`append()`	да	нет
`remove()`	да	нет

Используйте `list`, когда набор должен изменяться:

```
tasks = ["Учить Python", "Сделать проект"]
tasks.append("Прочитать документацию")
```

Используйте `tuple`, когда значения образуют фиксированную группу:

```
point = (10, 20)
color = (255, 120, 40)
size = (1920, 1080)
```

> Выбирайте структуру по смыслу
>
> Не нужно заменять все списки кортежами. `list` и `tuple` решают разные задачи.

Распаковка кортежа

Кортеж можно разложить по нескольким переменным:

```
point = (10, 20)

x, y = point

print(x)
print(y)
```

Результат:

```
10
20
```

Это называется **распаковкой**:

```
(10, 20)
   ↓
 x    y
 ↓    ↓
10   20
```

Порядок важен: первый элемент попадает в первую переменную, второй — во вторую.

Список 2.7 `tuple-unpacking.py`

```
point = (10, 20)

x, y = point

print("x:", x)
print("y:", y)

color = (255, 120, 40)

red, green, blue = color

print(red)
print(green)
print(blue)
```



Рисунок 2.4: Распаковка кортежа

Распаковка трёх значений

Например:

```
color = (255, 120, 40)

red, green, blue = color

print(red)
print(green)
print(blue)
```

Результат:

```
255
120
40
```

Здесь:

```
255 → red
120 → green
40  → blue
```

Количество переменных должно совпадать

При простой распаковке количество переменных должно совпадать с количеством элементов:

```
point = (10, 20)
x, y, z = point
```

Python сообщит:

```
ValueError
```

> ⚠ Следите за количеством значений
>
> При простой распаковке:
>
> ```
> x, y = (10, 20)
> ```
>
> количество переменных должно соответствовать количеству элементов.
>
> ```
> 2 элемента
> ↓
> 2 переменные
> ```

Кортеж можно распаковать сразу

Отдельная переменная для кортежа не обязательна:

```
x, y = (10, 20)

print(x)
print(y)
```

Результат:

```
10
20
```

Можно встретить и запись без скобок:

```
x, y = 10, 20
```

Результат тот же: несколько значений распределяются по нескольким переменным.

Обмен значений переменных

Распаковка позволяет удобно поменять значения местами:

```
a = 10
b = 20

a, b = b, a

print(a)
print(b)
```

Результат:

```
20
10
```

Функция может вернуть несколько значений

В предыдущей главе мы изучили функции. Если функция возвращает несколько значений через запятую, Python собирает их в кортеж:

```
def get_point():
    return 10, 20
```

Список 2.8 `swap-values.py`

```
a = 10
b = 20

print("До:")
print("a =", a)
print("b =", b)

a, b = b, a

print("После:")
print("a =", a)
print("b =", b)
```

Вызов:

```
point = get_point()

print(point)
print(type(point))
```

Результат:

```
(10, 20)
<class 'tuple'>
```

Результат функции можно сразу распаковать:

```
def get_point():
    return 10, 20


x, y = get_point()

print(x)
print(y)
```

Результат:

```
10
20
```

Получается цепочка:

```
функция
   ↓
return 10, 20
   ↓
(10, 20)
   ↓
распаковка
   ↓
x = 10
y = 20
```

Так функции и кортежи хорошо работают вместе.

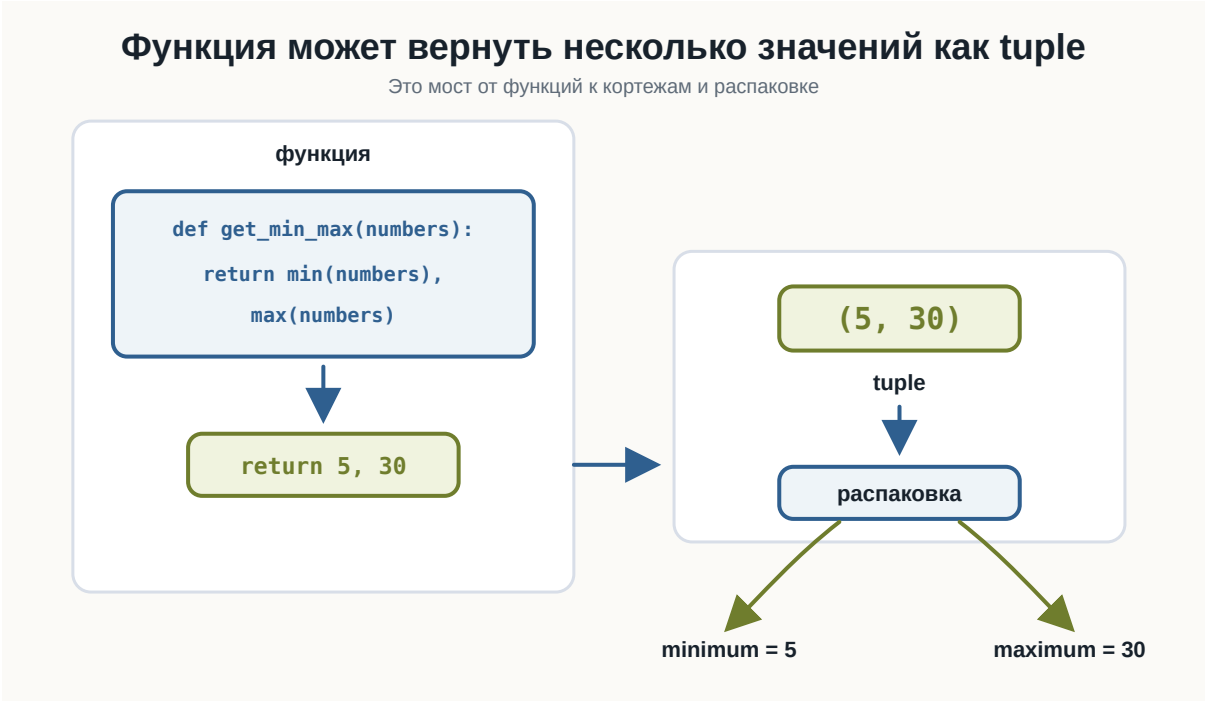


Рисунок 2.5: Функция возвращает tuple

Пример: минимальное и максимальное значение

Функция может вернуть два связанных результата:

```
def get_min_max(numbers):
    return min(numbers), max(numbers)


numbers = [12, 5, 30, 8]
```

```
minimum, maximum = get_min_max(numbers)

print("Минимум:", minimum)
print("Максимум:", maximum)
```

Результат:

```
Минимум: 5
Максимум: 30
```

Список 2.9 `function-return-tuple.py`

```
def get_min_max(numbers):
    return min(numbers), max(numbers)


numbers = [12, 5, 30, 8]

minimum, maximum = get_min_max(numbers)

print("Минимум:", minimum)
print("Максимум:", maximum)
print(type(get_min_max(numbers)))
```

Кортежи внутри списка

Кортежи можно хранить внутри списка:

```
points = [
    (10, 20),
    (30, 40),
    (50, 60)
]
```

Здесь `points` — изменяемый список точек, а каждая точка — фиксированный кортеж координат.

Перебор списка кортежей

Можно вывести каждую точку целиком:

```
points = [
    (10, 20),
    (30, 40),
    (50, 60)
]

for point in points:
    print(point)
```

Результат:

```
(10, 20)
(30, 40)
(50, 60)
```

Распаковка прямо в for

Или сразу распаковать каждую точку в цикле:

```
points = [
    (10, 20),
    (30, 40),
    (50, 60)
]

for x, y in points:
    print("x:", x, "y:", y)
```

Результат:

```
x: 10 y: 20
x: 30 y: 40
x: 50 y: 60
```

На каждой итерации очередной кортеж распаковывается в x и y.

Список 2.10 `points-list.py`

```
points = [
    (10, 20),
    (30, 40),
    (50, 60)
]

for x, y in points:
    print("x =", x, "y =", y)
```

Где list и tuple работают вместе

Частый вариант: список хранит изменяемый набор объектов, а кортеж описывает один фиксированный объект.

```
points = [
    (2, 5),
    (10, 3),
    (7, 8)
]

for x, y in points:
    print("Координаты:", x, y)
```

Ещё пример:

```
resolutions = [
    (1920, 1080),
    (2560, 1440),
    (3840, 2160)
]

for width, height in resolutions:
    print(width, "x", height)
```

Смысл тот же: `list` хранит набор, который может расти или уменьшаться, а `tuple` хранит фиксированную пару значений.

bool кортежа

Как и другие коллекции, пустой кортеж:

```
()
```

считается ложным значением:

```
print(bool(()))
```

Результат:

```
False
```

Непустой:

```
print(bool((10, 20)))
```

Результат:

```
True
```

Поэтому можно встретить:

```
if coordinates:
    print("Координаты заданы")
```

Итоговый ориентир

```
Будут ли элементы набора изменяться?
          ↓
      да      нет
      ↓        ↓
    list     tuple
```

Это не жёсткое правило на все случаи. Главное — понимать смысл данных и выбирать структуру осознанно.

Попробуйте сами

Список 2.11 `tuple-analysis.py`

```
def analyze_numbers(numbers):
    minimum = min(numbers)
    maximum = max(numbers)
    average = sum(numbers) / len(numbers)

    return minimum, maximum, average


numbers = [10, 20, 30, 40]

minimum, maximum, average = analyze_numbers(numbers)

print("Минимум:", minimum)
print("Максимум:", maximum)
print("Среднее:", average)
```

Эксперимент

Создайте:

```
values = (10, 20, 30, 40, 50)
```

Перед запуском предскажите результаты:

```
print(values[0])
print(values[-1])
print(values[1:4])
print(len(values))
print(30 in values)
```

Затем попробуйте:

```
values[0] = 100
```

и посмотрите, какую ошибку сообщит Python.
После этого сравните тот же эксперимент со списком:

```
values = [10, 20, 30, 40, 50]
values[0] = 100
```

Обратите внимание на различие между `tuple` и `list`.

Что важно запомнить

Кортеж имеет тип:

```
tuple
```

Пример:

```
point = (10, 20)
```

Он хранит несколько элементов:

```
(10, 20)
```

Поддерживает индексы:

```
point[0]
point[-1]
```

Длину:

```
len(point)
```

Срезы:

```
point[start:stop:step]
```

Перебор:

```
for value in values:
    ...
```

Проверку:

```
value in values
value not in values
```

Основные методы:

```
count()
index()
```

Главное отличие:

```
list
→ изменяемый

tuple
→ неизменяемый
```

Особый случай:

```
(10)  → int
(10,) → tuple
```

Кортеж можно распаковать:

```
x, y = (10, 20)
```

Функция также может вернуть несколько значений:

```
def get_point():
    return 10, 20
```

И самое важное:

> **Кортеж удобно использовать для фиксированной упорядоченной группы связанных значений.**

Практические задания

Задание 1. Первый кортеж

Создайте кортеж:

```
Python
Java
Go
```

Выведите его и его тип.

Ответ

```
languages = ("Python", "Java", "Go")

print(languages)
print(type(languages))
```

Результат:

```
('Python', 'Java', 'Go')
<class 'tuple'>
```

Задание 2. Первый и последний элемент

Дан:

```
numbers = (10, 20, 30, 40)
```

Выведите первый и последний элементы.

Ответ

```
numbers = (10, 20, 30, 40)

print(numbers[0])
print(numbers[-1])
```

Результат:

```
10
40
```

Задание 3. Длина кортежа

Дан:

```
cities = ("Москва", "Казань", "Самара", "Омск")
```

Выведите количество элементов.

Ответ

```
cities = ("Москва", "Казань", "Самара", "Омск")

print(len(cities))
```

Результат:

```
4
```

Задание 4. Кортеж из одного элемента

Как создать кортеж, содержащий только:

```
Python
```

Проверьте результат через `type()`.

Ответ

```
language = ("Python",)

print(language)
print(type(language))
```

Важно оставить запятую:

```
("Python",)
```

Без неё:

```
("Python")
```

получится обычная строка.

Задание 5. Срез

Дан:

```
numbers = (10, 20, 30, 40, 50)
```

Получите:

```
(20, 30, 40)
```

с помощью среза.

> **Ответ**
>
> ```
> numbers = (10, 20, 30, 40, 50)
>
> print(numbers[1:4])
> ```

Задание 6. Проверка элемента

Дан:

```
languages = ("Python", "Java", "Go")
```

Попросите пользователя ввести язык и проверьте, содержится ли он в кортеже.

> **Ответ**
>
> ```
> languages = ("Python", "Java", "Go")
>
> language = input("Введите язык: ")
>
> if language in languages:
>     print("Язык найден")
> else:
>     print("Языка нет")
> ```

Задание 7. Перебор кортежа

Дан:

```
colors = ("red", "green", "blue")
```

Выведите каждый элемент на отдельной строке.

> **Ответ**
>
> ```
> colors = ("red", "green", "blue")
>
> for color in colors:
>     print(color)
> ```

Задание 8. Найдите ошибку

Почему этот код не работает?

```
point = (10, 20)

point[0] = 100
```

> **Ответ**
>
> Кортежи неизменяемые.
> После создания нельзя изменить отдельный элемент:
>
> ```
> point[0] = 100
> ```
>
> Python сообщит:
>
> ```
> TypeError
> ```
>
> Если нужны новые координаты, можно создать новый кортеж:
>
> ```
> point = (100, 20)
> ```

Задание 9. Распаковка

Дан:

```
point = (15, 25)
```

Распакуйте его в переменные:

```
x
y
```

и выведите их.

Ответ

```
point = (15, 25)

x, y = point

print("x:", x)
print("y:", y)
```

Результат:

```
x: 15
y: 25
```

Задание 10. RGB

Дан цвет:

```
color = (255, 128, 64)
```

Распакуйте его в:

```
red
green
blue
```

Ответ

```
color = (255, 128, 64)

red, green, blue = color

print("Red:", red)
print("Green:", green)
print("Blue:", blue)
```

Задание 11. count()

Дан:

```
numbers = (10, 20, 10, 30, 10, 40)
```

Определите, сколько раз встречается число `10`.

Ответ

```
numbers = (10, 20, 10, 30, 10, 40)

print(numbers.count(10))
```

Результат:

```
3
```

Задание 12. index()

Дан:

```
languages = ("Python", "Java", "Go", "Rust")
```

Найдите индекс:

```
Go
```

Ответ

```
languages = ("Python", "Java", "Go", "Rust")

print(languages.index("Go"))
```

Результат:

```
2
```

Задание 13. Функция с двумя результатами

Создайте функцию:

```
get_min_max(numbers)
```

которая возвращает минимальное и максимальное значения списка.

Затем распакуйте результат в две переменные.

Ответ

```
def get_min_max(numbers):
    return min(numbers), max(numbers)


numbers = [12, 5, 30, 8]

minimum, maximum = get_min_max(numbers)

print("Минимум:", minimum)
print("Максимум:", maximum)
```

Результат:

```
Минимум: 5
Максимум: 30
```

Функция возвращает:

```
(5, 30)
```

который затем распаковывается.

Задание 14. Список координат

Дан список:

```
points = [
    (10, 20),
    (30, 40),
    (50, 60)
]
```

С помощью распаковки внутри `for` выведите:

```
x = 10, y = 20
x = 30, y = 40
x = 50, y = 60
```

Ответ

```
points = [
    (10, 20),
    (30, 40),
    (50, 60)
]

for x, y in points:
    print("x =", x, ", y =", y)
```

На каждой итерации очередной кортеж:

```
(x, y)
```

распаковывается в две переменные.

Задание 15. List или tuple?

Выберите подходящую структуру для каждого случая:

1. список покупок;
2. координаты точки `(x, y)`;
3. список текущих задач;
4. размер экрана `(width, height)`;
5. список пользователей.

Ответ

Один из разумных вариантов:

```
список покупок
→ list

координаты точки
→ tuple

список задач
→ list

размер экрана
→ tuple

список пользователей
→ list
```

Причина:

```
list
→ набор предполагается изменять

tuple
→ фиксированная группа связанных значений
```

Это не абсолютное правило для всех программ, но хороший ориентир на текущем этапе.

Задание 16. Задача со звёздочкой

Создайте функцию:

```
analyze_numbers(numbers)
```

которая получает список чисел и возвращает кортеж:

```
(минимум, максимум, среднее)
```

Затем распакуйте результат.

Например:

```
numbers = [10, 20, 30]
```

должно дать:

```
Минимум: 10
Максимум: 30
Среднее: 20.0
```

Ответ

```
def analyze_numbers(numbers):
    minimum = min(numbers)
    maximum = max(numbers)
    average = sum(numbers) / len(numbers)

    return minimum, maximum, average


numbers = [10, 20, 30]

minimum, maximum, average = analyze_numbers(numbers)

print("Минимум:", minimum)
print("Максимум:", maximum)
print("Среднее:", average)
```

Функция возвращает три значения:

```
return minimum, maximum, average
```

Python объединяет их в кортеж:

```
(10, 30, 20.0)
```

После чего результат распаковывается в три переменные.

Что дальше?

Кортеж помогает хранить несколько значений в определённом порядке:

```
color = (255, 120, 40)
```

Причём этот набор после создания не изменяется.

Но иногда порядок вообще не важен.

Вместо этого нужно другое свойство:

```
каждое значение должно встречаться только один раз
```

Например:

```
уникальные теги
уникальные категории
уникальные посетители
уникальные значения из списка
```

Для таких задач Python предоставляет ещё одну структуру данных — **множество**.

В следующей главе познакомимся с типом:

```
set
```

и узнаем, как Python работает с уникальными значениями.

2 4.2 Множества

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как хранить уникальные значения и выполнять операции над группами данных?**

В предыдущей главе мы познакомились с кортежами:

```
point = (10, 20)
```

Кортеж сохраняет порядок элементов и подходит для фиксированного набора значений.

Теперь рассмотрим другую задачу.

Представим список языков:

```
languages = ["Python", "Java", "Python", "Go", "Java"]
```

Некоторые значения повторяются.

Если нас интересует только то, **какие языки присутствуют**, а количество повторений не имеет значения, удобно использовать множество:

```
languages = {"Python", "Java", "Go"}
```

В Python множества имеют тип `set`.

Что такое множество

Множество — коллекция уникальных элементов.

Создадим первое множество:

```
languages = {"Python", "Java", "Go"}

print(languages)
print(type(languages))
```

Тип:

```
<class 'set'>
```

Главное свойство видно на примере:

```
numbers = {10, 20, 10, 30, 20}

print(numbers)
```

В множестве останутся только уникальные значения:

```
10
20
30
```

Повторные 10 и 20 не создают дополнительные элементы.

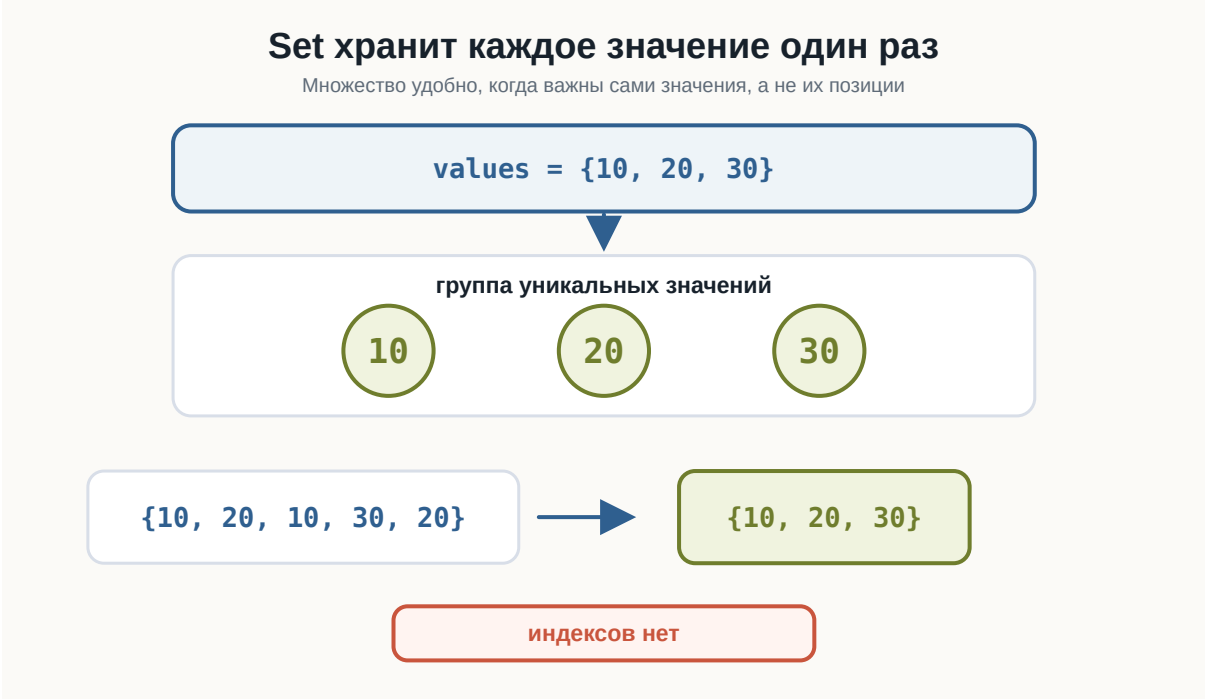


Рисунок 2.1: Устройство множества

> **Порядок элементов**
>
> Не следует рассчитывать на определённый порядок элементов `set`.
> Например, множество:
>
> ```
> {"Python", "Java", "Go"}
> ```
>
> при выводе может отображаться не в том порядке, в котором значения были записаны.
> Если порядок данных важен, нужно выбрать другую структуру.

Создание множества

Множество с элементами можно создать с помощью фигурных скобок:

```
numbers = {10, 20, 30}
```

Строки:

```
languages = {"Python", "Java", "Go"}
```

Логические значения:

```
answers = {True, False}
```

В одном `set` технически могут находиться значения разных типов:

```
values = {10, "Python", True}
```

Но на практике понятнее, когда множество представляет одну группу однородных по смыслу данных.

Пустое множество

Здесь есть важная особенность.

Такая запись:

```
items = {}
```

не создаёт множество.

Проверим:

```
items = {}

print(type(items))
```

Результат:

```
<class 'dict'>
```

С фигурными скобками без элементов Python создаёт словарь.

Для пустого множества используется:

```
items = set()
```

Проверим:

```
items = set()

print(items)
print(type(items))
```

Результат:

```
set()
<class 'set'>
```

⚠ Не путайте {} и set()

```
{}
```

— пустой словарь.

```
set()
```

— пустое множество.
Это различие важно запомнить.

Список 2.1 `empty-set.py`

```
wrong = {}
correct = set()

print(type(wrong).__name__)
print(type(correct).__name__)
```

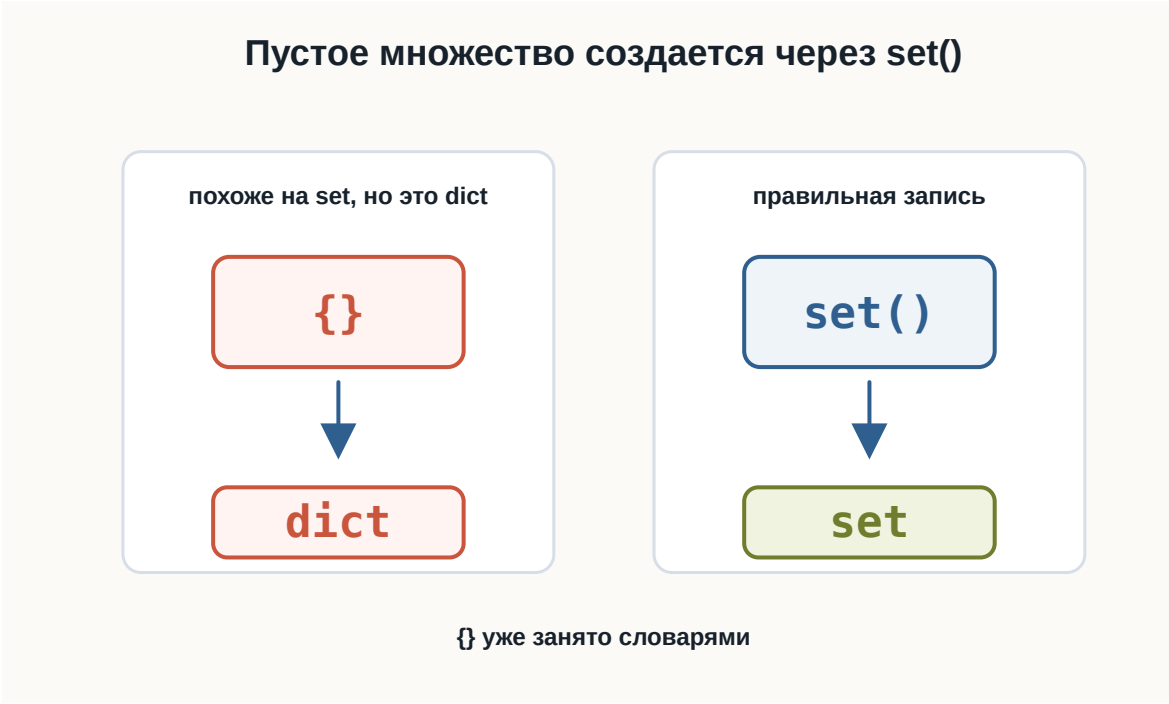


Рисунок 2.2: Пустое множество

Создание set из другой коллекции

Функция `set()` умеет принимать другую коллекцию.

Например, список:

```
numbers = [10, 20, 10, 30, 20]

unique_numbers = set(numbers)

print(unique_numbers)
```

Повторения исчезнут.

То же можно сделать со строками:

```
letters = set("banana")

print(letters)
```

В результате множество будет содержать уникальные символы строки.

Это один из самых распространённых способов применения `set`: получить уникальные значения из существующих данных.

Список 2.2 `remove-duplicates.py`

```
visits = [
    "user_17",
    "user_42",
    "user_17",
    "user_91",
    "user_42"
]

unique_users = set(visits)

print("Записей:", len(visits))
print("Пользователей:", len(unique_users))
```

Сколько уникальных значений

После преобразования можно использовать `len()`:

```
cities = ["Москва", "Казань", "Москва", "Омск", "Казань"]

unique_cities = set(cities)

print(len(unique_cities))
```

Результат:

```
3
```

Таким способом можно узнать количество **различных** значений.

Проверка элемента

Множества хорошо подходят для проверки наличия значения.

Используется уже знакомый оператор `in`:

```
allowed_roles = {"admin", "editor", "moderator"}

print("admin" in allowed_roles)
print("guest" in allowed_roles)
```

Результат:

```
True
False
```

Можно использовать условие:

```
role = "editor"

if role in allowed_roles:
    print("Роль разрешена")
```

Для проверки отсутствия используется `not in`:

```
if "guest" not in allowed_roles:
    print("Такой роли нет")
```

Список 2.3 `set-membership.py`

```
allowed_roles = {"admin", "editor", "moderator"}

role = "editor"

if role in allowed_roles:
    print("Роль разрешена")
else:
    print("Роль не разрешена")
```

У множества нет индексов

У списка можно получить элемент по позиции:

```
languages = ["Python", "Java", "Go"]

print(languages[0])
```

У множества такой операции нет:

```
languages = {"Python", "Java", "Go"}

print(languages[0])
```

Этот код приведёт к:

```
TypeError
```

Причина проста: `set` не предназначен для работы с позициями элементов.

Поэтому для множества нет смысла спрашивать:

```
какой элемент первый?
какой элемент последний?
что находится по индексу 2?
```

Если такая информация нужна, `set` — скорее всего, не та структура данных.

Добавление элемента

Для добавления используется метод:

```
add()
```

Например:

```
skills = {"Python", "Git"}

skills.add("SQL")

print(skills)
```

Теперь множество содержит:

```
Python
Git
SQL
```

Если добавить существующее значение:

```
skills.add("Python")
```

второй `"Python"` не появится.

Это естественное следствие уникальности элементов `set`.

Список 2.4 `set-modification.py`

```
skills = {"Python", "Git"}

skills.add("SQL")
skills.add("Python")

print(skills)
```

Удаление через remove()

Метод `remove()` удаляет указанное значение:

```
blocked_users = {"alex", "maria", "john"}

blocked_users.remove("maria")

print(blocked_users)
```

Но есть важная деталь.

Если элемента нет:

```
blocked_users.remove("anna")
```

возникнет:

```
KeyError
```

Поэтому `remove()` удобно использовать, когда мы уверены, что значение присутствует.

Удаление через discard()

Метод `discard()` тоже удаляет значение:

```
blocked_users = {"alex", "maria", "john"}

blocked_users.discard("maria")
```

Главное отличие проявляется, если элемента нет:

```
blocked_users.discard("anna")
```

Ошибки не будет.

Сравним:

Метод	Элемент существует	Элемента нет
`remove()`	удаляет	`KeyError`
`discard()`	удаляет	ничего не происходит

Список 2.5 `set-removal.py`

```
blocked = {"alex", "maria", "john"}

blocked.remove("maria")
blocked.discard("anna")

print(blocked)

# remove() для отсутствующего элемента приведет к KeyError.
# discard() для отсутствующего элемента ошибки не вызывает.
```



Рисунок 2.3: remove() и discard()

remove() или discard()?

Используйте:

```
remove()
```

если отсутствие элемента само по себе означает проблему.
Используйте:

```
discard()
```

если нужно просто убедиться, что значения больше нет.

Метод pop()

У множества есть метод:

```
pop()
```

Он удаляет и возвращает **произвольный элемент**:

```
values = {"Python", "Java", "Go"}

removed = values.pop()

print("Удалено:", removed)
print(values)
```

Не следует использовать `pop()` в расчёте на удаление «первого» или «последнего» элемента.

У `set` нет такого позиционного смысла.

Очистка множества

Чтобы удалить все элементы, используется:

```
clear()
```

Например:

```
active_filters = {"python", "junior", "remote"}

active_filters.clear()

print(active_filters)
```

Результат:

```
set()
```

Сам объект множества остаётся, но становится пустым.

Перебор множества

Множество можно перебирать циклом `for`:

```
technologies = {"Python", "Docker", "Git"}

for technology in technologies:
    print(technology)
```

Каждый элемент будет обработан один раз.

Но порядок обхода не следует считать фиксированным.

Поэтому такой код подходит, когда важны сами элементы:

```
Python
Docker
Git
```

а не их позиции.

Объединение множеств

Теперь начинается одна из самых полезных возможностей `set`.

Есть две команды разработчиков:

```
backend = {"Python", "PostgreSQL", "Docker"}
devops = {"Docker", "Linux", "Kubernetes"}
```

Нужно получить все технологии, встречающиеся хотя бы в одной команде.

Используем оператор:

```
|
```

```
all_technologies = backend | devops

print(all_technologies)
```

Получится множество примерно такого содержания:

```
Python
PostgreSQL
Docker
Linux
Kubernetes
```

`Docker` встречался в обоих множествах, но в результате присутствует один раз.

Операция называется **объединением**.

То же самое можно записать:

```
backend.union(devops)
```

Список 2.6 `set-union.py`

```
backend = {"Python", "PostgreSQL", "Docker"}
devops = {"Docker", "Linux", "Kubernetes"}

result = backend | devops

print(result)
```

Пересечение множеств

Теперь найдём технологии, которые используются **обеими** командами.

Оператор:

```
&
```

Пример:

```
backend = {"Python", "PostgreSQL", "Docker"}
devops = {"Docker", "Linux", "Kubernetes"}

common = backend & devops

print(common)
```

Результат:

```
Docker
```

Это **пересечение** множеств.

Другой вариант записи:

```
backend.intersection(devops)
```

Смысл:

```
A & B
↓
значения, которые есть и в A, и в B
```

Список 2.7 `set-intersection.py`

```
backend = {"Python", "PostgreSQL", "Docker"}
devops = {"Docker", "Linux", "Kubernetes"}

result = backend & devops

print(result)
```

Разность множеств

Иногда нужно узнать, какие значения присутствуют в одном множестве, но отсутствуют в другом.

Используется:

```
-
```

Например:

```
backend = {"Python", "PostgreSQL", "Docker"}
devops = {"Docker", "Linux", "Kubernetes"}

only_backend = backend - devops

print(only_backend)
```

В результате останутся:

```
Python
PostgreSQL
```

`Docker` исключается, потому что присутствует и в `devops`.

То же самое:

```
backend.difference(devops)
```

Список 2.8 `set-difference.py`

```
required = {"Python", "Git", "SQL", "Docker"}
candidate = {"Python", "Git", "Linux"}

missing = required - candidate

print(missing)
```

Направление разности имеет значение

Сравним:

```
backend - devops
```

и:

```
devops - backend
```

Это разные вопросы.

Первый:

```
Что есть в backend, но нет в devops?
```

Второй:

```
Что есть в devops, но нет в backend?
```

Например:

```
backend = {"Python", "PostgreSQL", "Docker"}
devops = {"Docker", "Linux", "Kubernetes"}

print(backend - devops)
print(devops - backend)
```

В первом результате будут Python и PostgreSQL, во втором — Linux и Kubernetes.

Симметрическая разность

Есть ещё одна полезная операция:

```
^
```

Она оставляет значения, которые встречаются только в одном из двух множеств.

```
first = {"Python", "Git", "Docker"}
second = {"Python", "SQL", "Docker"}

difference = first ^ second

print(difference)
```

Общие значения:

```
Python
Docker
```

не попадут в результат.

Останутся:

```
Git
SQL
```

Эту операцию называют **симметрической разностью**.

На начальном этапе достаточно понимать её как:

> элементы, которые не являются общими.

Четыре операции рядом

Пусть:

Список 2.9 `set-symmetric-difference.py`

```
old = {"Python", "Linux", "Git", "SQL"}
current = {"Python", "Docker", "Git", "FastAPI"}

changed = old ^ current

print(changed)
```

```
a = {1, 2, 3}
b = {3, 4, 5}
```

Тогда:

```
a | b
```

даёт все уникальные значения:

```
{1, 2, 3, 4, 5}
```

```
a & b
```

даёт общие:

```
{3}
```

```
a - b
```

даёт значения только из a:

```
{1, 2}
```

```
a ^ b
```

даёт значения, которые не являются общими:

```
{1, 2, 4, 5}
```

Удобная памятка:

Операция	Оператор	Вопрос
Объединение	`A | B`	Что есть хотя бы в одном?
Пересечение	`A & B`	Что есть в обоих?
Разность	`A - B`	Что есть в A, но нет в B?
Симметрическая разность	`A ^ B`	Что не является общим?



Рисунок 2.4: Операции над множествами

Пример: интересы пользователей

Допустим, два пользователя выбрали интересующие их темы:

```
anna = {"Python", "SQL", "Linux"}
maxim = {"Python", "Docker", "Git"}
```

Общие интересы:

```
common = anna & maxim

print(common)
```

Получим:

```
Python
```

Все темы двух пользователей:

```
all_topics = anna | maxim
```

Темы Анны, которых нет у Максима:

```
only_anna = anna - maxim
```

Одни и те же данные позволяют отвечать на разные вопросы с помощью операций над множествами.

Пример: уникальные посетители

Представим, что система сохранила идентификаторы посещений:

```
visits = [
    "user_17",
    "user_42",
    "user_17",
    "user_91",
    "user_42"
]
```

Количество записей:

```
print(len(visits))
```

равно:

```
5
```

Но пользователей меньше.

Получим уникальных посетителей:

```
unique_users = set(visits)

print(len(unique_users))
```

Результат:

```
3
```

Здесь `list` отвечает на вопрос:

```
Сколько было записей о посещениях?
```

а `set`:

```
Сколько разных пользователей посетили систему?
```

Итоговый пример: студенты двух курсов

Множества удобно использовать, когда нужно сравнить две группы.

Например, есть студенты курса Python и курса SQL. По этим двум наборам можно найти общих студентов, всех уникальных студентов и тех, кто посещает только один курс.

Set, list и tuple

Теперь мы знаем три структуры для нескольких значений.

Структура	Порядок	Можно изменять	Повторы
`list`	да	да	да
`tuple`	да	нет	да
`set`	не полагаемся на порядок	да	нет

Список 2.10 `set-analysis.py`

```
python_students = {
    "Анна",
    "Максим",
    "Игорь",
    "София",
    "Денис"
}

sql_students = {
    "Игорь",
    "София",
    "Мария",
    "Алексей"
}

both = python_students & sql_students
all_students = python_students | sql_students
only_python = python_students - sql_students
only_sql = sql_students - python_students
one_course = python_students ^ sql_students

print("Оба курса:", both)
print("Всего уникальных студентов:", len(all_students))
print("Только Python:", only_python)
print("Только SQL:", only_sql)
print("Один курс:", one_course)
```

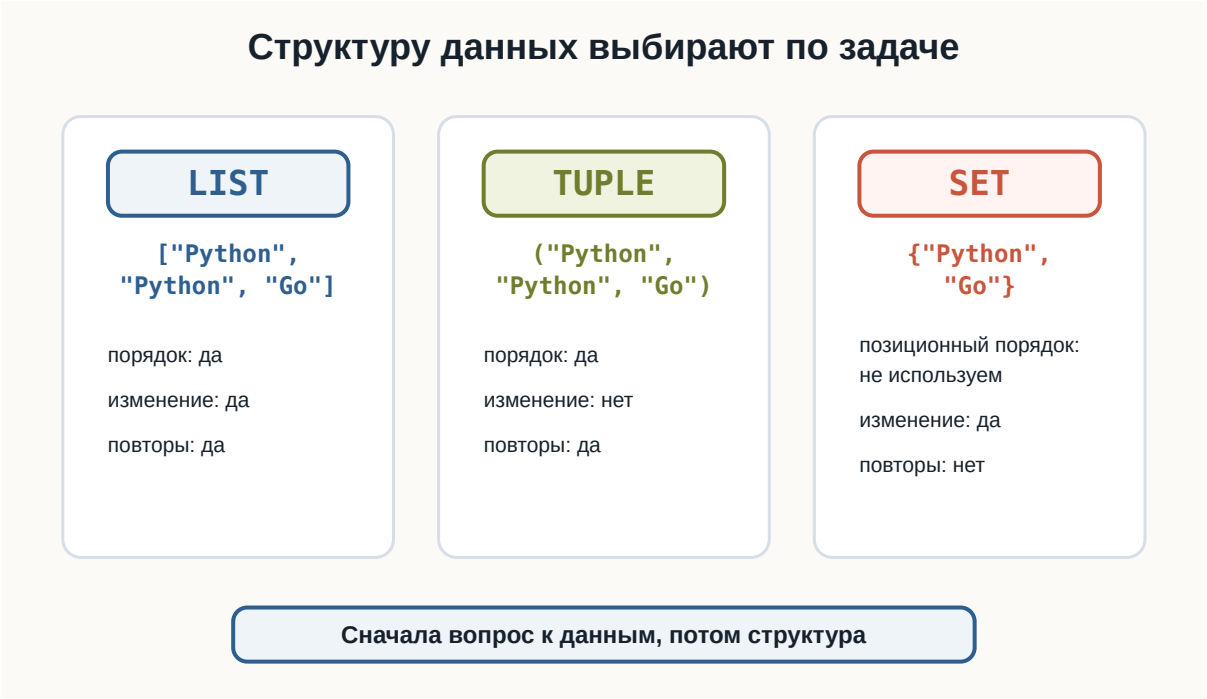


Рисунок 2.5: List, tuple и set

Выбор зависит от задачи.

Нужна изменяемая последовательность:

```
tasks = ["Купить кофе", "Написать код"]
```

Используем:

```
list
```

Нужна фиксированная группа:

```
resolution = (1920, 1080)
```

Используем:

```
tuple
```

Нужны уникальные значения:

```
permissions = {"read", "write", "delete"}
```

Подходит:

```
set
```

Что важно запомнить

Множество имеет тип:

```
set
```

Множество с данными:

```
languages = {"Python", "Java", "Go"}
```

Пустое множество:

```
languages = set()
```

`set` хранит уникальные элементы и не предоставляет доступ по индексу.

Основные операции изменения:

```
values.add(item)
values.remove(item)
values.discard(item)
values.pop()
values.clear()
```

Проверка:

```
item in values
item not in values
```

Основные операции между множествами:

```
a | b
a & b
a - b
a ^ b
```

Они позволяют получать объединение, пересечение, разность и симметрическую разность.

Практические задания

Задание 1. Участники конференции

После регистрации получился список:

```
participants = [
    "Анна",
    "Иван",
    "Мария",
    "Анна",
    "Олег",
    "Иван",
    "София"
]
```

Определите:

- сколько всего регистраций;
- сколько уникальных участников.

Ответ

```
participants = [
    "Анна",
    "Иван",
    "Мария",
    "Анна",
    "Олег",
    "Иван",
    "София"
]

unique_participants = set(participants)

print("Регистраций:", len(participants))
print("Участников:", len(unique_participants))
```

Результат:

```
Регистраций: 7
Участников: 5
```

Задание 2. Доступ в лабораторию

Разрешённые пропуска:

```
allowed = {"A102", "B205", "C310", "D404"}
```

Попросите пользователя ввести номер пропуска и сообщите, разрешён ли доступ.

Ответ

```
allowed = {"A102", "B205", "C310", "D404"}

card = input("Номер пропуска: ")

if card in allowed:
    print("Доступ разрешён")
else:
    print("Доступ запрещён")
```

Задание 3. Список стоп-слов

Дано множество:

```
stop_words = {"реклама", "спам", "акция"}
```

Добавьте слово `"скидка"`.

Удалите `"акция"` так, чтобы программа не завершилась ошибкой, даже если этого слова уже нет.

Ответ

```
stop_words = {"реклама", "спам", "акция"}

stop_words.add("скидка")
stop_words.discard("акция")

print(stop_words)
```

Для удаления используется `discard()`, потому что отсутствие элемента не приводит к `KeyError`.

Задание 4. Два музыкальных плейлиста

Есть любимые жанры двух пользователей:

```
user_1 = {"rock", "jazz", "blues", "electronic"}
user_2 = {"jazz", "hip-hop", "electronic", "classical"}
```

Найдите жанры, которые нравятся обоим.

Ответ

```
user_1 = {"rock", "jazz", "blues", "electronic"}
user_2 = {"jazz", "hip-hop", "electronic", "classical"}

common = user_1 & user_2

print(common)
```

В результате останутся:

```
jazz
electronic
```

Задание 5. Отсутствующие навыки

Для вакансии нужны:

```
required = {"Python", "Git", "SQL", "Docker"}
```

Кандидат знает:

```
candidate = {"Python", "Git", "Linux"}
```

Определите, какие навыки ему ещё нужно изучить.

Ответ

```
required = {"Python", "Git", "SQL", "Docker"}
candidate = {"Python", "Git", "Linux"}

missing = required - candidate

print(missing)
```

Результат содержит:

```
SQL
Docker
```

Здесь важен порядок операции:

```
required - candidate
```

Задание 6. Каталог двух магазинов

Первый магазин продаёт:

```
shop_a = {"ноутбук", "мышь", "клавиатура", "монитор"}
```

Второй:

```
shop_b = {"монитор", "наушники", "мышь", "веб-камера"}
```

Получите единый каталог товаров без повторений.

Ответ

```
shop_a = {"ноутбук", "мышь", "клавиатура", "монитор"}
shop_b = {"монитор", "наушники", "мышь", "веб-камера"}

catalog = shop_a | shop_b

print(catalog)
```

Используется объединение множеств.

Задание 7. Изменения подписок

В прошлом месяце пользователь был подписан на:

```
old = {"Python", "Linux", "Git", "SQL"}
```

Сейчас:

```
current = {"Python", "Docker", "Git", "FastAPI"}
```

Найдите темы, состояние подписки на которые изменилось: пользователь либо отказался от них, либо добавил их.

Ответ

```
old = {"Python", "Linux", "Git", "SQL"}
current = {"Python", "Docker", "Git", "FastAPI"}

changed = old ^ current

print(changed)
```

Общие `"Python"` и `"Git"` исключаются.
В результате останутся темы, присутствующие только в одном из множеств.

Задание 8. Анализ текста

Попросите пользователя ввести строку.

Определите количество различных символов в ней, не учитывая пробелы.

Например:

```
hello world
```

Программа должна работать с введённым пользователем текстом, а не с заранее заданным набором символов.

Ответ

```
text = input("Введите текст: ")

characters = set(text)
characters.discard(" ")

print("Уникальных символов:", len(characters))
```

Здесь строка преобразуется в `set`, поэтому повторяющиеся символы автоматически исключаются.

Задание 9. Проверка прав

Для выполнения операции нужны права:

```
required = {"read", "write"}
```

Права пользователя:

```
permissions = {"read", "write", "download"}
```

Напишите программу, которая проверяет, присутствуют ли **все необходимые права**.

Попробуйте решить задачу, используя уже изученные операции над множествами.

> **Ответ**
>
> Один из вариантов:
>
> ```
> required = {"read", "write"}
> permissions = {"read", "write", "download"}
>
> missing = required - permissions
>
> if len(missing) == 0:
>     print("Все необходимые права есть")
> else:
>     print("Не хватает прав:", missing)
> ```
>
> Мы вычитаем из обязательных прав те, которые уже есть у пользователя. Если результат пуст:
>
> ```
> set()
> ```
>
> значит отсутствующих прав нет.

Задание 10. Анализ двух групп

На два курса записались студенты:

```
python_students = {
    "Анна",
    "Максим",
    "Игорь",
    "София",
    "Денис"
```

```
}

sql_students = {
    "Игорь",
    "София",
    "Мария",
    "Алексей"
}
```

Выведите:

1. студентов, посещающих оба курса;
2. всех уникальных студентов;
3. студентов только курса Python;
4. студентов только курса SQL;
5. студентов, посещающих ровно один из двух курсов;
6. общее количество уникальных студентов.

Ответ

```
python_students = {
    "Анна",
    "Максим",
    "Игорь",
    "София",
    "Денис"
}

sql_students = {
    "Игорь",
    "София",
    "Мария",
    "Алексей"
}

both = python_students & sql_students
all_students = python_students | sql_students
only_python = python_students - sql_students
only_sql = sql_students - python_students
one_course = python_students ^ sql_students

print("Оба курса:", both)
print("Все студенты:", all_students)
print("Только Python:", only_python)
print("Только SQL:", only_sql)
print("Только один курс:", one_course)
print("Всего студентов:", len(all_students))
```

Эта задача объединяет четыре основные операции над множествами в одном сценарии.

Что дальше?

Мы уже умеем хранить данные несколькими способами:

```
list
→ последовательность, которую можно изменять

tuple
→ фиксированная последовательность
```

```
set
→ уникальные значения
```

Но во всех этих случаях мы в основном работаем с отдельными элементами.

Следующая задача будет другой.

Представим данные пользователя:

```
имя        → Анна
возраст    → 25
город      → Казань
профессия → разработчик
```

Здесь каждому значению хочется дать собственное имя.

Для хранения таких пар:

```
ключ → значение
```

Python предоставляет **словари (`dict`)**.

Следующая глава — **«Словари»**.

2 4.3 Словари

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как хранить связанные данные в формате «ключ значение»?**

В списках, кортежах и множествах мы работали с отдельными элементами:

```
languages = ["Python", "Java", "Go"]
```

Но данные одного объекта часто состоят из именованных характеристик:

```
имя     → Анна
возраст → 25
город   → Казань
```

Список такую информацию хранить может:

```
user = ["Анна", 25, "Казань"]
```

Но тогда нужно помнить, что означает каждая позиция. Словарь позволяет обратиться к данным по смыслу:

```
user["name"]
user["age"]
user["city"]
```

Что такое словарь

Словарь хранит данные в виде пар:

```
ключ → значение
```

Пример:

```
user = {
    "name": "Анна",
    "age": 25,
    "city": "Казань"
}
```

Здесь ключи `"name"`, `"age"` и `"city"` указывают, что означает каждое значение.

Тип словаря называется `dict`. Количество пар возвращает `len()`:

```
user = {
    "name": "Анна",
    "age": 25,
    "city": "Казань"
}

print(user)
print(type(user))
print(len(user))
```

Результат:

```
{'name': 'Анна', 'age': 25, 'city': 'Казань'}
<class 'dict'>
3
```

Список 2.1 `dict-basics.py`

```
user = {
    "name": "Анна",
    "age": 25,
    "city": "Казань"
}

print(user)
print(type(user))
print(len(user))
```

Ключ и значение

Каждая запись словаря состоит из ключа и значения:

```
"name": "Анна"
```

Слева находится **ключ**, справа — **значение**. Между ними ставится двоеточие, а пары разделяются запятыми:

```
user = {
    "name": "Анна",
    "age": 25,
    "city": "Казань"
}
```

Ключ позволяет понять, что означает значение, без знания его позиции.



Рисунок 2.1: Устройство словаря

Создание словаря

Небольшой словарь можно записать в одну строку:

```
product = {"name": "Ноутбук", "price": 85000}
```

Если полей несколько, удобнее использовать несколько строк:

```
product = {
    "name": "Ноутбук",
    "price": 85000,
    "available": True
}
```

Так структуру легче читать.

Пустой словарь

Пустой словарь создаётся фигурными скобками:

```
settings = {}
```

Проверим:

```
settings = {}

print(settings)
print(type(settings))
```

Результат:

```
{}
<class 'dict'>
```

Связь с предыдущей главой:

```
{}    → dict
set() → set
```

Для пустого множества по-прежнему используется `set()`.

Список 2.2 `empty-dict.py`

```
settings = {}

print(settings)
print(type(settings))

empty_set = set()

print(empty_set)
print(type(empty_set))
```

Получение значения по ключу

Чтобы получить значение, после имени словаря указывают ключ в квадратных скобках:

```
dictionary[key]
```

Пример:

```
product = {
    "name": "Ноутбук",
    "price": 85000,
    "available": True
}

print(product["name"])
print(product["price"])
print(product["available"])
```

Результат:

```
Ноутбук
85000
True
```

Главная идея: мы обращаемся не к позиции, а к конкретному ключу.

Ключи — не индексы

У списка значение получают по индексу:

Список 2.3 `dict-access.py`

```
product = {
    "name": "Ноутбук",
    "price": 85000,
    "available": True
}

print(product["name"])
print(product["price"])
print(product["available"])
```

```
user = ["Анна", 25, "Казань"]

print(user[0])
```

У словаря значение получают по ключу:

```
user = {
    "name": "Анна",
    "age": 25,
    "city": "Казань"
}

print(user["name"])
```

Индекс отвечает на вопрос «где?». Ключ отвечает на вопрос «что это?».

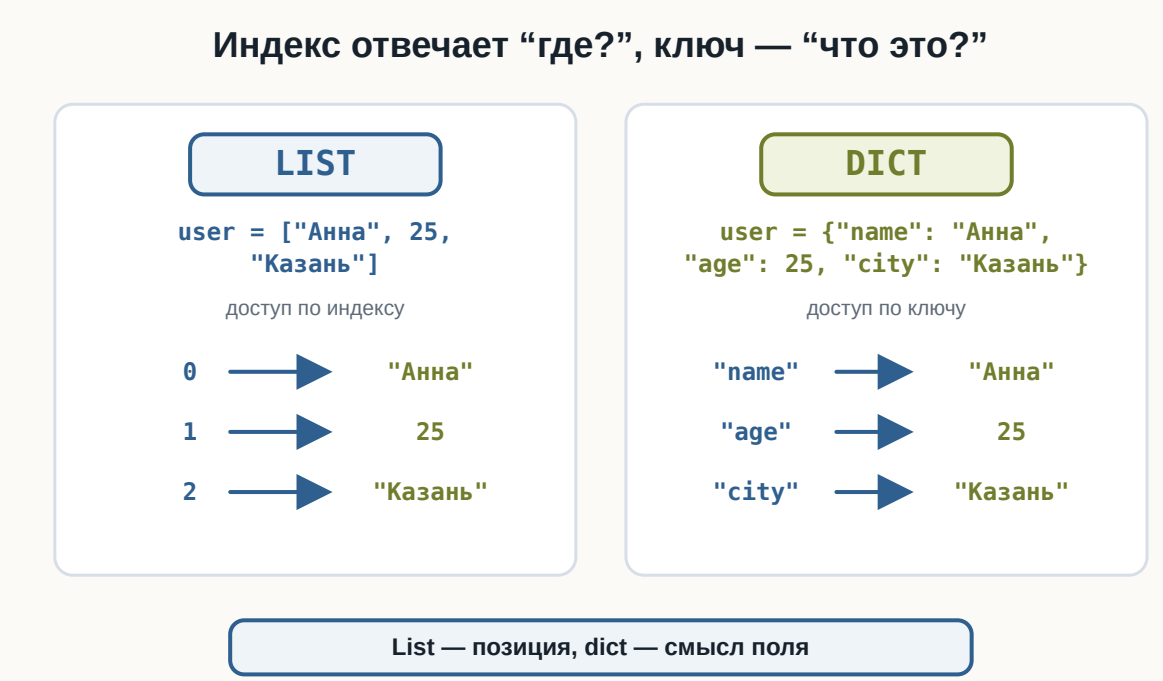


Рисунок 2.2: List и dict

Запись:

```
user[0]
```

не означает «первое значение словаря». Она означает: найти значение с ключом `0`. Если такого ключа нет, возникнет ошибка.

Если ключа нет

Прямое обращение через квадратные скобки предполагает, что ключ существует:

```
user = {
    "name": "Анна",
    "age": 25
}

print(user["email"])
```

Ключа `"email"` нет, поэтому Python сообщит:

```
KeyError
```

Список 2.4 `dict-key-error.py`

```
user = {
    "name": "Анна",
    "age": 25
}

print(user["email"])
```

⚠ Несуществующий ключ

Обращение:

```
data["key"]
```

безопасно только тогда, когда такой ключ действительно есть в словаре.

Добавление новой пары

Чтобы добавить новую пару, присвойте значение новому ключу:

```
profile = {
    "nickname": "Neo",
    "level": 12
}

profile["server"] = "EU-1"

print(profile)
```

Если ключа ещё нет, создаётся новая пара `ключ → значение`.

Изменение значения

Если ключ уже существует, та же запись изменит значение:

Список 2.5 `dict-add.py`

```
profile = {
    "nickname": "Neo",
    "level": 12
}

profile["server"] = "EU-1"

print(profile)
```

```
game = {
    "title": "Python Quest",
    "score": 1200
}

game["score"] = 1450

print(game)
```

Результат:

```
{'title': 'Python Quest', 'score': 1450}
```

Список 2.6 `dict-update-value.py`

```
game = {
    "title": "Python Quest",
    "score": 1200
}

game["score"] = 1450

print(game)
```

Один синтаксис — два результата

Запись:

```
dictionary[key] = value
```

работает по-разному в зависимости от ключа:

```
ключа нет  → добавление новой пары
ключ есть → изменение значения
```

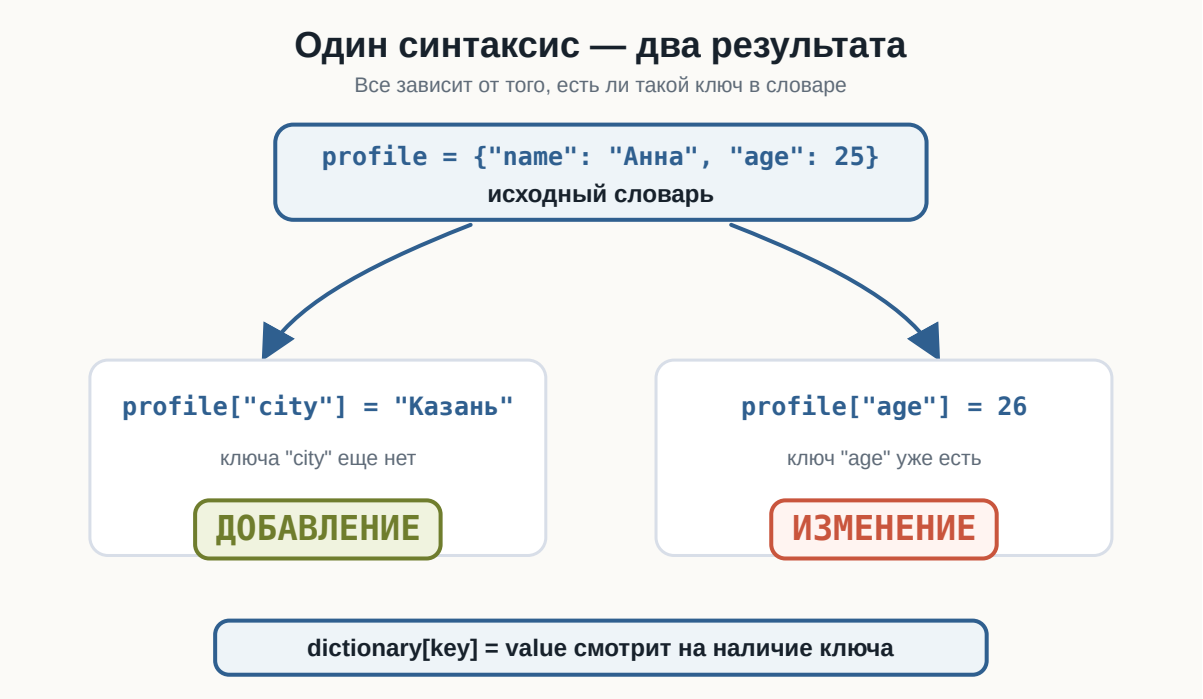


Рисунок 2.3: Добавление и изменение в dict

Значения могут иметь разные типы

Один словарь может описывать объект с разными характеристиками:

```
course = {
    "title": "Python",
    "lessons": 42,
    "price": 19.99,
    "available": True
}
```

Здесь значения имеют разные типы:

```
"title"     → str
"lessons"   → int
"price"     → float
"available" → bool
```

Ключи должны быть уникальными

В одном словаре каждый ключ обозначает одно значение.

Не следует создавать словарь с повторяющимися ключами:

```
data = {
    "name": "Анна",
    "name": "Мария"
}
```

Одинаковые ключи не создают две независимые записи. Последнее значение заменяет предыдущее:

```
data = {
    "name": "Анна",
    "name": "Мария"
}

print(data["name"])
```

Результат:

```
Мария
```

Количество пар

Для словаря работает `len()`:

```
user = {
    "name": "Анна",
    "age": 25,
```

```
    "city": "Казань"
}

print(len(user))
```

Результат:

```
3
```

`len()` возвращает количество пар `ключ → значение`.

Проверка наличия ключа

Оператор `in` проверяет наличие **ключа**:

```
device = {
    "name": "Sensor A",
    "status": "online"
}

if "temperature" in device:
    print(device["temperature"])
else:
    print("Температура неизвестна")
```

Для проверки отсутствия используется `not in`:

```
if "email" not in user:
    print("Email не указан")
```

> **in проверяет ключи**
>
> В выражении:
>
> ```
> "name" in user
> ```
>
> Python проверяет наличие ключа `"name"`. Значения словаря здесь не проверяются.

Список 2.7 `dict-membership.py`

```
device = {
    "name": "Sensor A",
    "status": "online"
}

if "temperature" in device:
    print(device["temperature"])
else:
    print("Температура неизвестна")
```

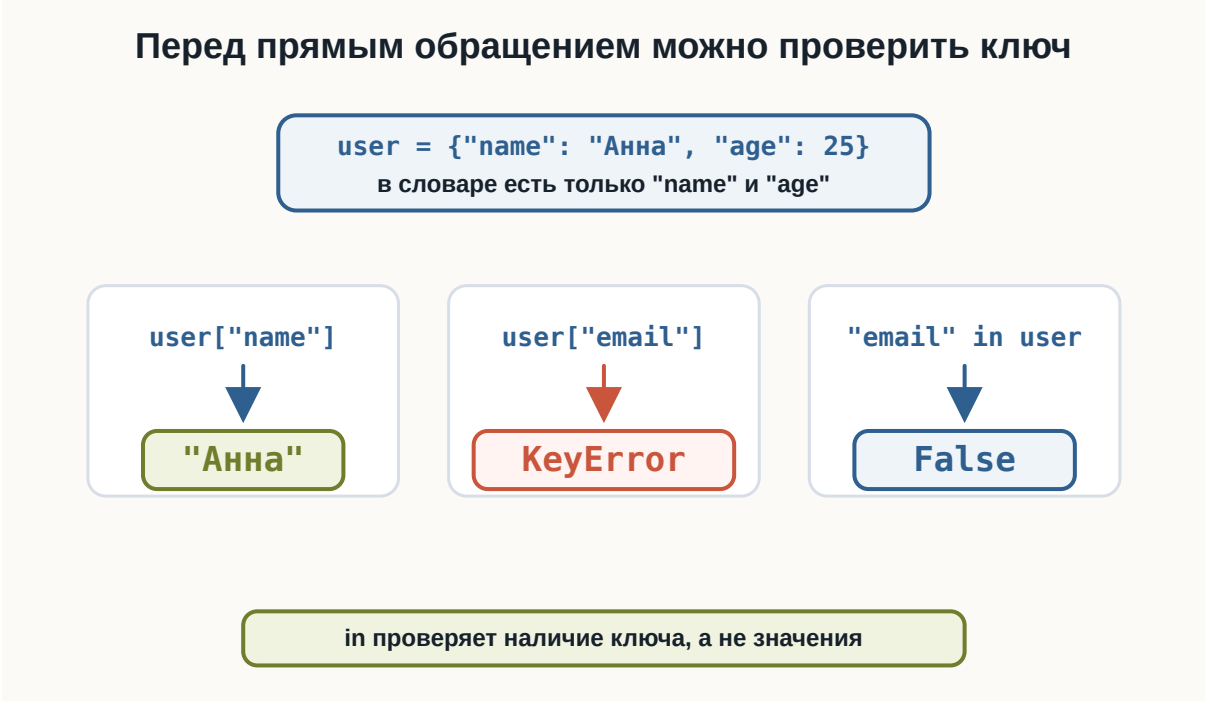


Рисунок 2.4: KeyError и проверка ключа

Удаление пары

Пару можно удалить с помощью `del`:

```
session = {
    "player": "Alex",
    "score": 1250,
    "temporary_code": "X7F9"
}
```

```
del session["temporary_code"]

print(session)
```

После удаления ключа `"temporary_code"` в словаре больше нет.

Если попытаться удалить отсутствующий ключ, возникнет `KeyError`.

Список 2.8 `dict-delete.py`

```
session = {
    "player": "Alex",
    "score": 1250,
    "temporary_code": "X7F9"
}

del session["temporary_code"]

print(session)
```

Пример: карточка товара

Словарь удобно использовать для описания одного объекта:

```
product = {
    "name": "Механическая клавиатура",
    "price": 7490,
    "available": True
}

print("Товар:", product["name"])
print("Цена:", product["price"])

product["price"] = 6990
product["rating"] = 4.8

print(product)
```

Здесь мы получили значения по ключам, изменили цену и добавили новое поле.

Список 2.9 `product-card.py`

```
product = {
    "name": "Механическая клавиатура",
    "price": 7490,
    "available": True
}

print("Товар:", product["name"])
print("Цена:", product["price"])

product["price"] = 6990
product["rating"] = 4.8

print(product)
```

Итоговый пример

Соберём базовые операции словаря в одном сценарии:

```
spaceship = {
    "name": "Odyssey",
    "fuel": 80,
    "crew": 4,
    "active": True
}

spaceship["fuel"] = spaceship["fuel"] - 15
spaceship["destination"] = "Mars"

if "autopilot" not in spaceship:
    spaceship["autopilot"] = False

print("Корабль:", spaceship["name"])
print("Топливо:", spaceship["fuel"])
print("Характеристик:", len(spaceship))
print(spaceship)
```

Список или словарь

Список удобен, когда важна последовательность элементов:

Список 2.10 `dict-practice.py`

```
spaceship = {
    "name": "Odyssey",
    "fuel": 80,
    "crew": 4,
    "active": True
}

spaceship["fuel"] = spaceship["fuel"] - 15
spaceship["destination"] = "Mars"

if "autopilot" not in spaceship:
    spaceship["autopilot"] = False

print("Корабль:", spaceship["name"])
print("Топливо:", spaceship["fuel"])
print("Характеристик:", len(spaceship))
print(spaceship)
```

```
temperatures = [18, 20, 19, 22]
```

Словарь удобен, когда значения имеют собственные имена:

```
weather = {
    "city": "Казань",
    "temperature": 20,
    "wind": "северный"
}
```

Общий ориентир:

Структура	Когда использовать	Пример
`list`	изменяемая последовательность	`["Python", "Go"]`
`tuple`	фиксированная последовательность	`(1920, 1080)`
`set`	уникальные значения	`{"Python", "Go"}`
`dict`	пары `ключ → значение`	`{"name": "Анна", "age": 25}`



Рисунок 2.5: Выбор структуры данных

Что важно запомнить

Словарь имеет тип `dict` и хранит пары:

```
ключ → значение
```

Базовые действия:

```
user = {"name": "Анна", "age": 25}

print(user["name"])

user["city"] = "Казань"
user["age"] = 26

print("name" in user)
print(len(user))

del user["city"]
```

Если обратиться к отсутствующему ключу:

```
user["email"]
```

возникнет `KeyError`.

Практические задания

Задание 1. Космический аппарат

Создайте словарь с информацией о космическом аппарате:

```
name     → Voyager 1
launched → 1977
active   → True
```

Выведите название аппарата и год запуска.

> Ответ

```
spacecraft = {
    "name": "Voyager 1",
    "launched": 1977,
    "active": True
}

print(spacecraft["name"])
print(spacecraft["launched"])
```

Задание 2. Игровой персонаж

Дан персонаж:

```
player = {
    "name": "Raven",
    "health": 100,
    "level": 7
}
```

После боя здоровье уменьшилось до 65. Измените словарь и выведите новое значение здоровья.

Ответ

```
player = {
    "name": "Raven",
    "health": 100,
    "level": 7
}

player["health"] = 65

print(player["health"])
```

Задание 3. Фильм

Дан фильм:

```
movie = {
    "title": "Interstellar",
    "year": 2014
}
```

Добавьте рейтинг 8 . 7, а затем выведите весь словарь.

Ответ

```
movie = {
    "title": "Interstellar",
    "year": 2014
}

movie["rating"] = 8.7

print(movie)
```

Задание 4. Компьютер

Дано описание компьютера:

```
computer = {
    "cpu": "Ryzen 7",
    "ram": 32,
    "ssd": 1000
}
```

Проверьте, присутствует ли ключ `"gpu"`. Если его нет, выведите:

```
Видеокарта не указана
```

> **Ответ**
>
> ```
> computer = {
>     "cpu": "Ryzen 7",
>     "ram": 32,
>     "ssd": 1000
> }
>
> if "gpu" not in computer:
>     print("Видеокарта не указана")
> ```

Задание 5. Игровая сессия

Дана игровая сессия:

```
session = {
    "player": "Alex",
    "score": 1250,
    "temporary_code": "X7F9"
}
```

Удалите `"temporary_code"` и выведите словарь.

Ответ

```
session = {
    "player": "Alex",
    "score": 1250,
    "temporary_code": "X7F9"
}

del session["temporary_code"]

print(session)
```

Задание 6. Автомобиль

Дан автомобиль:

```
car = {
    "brand": "Volvo",
    "model": "XC60",
    "year": 2024,
    "electric": False
}
```

Определите количество характеристик с помощью Python, не считая их вручную.

Ответ

```
car = {
    "brand": "Volvo",
    "model": "XC60",
    "year": 2024,
    "electric": False
}

print(len(car))
```

Результат:

```
4
```

Задание 7. Склад

На складе хранится информация:

```
item = {
    "name": "Монитор",
    "price": 32000,
    "quantity": 4
}
```

Поступило ещё 3 монитора. Не записывайте итоговое количество вручную: измените `"quantity"`, используя его текущее значение.

Ответ

```
item = {
    "name": "Монитор",
    "price": 32000,
    "quantity": 4
}

item["quantity"] = item["quantity"] + 3

print(item["quantity"])
```

Результат:

```
7
```

Задание 8. Профиль игрока

Создайте словарь:

```
nickname → Neo
level    → 12
coins    → 450
premium  → False
```

Затем выполните последовательно:

1. увеличьте `level` на 1;
2. добавьте `150` монет к текущему количеству;
3. измените `premium` на `True`;

4. добавьте новое поле `"server"` со значением `"EU-1"`;
5. выведите итоговый словарь.

Ответ

```
player = {
    "nickname": "Neo",
    "level": 12,
    "coins": 450,
    "premium": False
}

player["level"] = player["level"] + 1
player["coins"] = player["coins"] + 150
player["premium"] = True
player["server"] = "EU-1"

print(player)
```

Задание 9. Датчик

Дан словарь:

```
device = {
    "name": "Sensor A",
    "status": "online"
}
```

Напишите код, который выводит значение `"temperature"` только в том случае, если такой ключ существует. Если ключа нет, вывести:

```
Температура неизвестна
```

Ответ

```
device = {
    "name": "Sensor A",
    "status": "online"
}

if "temperature" in device:
    print(device["temperature"])
else:
    print("Температура неизвестна")
```

Задание 10. Книга

Создайте словарь, описывающий книгу. В нём должны быть поля:

```
title
author
year
available
```

После создания:

1. выведите название и автора;
2. измените `available`;
3. добавьте поле `pages`;
4. проверьте наличие поля `rating`;
5. если его нет — добавьте рейтинг;
6. удалите `year`;
7. выведите количество оставшихся характеристик и итоговый словарь.

Ответ

```
book = {
    "title": "1984",
    "author": "George Orwell",
    "year": 1949,
    "available": True
}

print(book["title"])
print(book["author"])

book["available"] = False
book["pages"] = 328

if "rating" not in book:
    book["rating"] = 4.8

del book["year"]

print("Характеристик:", len(book))
print(book)
```

Что дальше?

Теперь мы умеем создать словарь, получить значение по ключу, добавить новую пару, изменить значение, проверить наличие ключа и удалить пару.

Следующий естественный вопрос:

```
Как удобнее и безопаснее работать со словарём целиком?
```

На него отвечает следующая глава — **«Работа со словарями»**.

Словари Python: краткое резюме

Словарь (`dict`) в Python хранит данные в формате **«ключ значение»**. Такой формат удобен, когда отдельным характеристикам нужно дать понятные

имена: например, имя пользователя, цену товара, статус заказа или настройки программы.

Словарь создаётся с помощью фигурных скобок `{}`. Значение можно получить через `dictionary[key]`, новую пару добавить присваиванием, а существующее значение изменить через тот же ключ. Оператор `in` позволяет проверить наличие ключа, `len()` возвращает количество пар, а `del` удаляет запись.

Если ключ отсутствует, обращение через `dictionary[key]` приводит к `KeyError`. Поэтому перед прямым обращением полезно проверять ключ через `in`, когда отсутствие ключа является нормальной ситуацией.

2 4.4 Работа со словарями

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как удобнее получать, перебирать, обновлять и очищать словарь?**

В предыдущей главе мы познакомились со словарём `dict`: ключами, значениями, созданием словаря, прямым доступом через `dictionary[key]`, добавлением, изменением, `len()`, проверкой ключа через `in` и удалением через `del`.

Теперь разберём методы и приёмы, которые нужны для работы со словарём целиком.

Безопасное получение через get()

Прямой доступ требует, чтобы ключ существовал. Если ключа нет, будет `KeyError`.

Метод `get()` работает мягче: он возвращает значение, если ключ есть, и `None`, если ключ отсутствует.

```
user = {
    "name": "Анна",
    "age": 25
}

print(user.get("name"))
print(user.get("email"))
```

Результат:

```
Анна
None
```

Главная идея: `get()` не вызывает `KeyError` для отсутствующего ключа.

Список 2.1 `dict-get.py`

```
user = {
    "name": "Анна",
    "age": 25
}

print(user.get("name"))
print(user.get("email"))
```

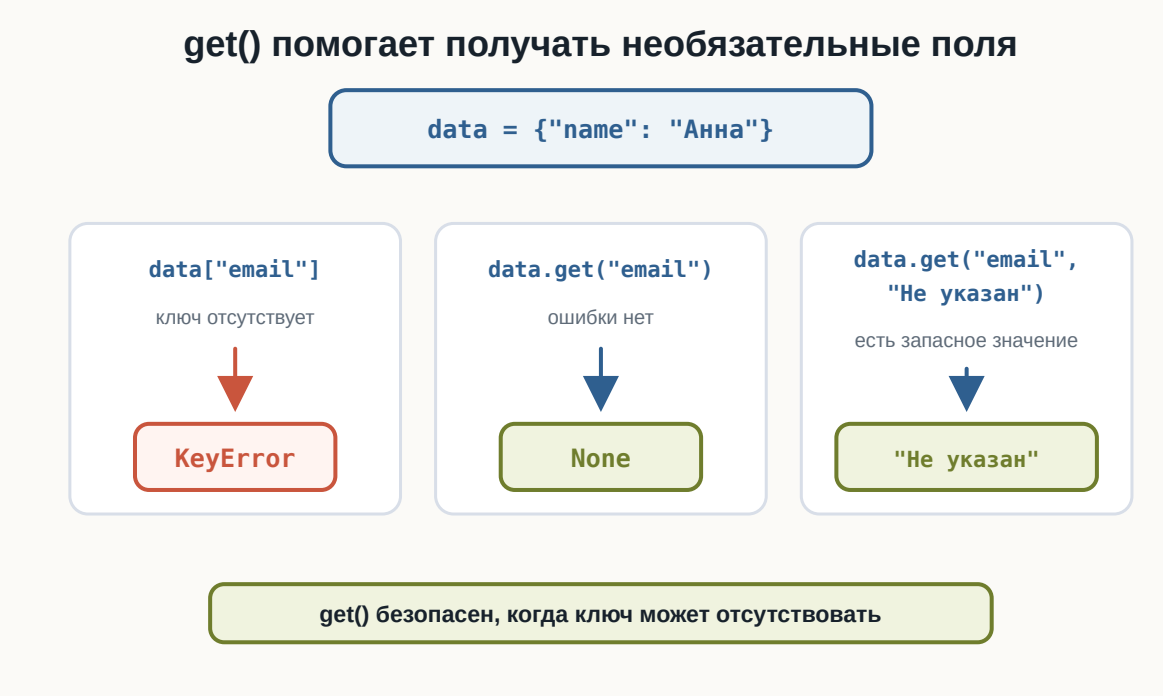


Рисунок 2.1: get() и отсутствующий ключ

get() со значением по умолчанию

Вторым аргументом `get()` можно передать значение, которое нужно вернуть при отсутствии ключа:

```
dictionary.get(key, default)
```

Пример:

```
config = {
    "theme": "dark"
}

theme = config.get("theme", "light")
font_size = config.get("font_size", 14)

print(theme)
print(font_size)
```

Результат:

```
dark
14
```

`default` используется только тогда, когда ключа нет. Для `"theme"` вернулось настоящее значение `"dark"`, а для отсутствующего `"font_size"` — 14.

Список 2.2 `dict-get-default.py`

```
config = {
    "theme": "dark"
}

theme = config.get("theme", "light")
font_size = config.get("font_size", 14)

print(theme)
print(font_size)
```

Ключи через keys()

Метод `keys()` предоставляет ключи словаря.

```
user = {
    "name": "Анна",
```

```
    "age": 25,
    "city": "Казань"
}

for key in user.keys():
    print(key)
```

Результат:

```
name
age
city
```

На текущем этапе достаточно понимать:

```
keys()
→ ключи словаря
```

Список 2.3 `dict-keys.py`

```
user = {
    "name": "Анна",
    "age": 25,
    "city": "Казань"
}

for key in user.keys():
    print(key)
```

Значения через values()

Метод `values()` удобен, когда ключи не нужны и нужно обработать только значения.

```
scores = {
    "Python": 90,
    "SQL": 82,
    "Git": 95
}
```

```
total = 0

for score in scores.values():
    total += score

print(total)
```

Результат:

```
267
```

Здесь цикл видит только числа 90, 82 и 95.

Список 2.4 `dict-values.py`

```
scores = {
    "Python": 90,
    "SQL": 82,
    "Git": 95
}

total = 0

for score in scores.values():
    total += score

print(total)
```

Пары через items()

Метод `items()` предоставляет пары `ключ → значение`.

```
user = {
    "name": "Анна",
    "age": 25,
    "city": "Казань"
}

for key, value in user.items():
    print(key, "→", value)
```

Результат:

```
name → Анна
age → 25
city → Казань
```

Каждый элемент `items()` можно распаковать в две переменные: `key` и `value`.

Список 2.5 `dict-items.py`

```
user = {
    "name": "Анна",
    "age": 25,
    "city": "Казань"
}

for key, value in user.items():
    print(key, "→", value)
```

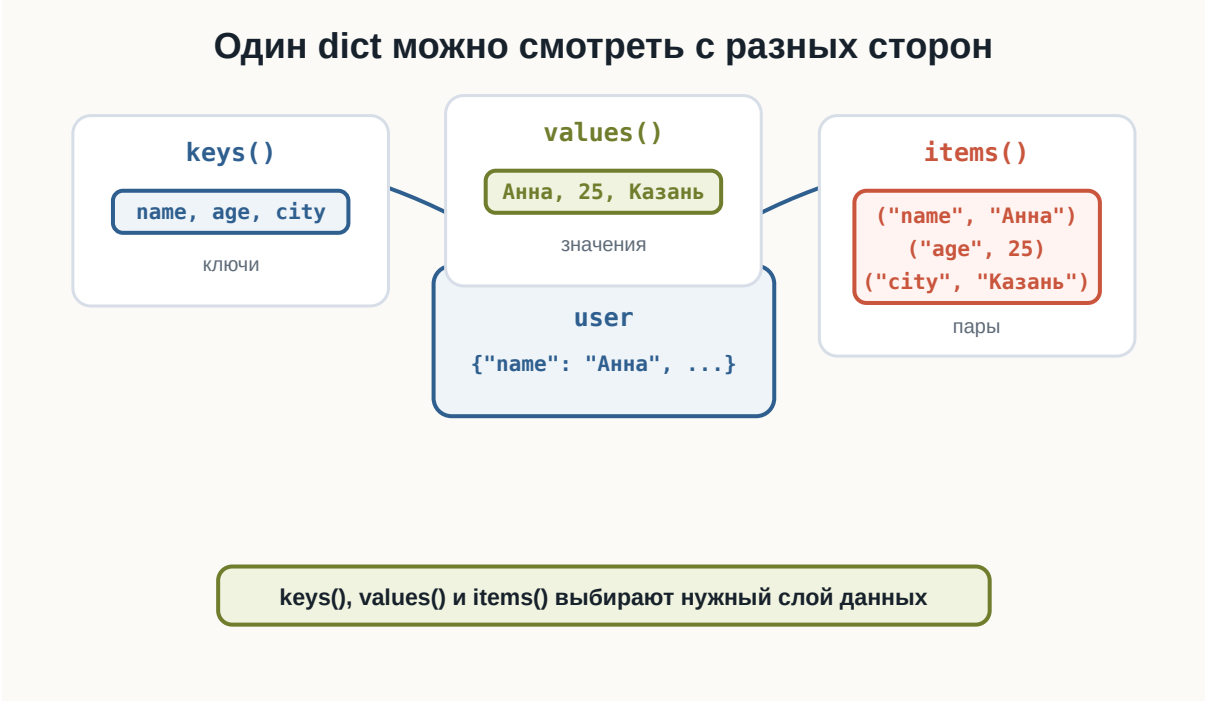


Рисунок 2.2: keys(), values() и items()

Перебор словаря через for

Если перебирать сам словарь, Python перебирает его ключи:

```
settings = {
    "theme": "dark",
    "language": "ru",
    "notifications": True
}

for key in settings:
    print(key, "=", settings[key])
```

Результат:

```
theme = dark
language = ru
notifications = True
```

Запись `for key in settings:` означает перебор ключей. Значение можно получить через `settings[key]`.

В файле примера есть подсказка: если нужны и ключ, и значение, `items()` обычно понятнее.

Список 2.6 `dict-iteration.py`

```
settings = {
    "theme": "dark",
    "language": "ru",
    "notifications": True
}

for key in settings:
    print(key, "=", settings[key])

# Если нужны и ключ, и значение, items() обычно понятнее.
```

Перебор ключей и значений

Когда в цикле нужны сразу ключ и значение, обычно используют `items()`:

```
for key, value in user.items():
    print(key, value)
```

Это связано с уже знакомой распаковкой кортежей:

```
key, value = ("name", "Анна")
```

На каждой итерации новая пара из `items()` раскладывается по двум переменным.

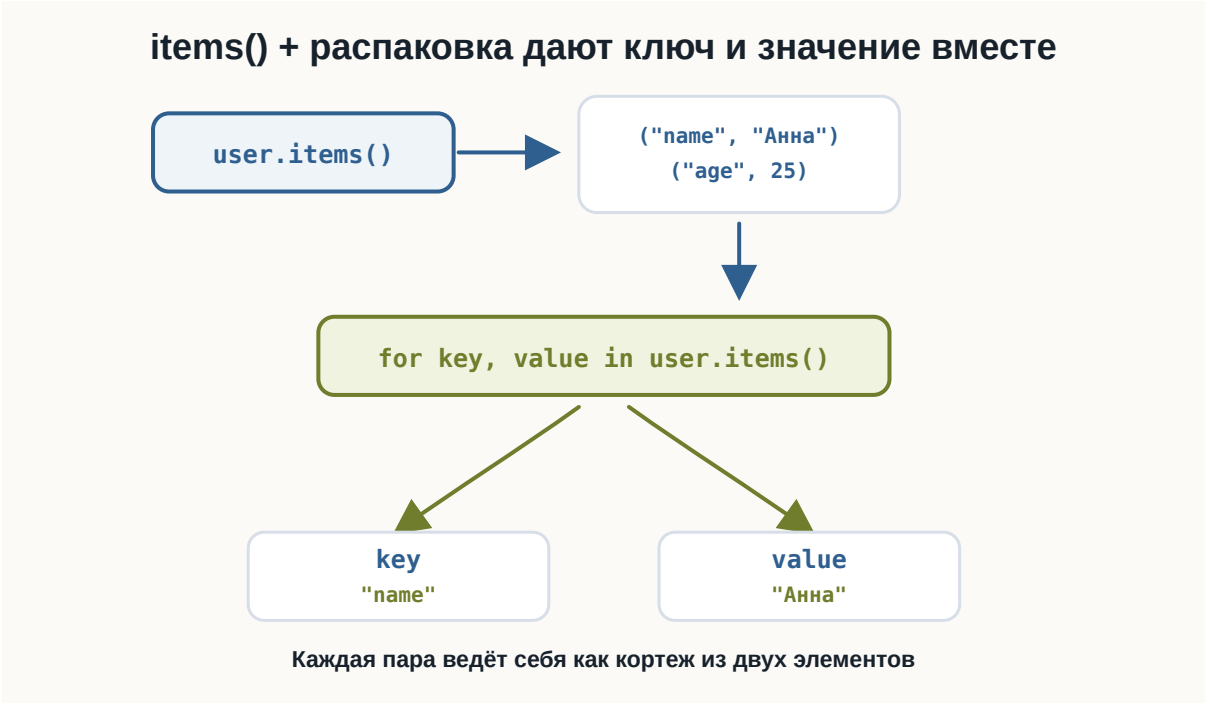


Рисунок 2.3: items() и распаковка

Обновление через update()

Метод `update()` применяет сразу несколько изменений:

```
profile = {
    "name": "Анна",
    "city": "Казань"
}

changes = {
    "city": "Москва",
    "age": 25
}

profile.update(changes)

print(profile)
```

В результате `"city"` изменится на `"Москва"`, а новый ключ `"age"` добавится.

Список 2.7 `dict-update.py`

```
profile = {
    "name": "Анна",
    "city": "Казань"
}

changes = {
    "city": "Москва",
    "age": 25
}

profile.update(changes)

print(profile)
```

Правило такое же, как при присваивании:

```
ключ уже есть → значение изменится
ключа нет     → пара добавится
```

Удаление через pop()

В предыдущей главе было удаление через `del`. Метод `pop()` отличается тем, что возвращает удалённое значение.

```
user = {
    "name": "Анна",
    "age": 25,
    "city": "Казань"
}

city = user.pop("city")

print(city)
print(user)

missing = user.pop("email", "Не указан")

print(missing)
```

Результат:

```
Казань
{'name': 'Анна', 'age': 25}
Не указан
```

Если ключ может отсутствовать, передавайте значение по умолчанию:

```
dictionary.pop(key, default)
```

Тогда отсутствующий ключ не приведёт к `KeyError`.

Список 2.8 `dict-pop.py`

```
user = {
    "name": "Анна",
    "age": 25,
    "city": "Казань"
}

city = user.pop("city")

print(city)
print(user)

missing = user.pop("email", "Не указан")

print(missing)
```

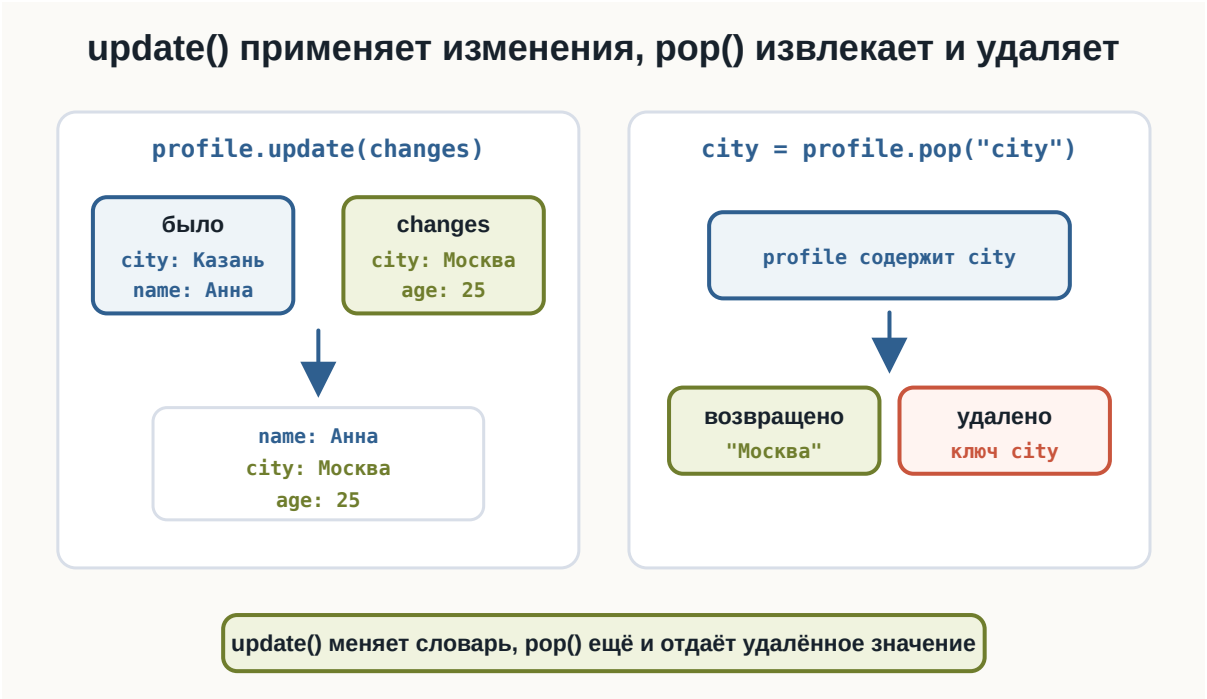


Рисунок 2.4: update() и pop()

Очистка через clear()

Метод `clear()` удаляет все пары из словаря:

```
cache = {
    "user": "Анна",
    "token": "ABC123"
}

cache.clear()

print(cache)
```

Результат:

```
{}
```

Словарь остаётся существовать, но становится пустым.

Список 2.9 `dict-clear.py`

```
cache = {
    "user": "Анна",
    "token": "ABC123"
}

cache.clear()

print(cache)
```

Изменение значений во время перебора

Во время перебора можно менять значения существующих ключей, если набор ключей не меняется.

```
prices = {
    "Кофе": 200,
    "Чай": 150,
```

```
    "Сок": 180
}

for product in prices:
    prices[product] += 20

print(prices)
```

Результат:

```
{'Кофе': 220, 'Чай': 170, 'Сок': 200}
```

Здесь мы не добавляем и не удаляем ключи. Меняются только значения.

Список 2.10 `dict-update-values-loop.py`

```
prices = {
    "Кофе": 200,
    "Чай": 150,
    "Сок": 180
}

for product in prices:
    prices[product] += 20

print(prices)
```

Не меняйте размер dict во время for

Во время прямого перебора словаря не стоит добавлять или удалять ключи этого же словаря.

Такой код не используйте как рабочий:

```
for key in data:
    del data[key]
```

И такой тоже:

```
for key in data:
    data["new"] = 100
```

Изменение количества ключей во время итерации может привести к ошибке:

```
RuntimeError: dictionary changed size during iteration
```

> ⚠ Главное правило
>
> Значения существующих ключей менять можно.
> Количество ключей во время прямого перебора менять не стоит.

Пример: дорогие товары

С помощью `items()` можно фильтровать словарь по значениям.

```
prices = {
    "Мышь": 2500,
    "Клавиатура": 7000,
    "Монитор": 32000,
    "Кабель": 800
}

for product, price in prices.items():
    if price >= 5000:
        print(product, "→", price)
```

Результат:

```
Клавиатура → 7000
Монитор → 32000
```

Здесь `items()` даёт пару, распаковка разделяет её на `product` и `price`, а `if` оставляет только нужные товары.

Итоговый пример: анализ склада

Соберём `get()`, `items()`, `update()` и `pop()` в одном сценарии.

Список 2.11 `expensive-products.py`

```
prices = {
    "Мышь": 2500,
    "Клавиатура": 7000,
    "Монитор": 32000,
    "Кабель": 800
}

for product, price in prices.items():
    if price >= 5000:
        print(product, "→", price)
```

```
stock = {
    "Ноутбук": 4,
    "Монитор": 0,
    "Клавиатура": 12,
    "Мышь": 0,
    "Наушники": 7
}

total_quantity = 0

for product, quantity in stock.items():
    total_quantity += quantity

    if quantity > 0:
        print(product, "есть в наличии")

print("Всего единиц:", total_quantity)

webcam = stock.get("Веб-камера", 0)

print("Веб-камер:", webcam)

stock.update({
    "Монитор": 3,
    "Веб-камера": 5
})

removed = stock.pop("Мышь", 0)

print("Удалено мышей:", removed)
print(stock)
```

В этом примере словарь не просто хранит данные: программа считает остатки, проверяет отсутствующий ключ, применяет поставку и удаляет позицию.

Список 2.12 `dictionary-analysis.py`

```
stock = {
    "Ноутбук": 4,
    "Монитор": 0,
    "Клавиатура": 12,
    "Мышь": 0,
    "Наушники": 7
}

total_quantity = 0

for product, quantity in stock.items():
    total_quantity += quantity

    if quantity > 0:
        print(product, "есть в наличии")

print("Всего единиц:", total_quantity)

webcam = stock.get("Веб-камера", 0)

print("Веб-камер:", webcam)

stock.update({
    "Монитор": 3,
    "Веб-камера": 5
})

removed = stock.pop("Мышь", 0)

print("Удалено мышей:", removed)
print(stock)
```

Что использовать

Краткая памятка:

Операция	Когда нужна
`data.get(key)`	получить необязательное значение
`data.get(key, default)`	заменить `None` своим значением
`data.keys()`	пройти по ключам

Операция	Когда нужна
`data.values()`	пройти по значениям
`data.items()`	пройти по парам `key, value`
`for key in data`	перебрать ключи короткой записью
`data.update(changes)`	применить несколько изменений
`data.pop(key)`	удалить ключ и получить значение
`data.pop(key, default)`	удалить ключ без ошибки при отсутствии
`data.clear()`	очистить словарь

Что важно запомнить

`get()` безопасно получает значение:

```
data.get("email")
data.get("email", "Не указан")
```

`keys()`, `values()` и `items()` дают разные взгляды на один словарь:

```
data.keys()
data.values()
data.items()
```

Цикл по самому словарю перебирает ключи:

```
for key in data:
    ...
```

`items()` удобно распаковывать в две переменные:

```
for key, value in data.items():
    ...
```

`update()` применяет изменения, `pop()` удаляет и возвращает значение, `clear()` очищает словарь.

Во время перебора можно менять значения существующих ключей, но не стоит добавлять или удалять ключи того же словаря.

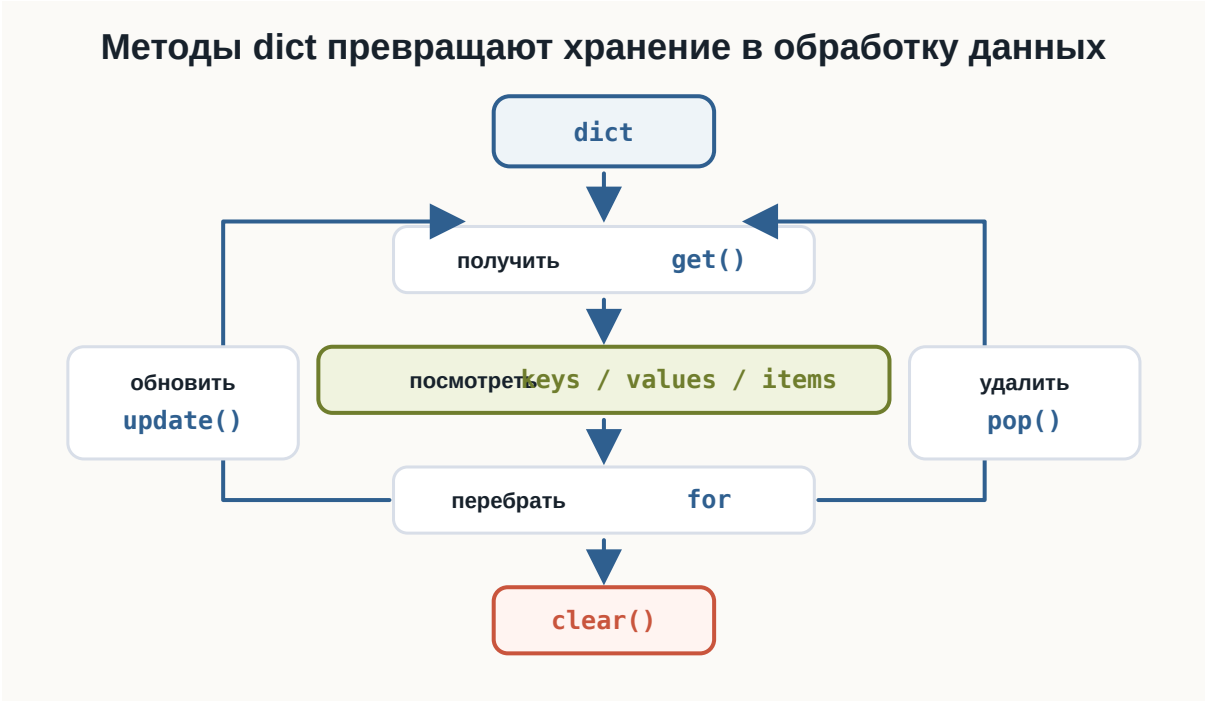


Рисунок 2.5: Карта методов dict

Практические задания

Задание 1. Погода

Дан словарь:

```
weather = {
    "temperature": 18,
    "wind": 4
}
```

Получите `"humidity"` через `get()`. Если поля нет, используйте значение `"неизвестно"`.

Ответ

```
weather = {
    "temperature": 18,
    "wind": 4
}

humidity = weather.get("humidity", "неизвестно")

print("Влажность:", humidity)
```

Задание 2. Системные параметры

Дан словарь:

```
system = {
    "os": "Linux",
    "ram": 32,
    "cores": 8
}
```

Выведите только названия параметров.

Ответ

```
system = {
    "os": "Linux",
    "ram": 32,
    "cores": 8
}

for key in system.keys():
    print(key)
```

Задание 3. Выручка

Даны продажи:

```
sales = {
    "Понедельник": 12000,
    "Вторник": 15000,
    "Среда": 11000,
    "Четверг": 17000
}
```

Вычислите общую выручку, перебирая только значения.

Ответ

```
sales = {
    "Понедельник": 12000,
    "Вторник": 15000,
    "Среда": 11000,
    "Четверг": 17000
}

total = 0

for value in sales.values():
    total += value

print("Выручка:", total)
```

Задание 4. Игровой инвентарь

Дан словарь:

```
inventory = {
    "Зелье": 5,
    "Стрела": 40,
    "Ключ": 2
}
```

Выведите все позиции в формате `предмет : количество`, используя `items()`.

Ответ

```
inventory = {
    "Зелье": 5,
    "Стрела": 40,
    "Ключ": 2
}

for item, quantity in inventory.items():
    print(item, ":", quantity)
```

Задание 5. Конфигурация сервера

Изначально:

```
server = {
    "host": "localhost",
    "port": 8000,
    "debug": True
}
```

Новые настройки:

```
changes = {
    "port": 8080,
    "debug": False,
    "workers": 4
}
```

Примените изменения одной операцией.

Ответ

```
server = {
    "host": "localhost",
    "port": 8000,
    "debug": True
}

changes = {
    "port": 8080,
    "debug": False,
    "workers": 4
}

server.update(changes)

print(server)
```

Задание 6. Корзина

Дана корзина:

```
cart = {
    "Ноутбук": 1,
    "Мышь": 2,
    "Кабель": 3
}
```

Удалите `"Кабель"` через `pop()` и сохраните его количество в переменную.

Ответ

```
cart = {
    "Ноутбук": 1,
    "Мышь": 2,
    "Кабель": 3
}

quantity = cart.pop("Кабель")

print("Удалено:", quantity)
print(cart)
```

Задание 7. Сессия

Дан словарь:

```
session = {
    "user": "alex",
    "active": True
}
```

Удалите `"token"` через `pop()` так, чтобы отсутствие ключа не вызвало ошибку.

> Ответ

```
session = {
    "user": "alex",
    "active": True
}

token = session.pop("token", None)

print(token)
print(session)
```

Задание 8. Изменение цен

Дан прайс:

```
prices = {
    "Кофе": 200,
    "Чай": 150,
    "Какао": 220
}
```

Увеличьте каждую цену на 10%.

Ответ

```
prices = {
    "Кофе": 200,
    "Чай": 150,
    "Какао": 220
}

for product in prices:
    prices[product] = prices[product] * 1.1

print(prices)
```

Задание 9. Результаты игроков

Даны результаты:

```
scores = {
    "Alex": 1250,
    "Maria": 1840,
    "John": 990,
    "Sofia": 1720
}
```

С помощью `items()` найдите игрока с максимальным результатом. Не используйте `max()` как готовое решение.

Ответ

```
scores = {
    "Alex": 1250,
    "Maria": 1840,
    "John": 990,
    "Sofia": 1720
}

best_player = None
best_score = -1

for player, score in scores.items():
    if score > best_score:
        best_player = player
        best_score = score

print("Игрок:", best_player)
print("Результат:", best_score)
```

Задание 10. Анализ склада

Дан склад:

```
stock = {
    "Ноутбук": 4,
    "Монитор": 0,
    "Клавиатура": 12,
    "Мышь": 0,
    "Наушники": 7
}
```

Нужно:

1. подсчитать общее количество единиц товара;
2. вывести товары, которые есть в наличии;
3. получить количество `"Веб-камера"` через `get()` со значением `0`;
4. применить обновление для `"Монитор"` и `"Веб-камера"`;
5. удалить `"Мышь"` через `pop()` со значением по умолчанию;
6. вывести итоговый словарь.

Ответ

```
stock = {
    "Ноутбук": 4,
    "Монитор": 0,
    "Клавиатура": 12,
    "Мышь": 0,
    "Наушники": 7
}

total_quantity = 0

for product, quantity in stock.items():
    total_quantity += quantity

    if quantity > 0:
        print(product, "есть в наличии")

print("Всего единиц:", total_quantity)

webcam = stock.get("Веб-камера", 0)

print("Веб-камер:", webcam)

stock.update({
    "Монитор": 3,
    "Веб-камера": 5
})

removed = stock.pop("Мышь", 0)

print("Удалено мышей:", removed)
print(stock)
```

Что дальше?

Теперь мы умеем обрабатывать один словарь: безопасно получать данные, перебирать ключи и значения, обновлять пары, удалять элементы и очищать структуру.

Следующий шаг — объединить уже знакомые структуры:

```
users = [
    {"name": "Анна", "age": 25},
    {"name": "Максим", "age": 30}
]
```

Такие данные состоят из нескольких уровней. Это тема следующей главы — **«Вложенные структуры данных»**.

Работа со словарями Python: краткое резюме

Работа со словарями в Python строится вокруг нескольких практичных методов. `get()` помогает безопасно получать необязательные значения, `keys()` даёт ключи, `values()` — значения, а `items()` — пары, которые удобно распаковывать в цикле `for`.

Метод `update()` применяет сразу несколько изменений, `pop()` удаляет ключ и возвращает его значение, а `clear()` полностью очищает словарь. При переборе словаря можно менять значения существующих ключей, но не стоит добавлять или удалять ключи в том же прямом цикле.

2 4.5 Вложенные структуры данных

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как объединять списки и словари, чтобы описывать более сложные данные?**

До сих пор мы работали с отдельными структурами.

Список:

```
languages = ["Python", "Java", "Go"]
```

Словарь:

```
user = {
    "name": "Анна",
    "age": 25
}
```

Но реальные данные часто состоят из нескольких объектов.

Например, нужно хранить нескольких пользователей:

```
Анна → 25 лет
Максим → 30 лет
Олег → 27 лет
```

Один словарь описывает одного пользователя.

Несколько пользователей удобно хранить в списке:

```
users = [
    {
        "name": "Анна",
```

```
        "age": 25
    },
    {
        "name": "Максим",
        "age": 30
    },
    {
        "name": "Олег",
        "age": 27
    }
]
```

Так появляются **вложенные структуры данных**.

Что такое вложенная структура

Вложенная структура — это структура данных, внутри которой находятся другие структуры.

Например:

```
users = [
    {"name": "Анна", "age": 25},
    {"name": "Максим", "age": 30}
]
```

Здесь:

```
users
↓
list
↓
каждый элемент
↓
dict
```

Получается:

```
list
├── dict
└── dict
```

Список словарей

Одна из самых распространённых моделей:

```
products = [
    {
        "name": "Ноутбук",
        "price": 85000
    },
    {
        "name": "Монитор",
        "price": 32000
    },
    {
        "name": "Мышь",
        "price": 2500
    }
]
```

Здесь:

```
products
→ список товаров

каждый элемент списка
→ словарь одного товара
```

Список хранит коллекцию объектов, а словарь описывает свойства каждого объекта.

Список 2.1 `list-of-dicts.py`

```
products = [
    {
        "name": "Ноутбук",
        "price": 85000
    },
    {
        "name": "Монитор",
        "price": 32000
    },
    {
        "name": "Мышь",
        "price": 2500
    }
]

print(products)

print(type(products))
print(type(products[0]))
```

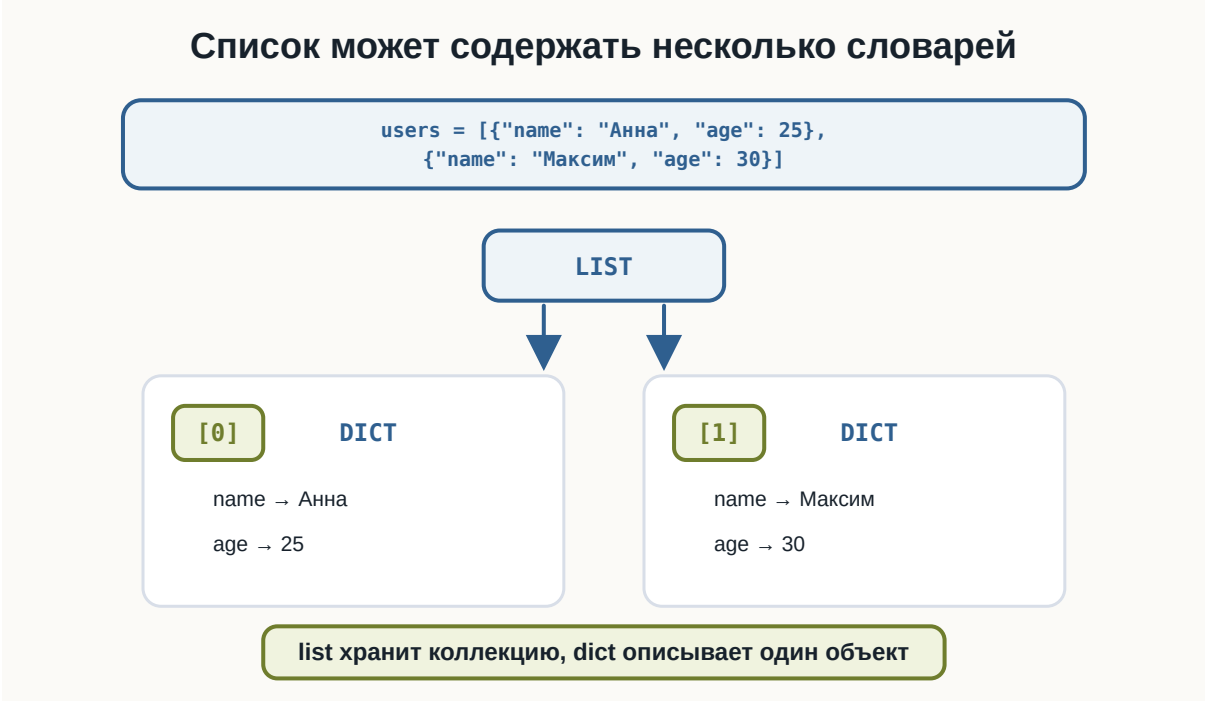


Рисунок 2.1: Список словарей

Доступ к одному объекту

Получим первый элемент списка:

```
products[0]
```

Результат:

```
{
    "name": "Ноутбук",
    "price": 85000
}
```

Теперь можно получить поле этого словаря:

```
products[0]["name"]
```

Результат:

```
Ноутбук
```

Полная цепочка:

```
products
↓
[0]
↓
первый словарь
↓
["name"]
↓
"Ноутбук"
```

Читаем выражение слева направо

Рассмотрим:

```
products[1]["price"]
```

Читаем по шагам:

```
products
↓
элемент с индексом 1
↓
словарь товара
↓
ключ "price"
↓
32000
```

Такой способ чтения помогает разбирать вложенные выражения без угадывания.

Если написать индекс, которого нет, например `products[10]`, Python получит несуществующий элемент списка и выдаст `IndexError`.

Если элемент есть, но внутри словаря нет нужного ключа, например `products[0]["missing"]`, будет `KeyError`.

> **Разбирайте доступ по шагам**
>
> Если выражение кажется сложным:
>
> ```
> products[1]["price"]
> ```
>
> мысленно разбейте его:
>
> ```
> product = products[1]
> price = product["price"]
> ```
>
> Результат тот же, но структура становится понятнее.

Список 2.2 `nested-access.py`

```
products = [
    {"name": "Ноутбук", "price": 85000},
    {"name": "Монитор", "price": 32000}
]

print(products[0]["name"])
print(products[1]["price"])
```



Рисунок 2.2: Вложенный доступ

Изменение вложенного значения

Элемент внутри вложенного словаря можно изменить:

```
products = [
    {
        "name": "Ноутбук",
        "price": 85000
    },
    {
        "name": "Монитор",
        "price": 32000
    }
]

products[1]["price"] = 30000

print(products[1])
```

Теперь второй товар содержит:

```
price → 30000
```

Мы:

```
нашли элемент списка
↓
получили словарь
↓
изменили значение по ключу
```

Добавление поля во вложенный словарь

Можно добавить новое поле:

```
products[0]["available"] = True
```

Теперь первый товар содержит:

```
{
    "name": "Ноутбук",
    "price": 85000,
    "available": True
}
```

Это обычное изменение словаря, только сам словарь находится внутри списка.

Список 2.3 `nested-update.py`

```
products = [
    {"name": "Ноутбук", "price": 85000},
    {"name": "Монитор", "price": 32000}
]

products[1]["price"] = 30000
products[0]["available"] = True

print(products)
```

Перебор списка словарей

Чаще всего все объекты обрабатываются циклом:

```
products = [
    {"name": "Ноутбук", "price": 85000},
    {"name": "Монитор", "price": 32000},
    {"name": "Мышь", "price": 2500}
]

for product in products:
    print(product["name"], "→", product["price"])
```

Результат:

```
Ноутбук → 85000
Монитор → 32000
Мышь → 2500
```

На каждой итерации:

```
product
```

содержит один словарь.

Список 2.4 `nested-iteration.py`

```
products = [
    {"name": "Ноутбук", "price": 85000},
    {"name": "Монитор", "price": 32000},
    {"name": "Мышь", "price": 2500}
]

for product in products:
    print(product["name"], "→", product["price"])
```



Рисунок 2.3: Перебор списка словарей

Фильтрация объектов

Можно использовать условие внутри цикла:

```
products = [
    {"name": "Ноутбук", "price": 85000},
    {"name": "Монитор", "price": 32000},
    {"name": "Мышь", "price": 2500}
]

for product in products:
    if product["price"] >= 30000:
        print(product["name"])
```

Результат:

```
Ноутбук
Монитор
```

Теперь условие применяется к каждому объекту отдельно.

Список 2.5 `nested-filter.py`

```
products = [
    {"name": "Ноутбук", "price": 85000},
    {"name": "Монитор", "price": 32000},
    {"name": "Мышь", "price": 2500}
]

for product in products:
    if product["price"] >= 30000:
        print(product["name"])
```

Безопасное получение поля

Не все словари обязательно содержат одинаковые поля.

Например:

```
users = [
    {
        "name": "Анна",
        "city": "Казань"
    },
    {
        "name": "Максим"
    }
]
```

У второго пользователя нет `"city"`.

Поэтому:

```
user["city"]
```

может вызвать `KeyError`.

Используем уже знакомый `get()`:

```
for user in users:
    city = user.get("city", "Не указан")

    print(user["name"], "→", city)
```

Результат:

```
Анна → Казань
Максим → Не указан
```

Список 2.6 `optional-field.py`

```
users = [
    {
        "name": "Анна",
        "city": "Казань"
    },
    {
        "name": "Максим"
    }
]

for user in users:
    city = user.get("city", "Не указан")
    print(user["name"], "→", city)
```

Добавление нового объекта

Список остаётся обычным изменяемым списком.

Поэтому можно использовать `append()`:

```
users = [
    {"name": "Анна", "age": 25}
]

users.append({
    "name": "Максим",
    "age": 30
})

print(users)
```

Теперь список содержит два словаря.

Создание объекта перед append()

Иногда удобнее сначала создать словарь:

```
new_user = {
    "name": "Олег",
    "age": 27
}
```

а затем:

```
users.append(new_user)
```

Это особенно полезно, когда данные собираются постепенно.

Создание объекта из input()

Например:

```
users = []

name = input("Имя: ")
age = int(input("Возраст: "))

user = {
    "name": name,
    "age": age
}

users.append(user)

print(users)
```

Получается цепочка:

```
input()
↓
переменные
↓
dict
```

```
↓
append()
↓
list
```

Список 2.7 append-dict.py

```
users = []

name = input("Имя: ")
age = int(input("Возраст: "))

user = {
    "name": name,
    "age": age
}

users.append(user)

print(users)
```

Словарь со списком

Вложенность может работать и наоборот.

Например:

```
course = {
    "title": "Python",
    "topics": [
        "Переменные",
        "Условия",
        "Циклы"
    ]
}
```

Здесь:

```
course
↓
dict
```

```
↓
ключ "topics"
↓
list
```

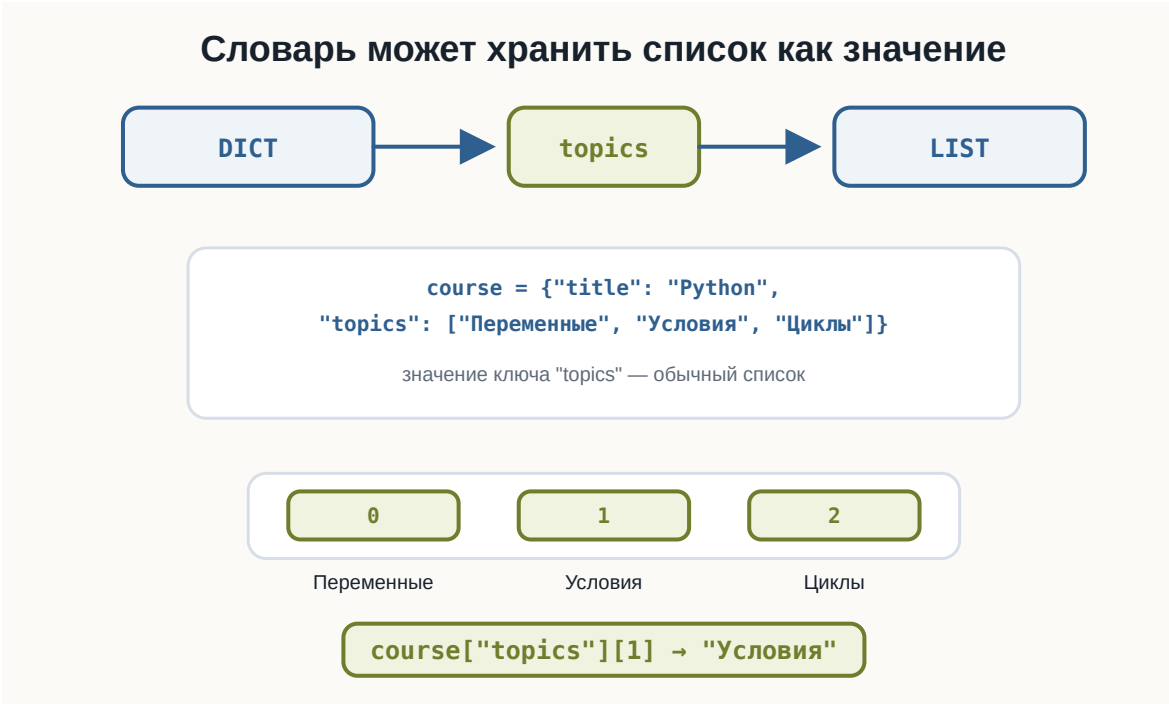


Рисунок 2.4: Словарь со списком

Доступ к элементу списка внутри словаря

Получим список тем:

```
course["topics"]
```

Получим первую тему:

```
course["topics"][0]
```

Результат:

```
Переменные
```

Читаем:

```
course
↓
ключ "topics"
↓
список
↓
индекс 0
↓
"Переменные"
```

Изменение списка внутри словаря

Поскольку `"topics"` содержит обычный список:

```
course["topics"].append("Функции")
```

Теперь:

```
print(course["topics"])
```

получим:

```
['Переменные', 'Условия', 'Циклы', 'Функции']
```

Методы списка работают как обычно.

Перебор списка внутри словаря

Например:

```
course = {
    "title": "Python",
    "topics": [
        "Переменные",
        "Условия",
        "Циклы"
```

```
    ]
}

for topic in course["topics"]:
    print(topic)
```

Цикл получает элементы вложенного списка.

Список 2.8 `dict-with-list.py`

```
course = {
    "title": "Python",
    "topics": [
        "Переменные",
        "Условия",
        "Циклы"
    ]
}

print(course["topics"][0])

course["topics"].append("Функции")

for topic in course["topics"]:
    print(topic)
```

Несколько списков внутри словаря

Словарь может хранить несколько списков:

```
project = {
    "name": "Website",
    "developers": ["Анна", "Максим"],
    "technologies": ["Python", "HTML", "CSS"]
}
```

Получить второго разработчика:

```
project["developers"][1]
```

Получить первую технологию:

```
project["technologies"][0]
```

Главное — каждый раз понимать, какой тип находится на текущем уровне.

Проверяйте тип структуры на каждом уровне

Рассмотрим:

```
project["developers"][1]
```

По шагам:

```
project
→ dict

project["developers"]
→ list

project["developers"][1]
→ str
```

А:

```
users[0]["name"]
```

разбирается иначе:

```
users
→ list

users[0]
→ dict

users[0]["name"]
→ str
```

Это один из самых полезных навыков при работе с вложенными данными.

Словарь внутри словаря

Можно вложить словарь в другой словарь:

```
user = {
    "name": "Анна",
    "contact": {
        "email": "anna@example.com",
        "city": "Казань"
    }
}
```

Получить email:

```
user["contact"]["email"]
```

Результат:

```
anna@example.com
```

В этой главе ограничимся простыми структурами глубиной 2–3 уровня.

Изменение вложенного словаря

Например:

```
user["contact"]["city"] = "Москва"
```

Получаем:

```
contact
└── city → Москва
```

Логика та же:

```
получить внешний словарь
↓
получить внутренний словарь
↓
изменить поле
```

Список 2.9 `nested-dict.py`

```
user = {
    "name": "Анна",
    "contact": {
        "email": "anna@example.com",
        "city": "Казань"
    }
}

print(user["contact"]["email"])

user["contact"]["city"] = "Москва"

print(user)
```

Пример: заказы

```
orders = [
    {
        "id": 101,
        "customer": "Анна",
        "total": 4500,
        "paid": True
    },
    {
        "id": 102,
        "customer": "Максим",
        "total": 7300,
        "paid": False
    }
]
```

Выведем неоплаченные заказы:

```
for order in orders:
    if not order["paid"]:
        print(
            "Заказ",
            order["id"],
            "→",
            order["customer"]
        )
```

Здесь список словарей уже напоминает данные реального приложения.

Пример: сумма заказов

Можно обработать числовое поле всех объектов:

```
orders = [
    {"id": 101, "total": 4500},
    {"id": 102, "total": 7300},
    {"id": 103, "total": 2100}
]

total = 0

for order in orders:
    total += order["total"]

print("Общая сумма:", total)
```

Результат:

```
Общая сумма: 13900
```

Поиск объекта

Например, найдём пользователя по имени:

```
users = [
    {"name": "Анна", "age": 25},
    {"name": "Максим", "age": 30},
    {"name": "Олег", "age": 27}
]

search_name = "Максим"

for user in users:
    if user["name"] == search_name:
        print(user)
```

Мы пока не усложняем логику досрочным выходом из цикла.

Главная идея — поиск нужного словаря по его полю.

Добавление данных в найденный объект

Например:

```
users = [
    {"name": "Анна", "active": True},
    {"name": "Максим", "active": False}
]

for user in users:
    if user["name"] == "Максим":
        user["active"] = True

print(users)
```

Изменяется конкретный словарь внутри списка.

Итоговый пример: команда

Теперь объединим несколько операций в одной модели:

```
team = {
    "name": "Backend",
    "members": [
        {"name": "Анна", "experience": 3},
        {"name": "Максим", "experience": 5},
        {"name": "Олег", "experience": 2}
    ]
}
```

Здесь внешний словарь описывает команду, а список `"members"` хранит словари участников.

Программа может посчитать средний опыт, выбрать участников по условию и изменить вложенное значение.

Не делайте вложенность глубже без необходимости

Технически можно добавлять новые уровни почти бесконечно, но такой код быстро становится трудно читать.

```
team["members"][0]["name"]
```

Такую цепочку ещё можно спокойно разобрать: словарь, список, словарь, поле.

А если выражение становится заметно длиннее, структуру стоит внимательно пересмотреть.

На текущем этапе хороший ориентир:

> Если доступ превращается в длинную цепочку из множества `[]`, структуру стоит внимательно пересмотреть.

⚠ Сложнее — не значит лучше

Вложенные структуры полезны, но чрезмерная вложенность усложняет:

- чтение;
- изменение;
- поиск ошибок;
- обработку данных.

Пока используйте 2–3 понятных уровня.

Как выбрать структуру

Рассмотрим несколько моделей.

Несколько товаров:

```
[
    {"name": "Ноутбук", "price": 85000},
    {"name": "Монитор", "price": 32000}
]
```

Подходит:

```
list
↓
dict
```

Один курс с несколькими темами:

```
{
    "title": "Python",
    "topics": ["Циклы", "Строки", "Списки"]
}
```

Подходит:

```
dict
↓
list
```

Пользователь с контактными данными:

```
{
    "name": "Анна",
    "contact": {
        "email": "anna@example.com"
    }
}
```

Подходит:

```
dict
↓
dict
```

Структуру выбирают исходя из смысла данных.

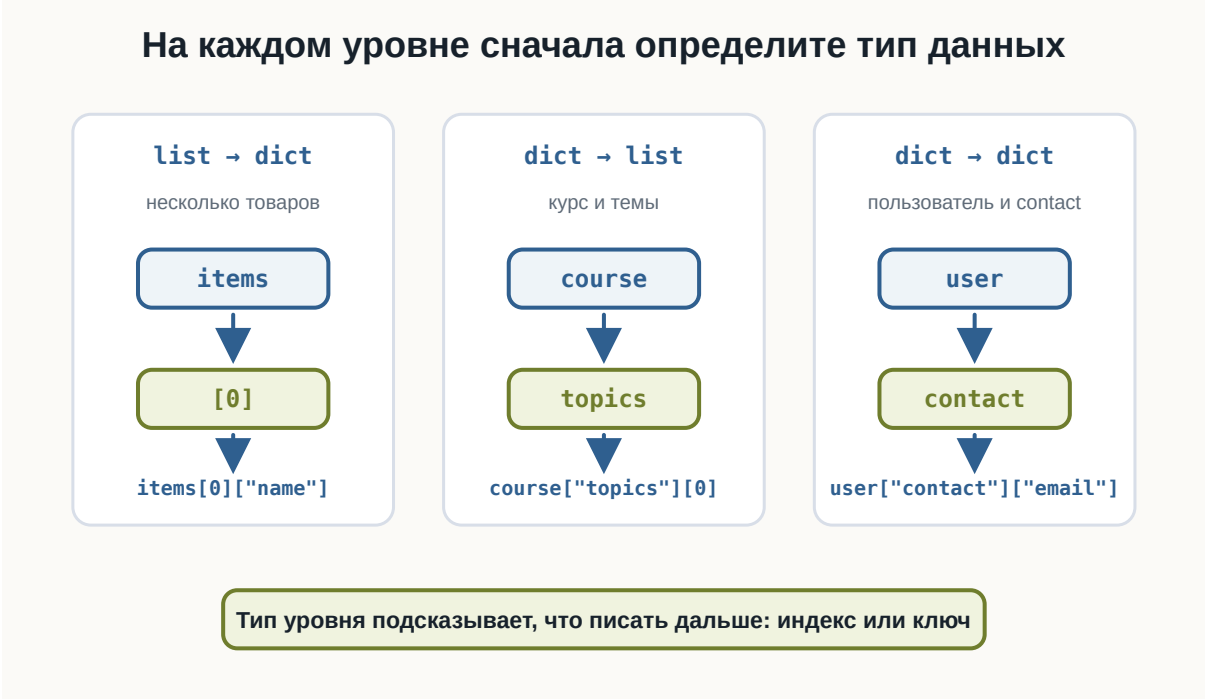


Рисунок 2.5: Карта вложенных структур

Что важно запомнить

Список словарей:

```
users = [
    {"name": "Анна"},
    {"name": "Максим"}
]
```

Доступ:

```
users[0]["name"]
```

Перебор:

```
for user in users:
    print(user["name"])
```

Словарь со списком:

```
course = {
    "topics": ["Условия", "Циклы"]
}
```

Доступ:

```
course["topics"][0]
```

Словарь внутри словаря:

```
user = {
    "contact": {
        "email": "anna@example.com"
    }
}
```

Доступ:

```
user["contact"]["email"]
```

Для необязательных полей словаря полезен:

```
get()
```

Главное правило:

> На каждом уровне понимайте, с каким типом данных вы сейчас работаете.

Практические задания

Задание 1. Фильмотека

Дан список:

```
movies = [
    {"title": "Interstellar", "year": 2014},
    {"title": "Dune", "year": 2021},
    {"title": "Arrival", "year": 2016}
]
```

Выведите названия всех фильмов.

Ответ

```
movies = [
    {"title": "Interstellar", "year": 2014},
    {"title": "Dune", "year": 2021},
    {"title": "Arrival", "year": 2016}
]

for movie in movies:
    print(movie["title"])
```

Задание 2. Изменение статуса сервера

Даны серверы:

```
servers = [
    {"name": "api-1", "online": True},
    {"name": "api-2", "online": False},
    {"name": "api-3", "online": True}
]
```

Измените статус `api-2` на `True`.

Ответ

```
servers = [
    {"name": "api-1", "online": True},
    {"name": "api-2", "online": False},
    {"name": "api-3", "online": True}
]

for server in servers:
    if server["name"] == "api-2":
        server["online"] = True

print(servers)
```

Задание 3. Студент и предметы

Дан словарь:

```
student = {
    "name": "Мария",
    "subjects": ["Python", "SQL", "Git"]
}
```

Выведите каждый предмет на отдельной строке.

Ответ

```
student = {
    "name": "Мария",
    "subjects": ["Python", "SQL", "Git"]
}

for subject in student["subjects"]:
    print(subject)
```

Задание 4. Новый навык

Дан профиль:

```
developer = {
    "name": "Alex",
    "skills": ["Python", "Git"]
}
```

Добавьте навык `"Docker"`.

Ответ

```
developer = {
    "name": "Alex",
    "skills": ["Python", "Git"]
}

developer["skills"].append("Docker")

print(developer)
```

Задание 5. Контактные данные

Дан:

```
employee = {
    "name": "Анна",
    "contact": {
        "email": "anna@example.com",
        "phone": "+123456789"
    }
}
```

Выведите email сотрудника.

Ответ

```
employee = {
    "name": "Анна",
    "contact": {
        "email": "anna@example.com",
        "phone": "+123456789"
    }
}

print(employee["contact"]["email"])
```

Задание 6. Средняя температура

Даны измерения:

```
measurements = [
    {"city": "Москва", "temperature": 18},
    {"city": "Казань", "temperature": 22},
    {"city": "Самара", "temperature": 20},
    {"city": "Омск", "temperature": 16}
]
```

Вычислите среднюю температуру.

Ответ

```
measurements = [
    {"city": "Москва", "temperature": 18},
    {"city": "Казань", "temperature": 22},
    {"city": "Самара", "temperature": 20},
    {"city": "Омск", "temperature": 16}
]

total = 0

for measurement in measurements:
    total += measurement["temperature"]

average = total / len(measurements)

print("Средняя температура:", average)
```

Задание 7. Только доступные билеты

Даны:

```
tickets = [
    {"event": "Python Meetup", "available": True},
    {"event": "Data Conference", "available": False},
    {"event": "Open Source Day", "available": True}
]
```

Выведите только мероприятия, на которые ещё есть билеты.

Ответ

```
tickets = [
    {"event": "Python Meetup", "available": True},
    {"event": "Data Conference", "available": False},
    {"event": "Open Source Day", "available": True}
]

for ticket in tickets:
    if ticket["available"]:
        print(ticket["event"])
```

Задание 8. Необязательное поле

Даны пользователи:

```
users = [
    {"name": "Анна", "country": "Россия"},
    {"name": "John"},
    {"name": "Maria", "country": "Испания"}
]
```

Выведите:

```
Анна → Россия
John → Не указана
Maria → Испания
```

Используйте `get()`.

Ответ

```
users = [
    {"name": "Анна", "country": "Россия"},
    {"name": "John"},
    {"name": "Maria", "country": "Испания"}
]

for user in users:
    country = user.get("country", "Не указана")

    print(user["name"], "→", country)
```

Задание 9. Добавление заказа

Есть список:

```
orders = [
    {"id": 101, "total": 1500},
    {"id": 102, "total": 2700}
]
```

Создайте новый заказ:

```
id    → 103
total → 4200
```

и добавьте его в список.

Ответ

```
orders = [
    {"id": 101, "total": 1500},
    {"id": 102, "total": 2700}
]

new_order = {
    "id": 103,
    "total": 4200
}

orders.append(new_order)

print(orders)
```

Задание 10. Анализ команды

Дана команда:

```
team = {
    "name": "Backend",
    "members": [
        {"name": "Анна", "experience": 3},
        {"name": "Максим", "experience": 5},
        {"name": "Олег", "experience": 2}
    ]
}
```

Нужно:

1. вывести название команды;
2. вывести всех участников;
3. найти суммарный опыт;
4. вычислить средний опыт;
5. вывести участников с опытом не меньше 3 лет;
6. увеличить опыт Олега на 1.

Ответ

```
team = {
    "name": "Backend",
    "members": [
        {"name": "Анна", "experience": 3},
        {"name": "Максим", "experience": 5},
        {"name": "Олег", "experience": 2}
    ]
}

print("Команда:", team["name"])

total_experience = 0

for member in team["members"]:
    print(member["name"])

    total_experience += member["experience"]

average = total_experience / len(team["members"])

print("Общий опыт:", total_experience)
print("Средний опыт:", average)

print("Опыт от 3 лет:")

for member in team["members"]:
    if member["experience"] >= 3:
        print(member["name"])

for member in team["members"]:
    if member["name"] == "Олег":
        member["experience"] += 1

print(team)
```

Что дальше?

Теперь мы умеем хранить данные в более реалистичном виде:

```
list
↓
dict
```

или:

```
dict
↓
list
```

Мы можем:

```
получать вложенные значения
↓
перебирать объекты
↓
изменять поля
↓
фильтровать данные
↓
добавлять новые объекты
```

Но при обработке данных часто возникает повторяющийся шаблон.

Например:

```
squares = []

for number in numbers:
    squares.append(number ** 2)
```

Такой код встречается очень часто.

Python позволяет записывать некоторые подобные преобразования короче.

Следующая глава — **«List comprehensions»**.

Вложенные структуры данных Python: краткое резюме

Вложенные структуры данных в Python позволяют объединять `list` и `dict` для описания более сложных объектов. Распространённый вариант — список

словарей, где каждый словарь представляет отдельный объект: пользователя, товар, заказ или задачу.

Доступ к вложенным данным выполняется последовательно: например, `users[0]["name"]` сначала получает элемент списка, а затем значение словаря. Аналогично `course["topics"][0]` сначала получает список по ключу, а затем его элемент.

При работе с вложенными структурами важно понимать тип данных на каждом уровне и избегать лишней глубины. Следующая глава посвящена теме **«List comprehensions»**.

Список 2.10 `orders-analysis.py`

```
orders = [
    {
        "id": 101,
        "customer": "Анна",
        "total": 4500,
        "paid": True
    },
    {
        "id": 102,
        "customer": "Максим",
        "total": 7300,
        "paid": False
    },
    {
        "id": 103,
        "customer": "Мария",
        "total": 2100,
        "paid": True
    }
]

total = 0

for order in orders:
    total += order["total"]

    if not order["paid"]:
        print(
            "Не оплачен:",
            order["id"],
            order["customer"]
        )

print("Сумма заказов:", total)
```

Список 2.11 `team-analysis.py`

```
team = {
    "name": "Backend",
    "members": [
        {"name": "Анна", "experience": 3},
        {"name": "Максим", "experience": 5},
        {"name": "Олег", "experience": 2}
    ]
}

print("Команда:", team["name"])

total_experience = 0

for member in team["members"]:
    total_experience += member["experience"]

    if member["experience"] >= 3:
        print(
            "Опытный участник:",
            member["name"]
        )

average = total_experience / len(team["members"])

print("Средний опыт:", average)

for member in team["members"]:
    if member["name"] == "Олег":
        member["experience"] += 1

print(team)
```

2 4.6 List comprehensions

> ❗ Важное уведомление
>
> Главный вопрос главы:
> **Как создавать новые списки на основе существующих без повторяющегося `for` и `append()`?**

В предыдущих главах мы часто создавали новый список так:

```
numbers = [1, 2, 3, 4, 5]

squares = []

for number in numbers:
    squares.append(number ** 2)

print(squares)
```

Результат:

```
[1, 4, 9, 16, 25]
```

Этот код понятен:

```
создать пустой список
↓
перебрать исходные данные
↓
преобразовать элемент
↓
добавить результат
```

В Python такой шаблон можно записать компактнее:

```
squares = [number ** 2 for number in numbers]
```

Это **list comprehension**.

Список 2.1 `for-vs-comprehension.py`

```
numbers = [1, 2, 3, 4, 5]

squares_loop = []

for number in numbers:
    squares_loop.append(number ** 2)

squares_comprehension = [
    number ** 2
    for number in numbers
]

print(squares_loop)
print(squares_comprehension)
```

Первый list comprehension

Рассмотрим полный пример:

```
numbers = [1, 2, 3, 4, 5]

squares = [number ** 2 for number in numbers]

print(squares)
```

Результат:

```
[1, 4, 9, 16, 25]
```

Общий синтаксис:

```
[expression for item in iterable]
```

Например:

```
[number ** 2 for number in numbers]
```

Здесь:

```
number ** 2
→ что добавить в новый список

for number in numbers
→ откуда брать элементы
```

Список 2.2 `basic-comprehension.py`

```
numbers = [1, 2, 3, 4, 5]

squares = [
    number ** 2
    for number in numbers
]

print(squares)
```

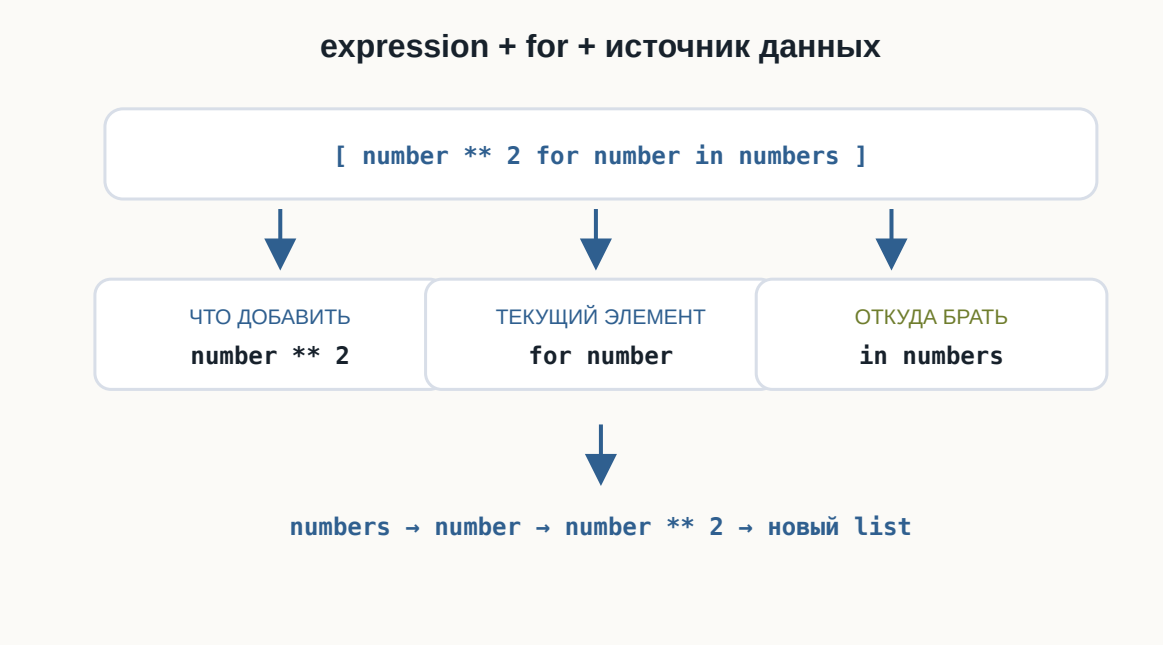


Рисунок 2.1: Строение list comprehension

Обычный цикл и list comprehension

Эти два варианта решают одну задачу.

Обычный цикл:

```
squares = []

for number in numbers:
    squares.append(number ** 2)
```

List comprehension:

```
squares = [number ** 2 for number in numbers]
```

Во втором варианте исчезают:

```
squares = []
```

и:

```
squares.append(...)
```

Python сразу создаёт новый список.

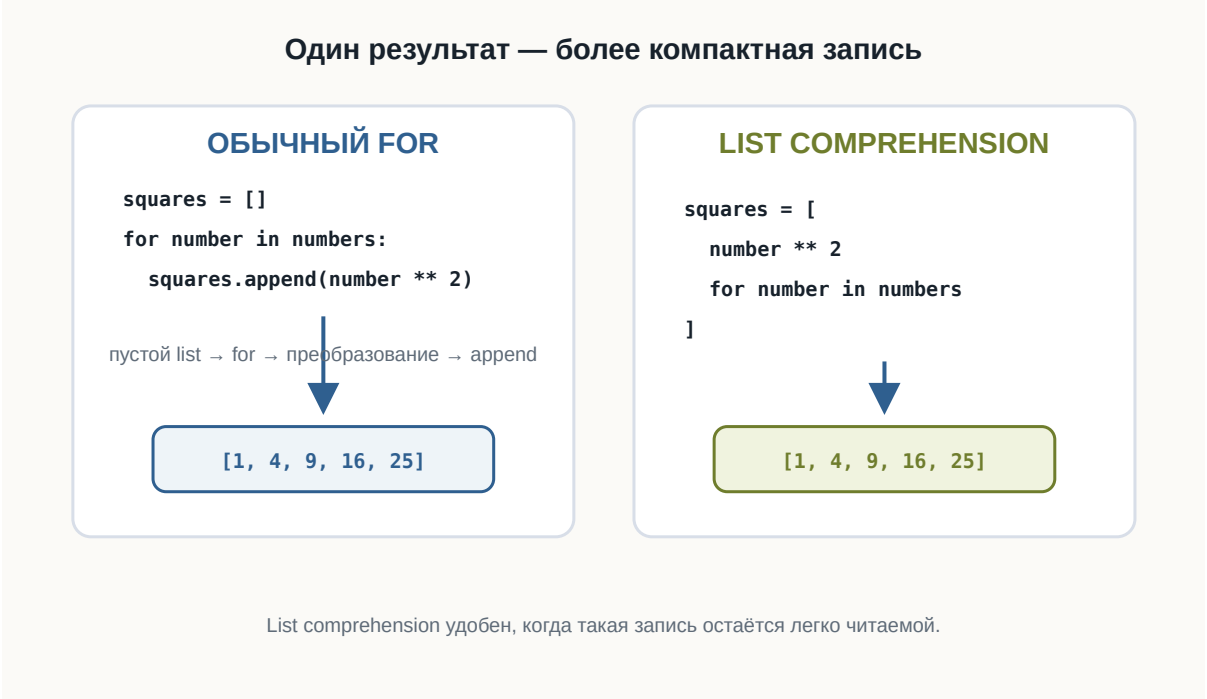


Рисунок 2.2: Обычный for и list comprehension

Как читать list comprehension

Рассмотрим:

```
[length * 2 for length in lengths]
```

Удобно читать начиная с `for`:

```
для каждого length из lengths
↓
вычислить length * 2
↓
добавить результат в новый список
```

Например:

```
lengths = [2, 5, 8]

doubled = [length * 2 for length in lengths]

print(doubled)
```

Результат:

```
[4, 10, 16]
```

> Сначала найдите for
>
> Если запись:
>
> ```
> [value * 2 for value in values]
> ```
>
> пока выглядит непривычно, сначала прочитайте:
>
> ```
> for value in values
> ```
>
> а затем посмотрите, какое выражение стоит слева:
>
> ```
> value * 2
> ```
>
> Так конструкцию проще разобрать.

Новый список создаётся отдельно

List comprehension не изменяет исходный список:

```
numbers = [1, 2, 3]

doubled = [number * 2 for number in numbers]

print(numbers)
print(doubled)
```

Результат:

```
[1, 2, 3]
[2, 4, 6]
```

`numbers` остаётся прежним.

Создаётся новый список:

```
doubled
```

Преобразование чисел

List comprehension особенно удобен для однотипных преобразований.

Например, перевод температур из градусов Цельсия в Фаренгейты:

Список 2.3 `number-transform.py`

```
celsius = [0, 10, 20, 30]

fahrenheit = [
    temperature * 9 / 5 + 32
    for temperature in celsius
]

print(fahrenheit)
```

Результат:

```
[32.0, 50.0, 68.0, 86.0]
```

Каждый элемент проходит через одно выражение:

```
temperature * 9 / 5 + 32
```

Преобразование строк

List comprehension работает не только с числами.

Например:

```
names = ["anna", "max", "olga"]

upper_names = [name.upper() for name in names]

print(upper_names)
```

Результат:

```
['ANNA', 'MAX', 'OLGA']
```

Для каждого элемента вызывается:

```
name.upper()
```

Получение длины строк

Можно получить новое значение из каждого элемента:

Результат:

```
[6, 2, 10]
```

Исходный список содержит строки:

Список 2.4 `string-transform.py`

```
languages = [
    "Python",
    "Go",
    "JavaScript"
]

lengths = [
    len(language)
    for language in languages
]

print(lengths)
```

```
Python
Go
JavaScript
```

Новый — числа:

```
6
2
10
```

Тип элементов нового списка не обязан совпадать с исходным.

Фильтрация с if

Иногда нужно добавить не все элементы.

Обычный вариант:

```
numbers = [3, 8, 12, 5, 20, 7]

large_numbers = []

for number in numbers:
    if number >= 10:
        large_numbers.append(number)

print(large_numbers)
```

Результат:

```
[12, 20]
```

С list comprehension:

```
large_numbers = [
    number
    for number in numbers
    if number >= 10
]
```

Синтаксис:

```
[expression for item in iterable if condition]
```

Список 2.5 `filter-numbers.py`

```
numbers = [3, 8, 12, 5, 20, 7]

large_numbers = [
    number
    for number in numbers
    if number >= 10
]

print(large_numbers)
```

Как работает условие

Рассмотрим:

```
[number for number in numbers if number >= 10]
```

Логика:

```
взять очередной number
↓
проверить number >= 10
↓
```

```
True?
├── да → добавить number
└── нет → пропустить
```

`if` здесь отвечает за то, попадёт ли элемент в новый список.

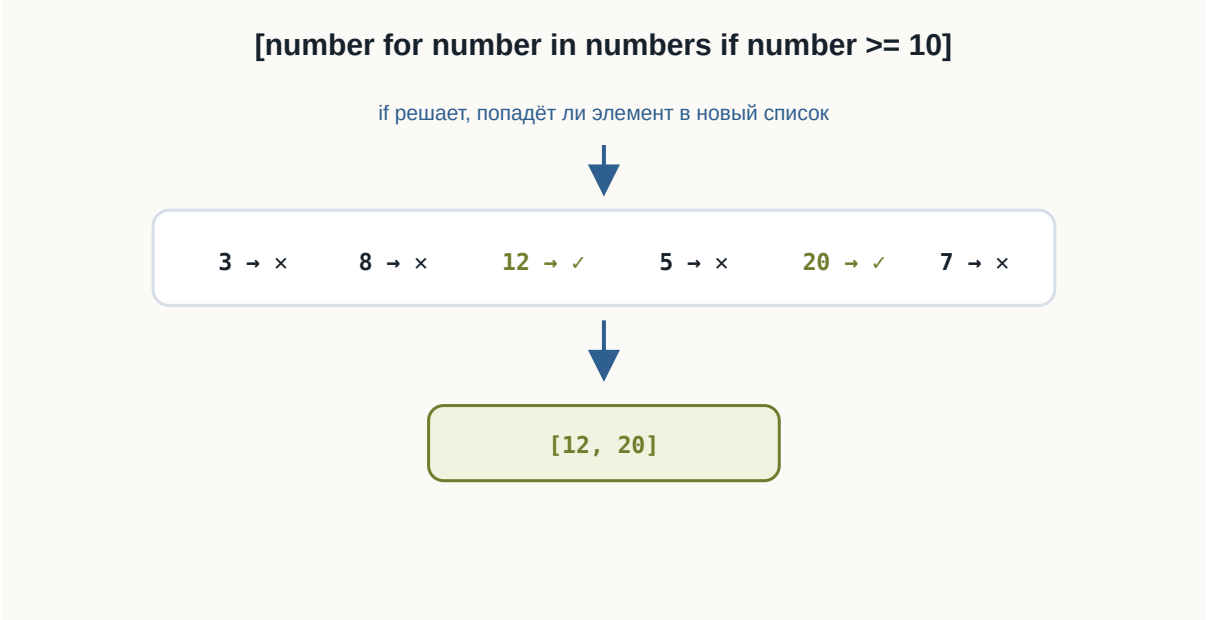


Рисунок 2.3: Фильтрация с if

Чётные числа

Например:

```python
numbers = [1, 2, 3, 4, 5, 6, 7, 8]

even_numbers = [
    number
    for number in numbers
    if number % 2 == 0
]

print(even_numbers)
```

Результат:

```
[2, 4, 6, 8]
```

Исходный список не изменяется.

Фильтрация строк

Условия могут работать и со строками:

```
words = [
    "Python",
    "Go",
    "JavaScript",
    "SQL",
    "TypeScript"
]

long_words = [
    word
    for word in words
    if len(word) >= 6
]

print(long_words)
```

Результат:

```
['Python', 'JavaScript', 'TypeScript']
```

Преобразование и фильтрация вместе

List comprehension может одновременно:

1. выбрать подходящие элементы;
2. преобразовать их.

Например:

```
numbers = [1, 2, 3, 4, 5, 6]

squares = [
    number ** 2
    for number in numbers
    if number % 2 == 0
]

print(squares)
```

Результат:

```
[4, 16, 36]
```

Сначала условие оставляет:

```
2
4
6
```

Затем выражение:

```
number ** 2
```

преобразует их:

```
4
16
36
```

Список 2.6 `transform-and-filter.py`

```
numbers = [1, 2, 3, 4, 5, 6]

squares = [
    number ** 2
    for number in numbers
    if number % 2 == 0
]

print(squares)
```



Рисунок 2.4: Преобразование и фильтрация

List comprehension со списком словарей

Теперь соединим тему с предыдущей главой.

Есть товары:

```
products = [
    {"name": "Ноутбук", "price": 85000},
    {"name": "Монитор", "price": 32000},
    {"name": "Мышь", "price": 2500}
]
```

Получим список названий:

```
names = [
    product["name"]
    for product in products
]

print(names)
```

Результат:

```
['Ноутбук', 'Монитор', 'Мышь']
```

Каждый:

```
product
```

по-прежнему является словарём.

Список 2.7 `dict-list-comprehension.py`

```
products = [
    {"name": "Ноутбук", "price": 85000},
    {"name": "Монитор", "price": 32000},
    {"name": "Мышь", "price": 2500}
]

names = [
    product["name"]
    for product in products
]

print(names)
```

Фильтрация списка словарей

Например, нужны товары дороже `10000`:

```
products = [
    {"name": "Ноутбук", "price": 85000},
    {"name": "Монитор", "price": 32000},
    {"name": "Мышь", "price": 2500}
]

expensive = [
    product
    for product in products
    if product["price"] > 10000
]

print(expensive)
```

В новый список попадут словари ноутбука и монитора.

Получение одного поля после фильтрации

Если нужны только названия:

```
expensive_names = [
    product["name"]
    for product in products
    if product["price"] > 10000
]

print(expensive_names)
```

Результат:

```
['Ноутбук', 'Монитор']
```

Здесь одновременно используются:

```
фильтрация
+
получение поля
+
создание нового списка
```

Список 2.8 `filter-dictionaries.py`

```
products = [
    {"name": "Ноутбук", "price": 85000},
    {"name": "Монитор", "price": 32000},
    {"name": "Мышь", "price": 2500}
]

expensive_names = [
    product["name"]
    for product in products
    if product["price"] > 10000
]

print(expensive_names)
```

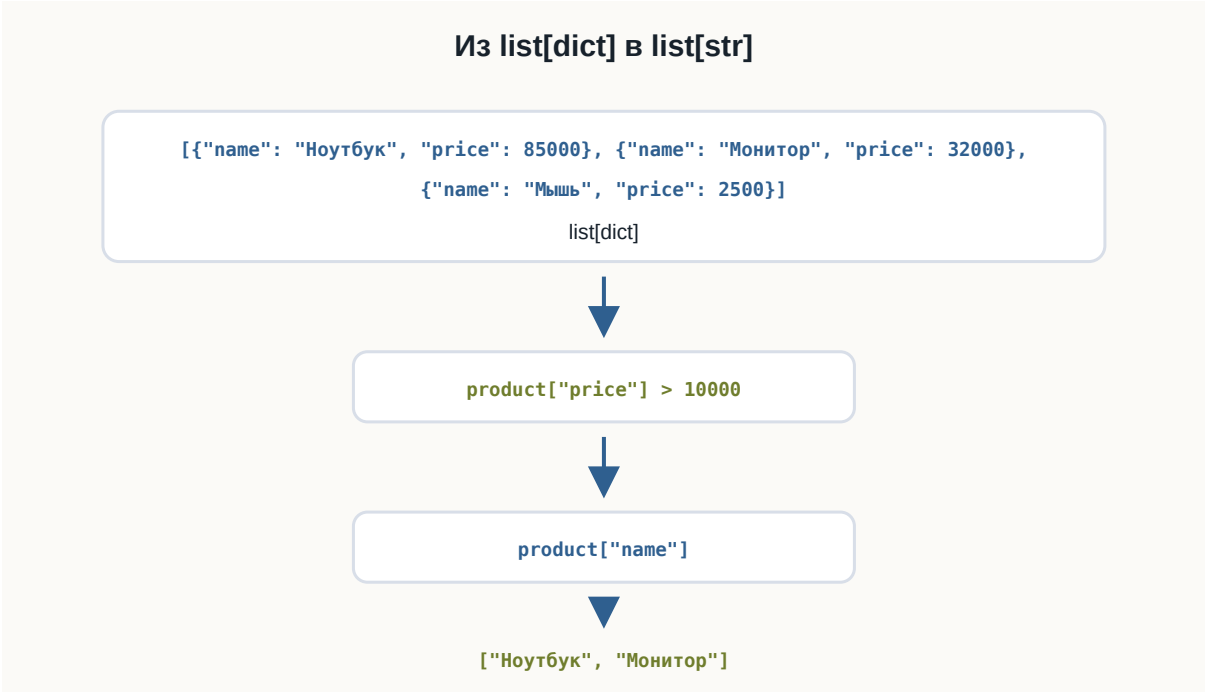


Рисунок 2.5: List comprehension для списка словарей

Можно использовать range()

List comprehension может работать с `range()`:

Список 2.9 `range-comprehension.py`

```
squares = [
    number ** 2
    for number in range(1, 6)
]

print(squares)
```

Результат:

```
[1, 4, 9, 16, 25]
```

Промежуточный список чисел создавать не требуется.

Несколько простых выборок из заказов

Не обязательно объединять все условия в одну конструкцию. Для одного набора заказов можно создать несколько понятных списков: оплаченные `id`, суммы больше `5000` и оплаченные заказы больше `5000`.

Список 2.10 `orders-comprehension.py`

```
orders = [
    {"id": 101, "total": 4500, "paid": True},
    {"id": 102, "total": 7300, "paid": False},
    {"id": 103, "total": 12000, "paid": True},
    {"id": 104, "total": 2800, "paid": True},
    {"id": 105, "total": 9500, "paid": False}
]

paid_ids = [
    order["id"]
    for order in orders
    if order["paid"]
]

large_totals = [
    order["total"]
    for order in orders
    if order["total"] > 5000
]

large_paid_ids = [
    order["id"]
    for order in orders
    if order["paid"] and order["total"] > 5000
]

print("Оплаченные:", paid_ids)
print("Суммы > 5000:", large_totals)
print("Оплаченные > 5000:", large_paid_ids)
```

Каждый список отвечает на один вопрос о заказах. Это проще читать и изменять, чем одну длинную строку.

Когда list comprehension удобен

Он хорошо подходит для простых операций:

```
преобразовать каждый элемент
```

```
отфильтровать элементы
```

```
отфильтровать + преобразовать
```

Например:

```
prices_with_tax = [
    price * 1.2
    for price in prices
]
```

или:

```
active_users = [
    user
    for user in users
    if user["active"]
]
```

Когда обычный for лучше

List comprehension не обязан заменять каждый цикл.

Рассмотрим:

```
result = []

for number in numbers:
    if number > 0:
        print("Обрабатываем:", number)

        value = number ** 2
        result.append(value)
```

Здесь есть несколько действий.

Переписывание такого кода в одну сложную конструкцию сделает его менее понятным.

> ⚠ Короткий код не всегда лучше
>
> Используйте list comprehension, когда преобразование или фильтрация остаются понятными с первого взгляда.
> Если внутри появляется сложная логика, несколько условий или побочные действия — обычный `for` часто читается лучше.

Однострочная или многострочная запись

Короткий вариант:

```
squares = [number ** 2 for number in numbers]
```

Если выражение становится длиннее, его можно оформить:

```
expensive_names = [
    product["name"]
    for product in products
    if product["price"] > 10000
]
```

Это всё тот же list comprehension.

Многострочная запись часто легче читается.

Типичные ошибки

Забыли квадратные скобки

Нам нужен список:

```
[number * 2 for number in numbers]
```

Квадратные скобки здесь являются частью синтаксиса list comprehension.

Перепутали порядок

Правильно:

```
[number * 2 for number in numbers]
```

а не:

```
[for number in numbers number * 2]
```

Удобная схема:

```
что добавить
↓
for
↓
откуда брать
↓
необязательный if
```

Слишком много логики

Если конструкция перестала быстро читаться, лучше вернуться к обычному циклу.

Что выбрать: for или list comprehension

Обычный `for`:

```
result = []

for item in items:
    result.append(...)
```

удобен, когда:

- действий несколько;
- нужна сложная логика;
- важны промежуточные шаги;
- компактная запись ухудшает понимание.

List comprehension:

```
result = [
    expression
    for item in items
]
```

удобен, когда задача сводится к понятному созданию нового списка.

Что важно запомнить

Базовая форма:

```
[expression for item in iterable]
```

Например:

```
[number ** 2 for number in numbers]
```

С условием:

```
[expression for item in iterable if condition]
```

Например:

```
[number for number in numbers if number > 0]
```

Преобразование и фильтрация:

```
[number ** 2 for number in numbers if number % 2 == 0]
```

Со словарями:

```
[user["name"] for user in users]
```

Фильтрация словарей:

```
[
    user
    for user in users
    if user["active"]
]
```

List comprehension создаёт **новый список**.

Используйте его тогда, когда результат остаётся легко читаемым.

Практические задания

Задание 1. Километры в метры

Дан список:

```
kilometers = [1, 3, 5, 10]
```

Создайте новый список со значениями в метрах.

Используйте list comprehension.

> Ответ

```
kilometers = [1, 3, 5, 10]

meters = [
    kilometer * 1000
    for kilometer in kilometers
]

print(meters)
```

Результат:

```
[1000, 3000, 5000, 10000]
```

Задание 2. Длины файлов

Даны имена:

```
files = [
    "main.py",
    "config.yaml",
    "README.md",
    "app.py"
]
```

Создайте список, содержащий длину каждого имени.

Ответ

```
files = [
    "main.py",
    "config.yaml",
    "README.md",
    "app.py"
]

lengths = [
    len(filename)
    for filename in files
]

print(lengths)
```

Задание 3. Положительные значения

Даны изменения баланса:

```
changes = [-150, 300, -50, 700, -20, 100]
```

Создайте новый список только с положительными значениями.

Ответ

```
changes = [-150, 300, -50, 700, -20, 100]

income = [
    value
    for value in changes
    if value > 0
]

print(income)
```

Результат:

```
[300, 700, 100]
```

Задание 4. Нечётные числа

Используя:

```
range(1, 21)
```

создайте список нечётных чисел от 1 до 20.

> Ответ

```
odd_numbers = [
    number
    for number in range(1, 21)
    if number % 2 != 0
]

print(odd_numbers)
```

Задание 5. Активные серверы

Даны:

```
servers = [
    {"name": "api-1", "online": True},
    {"name": "api-2", "online": False},
    {"name": "api-3", "online": True},
    {"name": "api-4", "online": False}
]
```

Создайте список словарей только с работающими серверами.

Ответ

```
servers = [
    {"name": "api-1", "online": True},
    {"name": "api-2", "online": False},
    {"name": "api-3", "online": True},
    {"name": "api-4", "online": False}
]

online_servers = [
    server
    for server in servers
    if server["online"]
]

print(online_servers)
```

Задание 6. Имена активных серверов

Используйте те же данные, но создайте список только с именами работающих серверов:

```
['api-1', 'api-3']
```

Ответ

```
servers = [
    {"name": "api-1", "online": True},
    {"name": "api-2", "online": False},
    {"name": "api-3", "online": True},
    {"name": "api-4", "online": False}
]

online_names = [
    server["name"]
    for server in servers
    if server["online"]
]

print(online_names)
```

Задание 7. Цены со скидкой

Даны цены:

```
prices = [1200, 3500, 8000, 15000]
```

Для товаров стоимостью от `5000` создайте новый список цен со скидкой `10%`.

В новый список должны попасть только товары, которые подходят под условие.

> Ответ

```
prices = [1200, 3500, 8000, 15000]

discounted = [
    price * 0.9
    for price in prices
    if price >= 5000
]

print(discounted)
```

Результат:

```
[7200.0, 13500.0]
```

Задание 8. Пользователи старше 18 лет

Даны:

```
users = [
    {"name": "Анна", "age": 17},
    {"name": "Максим", "age": 25},
    {"name": "Мария", "age": 19},
    {"name": "Олег", "age": 16}
]
```

Создайте список имён пользователей, которым не меньше 18 лет.

Ответ

```
users = [
    {"name": "Анна", "age": 17},
    {"name": "Максим", "age": 25},
    {"name": "Мария", "age": 19},
    {"name": "Олег", "age": 16}
]

adult_names = [
    user["name"]
    for user in users
    if user["age"] >= 18
]

print(adult_names)
```

Результат:

```
['Максим', 'Мария']
```

Задание 9. Расширения Python-файлов

Даны:

```
files = [
    "main.py",
    "index.html",
    "app.py",
    "styles.css",
    "test.py"
]
```

Создайте новый список только с Python-файлами.

Используйте метод строки `endswith()`.

Ответ

```
files = [
    "main.py",
    "index.html",
    "app.py",
    "styles.css",
    "test.py"
]

python_files = [
    filename
    for filename in files
    if filename.endswith(".py")
]

print(python_files)
```

Результат:

```
['main.py', 'app.py', 'test.py']
```

Задание 10. Анализ заказов

Даны:

```
orders = [
    {"id": 101, "total": 4500, "paid": True},
    {"id": 102, "total": 7300, "paid": False},
    {"id": 103, "total": 12000, "paid": True},
    {"id": 104, "total": 2800, "paid": True},
    {"id": 105, "total": 9500, "paid": False}
]
```

С помощью отдельных list comprehensions создайте:

1. список оплаченных заказов;
2. список `id` оплаченных заказов;
3. список сумм заказов дороже `5000`;
4. список `id` оплаченных заказов дороже `5000`.

Ответ

```
orders = [
    {"id": 101, "total": 4500, "paid": True},
    {"id": 102, "total": 7300, "paid": False},
    {"id": 103, "total": 12000, "paid": True},
    {"id": 104, "total": 2800, "paid": True},
    {"id": 105, "total": 9500, "paid": False}
]

paid_orders = [
    order
    for order in orders
    if order["paid"]
]

paid_ids = [
    order["id"]
    for order in orders
    if order["paid"]
]

large_totals = [
    order["total"]
    for order in orders
    if order["total"] > 5000
]

large_paid_ids = [
    order["id"]
    for order in orders
    if order["paid"] and order["total"] > 5000
]

print("Оплаченные:", paid_orders)
print("ID оплаченных:", paid_ids)
print("Суммы > 5000:", large_totals)
print("Оплаченные > 5000:", large_paid_ids)
```

Что дальше?

List comprehensions позволяют компактно создавать новые списки:

```
squares = [
    number ** 2
    for number in numbers
]
```

Похожий синтаксис существует и для других структур данных, но это отдельные темы.

В текущем опубликованном плане после этой главы нет подключённой следующей страницы. Следующая запланированная глава в структуре проекта — **«Обработка ошибок»**.

List comprehensions в Python: краткое резюме

List comprehension — компактный синтаксис Python для создания нового списка на основе существующей последовательности. Базовая форма записывается как `[expression for item in iterable]`, а необязательный `if` позволяет сразу отфильтровать элементы.

List comprehensions подходят для простых преобразований и фильтрации: вычисления новых значений, обработки строк, выбора элементов по условию и извлечения полей из списка словарей. При этом исходный список не изменяется — создаётся новый.

Если логика становится сложной или требует нескольких последовательных действий, обычный цикл `for` обычно остаётся более читаемым решением.

2 4.7 Обработка ошибок

> ❗ Важное уведомление
>
> **Главный вопрос главы:**
> Как сделать так, чтобы программа могла обработать ожидаемую ошибку и продолжить работу?

Представим программу, которая получает возраст пользователя:

```
age = int(input("Введите возраст: "))

print("Ваш возраст:", age)
```

Если пользователь введёт:

```
25
```

программа выполнится успешно.

Но что произойдёт, если ввести:

```
двадцать пять
```

Python не сможет преобразовать эту строку в число.

Возникнет исключение:

```
ValueError
```

Если исключение не обработано, выполнение программы прервётся.

Python позволяет заранее определить, что делать в подобных ситуациях.

Для этого используются `try` и `except`.

Ошибки и исключения

Не все ошибки одинаковы.

Рассмотрим два случая.

Синтаксическая ошибка:

```
if age >= 18
    print("Доступ разрешён")
```

Здесь пропущено двоеточие.

Python обнаружит `SyntaxError` ещё до выполнения программы.

Исключение во время выполнения:

```
number = int("hello")
```

Синтаксис правильный, но преобразовать `"hello"` в целое число невозможно.

Возникнет:

```
ValueError
```

Именно с такими исключениями мы будем работать в этой главе.

> **Важно**
>
> `try` и `except` не предназначены для исправления неправильного синтаксиса программы.
> Они позволяют обрабатывать исключения, возникающие при выполнении кода.

Знакомые исключения Python

Мы уже встречали ситуации, в которых Python сообщает об ошибке.

Исключение	Причина
`ValueError`	Значение не подходит для операции
`ZeroDivisionError`	Деление на ноль
`TypeError`	Операция с неподходящими типами
`KeyError`	Отсутствующий ключ словаря
`IndexError`	Индекс за пределами последовательности

Например:

```
numbers = [10, 20, 30]

print(numbers[5])
```

Результат:

```
IndexError: list index out of range
```

Или:

```
user = {"name": "Анна"}

print(user["email"])
```

Возникнет `KeyError`.

Необработанное исключение прерывает обычный ход выполнения программы.

Первая обработка ошибки: try и except

Рассмотрим пример:

```
try:
    age = int(input("Введите возраст: "))
    print("Ваш возраст:", age)

except ValueError:
    print("Ошибка: введите целое число")
```

Здесь:

```
try:
```

обозначает блок, в котором может возникнуть исключение.

А:

```
except ValueError:
```

указывает, какое исключение мы готовы обработать.

Если пользователь введёт:

```
abc
```

программа выведет:

```
Ошибка: введите целое число
```

Вместо необработанного исключения пользователь получит понятное сообщение.

Как работает try/except

Общая форма:

```
try:
    # Код, который может вызвать исключение

except ValueError:
    # Действия при ValueError
```

Порядок выполнения:

```
Начало try
    |
    v
Выполнение кода
    |
    +-- Ошибки нет --> продолжение программы
    |
    +-- ValueError --> блок except ValueError
```

Посмотрим на пример без пользовательского ввода:

```
try:
    number = int("42")
    print("Число:", number)

except ValueError:
    print("Не удалось получить число")

print("Программа завершена")
```

Результат:

```
Число: 42
Программа завершена
```

Блок `except` не выполняется, потому что исключение не возникло.

> Что стоит попробовать
>
> Замените:
>
> ```
> int("42")
> ```
>
> на:
>
> ```
> int("Python")
> ```
>
> И сравните результат.
> Это простой способ увидеть оба сценария работы `try/except`.

Что происходит после исключения

Если исключение возникло внутри `try`, оставшиеся инструкции этого блока пропускаются.

```
try:
    number = int("abc")

    print("Число:", number)

except ValueError:
    print("Неверный формат числа")

print("Работа продолжается")
```

Результат:

```
Неверный формат числа
Работа продолжается
```

Строка:

```
print("Число:", number)
```

не выполняется.

После обработки исключения программа переходит к коду после конструкции `try/except`.

Обработка деления на ноль

Ещё один распространённый случай:

```
a = 100
b = 0

try:
    result = a / b
    print(result)

except ZeroDivisionError:
    print("На ноль делить нельзя")
```

Результат:

```
На ноль делить нельзя
```

Измените:

```
b = 0
```

на:

```
b = 5
```

Теперь результатом будет:

```
20.0
```

Мы обрабатываем только конкретную ожидаемую проблему — деление на ноль.

Несколько блоков except

Одна операция может приводить к разным исключениям.

Например, программа получает два числа от пользователя:

```
try:
    a = int(input("Первое число: "))
    b = int(input("Второе число: "))

    result = a / b

    print("Результат:", result)

except ValueError:
    print("Нужно вводить целые числа")

except ZeroDivisionError:
    print("Делитель не может быть равен нулю")
```

Возможны три сценария.

Ввод	Результат
`10` и `2`	`5.0`
`abc` и `2`	Сообщение о неверном числе
`10` и `0`	Сообщение о делении на ноль

Каждый `except` отвечает за свой тип исключения.

Это позволяет показывать пользователю более точные сообщения.

Получение текста ошибки через as

Иногда полезно узнать подробности возникшего исключения.

Для этого используется `as`:

```
try:
    number = int("Python")

except ValueError as error:
    print("Ошибка:", error)
```

Переменная:

```
error
```

содержит объект исключения.

При выводе Python показывает его текстовое представление.

Это удобно при отладке, когда нужно понять причину ошибки.

Обработка нескольких типов одним except

Если действия для нескольких исключений одинаковые, их можно объединить:

```
try:
    value = int("abc")
    result = 100 / value

except (ValueError, ZeroDivisionError):
    print("Не удалось выполнить вычисление")
```

Здесь один блок обрабатывает два типа исключений.

Но если пользователю нужны разные сообщения, лучше использовать отдельные `except`.

Блок else

Иногда после успешного выполнения `try` нужно выполнить дополнительный код.

Для этого существует `else`:

```
try:
    number = int("25")

except ValueError:
    print("Неверное число")

else:
    print("Преобразование успешно")
    print("Результат:", number)
```

Результат:

```
Преобразование успешно
Результат: 25
```

`else` выполняется, только если блок `try` завершился без исключения.

Это позволяет отделить потенциально опасную операцию от дальнейшей обработки результата.

Блок finally

Блок `finally` предназначен для действий, которые нужно выполнить при завершении конструкции `try`.

```
try:
    number = int("abc")

except ValueError:
    print("Ошибка преобразования")

finally:
    print("Проверка завершена")
```

Результат:

```
Ошибка преобразования
Проверка завершена
```

Если заменить `"abc"` на `"42"`, `finally` тоже выполнится.

`finally` часто используют для освобождения ресурсов, например закрытия открытых соединений.

В этой главе достаточно понимать его основное назначение.

> **Особенность finally**
>
> Блок `finally` выполняется и при успешном завершении `try`, и при возникновении исключения, в том числе необработанного.
> При этом `finally` не означает, что исключение автоматически обработано.

Полная конструкция try/except/else/finally

Все части можно использовать вместе:

```
try:
    number = int("20")
    result = 100 / number

except ValueError:
    print("Неверный формат числа")

except ZeroDivisionError:
```

```
    print("Деление на ноль")

else:
    print("Результат:", result)

finally:
    print("Вычисление завершено")
```

Результат:

```
Результат: 5.0
Вычисление завершено
```

Необязательно использовать все блоки одновременно.

Чаще всего достаточно:

```
try:
    ...

except ValueError:
    ...
```

Дополнительные блоки нужны, когда у них есть конкретное назначение.

Самостоятельное создание исключения: raise

До сих пор исключения возникали автоматически.

Но иногда программа должна сама отклонить некорректные данные.

Например, количество товара не может быть отрицательным.

```
quantity = -5

if quantity < 0:
    raise ValueError("Количество не может быть отрицательным")
```

Здесь:

```
raise
```

создаёт исключение.

А:

```
ValueError(...)
```

описывает проблему.

Если исключение не обработано, выполнение программы прервётся.

raise вместе с try/except

Созданное нами исключение можно обработать:

```
try:
    quantity = -5

    if quantity < 0:
        raise ValueError("Количество не может быть отрицательным")

    print("Количество:", quantity)

except ValueError as error:
    print("Ошибка:", error)
```

Результат:

```
Ошибка: Количество не может быть отрицательным
```

Здесь важно различать две операции:

```
raise
```

создаёт исключение.

```
except
```

обрабатывает подходящее исключение.

Пример: проверка суммы перевода

Рассмотрим небольшую программу:

```
balance = 5000

try:
    amount = int(input("Сумма перевода: "))

    if amount <= 0:
        raise ValueError("Сумма должна быть положительной")

    if amount > balance:
        raise ValueError("Недостаточно средств")

except ValueError as error:
    print("Перевод отклонён:", error)

else:
    balance -= amount

    print("Перевод выполнен")
    print("Остаток:", balance)
```

Программа проверяет:

1. Можно ли преобразовать ввод в число.
2. Положительная ли сумма.
3. Достаточно ли средств.

Изменение баланса происходит только после успешной проверки.

Это пример того, как обработка исключений помогает управлять выполнением программы.

Почему не стоит использовать except без типа

В Python существует запись:

```
try:
    number = int("abc")

except:
    print("Произошла ошибка")
```

Она перехватывает слишком широкий круг исключений.

Из-за этого можно случайно скрыть проблему, которую программа не умеет правильно обрабатывать.

В учебных примерах лучше указывать конкретный тип:

```
except ValueError:
```

или:

```
except ZeroDivisionError:
```

> ⚠ Не скрывайте ошибки
>
> Не используйте без необходимости:
>
> ```
> except:
>     pass
> ```
>
> Такая конструкция может скрыть важную проблему, оставив программу в неожиданном состоянии.
> Обрабатывайте только те исключения, для которых действительно предусмотрено разумное действие.

Когда try/except не нужен

Не каждую ситуацию нужно превращать в исключение.

Например, проверить наличие ключа словаря можно через:

```
user = {"name": "Анна"}

email = user.get("email", "Не указан")
```

Это нормальный способ работы с необязательным полем.

Аналогично, если можно заранее проверить условие, иногда проще использовать `if`:

```
divisor = 0

if divisor != 0:
    print(100 / divisor)
else:
    print("Деление невозможно")
```

Выбор зависит от ситуации.

`if` проверяет ожидаемое условие.

`try/except` позволяет обработать исключение, которое возникло при выполнении операции.

Типичные ошибки начинающих

Слишком большой блок try

Не стоит помещать внутрь `try` всю программу.

Лучше ограничивать его операциями, которые действительно могут вызвать ожидаемые исключения.

Неправильный тип исключения

Например:

```
try:
    number = int("abc")

except ZeroDivisionError:
    print("Ошибка")
```

Такой обработчик не поможет, потому что возникает `ValueError`.

Использование переменной после неудачного преобразования

```
try:
    number = int("abc")

except ValueError:
    print("Ошибка")

print(number)
```

Если преобразование не удалось, переменная `number` не получила значение.

Последняя строка приведёт к новой ошибке.

Правильнее выполнять действия с результатом в `else`:

```
try:
    number = int("abc")

except ValueError:
    print("Ошибка")

else:
    print(number)
```

Что важно запомнить

Базовая обработка:

```
try:
    ...

except ValueError:
    ...
```

Несколько исключений:

```
except ValueError:
    ...

except ZeroDivisionError:
    ...
```

Получение сообщения:

```
except ValueError as error:
    print(error)
```

Действие при успешном выполнении:

```
else:
    ...
```

Завершающее действие:

```
finally:
    ...
```

Самостоятельное создание исключения:

```
raise ValueError("Некорректное значение")
```

Главный принцип:

Не пытайтесь скрыть любую ошибку. Обрабатывайте ожидаемые проблемы и оставляйте остальные видимыми для исправления.

Практические задания

Задание 1. Номер этажа

Сложность: начальная

Пользователь вводит номер этажа.

Преобразуйте введённое значение в целое число.

Если пользователь введёт текст вместо числа, выведите:

```
Номер этажа должен быть целым числом
```

Ответ

```
try:
    floor = int(input("Номер этажа: "))
    print("Выбран этаж:", floor)

except ValueError:
    print("Номер этажа должен быть целым числом")
```

Задание 2. Расход топлива

Сложность: начальная

Даны:

```
fuel = 35
distance = 0
```

Вычислите расход топлива на 100 километров:

```
fuel / distance * 100
```

Обработайте ситуацию, когда расстояние равно нулю.

Ответ

```
fuel = 35
distance = 0

try:
    consumption = fuel / distance * 100
    print("Расход:", consumption)

except ZeroDivisionError:
    print("Расстояние не может быть нулевым")
```

Задание 3. Выбор дня недели

Сложность: начальная

Дан список:

```
days = [
    "Понедельник",
    "Вторник",
    "Среда",
    "Четверг",
    "Пятница"
]
```

Пользователь вводит индекс.

Выведите соответствующий день.

Обработайте:

- `ValueError`, если введено не число;
- `IndexError`, если индекс выходит за пределы списка.

Отрицательные индексы в этой задаче также считайте недопустимыми.

Ответ

```
days = [
    "Понедельник",
    "Вторник",
    "Среда",
    "Четверг",
    "Пятница"
]

try:
    index = int(input("Индекс дня: "))

    if index < 0:
        raise IndexError("Отрицательный индекс")

    print(days[index])

except ValueError:
    print("Введите целое число")

except IndexError:
    print("Такого дня в списке нет")
```

Задание 4. Получение настроек

Сложность: средняя

Дан словарь:

```
settings = {
    "theme": "dark",
    "language": "ru",
    "autosave": True
}
```

Пользователь вводит название настройки.

Получите значение через квадратные скобки.

Если ключ отсутствует, обработайте `KeyError`.

Ответ

```
settings = {
    "theme": "dark",
    "language": "ru",
    "autosave": True
}

key = input("Название настройки: ")

try:
    print("Значение:", settings[key])

except KeyError:
    print("Настройка не найдена")
```

Задание 5. Проверка уровня заряда

Сложность: средняя

Пользователь вводит уровень заряда аккумулятора в процентах.

Допустимые значения — от `0` до `100`.

Используйте `raise ValueError`, если число выходит за пределы диапазона.

Обработайте исключение и выведите понятное сообщение.

Ответ

```
try:
    battery = int(input("Заряд аккумулятора: "))

    if battery < 0 or battery > 100:
        raise ValueError("Заряд должен быть от 0 до 100")

except ValueError as error:
    print("Ошибка:", error)

else:
    print("Заряд:", battery, "%")
```

Задание 6. Стоимость доставки

Сложность: средняя

Программа получает:

- вес посылки;
- стоимость доставки одного килограмма.

Оба значения вводятся пользователем.

Нужно:

1. Преобразовать их в `float`.
2. Проверить, что вес больше нуля.
3. Проверить, что тариф неотрицательный.
4. Вычислить стоимость.
5. Обработать `ValueError`.
6. При успешном расчёте вывести результат через `else`.
7. В `finally` вывести сообщение о завершении расчёта.

Ответ

```
try:
    weight = float(input("Вес посылки: "))
    rate = float(input("Тариф за кг: "))

    if weight <= 0:
        raise ValueError("Вес должен быть положительным")

    if rate < 0:
        raise ValueError("Тариф не может быть отрицательным")

    cost = weight * rate

except ValueError as error:
    print("Ошибка:", error)

else:
    print("Стоимость доставки:", cost)

finally:
    print("Расчёт завершён")
```

Задание 7. Распределение мест

Сложность: повышенная

В конференц-зале нужно распределить участников по столам.

Пользователь вводит:

- количество участников;
- количество столов.

Программа должна вычислить, сколько участников приходится на один стол при равномерном распределении без учёта остатка.

Требования:

1. Обработать некорректный числовой ввод.
2. Запретить отрицательное количество участников.
3. Запретить нулевое или отрицательное количество столов.
4. Использовать `raise` для проверки недопустимых значений.
5. Использовать `else` для вывода результата.
6. В `finally` сообщить о завершении проверки.

Ответ

```
try:
    participants = int(input("Количество участников: "))
    tables = int(input("Количество столов: "))

    if participants < 0:
        raise ValueError(
            "Количество участников не может быть отрицательным"
        )

    if tables <= 0:
        raise ValueError(
            "Количество столов должно быть положительным"
        )

    per_table = participants // tables
    remaining = participants % tables

except ValueError as error:
    print("Ошибка:", error)

else:
    print("Участников за столом:", per_table)
    print("Осталось распределить:", remaining)

finally:
    print("Проверка завершена")
```

Задание 8. Мини-проект: терминал покупки билетов

Сложность: повышенная

Разработайте небольшую программу покупки билетов.

Исходные данные:

```
tickets = {
    "standard": 1200,
    "premium": 2500,
    "vip": 5000
}

available = 20
```

Пользователь вводит категорию билета и количество.

Программа должна:

1. Проверить, существует ли категория.
2. Преобразовать количество в целое число.
3. Запретить нулевое и отрицательное количество.
4. Проверить наличие нужного количества билетов.
5. Рассчитать стоимость.
6. Уменьшить остаток только после успешной проверки.
7. Обработать ожидаемые исключения.
8. Вывести результат покупки или понятное сообщение об ошибке.

Ответ

```
tickets = {
    "standard": 1200,
    "premium": 2500,
    "vip": 5000
}

available = 20

try:
    category = input("Категория билета: ").lower()
    quantity = int(input("Количество билетов: "))

    if category not in tickets:
        raise ValueError("Неизвестная категория билета")

    if quantity <= 0:
        raise ValueError("Количество должно быть положительным")

    if quantity > available:
        raise ValueError("Недостаточно билетов")

    total = tickets[category] * quantity

except ValueError as error:
    print("Покупка отклонена:", error)

else:
    available -= quantity

    print("Покупка выполнена")
    print("Категория:", category)
    print("Количество:", quantity)
    print("Стоимость:", total)
    print("Осталось билетов:", available)
```

Что дальше?

Теперь мы умеем обрабатывать ожидаемые ошибки:

```
операция
    ↓
возникло исключение?
    ↓
обработка
    ↓
продолжение работы
```

Мы также научились создавать собственные исключения через `raise`.

Следующий важный шаг — научиться разделять программу на несколько файлов и повторно использовать написанный код.

В следующей главе перейдём к **«Модулям и импортам»**.

Обработка ошибок Python: краткое резюме

Обработка ошибок в Python позволяет управлять исключениями, возникающими во время выполнения программы. Конструкция `try/except` используется для перехвата ожидаемых исключений, например `ValueError`, `ZeroDivisionError`, `KeyError` и `IndexError`.

Блок `else` выполняется при успешном завершении `try`, а `finally` позволяет выполнить завершающие действия независимо от результата. Оператор `raise` используется для явного создания исключений, в том числе при проверке пользовательских данных.

Правильная обработка исключений помогает создавать более устойчивые программы, но не заменяет проверку логики и исправление ошибок в коде.